# RETI DI TELECOMUNICAZIONI

## Teoria, Architetture, Algoritmi e Modelli Prestazionali

**Corso del Corso di Laurea in Ingegneria Informatica e dell'Automazione**
Università degli Studi di Brescia

Docente: **Prof. Renato Lo Cigno**

*Dispensa accademica unificata e annotata a partire dal materiale didattico ufficiale, dalle lezioni frontali, dalle trascrizioni d'aula e dai contributi degli studenti. Copre le prime tre lezioni del corso (Modulo 1, parte introduttiva): evoluzione storica delle telecomunicazioni, panoramica e struttura commerciale di Internet, servizi e funzioni di rete, canali, topologie e architetture protocollari (ISO/OSI e TCP/IP).*

Anno Accademico 2024/2025

# Indice

**Convenzioni Notazionali e Simbologia Adottata** **1**

**1 Storia, Panoramica e Struttura di Internet** **2**
1.1 Breve storia delle telecomunicazioni . . . 2
1.1.1 I fondamenti matematici . . . 2
1.1.2 Applicazione, Protocollo e Standardizzazione . . . 3
1.2 Internet: Panoramica e Struttura . . . 3
1.2.1 Che cos'è Internet . . . 3
1.2.2 L'Architettura a Clessidra di IP . . . 4
1.2.3 Internet: La Rete delle Reti e la Gerarchia degli ISP . . . 4

**2 Servizi, Funzioni, Canali, Topologie e Architetture Protocollari** **6**
2.1 Definizioni Fondamentali: Comunicazione, Servizi e Funzioni . . . 6
2.1.1 Le Tre Funzioni Fondamentali della Rete . . . 6
2.2 Classificazione dei Servizi . . . 6
2.2.1 Banda, Capacità e Velocità: Tre Concetti Distinti . . . 7
2.2.2 Traffico Offerto, Throughput e Ritardo . . . 7
2.2.3 Perdita di Informazione . . . 8
2.3 Canali: Tipi e Modalità di Comunicazione . . . 9
2.3.1 Tipi di Canale . . . 9
2.3.2 Modalità di Comunicazione . . . 9
2.4 Topologie di Rete . . . 10
2.5 Tecniche di Commutazione: Circuito vs Pacchetto . . . 11
2.5.1 Commutazione di Circuito (*Circuit Switching*) . . . 11
2.5.2 Commutazione di Pacchetto (*Packet Switching*) . . . 11
2.6 Architetture Protocollari a Strati . . . 12
2.6.1 Origini ed Evoluzione delle Architetture Protocollari . . . 12
2.6.2 Architetture di Rete e Concetti di Stratificazione . . . 12
2.6.3 Creazione e Gestione delle PDU (Incapsulamento, Segmentazione e Concatenazione) . . . 13
2.6.4 Il Modello di Riferimento ISO/OSI (7 Livelli) e Confronto con TCP/IP . . . 13
2.6.5 Stack nei Nodi e Processamento degli Header . . . 14
2.7 Il Protocollo IPv4: Formato del Datagramma, Campi e Frammentazione . . . 15
2.7.1 Autonomia del Datagramma e Assenza di Delimitatori di Coda . . . 15
2.7.2 Struttura e Campi dell'Header IPv4 . . . 15
2.7.3 Tecniche di Tunneling . . . 16
2.7.4 MTU e Meccanica Analitica della Frammentazione . . . 17
2.8 Indirizzamento IPv4, Subnetting e Indirizzi Speciali . . . 17
2.8.1 L'Indirizzo IP Identifica una Scheda di Rete (NIC), Non un Host . . . 18
2.8.2 Notazione Dotted Decimal e Struttura Gerarchica . . . 18
2.8.3 Subnet Mask e Notazione CIDR . . . 18
2.8.4 Esercitazione d'Aula: Assegnazione Uniforme di Indirizzi IP e Rete di Campus . . . 19
2.8.5 Catalogo Sistematico degli Indirizzi IP Speciali . . . 22
2.8.6 Indirizzi Privati e Consegna Diretta vs Indiretta . . . 22
2.9 Protocolli di Supporto ad IP: ICMP e DHCP . . . 23
2.9.1 ICMP: Internet Control Message Protocol . . . 23
2.9.2 DHCP: Dynamic Host Configuration Protocol . . . 24
2.10 Network Address Translation (NAT) e NAPT . . . 26
2.10.1 Genesi, Ruolo e Architettura Stateful . . . 26
2.10.2 Basic NAT vs NAPT (Network Address and Port Translation) . . . 26
2.10.3 Il Meccanismo di Multiplexing basato sulla Quintupla . . . 26

2.10.4 Esercitazione d'Aula: Tabella NAT a Quintupla, Flussi Uscenti e Filtraggio Inbound . . 27
2.10.5 Esercizio d'Esame: Flussi Paralleli, Blind Forwarding e Riuso di Porte nel NAT (Esercizio 4) . . . 30
2.10.6 Gestione dei Timeout e Rilascio delle Mappature . . . 33
2.10.7 Costi Computazionali, Criticità Architetturali e Tecniche di Traversal . . . 33
2.10.8 Confronto Architetturale: NAT vs Proxy Applicativi . . . 33
2.11 Architettura Interna del Router e Forwarding Hardware . . . 33
2.11.1 Disaccoppiamento dei Piani e i Quattro Blocchi Funzionali . . . 34
2.12 Progettazione di Sottoreti ed Esercizio Pratico di Tabelle di Routing . . . 34
2.12.1 Principi Metodologici: Relazione tra LAN, Subnet e Link Punto-Punto . . . 34
2.12.2 Esercizio d'Esame: Tabelle di Inoltro (Forwarding), Subnetting /20 e /24 e Collegamento Punto-Punto /30 con ISP (Esercizio 5) . . . 34
2.13 Il Protocollo IPv6: Architettura, Header, Indirizzamento ed Estensioni . . . 38
2.13.1 Crisi di IPv4 e Genesi di IPv6: Motivazioni Architetturali . . . 38
2.13.2 L'Header Base a 40 Byte (RFC 8200) . . . 39
2.13.3 Architettura dello Spazio di Indirizzamento a 128 Bit . . . 40
2.13.4 Classificazione degli Indirizzi e Bando al Broadcast . . . 41
2.13.5 Regole di Frammentazione e Path MTU Discovery . . . 41
2.13.6 Architettura degli Extension Headers (RFC 8200) . . . 41
2.13.7 Tecniche di Transizione e Coesistenza con IPv4 . . . 42
2.13.8 Tabella Sinottica Comparativa Sintetica: IPv4 vs IPv6 . . . 42

**3 Livello di Collegamento Dati (*Data Link Layer*) 44**
3.1 Inquadramento, Obiettivi e Modello Architetturale . . . 44
3.1.1 L'Astrazione del "Tubo di Unità Dati" . . . 44
3.1.2 Visibilità della Rete: Ambito Locale vs. Globale . . . 44
3.1.3 I Limiti Fisici del Canale da Compensare . . . 45
3.2 Interfaccia con il Livello Fisico e Architettura Hardware (NIC) . . . 45
3.2.1 Cosa Offre il Livello Fisico al Data Link . . . 45
3.2.2 Collocazione Architetturale: Kernel Space vs. Scheda di Rete (NIC) . . . 45
3.2.3 Analisi Quantitativa: Vincoli di Latenza e Carico CPU a 10 Gbit/s . . . 46
3.3 Le Tre Classi di Servizio Offerte al Livello di Rete . . . 46
3.4 Tecniche di Delimitazione delle Trame (Framing) . . . 47
3.4.1 Definizione e Dipendenza dal Livello Fisico . . . 47
3.4.2 1. Character Count (Conteggio dei Caratteri) . . . 48
3.4.3 2. Character Stuffing (Byte Stuffing) . . . 48
3.4.4 3. Bit Stuffing (Inserimento di Bit) . . . 48
3.4.5 4. Violazioni di Codifica Fisica (Physical Layer Violations) . . . 48
3.4.6 5. Intervalli Temporali e Preambolo (Canali Broadcast) . . . 49
3.5 Rilevazione e Gestione degli Errori . . . 49
3.5.1 Tipologie di Guasto e Filosofia di Rilevazione a Livello 2 . . . 49
3.5.2 Codici a Ripetizione e Parità Semplice . . . 50
3.5.3 Codici di Parità Bidimensionale (Riga/Colonna) . . . 50
3.6 Codici di Ridondanza Ciclica (CRC) . . . 50
3.7 Controllo di Flusso e Negoziazione della Velocità . . . 51
3.7.1 Distinzione Concettuale: Negoziazione della Velocità vs. Controllo di Flusso . . . 51
3.7.2 Origine Fisica del Problema: La Saturazione dei Buffer del Ricevitore . . . 52
3.7.3 Meccanismi di Realizzazione a Livello 2 . . . 52
3.7.4 Ethernet Full-Duplex Flow Control (Standard IEEE 802.3x) e il PAUSE Frame . . . 52
3.8 Controllo di Accesso al Mezzo (MAC) — Inquadramento e Tassonomia Generale . . . 53
3.8.1 Sottolivello MAC nel Modello Architetturale IEEE 802 . . . 53
3.8.2 Tassonomia delle Tecniche di Allocazione del Canale . . . 53
3.8.3 Allocazione Statica del Mezzo (Partizionamento Fisico) . . . 54
3.9 Allocazione Dinamica del Mezzo: Protocolli a Turno vs. a Contesa . . . 54
3.9.1 Protocolli a Turno (Token Passing e Token Ring) . . . 54

3.9.2 Protocolli a Contesa (Random Access): Perché Hanno Vinto la Competizione Storica . . 55
3.10 Modello Stocastico per l'Analisi dei Protocolli a Contesa . . . 55
3.10.1 Ipotesi Fondamentali del Modello e "Regole del Gioco" . . . 56
3.10.2 Il Processo di Generazione di Poisson e Sovrapposizione degli Arrivi . . . 56
3.10.3 Carico Offerto Normalizzato ($G$) e Throughput ($S$) . . . 56
3.11 Protocollo ALOHA Puro (Abramson, 1970) . . . 57
3.11.1 Origine Storica e Principio di Funzionamento . . . 57
3.11.2 La Finestra di Vulnerabilità di ALOHA Puro ($2T$) . . . 57
3.11.3 Derivazione Analitica del Throughput e Limite Teorico del 18.4% . . . 57
3.12 Protocollo Slotted ALOHA (Roberts, 1972) . . . 58
3.12.1 Sincronizzazione Temporale e Dimezzamento del Periodo di Vulnerabilità . . . 58
3.12.2 Derivazione del Throughput e Raddoppio al 36.8% . . . 59
3.12.3 Ambiti di Utilizzo Moderni di Slotted ALOHA . . . 60
3.12.4 Valutazione delle Prestazioni e Dimensionamento di ALOHA e Slotted ALOHA . . . 60
3.13 Carrier Sense Multiple Access (CSMA) . . . 67
3.13.1 Principio Operativo del Carrier Sense . . . 67
3.13.2 Varianti di Persistenza del CSMA . . . 67
3.13.3 Il Ritardo di Vulnerabilità e il Parametro Adimensionale $a$ . . . 67
3.13.4 Il Limite Fisico del Sensing: Perché Collision Detection è Impossibile su Wi-Fi . . . 68
3.13.5 Modellazione Analitica Unificata e Prestazioni di CSMA (0-p, 1-p, p-p) . . . 68
3.14 Ethernet Classica (IEEE 802.3) e CSMA/CD su Cavo Coassiale . . . 77
3.14.1 Il Progetto Originale di Metcalfe (1973–1976) . . . 77
3.14.2 Round Trip Delay (RTD) e la Genesi Fisica della Trama Minima di 64 Byte . . . 77
3.14.3 Procedura di Collision Detection e la Sequenza di Jamming . . . 78
3.14.4 L'Algoritmo di Binary Exponential Backoff . . . 79
3.15 Formato della Trama Ethernet ed Evoluzione del Cablaggio Fisico . . . 80
3.15.1 Struttura della Trama Ethernet II e IEEE 802.3 . . . 80
3.15.2 Evoluzione Fisica dei Mezzi Trasmissivi . . . 80
3.16 Commutazione di Livello 2: Hub vs. Switch e Domini di Rete . . . 81
3.16.1 Dispositivi di Livello 1: Hub e Repeater . . . 81
3.16.2 Dispositivi di Livello 2: Bridge e Switch . . . 81
3.16.3 Algoritmo di Auto-Apprendimento (Backward Learning) . . . 81
3.16.4 Algoritmo di Inoltro e Filtraggio (Filtering and Forwarding) . . . 82
3.16.5 Isolamento dei Domini di Rete e Criticità delle Reti Commutate . . . 82
3.16.6 Esercizio d'Esame: Probabilità di Collisione, Backoff e Throughput in Ethernet con Hub 83
3.17 Il Protocollo Point-to-Point (PPP) . . . 87
3.17.1 Inquadramento e Assenza di Contesa . . . 87
3.17.2 Formato della Trama e Meccanismo di Byte Stuffing . . . 87
3.17.3 Architettura Modulare a Tre Livelli di PPP . . . 87
3.17.4 Cenni sulle Linee Asimmetriche xDSL (ADSL) . . . 88
3.18 Reti Wireless LAN: Architettura IEEE 802.11 (Wi-Fi) . . . 88
3.18.1 Inquadramento nello Standard IEEE 802 e Caratteristiche del Canale Radio . . . 88
3.18.2 Evoluzione Cronologica degli Standard IEEE 802.11 . . . 88
3.18.3 Entità Architetturali di una WLAN: BSS, ESS e Access Point . . . 89
3.18.4 Identificatori di Rete: SSID vs. BSSID e Trame Beacon . . . 89
3.19 Protocollo di Accesso Wi-Fi: CSMA/CA e Meccanismi di Risoluzione . . . 90
3.19.1 L'Impossibilità Fisica di Collision Detection su Canale Radio . . . 90
3.19.2 Spazi Inter-Trama Prioritari (IFS) e Gestione su Canale Libero . . . 90
3.19.3 Contesa su Canale Occupato e la Regola del Congelamento (Freeze) . . . 91
3.19.4 Riscontro Esplicito (ACK) e Raddoppio della Contention Window . . . 91
3.19.5 Geometrie Critiche del Mezzo Radio: Terminale Nascosto ed Esposto . . . 92
3.19.6 Meccanismi ad Alte Prestazioni: TXOP e Aggregazione Trame . . . 93
3.19.7 Esercizio d'Esame: Collision Avoidance, Backoff e Throughput in Reti Wi-Fi . . . 93
3.20 Risoluzione degli Indirizzi (ARP) e Funzioni di Rete Locale . . . 98

3.20.1 Incapsulamento e Duplicità dell'Indirizzamento: Indirizzi MAC vs. Indirizzi IP . . . . . 98
3.20.2 Modalità di Risoluzione: Consegna Diretta vs. Consegna Indiretta . . . . . 98
3.20.3 La Filosofia Broadcast-and-Select e il Protocollo ARP . . . . . 99
3.20.4 Formato Analitico del Pacchetto ARP (RFC 826) . . . . . 99
3.20.5 Gestione della Tabella/Cache ARP e Caching Opportunistico . . . . . 100
3.20.6 Broadcast Storm nelle Reti Commutate e Proxy ARP . . . . . 100

**4 Livello di Trasporto: Servizi, UDP, Trasferimento Affidabile e Protocollo TCP 101**
4.1 Il Livello di Trasporto nell'Architettura Internet . . . . . 101
4.1.1 Posizionamento Architetturale e Visione End-to-End . . . . . 101
4.1.2 Panoramica dei Protocolli di Trasporto in Internet . . . . . 102
4.2 Indirizzamento a Livello Trasporto: Multiplazione e Demultiplazione . . . . . 103
4.2.1 Il Ruolo Fondamentale delle Porte . . . . . 103
4.2.2 Identificazione di un Flusso: La 5-upla . . . . . 103
4.3 Il Protocollo UDP (User Datagram Protocol — RFC 768) . . . . . 104
4.3.1 Caratteristiche Fondamentali e Motivazioni d'Uso . . . . . 104
4.3.2 Perché UDP è relativamente poco diffuso rispetto a TCP? . . . . . 104
4.3.3 Formato del Segmento UDP . . . . . 105
4.3.4 L'Algoritmo dell'Internet Checksum e la Violazione dello Pseudo-Header . . . . . 105
4.4 Principi del Trasferimento Dati Affidabile (RDT) . . . . . 106
4.4.1 La Natura del Problema del Trasporto Affidabile . . . . . 106
4.4.2 Tassonomia delle Imperfezioni del Canale Fisico . . . . . 106
4.5 Il Meccanismo Base Stop-and-Wait e i Suoi Limiti . . . . . 107
4.5.1 Funzionamento del Protocollo Stop-and-Wait . . . . . 107
4.5.2 Analisi Analitica dell'Efficienza dello Stop-and-Wait . . . . . 107
4.6 Protocolli a Finestra Scorrevole (*Sliding Window*) e Pipelining . . . . . 108
4.6.1 Il Concetto di Pipelining . . . . . 108
4.6.2 Definizione e Dinamica delle Finestre: $W_T$ e $W_R$ . . . . . 109
4.6.3 Relazione Critica tra le Finestre: Prevenzione delle Perdite Sistematiche . . . . . 109
4.6.4 Il Protocollo Go-Back-N ($W_R = 1$) . . . . . 109
4.6.5 Il Protocollo Selective Repeat ($W_R > 1$) . . . . . 110
4.6.6 Semantica degli Acknowledgement: Individuali vs Cumulativi . . . . . 110
4.6.7 Politiche di Ritrasmissione al Timeout: Conservativa vs Aggressiva . . . . . 110
4.7 Il Teorema dello Spazio di Numerazione Modulare . . . . . 111
4.7.1 Analisi Dettagliata del Controesempio Critico: $W_T = 3$, $W_R = 2$, $k = 2$ . . . . . 112
4.8 Approfondimento Operativo: Switch Cut-Through nei Data Center . . . . . 113
4.8.1 Commutazione Store-and-Forward vs Cut-Through . . . . . 113
4.8.2 Il Compromesso di Affidabilità e l'Ordine di Grandezza Fisico . . . . . 114
4.9 Il Protocollo TCP: Architettura, Servizi e Ruoli End-to-End . . . . . 114
4.9.1 Caratteristiche Fondamentali del Canale TCP . . . . . 114
4.9.2 Architettura Funzionale del Trasmettitore e del Ricevitore . . . . . 115
4.9.3 Regole Formali per la Generazione degli ACK . . . . . 116
4.10 La PDU di TCP: Struttura Dettagliata dell'Header a 32 bit . . . . . 116
4.10.1 Rappresentazione Architetturale dell'Header . . . . . 116
4.10.2 Disamina Analitica dei Singoli Campi . . . . . 116
4.11 Gestione del Ciclo di Vita della Connessione . . . . . 118
4.11.1 Apertura della Connessione: Il Three-Way Handshake (3WHS) . . . . . 118
4.11.2 Dimensionamento dell'MSS e Path MTU Discovery . . . . . 119
4.11.3 Chiusura della Connessione: Il Teardown a Quattro Vie (Half-Close) . . . . . 120
4.11.4 Chiusura Forzata tramite Flag RST (Reset) . . . . . 120
4.12 Stima del Round-Trip Time e Calcolo Adattativo del Timeout (RTO) . . . . . 120
4.12.1 La Natura Stocastica dell'RTT nelle Reti a Pacchetto . . . . . 121
4.12.2 Il Dilemma di Karn-Partridge e i Timestamp TCP . . . . . 121
4.12.3 L'Algoritmo di Jacobson per la Stima di RTO (RFC 2988) . . . . . 121
4.13 Il Controllo di Flusso End-to-End . . . . . 122

4.13.1 Protezione del Buffer del Ricevitore . . . 123
4.13.2 Il Problema dello Stallo con Finestra a Zero e i Pacchetti Sonda (Probing) . . . 123
4.14 I Fondamenti del Controllo di Congestione: Cause, Costi ed Approccio End-to-End . . . 124
4.14.1 Che cos'è la Congestione di Rete? . . . 124
4.14.2 Analisi dei Costi della Congestione: I Tre Scenari Canonici . . . 124
4.14.3 La Filosofia End-to-End di TCP . . . 124
4.15 Finestra di Congestione e Algoritmi di Crescita: Slow Start e Congestion Avoidance . . . 125
4.15.1 La Variabile Dinamica CWND e il Principio del Self-Clocking . . . 125
4.15.2 L'Algoritmo di Slow Start (Partenza Lenta) . . . 125
4.15.3 L'Algoritmo di Congestion Avoidance (Prevenzione della Congestione) . . . 126
4.15.4 Tracciamento Passo-Passo dei Puntatori (LP e UP) ed Esempio d'Aula . . . 126
4.16 Meccanismi di Recupero delle Perdite: RTO, Fast Retransmit e Fast Recovery . . . 128
4.16.1 Tassonomia degli Eventi di Perdita: Timeout vs Duplicate ACK . . . 128
4.16.2 La Reazione Drastica al Timeout (RTO) . . . 128
4.16.3 L'Algoritmo di Fast Retransmit (Ritrasmissione Rapida) . . . 128
4.16.4 Evoluzione Storica: Il Confronto tra TCP Tahoe e TCP Reno . . . 128
4.16.5 Cenni alle Evoluzioni Moderne: TCP CUBIC e TCP Compound . . . 130
4.17 Modellazione Analitica del Throughput ed Equità (*Fairness*) di Rete . . . 130
4.17.1 Dalla Finestra Fissa alla Finestra Variabile: L'Integrale del Throughput . . . 131
4.17.2 Stima Macroscopica del Throughput Medio a Regime AIMD . . . 131
4.17.3 La Relazione con la Probabilità di Perdita: La Formula di Mathis . . . 131
4.17.4 L'Equità di Rete: La Dimostrazione Geometrica di Chiu-Jain . . . 132
4.17.5 L'Iniquità Strutturale di TCP con RTT Eterogenei *()* . . . 133

**5 Livello di Applicazione (*Application Layer*) 135**
5.1 Principi delle Applicazioni di Rete e Paradigmi Architetturali . . . 135
5.1.1 Ruolo e Funzione dei Protocolli Applicativi . . . 135
5.1.2 Evoluzione del Traffico e Dinamiche di Rete . . . 135
5.1.3 Tassonomia delle Architetture Applicative . . . 136
5.1.4 Processi Comunicanti e Socket . . . 137
5.1.5 Requisiti di Servizio Richiesti al Livello di Trasporto . . . 137
5.2 Il Sistema dei Nomi di Dominio (DNS – Domain Name System) . . . 138
5.2.1 Motivazioni e Definizione del Servizio . . . 138
5.2.2 Perché il DNS non è Centralizzato? . . . 138
5.2.3 Nomi di Dominio: FQDN e PQDN . . . 139
5.2.4 Gerarchia dei Server DNS . . . 139
5.2.5 Modalità di Risoluzione: Query Iterative vs. Ricorsive . . . 139
5.2.6 Caching e Aggiornamento dei Record . . . 140
5.2.7 Record di Risorsa DNS (Resource Record – RR) . . . 140
5.2.8 Formato dei Messaggi DNS . . . 141
5.3 Il Web e il Protocollo HTTP . . . 141
5.3.1 Distinzione Fondamentale tra HTML e HTTP . . . 141
5.3.2 Modello di Servizio e Assenza di Stato (*Statelessness*) . . . 142
5.3.3 Connessioni Non Persistenti vs. Connessioni Persistenti . . . 142
5.3.4 Formato dei Messaggi HTTP . . . 143
5.3.5 Gestione dello Stato: i Cookie . . . 144
5.3.6 Web Caching, Proxy Server e Richieste Condizionali . . . 145
5.3.7 Evoluzione ad Alte Prestazioni: HTTP/2 (RFC 7540) . . . 146
5.4 Altri Protocolli di Rete: Posta Elettronica, FTP e Flussi Multimediali . . . 146
5.4.1 L'Architettura della Posta Elettronica . . . 146
5.4.2 Il Protocollo SMTP (RFC 2821) . . . 147
5.4.3 Confronto Architetturale: HTTP vs. SMTP . . . 148
5.4.4 Formato del Messaggio di Posta ed Estensioni MIME . . . 148
5.4.5 Protocolli di Accesso alla Posta: POP3, IMAP e Webmail . . . 148
5.4.6 Il Protocollo FTP (File Transfer Protocol, RFC 959) . . . 149

5.4.7 Cenni a Protocolli Applicativi Moderni: Messaggistica e Streaming . . . . . 149

# Convenzioni Notazionali e Simbologia Adottata

| Simbolo | Grandezza Fisica | Unità di Misura e Definizione |
|---|---|---|
| $R$ | Frequenza di cifra / Bit rate | [bit/s], velocità con cui i bit sono emessi sul canale. |
| $B$ | Larghezza di banda analogica | [Hz], intervallo spettrale delle frequenze del canale. |
| $C$ | Capacità teorica di canale | [bit/s], limite superiore asintotico di Shannon. |
| $M$ | Numero di livelli di segnale | [adim.], simboli distinti; contribuisce a $C_{\text{Nyquist}} = 2B\log_2 M$. |
| SNR | Rapporto Segnale/Rumore | [adim.] o [dB], $S/N$ o $10\log_{10}(S/N)$; misura la qualità del canale. |
| $f_s$ | Frequenza di campionamento | [Hz]; deve soddisfare $f_s \geq 2f_{\max}$ (Nyquist). |
| $k$ | Bit per campione (PCM) | [adim.], profondità di quantizzazione; $R_{\text{PCM}} = f_s \cdot k$. |
| $L$ | Lunghezza pacchetto / PDU | [bit] o [byte], con 1 byte = 8 bit. |
| $d$ | Distanza fisica collegamento | [m] o [km], lunghezza del cavo o tratta radio. |
| $v$ | Velocità di propagazione | [m/s], tipic. $2{\cdot}10^8$ m/s in fibra/rame, $3{\cdot}10^8$ m/s in aria. |
| $d_{\text{trans}}$ | Ritardo di trasmissione | [s], tempo per trasmettere un pacchetto: $L/R$. |
| $d_{\text{prop}}$ | Ritardo di propagazione | [s], tempo di transito del fronte d'onda: $d/v$. |
| $d_{\text{proc}}$ | Ritardo di elaborazione | [s], ispezione header e lookup di routing. |
| $d_{\text{queue}}$ | Ritardo di accodamento | [s], attesa nei buffer prima della trasmissione. |
| RTT | Round-Trip Time | [s], tempo andata e ritorno di un pacchetto. |
| $\lambda$ | Tasso di arrivo pacchetti | [pacchetti/s], intensità del processo di arrivo (Poisson). |
| $\mu$ | Tasso di servizio | [pacchetti/s], velocità con cui un link smaltisce pacchetti. |
| $A$ | Intensità di traffico telefonico | [Erlang], prodotto $\lambda \cdot t_m$ (tasso di arrivo × durata media). |
| $I$ | Intensità di traffico dati | [adim.], rapporto $\lambda L/R$; deve essere $< 1$ per stabilità. |
| $W$ | Finestra TCP (*window*) | [byte], byte in volo non ancora riscontrati (ACK). |
| TTL | Time to Live | [hop], campo IP decrementato ad ogni router; a 0 il pacchetto è scartato. |
| MTU | Maximum Transfer Unit | [byte], frame massimo trasmissibile (es. 1500 byte Ethernet). |

Tabella 1: Tavola delle grandezze e dei simboli standard adoperati nella trattazione analitica.

**Distinzione Rigorosa tra Bit e Byte**

Nelle reti di telecomunicazioni, le velocità di trasmissione e le capacità dei canali si esprimono tassativamente in multipli **decimali** di bit al secondo: kbit/s $= 10^3$ bit/s, Mbit/s $= 10^6$ bit/s, Gbit/s $= 10^9$ bit/s. Le dimensioni dei dati in memoria si indicano in byte o multipli **binari**: KiB $= 2^{10}$ byte $= 1024$ byte. Confondere bit e byte comporta un errore sistematico di un fattore pari a 8.

# Capitolo 1

# Storia, Panoramica e Struttura di Internet

## 1.1 Breve storia delle telecomunicazioni

**Definizione: Telecomunicazione**

Il trasporto dell'informazione attraverso sistemi di tipo elettromagnetico **nello spazio** (tra due punti geograficamente distinti) o **nel tempo** (memorizzazione per fruizione futura).

Le principali tappe evolutive delle telecomunicazioni:

- **1837 – Telegrafo (Morse)**: prima trasmissione numerica punto-punto a distanza con simboli discreti. Il codice assegna sequenze più brevi alle lettere più frequenti ($E$), anticipando la compressione di sorgente formalizzata un secolo dopo da Shannon.
- **1876 – Telefono (Bell e Meucci)**: trasduzione acustico-elettrica della voce umana. Introduce le **centrali di commutazione** per connettere dinamicamente gli utenti a commutazione di circuito, evitando l'impraticabile maglia completa.
- **Inizio '900 – Onde Elettromagnetiche e Radio (Marconi e Tesla)**: trasmissione wireless nello spazio libero ad alta frequenza, pilastro di radiodiffusione, radar e reti mobili.
- **Il Vincolo della Legacy**: le reti si sono sviluppate per stratificazioni progressive senza poter azzerare gli apparati preesistenti. Ogni generazione garantisce retrocompatibilità: ad esempio, il **5G** mantiene fallback verso il **GSM** (2G) per assicurare chiamate d'emergenza in zone montane o impervie.

### 1.1.1 I fondamenti matematici

**Erlang** (anni '10–'20 del Novecento): applicò il calcolo delle probabilità e la teoria delle code al dimensionamento delle centrali telefoniche per servire $N$ utenti con probabilità di blocco controllata:

$$A = \lambda \cdot t_m \quad [\text{Erlang}] \tag{1.1}$$

dove $\lambda$ è il tasso di arrivo e $t_m$ la durata media della conversazione.

**Shannon** formalizzò il limite teorico di trasmissione su canale a rumore bianco gaussiano additivo (AWGN):

**Principio / Fondamento: Teorema di Nyquist (canale ideale privo di rumore)**

$$C_{\text{Nyquist}} = 2B \log_2 M \quad [\text{bit/s}] \tag{1.2}$$

dove $B$ è la banda analogica [Hz] e $M$ il numero di simboli distinguibili.

**Principio / Fondamento: Teorema di Shannon–Hartley (canale AWGN)**

$$C_{\text{Shannon}} = B \log_2\left(1 + \frac{S}{N}\right) \quad [\text{bit/s}] \tag{1.3}$$

dove $S/N$ è il rapporto segnale/rumore. Limite asintotico invalicabile per comunicazioni affidabili.

La conversione analogico-digitale (A/D) si articola in tre passaggi:

1. **Campionamento**: prelievo di campioni a frequenza $f_s \geq 2f_{\max}$ (teorema di Nyquist-Shannon).
2. **Quantizzazione**: discretizzazione dei valori continui in $M = 2^k$ livelli (con rumore di quantizzazione).
3. **Codifica**: associazione di ciascun livello discreto a una parola di $k$ bit.

**Kleinrock** (anni '60–'70): applicò la teoria delle code alla commutazione di pacchetto, dimostrando la superiorità del multiplexing statistico sul traffico intermittente (*bursty*) e aprendo la strada ad ARPANET.

### 1.1.2 Applicazione, Protocollo e Standardizzazione

È fondamentale distinguere tra il **protocollo applicativo** (specifica formale e pubblica su messaggi e sintassi, es. HTTP) e l'**applicazione concreta** (software utente, es. Chrome, Firefox): tale separazione tutela la libera concorrenza garantendo interoperabilità globale.

Le reti richiedono standard formali per garantire la cooperazione tra costruttori eterogenei:

| | **ITU** | **IETF** |
|---|---|---|
| Natura | Agenzia tecnica ONU | Organizzazione tecnica aperta *de facto* |
| Documenti | *Recommendation* (G.711, H.264) | RFC (*Request For Comments*) |
| Processo | Delegazioni nazionali, voto formale | Consenso tecnico aperto (*rough consensus*) |

La maggior parte del corso fa riferimento agli standard IETF (RFC), cardine dell'architettura Internet.

## 1.2 Internet: Panoramica e Struttura

### 1.2.1 Che cos'è Internet

Da un punto di vista strutturale Internet è composta da tre elementi:

- **Host** (sistemi terminali): PC, server, smartphone, dispositivi IoT.
- **Collegamenti**: rame, fibra ottica, onde elettromagnetiche (Wi-Fi, 4G/5G).
- **Router**: nodi di commutazione che instradano i pacchetti verso la destinazione.

Dal punto di vista dell'utente finale, Internet coincide con i **servizi applicativi**: il Web, la messaggistica istantanea, la posta elettronica, lo streaming multimediale, i giochi online. Dal punto di vista infrastrutturale ed ingegneristico, invece, Internet è un'**infrastruttura di telecomunicazione** che offre alle applicazioni due servizi di trasporto fondamentali:

- **Servizio Orientato alla Connessione (Affidabile)**: implementato dal protocollo **TCP**. Prima di iniziare a trasmettere i dati, sorgente e destinazione aprono una connessione *logica* (handshake a tre vie), accertando preventivamente l'esistenza e la disponibilità dell'interlocutore. Durante il trasferimento, il protocollo garantisce la consegna ordinata e integra dei dati mediante conferme di ricezione (ACK) e ritrasmissioni automatiche delle perdite.
- **Servizio Senza Connessione (Best Effort)**: implementato direttamente da **IP** (e da **UDP** a livello trasporto). I pacchetti (detti **datagrammi**) vengono immessi in rete senza alcuna negoziazione preliminare: la rete non sa se la destinazione sia attiva o raggiungibile, ma compie il "massimo sforzo" (*best effort*) per recapitare ciascun pacchetto in modo indipendente, senza garantire tempi di consegna, ordine o assenza di perdite.

**Definizione: Protocollo**

Un protocollo definisce formalmente la **sintassi** (il formato e la codifica dei campi), la **semantica** (il significato e l'interpretazione dei messaggi) e la **temporizzazione** (*timing*, l'ordine temporale e i timeout di attesa) dei messaggi scambiati tra due o più entità paritetiche (*peer*) in comunicazione, nonché le **azioni** intraprese alla ricezione di ciascun messaggio o evento (es. ritrasmissione se scade un timer di ACK). Senza protocolli condivisi due entità non possono comunicare, anche se collegate da un cavo ideale.

Le due architetture logiche fondamentali per la cooperazione tra host sono:

- **Client/Server**: esiste una netta asimmetria di ruoli. Il **client** (tipicamente il terminale utente) inizia attivamente la comunicazione richiedendo un servizio; il **server** (installato su macchine ad elevate prestazioni) rimane in ascolto passivo su un indirizzo IP e una porta noti e stabili, rispondendo alle richieste. La capacità complessiva del sistema è limitata dalle risorse hardware e di banda del server centrale (collo di bottiglia e SPOF).
- **Peer-to-Peer (P2P)**: non esiste una gerarchia predeterminata: ogni nodo della rete (*peer*) agisce contemporaneamente sia da client che da server verso gli altri nodi. Due host cooperano scambiandosi risorse (banda, file, capacità di calcolo) direttamente tra loro. Il modello P2P opera come una rete distribuita auto-scalante, poiché ogni nuovo nodo apporta non solo nuova domanda, ma anche nuova capacità di servizio al sistema.

### 1.2.2 L'Architettura a Clessidra di IP

L'architettura di Internet si rappresenta con la metafora della **clessidra**: molte tecnologie fisiche e molti protocolli applicativi convergono in un unico denominatore comune universale – il protocollo **IP**. Chi vuole aggiungere una nuova tecnologia fisica (es. 5G, fibra spaziale laser) deve solo implementare IP su di essa; chi vuole una nuova applicazione (es. HTTP/3 su QUIC) si appoggia a TCP o UDP senza toccare l'infrastruttura. IP non cambia, garantendo l'interoperabilità universale. Questa è la scelta di design più importante di Internet.

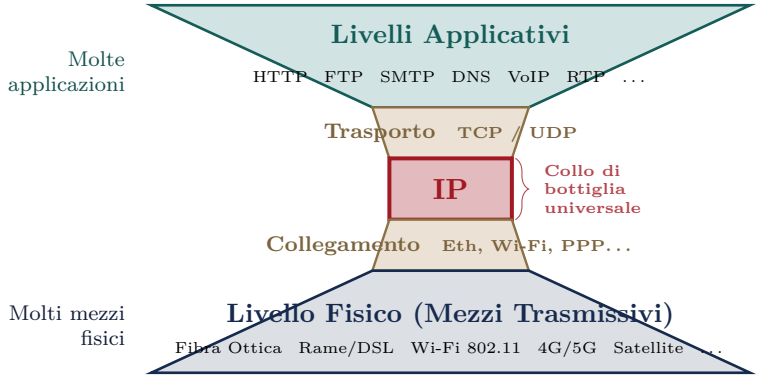



Figura 1.1: L'architettura a clessidra di Internet: la varietà dei mezzi fisici (basso) e dei protocolli applicativi (alto) converge sull'unico denominatore comune IP, che rimane invariato.

### 1.2.3 Internet: La Rete delle Reti e la Gerarchia degli ISP

Internet è organizzata gerarchicamente in ISP (*Internet Service Provider*) su tre livelli commerciali, distinti dalla portata geografica e dagli accordi economici:

**Tier 1 (Backbone Globale)** Copertura internazionale (AT&T, Verizon, Lumen, Telia, ...). Si connettono *in peering gratuito* tra loro (*settlement-free peering*): nessuno paga l'altro. Ne esistono ≈6–8 al mondo. Per definizione un ISP Tier 1 può raggiungere qualunque destinazione Internet senza pagare transito ad altri.

**Tier 2 (Nazionali/Regionali)** Pagano transito agli ISP Tier 1 e possono stabilire peering con altri Tier 2. Esempi: GÉANT (rete europea della ricerca), Telecom Italia Sparkle.

**Tier 3 (Accesso)** Reti di ultimo miglio verso gli utenti finali. Esempi italiani: TIM, Fastweb, Wind Tre, GARR (per le università). Un pacchetto tra due utenti di ISP diversi attraversa tipicamente 15–20 router.

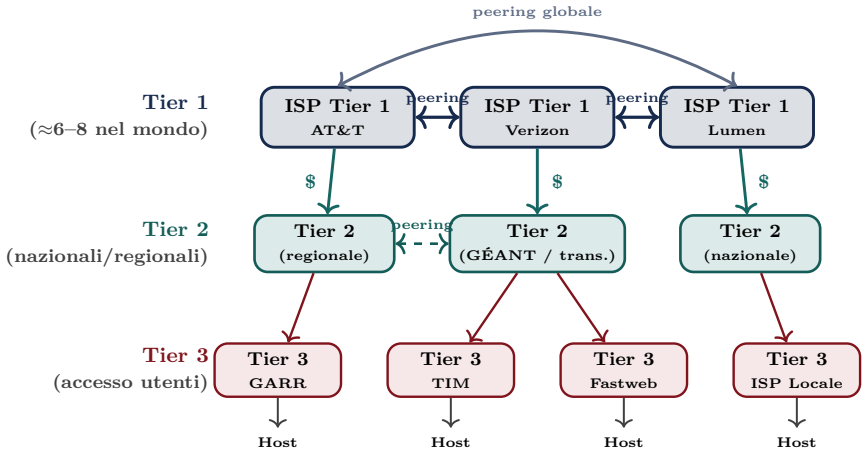



Figura 1.2: Struttura gerarchica di Internet: Tier 1 in peering gratuito, Tier 2 in transito pagato, Tier 3 di accesso verso gli utenti finali.

**Internet Exchange Point (IXP) e CDN**

Per ridurre i costi di transito, gli ISP si attestano presso **IXP** (Internet Exchange Point): punti neutri dove scambiano traffico direttamente senza passare per un Tier 1. In Italia: il **MIX** di Milano e il **NaMeX** di Roma. I *Content Delivery Network* (CDN — Akamai, Google, Cloudflare, Netflix) portano i propri server (*cache edge*) direttamente nei datacenter degli ISP di accesso, *appiattendo* progressivamente la gerarchia tradizionale: oggi il 70% del traffico Internet mondiale non attraversa mai un Tier 1, ma rimane all'interno

di accordi di peering diretti tra CDN e ISP. Questo fenomeno ha radicalmente cambiato l'economia di Internet rispetto al modello gerarchico originale.

**Accordi di Peering ed Asimmetria delle Rotte**

Un **contratto di peering** viene stipulato tra due ISP di pari rango (tipicamente due Tier 2 o Tier 3) per pura convenienza economica, quando tra le loro rispettive reti sussiste un flusso cospicuo e bilanciato di traffico reciproco. Attraverso un collegamento diretto (o attestandosi presso un punto neutro IXP), i due provider scambiano direttamente il traffico destinato ai rispettivi clienti finali senza dover corrispondere alcun canone di transito a monte verso gli ISP Tier 1.
Tuttavia, la natura distribuita del routing BGP e le politiche commerciali introducono frequentemente il fenomeno dell'**asimmetria dei percorsi**:

- Nella tratta di andata, il pacchetto emesso dall'Host cliente dell'ISP A e diretto all'Host dell'ISP B viene instradato direttamente attraverso il canale di peering locale concordato tra i due provider.
- Nella tratta di risposta, se il percorso prescelto dalle tabelle di routing risale verso un operatore di transito Tier 1, tale operatore di dorsale ignora completamente l'accordo bilaterale privato esistente a livello locale tra A e B. Il pacchetto di ritorno viene pertanto instradato attraverso la normale gerarchia di transito a pagamento, generando una rotta asimmetrica (percorso di andata diverso dal percorso di ritorno).

# Capitolo 2

# Servizi, Funzioni, Canali, Topologie e Architetture Protocollari

## 2.1 Definizioni Fondamentali: Comunicazione, Servizi e Funzioni

**Definizione: Comunicazione**

Il trasferimento di informazioni tra due o più entità secondo convenzioni prestabilite (protocolli).

**Definizione: Telecomunicazione (definizione ITU)**

Qualsiasi trasmissione e ricezione di segnali che rappresentano segni, scrittura, immagini e suono, informazioni di qualsiasi natura, attraverso cavi, radio o altri sistemi ottici ed elettromagnetici.

Una rete di telecomunicazioni separa nettamente due piani distinti:

- **Servizi Utente**: risiedono e sono fruiti **alla periferia** della rete (*edge*). Rappresentano le applicazioni concrete richieste: navigazione web, messaggistica, telefonia vocale, videoconferenza, download file.
- **Funzioni di Rete**: risiedono e sono implementate **nel nucleo** della rete (*core*). L'utente non interagisce direttamente con esse, ma esse sono indispensabili per l'erogazione del servizio.




Figura 2.1: Separazione tra il Servizio Utente (periferia) e le Funzioni di Rete (nucleo). UNI = User-Network Interface.

### 2.1.1 Le Tre Funzioni Fondamentali della Rete

- **Trasmissione**: il trasferimento di segnali che supportano informazione da un punto a uno o più altri punti. Include modulazione, amplificazione, rigenerazione e codifica di canale.
- **Commutazione**: il processo di interconnessione di unità funzionali, canali di trasmissione o circuiti, per il tempo necessario al trasferimento delle informazioni. Instrada ciascuna unità informativa verso la porta di uscita corretta.
- **Segnalazione**: lo scambio di informazioni per l'apertura, il controllo e la chiusura di connessioni e la gestione della rete (calcolo tabelle di routing, gestione delle risorse).

## 2.2 Classificazione dei Servizi

I servizi si classificano secondo criteri **qualitativi**:

**Orientati alla Connessione vs Connectionless**
: Nei servizi *connection-oriented* (es. TCP) le entità stabiliscono uno stato condiviso prima del trasferimento: si garantiscono ordinamento e controllo d'errore. Nei servizi *connectionless* (es. UDP, IP) ogni pacchetto è un **datagramma** autonomo, senza negoziazione preliminare.

**Diffusivi (Broadcast/Multicast) vs Interattivi (Unicast)**
: I servizi diffusivi distribuiscono la stessa informazione a una moltitudine di destinatari senza possibilità per i riceventi di influenzare il flusso (es. DVB-T, radio). I servizi interattivi consentono comunicazione bidirezionale punto-punto.

**Conversazionali vs Non Conversazionali**
I servizi conversazionali (telefonia, videoconferenza) impongono un vincolo stringente sul ritardo massimo end-to-end ($d \leq 150-200\,\text{ms}$) e bassissimo jitter. I servizi non conversazionali (e-mail, download HTTP) tollerano ritardi di minuti o ore.

**Monomediali vs Multimediali**
I servizi monomediali trattano un'unica tipologia di flusso (audio puro). I multimediali integrano flussi multipli sincronizzati (audio, video, sottotitoli).

**Servizi di Base vs a Valore Aggiunto (VAS)**
I servizi di base forniscono il semplice trasporto trasparente dei bit. I VAS integrano storage, crittografia, transcodifica e caching distribuito (CDN).

### 2.2.1 Banda, Capacità e Velocità: Tre Concetti Distinti

È fondamentale non confondere tre grandezze spesso usate come sinonimi nel linguaggio comune:

- nella **teoria dei segnali**, la **banda** $B$ è l'ampiezza spettrale del canale in Hertz;
- nelle **reti di telecomunicazione**, con "banda" si intende colloquialmente la quantità di dati per unità di tempo in Mbit/s – uso improprio ma estremamente comune;
- la **capacità di un canale** $C$ è la massima velocità trasmissiva (bit/s), soggetta al limite di Shannon (equazione 1.3, Capitolo 1).

### 2.2.2 Traffico Offerto, Throughput e Ritardo

**Definizione: Traffico offerto e throughput**

Il **traffico offerto** è la quantità di dati per unità di tempo (bit/s) che una sorgente cerca di inviare in rete. Il **throughput** è la porzione di traffico offerto consegnata correttamente alla destinazione. Valgono sempre:

$$\text{Throughput} \leq \text{capacità del link collo di bottiglia}$$
$$\text{Throughput} \leq \text{traffico offerto}$$

Il **ritardo complessivo end-to-end** percepito dall'utente è in realtà la combinazione di due macro-contributi: il **ritardo proprio della rete** (il tempo fisico di transito attraverso i nodi e i link) e il **ritardo proprio degli host e delle applicazioni**:

- **Ritardi di Host e Applicativi**: includono i tempi di codifica/compressione sorgente, pacchettizzazione, accodamento nei buffer di trasmissione del sistema operativo e attesa nel *buffer di playout* ricevente (*jitter buffer*, impiegato nei flussi real-time per assorbire le fluttuazioni del ritardo di rete).
- **Ritardo Nodale della Rete**: introdotto da ciascun nodo intermedio lungo il cammino, scomposto in quattro componenti fisiche distinte.

1. **Ritardo di trasmissione** $d_{\text{trans}} = L/R$: tempo impiegato dal trasmettitore per emettere sul canale tutti gli $L$ bit del pacchetto alla frequenza di cifra $R$. Dipende unicamente dalla dimensione del pacchetto e dalla velocità del link.
2. **Ritardo di propagazione** $d_{\text{prop}} = d/v$: tempo necessario al fronte d'onda elettromagnetico per percorrere la distanza fisica $d$ del mezzo alla velocità di propagazione $v$. È una costante puramente fisica (circa 200.000 km/s in fibra di vetro/rame, 300.000 km/s nello spazio libero/aria) e non dipende in alcun modo dalla velocità $R$ del canale.
3. **Ritardo di elaborazione** $d_{\text{proc}}$: tempo impiegato dal router per verificare l'integrità del pacchetto (checksum), ispezionare l'header IP e consultare la tabella di inoltro per individuare la porta di uscita. Nei moderni apparati hardware è dell'ordine di grandezza dei microsecondi.
4. **Ritardo di accodamento** $d_{\text{queue}}$: tempo trascorso dal pacchetto all'interno dei buffer di memoria del router in attesa che la porta di uscita prescelta diventi libera. È l'unica componente intrinsecamente **stocastica e variabile**, legata alla contesa istantanea delle risorse e all'intensità del traffico concorrente $I = \lambda L/R$.

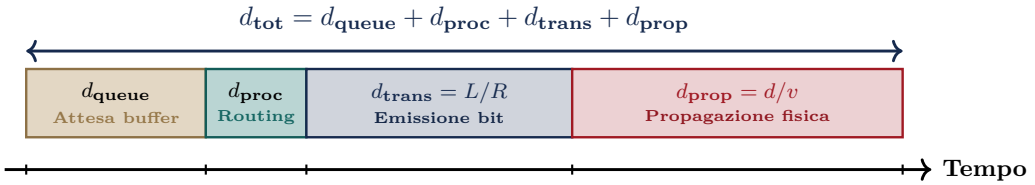



Figura 2.2: Scomposizione del ritardo nodale in un router: le quattro componenti additive. Il ritardo di accodamento è variabile e dipende dal carico istantaneo; il ritardo di propagazione è deterministico e dipende dalla distanza fisica.

**Ritardo di trasmissione vs propagazione: quando dominano?**

Consideriamo un pacchetto di $L = 1500\,\text{byte} = 12000\,\text{bit}$ su un link da $R = 100\,\text{Mbit/s}$:

$$d_{\text{trans}} = \frac{12000\,\text{bit}}{10^8\,\text{bit/s}} = 0.12\,\text{ms}.$$

Se il link collega Milano a Roma ($d \approx 500\,\text{km}$) via fibra ($v = 2 \cdot 10^8\,\text{m/s}$):

$$d_{\text{prop}} = \frac{500 \cdot 10^3\,\text{m}}{2 \cdot 10^8\,\text{m/s}} = 2.5\,\text{ms}.$$

Il ritardo di propagazione domina di oltre 20× su quello di trasmissione: aumentare la velocità del link da 100 Mbit/s a 1 Gbit/s riduce $d_{\text{trans}}$ da 0.12 ms a 0.012 ms, ma non cambia di un microsecondo $d_{\text{prop}}$, che è imposto dalla fisica del mezzo.

### 2.2.3 Perdita di Informazione

Nelle reti a commutazione di pacchetto la degradazione dell'informazione si manifesta principalmente in due forme distinte:

- **Errori sui singoli bit**: alterazioni di uno o più bit all'interno di un pacchetto durante la trasmissione sul canale fisico, dovute a rumore termico, interferenze o attenuazione del segnale. In genere questo tipo di errore è *irrilevante a livello di rete* perché viene rilevato e gestito dai meccanismi di controllo d'errore del livello di collegamento (CRC) o, in caso residuo, segnalato ai livelli superiori.
- **Perdita di pacchetti**: i pacchetti possono essere scartati quando i buffer di memoria di un router intermedio o dell'host ricevente sono saturi. Di fronte alla perdita di pacchetti si possono adottare due strategie:
  - **Ritrasmissione** (comunicazioni affidabili): il livello di trasporto affidabile (TCP) rileva la perdita (mancata ricezione di riscontri ACK entro un timeout) e ritrasmette automaticamente le copie mancanti.
  - **Accettare una degradazione del servizio**: per applicazioni real-time e interattive (VoIP, videoconferenza, streaming live), ritrasmettere un pacchetto perso è inutile o dannoso poiché giungerebbe troppo tardi per la riproduzione. L'applicazione preferisce accettare la perdita e gestirne l'impatto percettivo (es. interpolazione audio, perdita di un fotogramma video).

La perdita di informazione si esprime quantitativamente mediante due metriche complementari:

- **Probabilità di perdita** (misura *a priori*): parametro teorico o di progetto che caratterizza la qualità intrinseca della telecomunicazione e del canale indipendentemente dal traffico istantaneo. Viene impiegata per il dimensionamento della rete e per la stipula dei contratti di livello di servizio (*Service Level Agreement*, SLA).
- **Tasso di perdita** (*packet loss rate*, misura *a posteriori*): frazione di pacchetti effettivamente persi rispetto al totale di quelli trasmessi, misurata a consuntivo su un intervallo di osservazione reale durante il funzionamento operativo della rete.

## 2.3 Canali: Tipi e Modalità di Comunicazione

**Definizione: Rete di telecomunicazione**

Un insieme di nodi e canali che fornisce collegamento tra due o più punti per permettere la telecomunicazione. Si rappresenta formalmente come un **grafo** $G = (V, A)$, dove $V$ è l'insieme dei nodi (router, switch, host) e $A$ l'insieme dei canali (archi).

### 2.3.1 Tipi di Canale

I canali si classificano in base al numero di nodi che vi accedono:

**Canale Punto-Punto** Due soli nodi agli estremi. Le risorse trasmissive sono dedicate alla coppia; non vi è contesa di accesso. Esempio: fibra ottica tra due router, collegamento WAN.

**Canale Multi-Punto** Più nodi collegati allo stesso canale con un nodo *master* e più *slave*. Richiede un protocollo di arbitraggio dell'accesso.

**Canale Broadcast** Un unico mezzo condiviso: ogni trasmissione raggiunge tutti. I frame devono contenere l'indirizzo di destinazione. Esempi: Ethernet a bus, Wi-Fi, DVB-T.

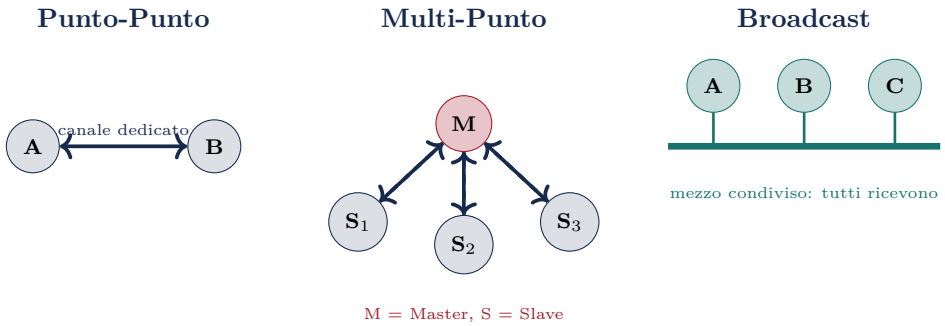



Figura 2.3: Tipologie di canale: punto-punto (risorse dedicate), multi-punto (master/slave con arbitraggio), broadcast (mezzo condiviso).

### 2.3.2 Modalità di Comunicazione

**Definizione: Modalità di Comunicazione**

- **Simplex**: comunicazione in un **solo verso**, da mittente a ricevente. Il canale è unidirezionale. Esempi: trasmissione televisiva terrestre DVB-T, sensore IoT verso gateway.
- **Half-Duplex**: comunicazione in **entrambi i versi, non simultaneamente**. I nodi si alternano nel ruolo di trasmettitore/ricevente. Esempi: walkie-talkie, Wi-Fi 802.11 (CSMA/CA).
- **Full-Duplex**: comunicazione in **entrambi i versi simultaneamente** su canali fisici o logici separati. Esempi: telefonia VoIP, Ethernet punto-punto 100BASE-TX.

**Multiplexing e condivisione del canale**

Un singolo mezzo fisico può ospitare più canali logici simultanei grazie al **multiplexing**:

- **FDM** (*Frequency Division Multiplexing*): ogni canale logico occupa una banda distinta del mezzo. Usato nelle reti radio, ADSL, DOCSIS.
- **TDM** (*Time Division Multiplexing*): ogni canale trasmette a turno in un *time slot* fisso periodico. Usato nella telefonia digitale (SDH/SONET, E1/T1).
- **CDM/CDMA** (*Code Division Multiplexing*): canali logici separati tramite sequenze di spreading ortogonali. Usato in 3G e GPS.

Il **multiplexing statistico** – alla base della commutazione di pacchetto di Internet – è una variante adattiva di TDM: i time slot non sono preassegnati ma attribuiti dinamicamente in base alla domanda istantanea. Questo è possibile perché il traffico reale è tipicamente *bursty*: gli utenti non trasmettono in modo continuo, ma a raffiche; nei periodi di silenzio le risorse sono liberate e messe a disposizione di altri flussi. Il risultato è che sullo stesso link fisico possono transitare molti più flussi rispetto a quanti sarebbero possibili con allocazione statica.

## 2.4 Topologie di Rete

La disposizione di nodi e canali definisce la **topologia** della rete. Formalmente è un grafo $G = (V, A)$ con $N = |V|$ nodi e $C = |A|$ canali (bidirezionali contati una sola volta).

**Maglia completa**
$C = N(N-1)/2$. Massima tolleranza ai guasti; costo quadratico proibitivo. Usata tipicamente per interconnessioni logiche (overlay P2P) o per il nucleo di backbone ad altissima affidabilità con un numero ridotto di nodi.

**Maglia parziale (*mesh*)**
$N - 1 < C < N(N-1)/2$. Ottimo bilanciamento ingegneristico tra costo di cablaggio e ridondanza dei percorsi. Rappresenta la topologia d'elezione per le reti geografiche (Internet core, dorsali di telecomunicazione).

**Albero (gerarchica)**
$C = N - 1$. Costo di infrastruttura minimo; esiste un unico cammino semplice tra qualsiasi coppia di nodi. Vulnerabile al partizionamento: il guasto di un nodo interno isola interi sottoalberi (nessun percorso alternativo). Il bus attivo è un caso particolare di albero con $C = N - 1$.

**Stella**
$C = N - 1$. Tutta la complessità di commutazione è accentrata nel nodo hub centrale. Presenta un evidente *Single Point of Failure* (SPOF): se cade l'hub la rete collassa. Topologia universale per LAN switched, reti d'accesso cellulari e satellitari.

**Anello bidirezionale**
$C = N$. Ogni nodo è connesso a due nodi adiacenti. In caso di tranciamento di un link, il traffico viene istantaneamente re-istradato lungo l'arco opposto con capacità residua dimezzata. È la configurazione standard per gli anelli metropolitani (MAN) e di distribuzione distrettuale.

**Bus** Il bus esiste in due varianti distinte:

- **Bus passivo** ($C = 1$): mezzo trasmissivo broadcast condiviso tra tutti gli host, privo di nodi intermediari attivi. Esiste una sola scelta possibile di percorso tra ogni coppia di nodi. Usato storicamente nelle prime LAN Ethernet (10BASE2/10BASE5, ormai sempre meno in uso) e nelle comunicazioni interne ai computer (bus di sistema).
- **Bus attivo** ($C = N - 1$): caso particolare di albero in cui ogni nodo è connesso al bus tramite un ripetitore attivo; il numero di canali è pari al numero di nodi meno uno ($C = N - 1$).

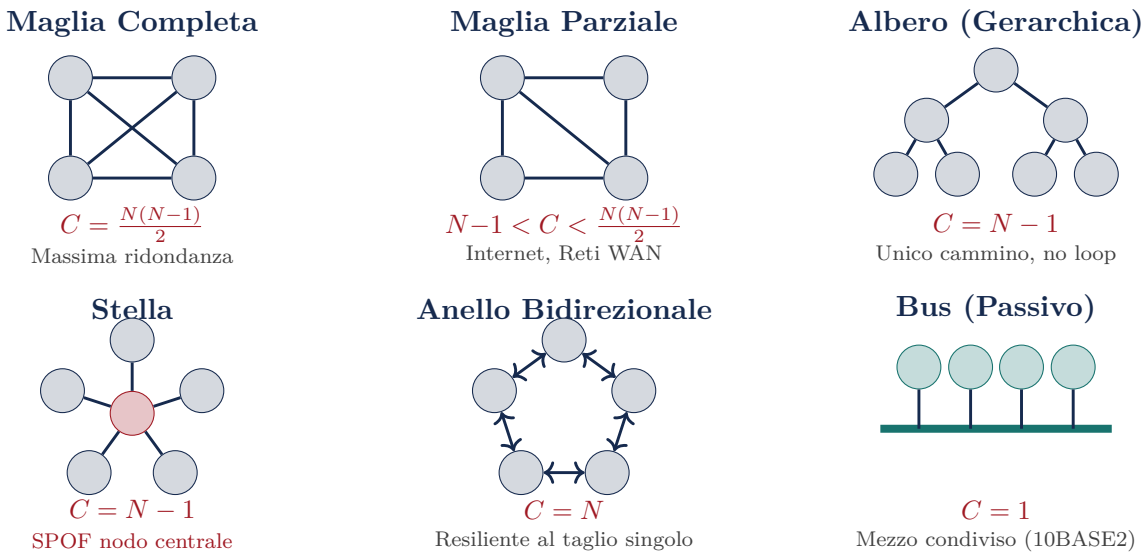



Figura 2.4: Topologie di rete fondamentali a confronto: formule di connettività $C$ e caratteristiche di affidabilità.

**Perché l'anello bidirezionale è diffuso nelle reti di accesso**

L'anello bidirezionale è la più semplice topologia che garantisca sopravvivenza in caso di guasto di un singolo collegamento – cosa che non vale per l'anello unidirezionale. Nelle reti distrettuali di accesso, la topologia ad anello è diffusa proprio perché un singolo guasto fisico (es. tranciamento accidentale

della fibra durante lavori di scavo) non isola la connettività ma riduce unicamente la capacità residua, garantendo la continuità operativa del servizio.

## 2.5 Tecniche di Commutazione: Circuito vs Pacchetto

Il cuore operativo di qualsiasi rete risiede nella modalità con cui i nodi intermedi instradano il traffico e allocano le risorse trasmissive.

### 2.5.1 Commutazione di Circuito (*Circuit Switching*)

Storicamente adottata dalla PSTN (rete telefonica pubblica). Richiede tre fasi sequenziali obbligatorie:

1. **Instaurazione (*Call Setup*)**: i nodi intermedi riservano un canale dedicato lungo un percorso fisso sorgente-destinazione. Se le risorse su un collegamento sono sature, la chiamata viene **bloccata** (segnale di occupato).
2. **Trasferimento**: i dati fluiscono lungo il circuito dedicato con banda garantita e costante, senza alcun ritardo di accodamento intermedio.
3. **Abbattimento (*Teardown*)**: al termine, le risorse vengono esplicitamente rilasciate e rese disponibili a nuovi utenti.

Le risorse sono allocate con **FDM** (ogni circuito occupa una porzione di frequenza permanente) o con **TDM** (ogni circuito riceve uno slot temporale fisso a intervalli regolari).

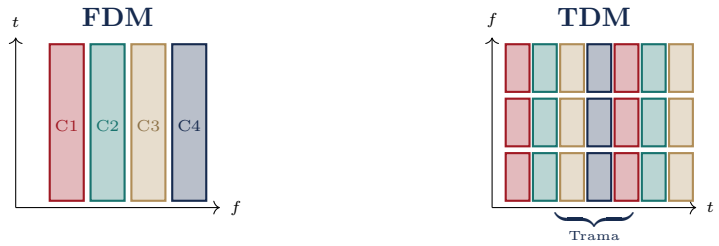



Figura 2.5: Commutazione di circuito: allocazione FDM (banda di frequenza fissa) e TDM (slot temporale fisso). In entrambi i casi le risorse sono riservate per l'intera durata della comunicazione.

### 2.5.2 Commutazione di Pacchetto (*Packet Switching*)

Fondamento di Internet: i dati vengono suddivisi in **pacchetti** di lunghezza limitata, ciascuno composto da **intestazione (header) + carico utile (payload)**.

I nodi intermedi (router) operano con **Store-and-Forward**: il router deve ricevere l'**intero pacchetto** prima di poter iniziare a trasmetterlo sulla linea di uscita. I pacchetti condividono i link con **multiplexing statistico**: le risorse sono allocate **su domanda**.

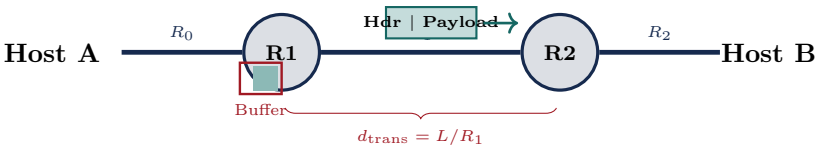



Figura 2.6: Store-and-Forward: il router R1 riceve l'intero pacchetto prima di iniziare a trasmetterlo sul link successivo. Il ritardo totale su $k$ hop è $k \cdot L/R$ (trascurando propagazione e accodamento).

| Parametro | Commutazione di Circuito | Commutazione di Pacchetto |
|---|---|---|
| **Setup** | Obbligatorio prima dell'invio. | Assente (datagramma puro). |
| **Allocazione Risorse** | Esclusiva e deterministica (FDM/TDM). | Su domanda (multiplexing statistico). |
| **Dati *bursty*** | Molto inefficiente (circuito inattivo nei silenzi). | Massima efficienza. |
| **Ritardo di Accodamento** | Nullo durante la conversazione. | Variabile, dipende dal carico. |
| **A Saturazione** | **Blocco**: nuove chiamate rifiutate; quelle in corso non degradano. | **Degrado/Perdita**: buffer saturi ⇒ pacchetti scartati. |

Tabella 2.1: Confronto sistematico tra commutazione di circuito e commutazione di pacchetto.

## 2.6 Architetture Protocollari a Strati

**Definizione: Architettura protocollare**

L'insieme dei protocolli e delle loro inter-relazioni che definiscono un'architettura logica e fisica di comunicazione. L'architettura è **stratificata**: ogni livello fornisce servizi allo strato superiore sfruttando i servizi dello strato inferiore e le proprie funzioni interne, nascondendo completamente i dettagli implementativi.

### 2.6.1 Origini ed Evoluzione delle Architetture Protocollari

La stratificazione protocollare applica i principi di modularità dell'ingegneria del software: ogni livello costituisce un modulo specializzato con interfaccia formale, indipendente dall'implementazione interna degli strati adiacenti, costruendo i propri servizi unicamente sulle funzioni offerte dal livello $N-1$ sottostante.

Lo studio delle telecomunicazioni moderne si fonda su due architetture di riferimento:

- **Il modello ISO/OSI** (*Open Systems Interconnection*): architettura teorica standardizzata a 7 livelli, introdotta come schema concettuale per definire una terminologia rigorosa e comprendere la modularità delle funzioni di rete.
- **L'architettura TCP/IP**: lo standard operativo reale su cui è interamente costruita la rete Internet globale.

### 2.6.2 Architetture di Rete e Concetti di Stratificazione

In termini astratti, una rete di telecomunicazione è composta da **sistemi** (host terminali, router, switch) interconnessi tramite mezzi trasmissivi. Ciascun sistema è strutturato come una gerarchia ordinata di **sottosistemi**, ciascuno corrispondente a un livello o strato (*layer*), all'interno del quale operano una o più **entità** ($N$-entità: processi software, demoni del sistema operativo o chip hardware dedicati) che ne realizzano le funzioni operative.

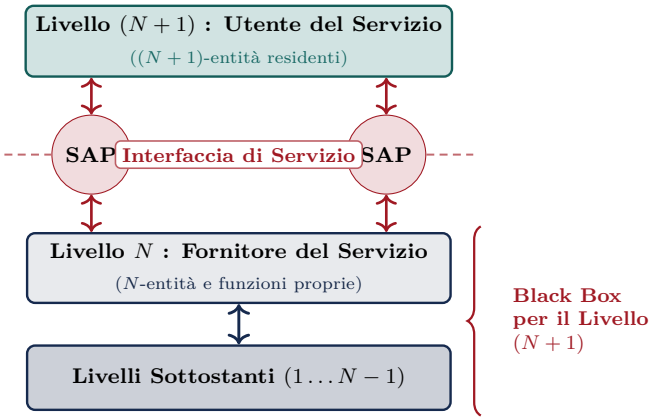



Figura 2.7: Relazione tra strati e SAP.

I cardini concettuali della stratificazione sono:

- **Fornitori di Servizio**: il livello $N$, operando congiuntamente con tutti i livelli inferiori $(1 \dots N-1)$, funge da fornitore del servizio per lo strato superiore.
- **Utenti del Servizio**: le $(N+1)$-entità residenti al livello superiore che invocano e fruiscono delle prestazioni erogate dal livello $N$.
- **Punti di Accesso al Servizio (SAP - *Service Access Point*)**: indirizzi logici e punti di interfaccia formale attraverso cui le $(N+1)$-entità scambiano dati e comandi con il livello $N$.
- **Principio della Black Box**: un'entità di livello $N+1$ percepisce l'insieme formato dallo strato $N$ e da tutti i livelli inferiori esclusivamente in termini di **servizio offerto**, senza alcun vincolo o visibilità sulle tecnologie, sui protocolli o sui mezzi fisici impiegati per realizzarlo.
- **Protocolli vs Interazioni Locali**:
  - Lo scambio di informazioni tra entità **omologhe** (*peer entities*) residenti su nodi remoti diversi avviene tramite un **$N$-protocollo** (comunicazione orizzontale logica via rete).
  - Il passaggio di dati tra livelli adiacenti all'interno dello **stesso sistema** (comunicazione verticale) **non è un protocollo**: è una semplice chiamata di funzione software locale (API o primitive di sistema operativo), istantanea e priva di trasmissione fisica su canale.

### 2.6.3 Creazione e Gestione delle PDU (Incapsulamento, Segmentazione e Concatenazione)

In un sistema a $M$ livelli, i dati generati dall'applicazione dell'utente attraversano la pila protocollare subendo progressive trasformazioni:

**Definizione: SDU, PCI e PDU**

- **SDU** (*Service Data Unit*): il blocco di dati ricevuto dal livello superiore, che rappresenta il carico utile (*payload*) da trasportare.
- **PCI** (*Protocol Control Information*): le informazioni di controllo (intestazione o *header*, ed eventualmente coda o *trailer*) generate dal livello corrente per consentire all'entità remota omologa di eseguire le proprie funzioni (indirizzamento, controllo di sequenza, rilevamento errori).
- **PDU** (*Protocol Data Unit*): l'unità dati completa scambiata tra due entità omologhe di livello $N$, formata dalla combinazione della PCI di livello $N$ e della SDU ricevuta:

$$\text{PDU}_N = \text{PCI}_N + \text{SDU}_N \tag{2.1}$$

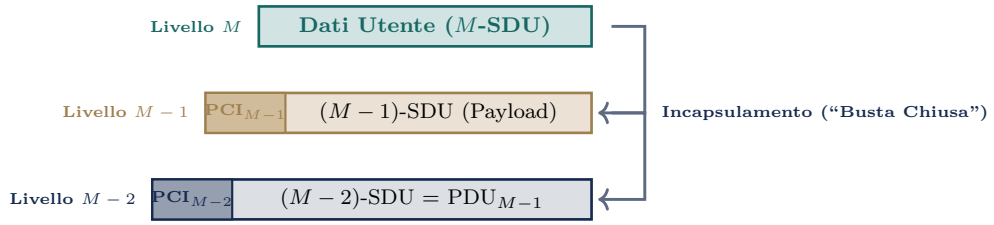



Figura 2.8: Il meccanismo dell'incapsulamento: ogni livello tratta la PDU ricevuta dallo strato superiore come una SDU (busta chiusa opaca), prefiggendovi la propria intestazione di controllo (PCI).

Il principio dell'**incapsulamento** stabilisce che ogni livello inferiore tratta la PDU dello strato superiore come una **busta chiusa**: non ne interpreta né modifica il contenuto interno, ma vi aggiunge semplicemente la propria intestazione. Nel passaggio dal livello $N$ al livello $N-1$:

$$\text{SDU}_{N-1} = \text{PDU}_N \quad \Longrightarrow \quad \text{PDU}_{N-1} = \text{PCI}_{N-1} + \text{SDU}_{N-1}$$

Oltre all'incapsulamento elementare, i protocolli possono eseguire due operazioni strutturali sulle unità dati:

- **Segmentazione**: suddivisione di una singola unità dati di grandi dimensioni in molteplici blocchi più piccoli. Si applica quando una $(N)$-SDU supera la dimensione massima trasportabile (*Maximum Transmission Unit*, MTU) della rete sottostante, generando più $(N)$-PDU distinte ciascuna con la propria intestazione.
- **Concatenazione**: unione di più unità dati di piccole dimensioni provenienti dallo strato superiore all'interno di un'unica $(N)$-PDU, per ottimizzare l'efficienza trasmissiva e ridurre l'overhead delle intestazioni.

### 2.6.4 Il Modello di Riferimento ISO/OSI (7 Livelli) e Confronto con TCP/IP

Il modello *Open Systems Interconnection* (OSI) è recepito negli standard internazionali ISO IS 7498 e CCITT X.200. Rappresenta la formalizzazione teorica della comunicazione di rete in 7 livelli:

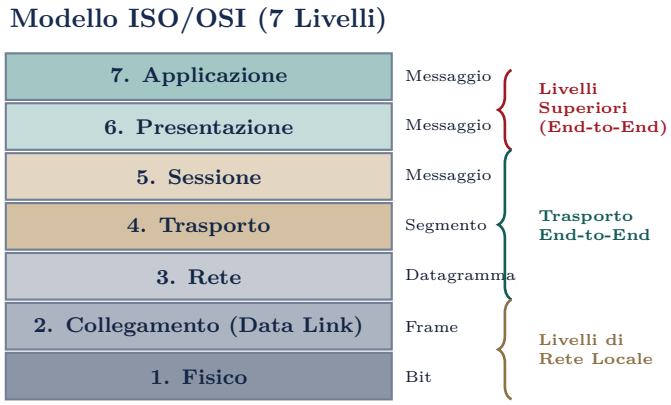



Figura 2.9: I sette livelli del modello ISO/OSI con le relative PDU. I livelli 5-7 risiedono esclusivamente sugli host end-to-end; il livello 3 governa l'instradamento nella rete; i livelli 1-2 operano tra nodi direttamente adiacenti.

I sette livelli ISO/OSI con le rispettive funzioni:

1. **Fisico**: trasmissione grezza dei singoli bit sul mezzo trasmissivo (specifiche elettriche, ottiche, modulazione, connettori fisici).
2. **Collegamento (*Data Link*)**: trasferimento affidabile dei frame tra due nodi direttamente adiacenti collegati dallo stesso link. Include delimitazione di trama, indirizzamento fisico (MAC) e rilevamento errori (CRC).
3. **Rete**: consegna end-to-end dei pacchetti dalla sorgente alla destinazione attraverso nodi intermedi, instradamento (*routing*) e controllo della congestione.
4. **Trasporto**: comunicazione logica di processo end-to-end tra applicazioni sorgente e destinazione (controllo di flusso, ordinamento, eventuale affidabilità e multiplazione su porte).
5. **Sessione**: instaurazione, mantenimento e sincronizzazione ordinata dei punti di dialogo tra applicazioni.
6. **Presentazione**: sintassi e semantica delle informazioni scambiate, conversione di codifica dei dati, compressione e cifratura.
7. **Applicazione**: interfaccia diretta e protocolli dedicati ai processi software dell'utente.

| Livello OSI | Livello TCP/IP | Protocolli Internet principali |
|---|---|---|
| 7 – Applicazione | | HTTP, HTTPS, SMTP, DNS, SSH, FTP, RTP, DHCP |
| 6 – Presentazione | Applicazione | TLS/SSL, MIME, JSON (integrati nell'applicazione) |
| 5 – Sessione | | RPC, WebRTC, Cookie (integrati nell'applicazione) |
| 4 – Trasporto | Trasporto | **TCP**, **UDP**, QUIC, SCTP |
| 3 – Rete | Rete (Internet) | **IP** (v4/v6), ICMP, OSPF, BGP, ARP |
| 2 – Collegamento | Link | Ethernet, Wi-Fi 802.11, PPP, DOCSIS |
| 1 – Fisico | Fisico | Rame, Fibra Ottica, Canale Radio |

Tabella 2.2: Corrispondenza tra i livelli ISO/OSI e l'architettura Internet TCP/IP a 5 livelli.

### 2.6.5 Stack nei Nodi e Processamento degli Header

L'elaborazione delle PDU varia radicalmente a seconda della tipologia di nodo attraversato lungo il cammino, come mostrato in Figura 2.10:

- **Host Sorgente**: implementa la pila a 5 livelli, eseguendo l'incapsulamento progressivo dall'alto verso il basso: Messaggio $M \rightarrow$ Segmento $[H_t \mid M] \rightarrow$ Datagramma $[H_n \mid H_t \mid M] \rightarrow$ Frame $[H_l \mid H_n \mid H_t \mid M]$.
- **Switch Ethernet (Livello 2)**: opera unicamente sui livelli Fisico e Collegamento. Ispeziona l'header MAC $H_l$ per selezionare la porta di uscita; l'intero datagramma IP ($H_n$) rimane completamente trasparente e immutato.
- **Router (Livello 3)**: opera fino al livello Rete. Rimuove l'header di frame d'ingresso $H_l$, esamina l'IP di destinazione in $H_n$, decrementa il campo TTL $\leftarrow$ TTL $-$ 1, ricalcola il checksum generando $H_n'$ e incapsula il pacchetto in un **nuovo frame** $H_l'$ idoneo al link fisico successivo.
- **Host Destinazione**: esegue il decapsulamento simmetrico inverso dal livello Fisico fino all'Applicazione, validando CRC, indirizzi e porte.

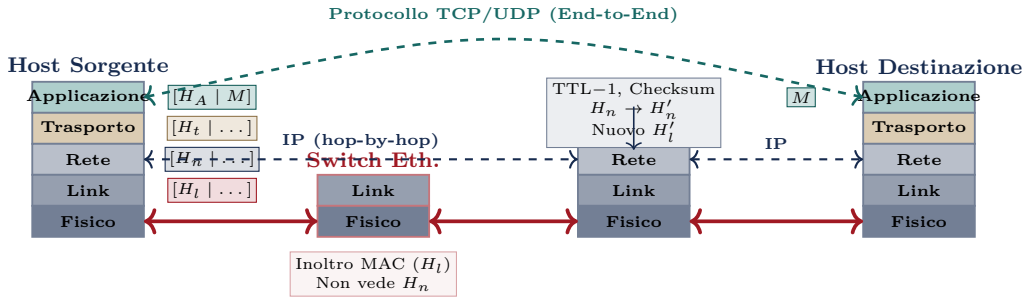



Figura 2.10: Stack protocollare nei nodi, incapsulamento ed evoluzione degli header lungo il cammino. Gli host implementano 5 livelli; lo switch L2 inoltra i frame in base all'indirizzo MAC $H_l$ senza accedere a IP; il router L3 decapsula il frame, decrementa il TTL, ricalcola il checksum in $H'_n$ e applica un nuovo header di frame $H'_l$ per il link successivo.

## 2.7 Il Protocollo IPv4: Formato del Datagramma, Campi e Frammentazione

Il protocollo `IP` (*Internet Protocol*) costituisce il fulcro dell'intera architettura telematica di Internet. Opera secondo un paradigma a **commutazione di pacchetto puro**, completamente **senza connessione** (*connectionless*) e rigorosamente **privo di memoria di stato** (*stateless*).

> **Principio / Fondamento: Servizio Best Effort**
>
> La rete IP offre un servizio di tipo **Best Effort** ("massimo sforzo"):
>
> - Il livello di rete garantisce di compiere qualsiasi tentativo fisicamente e logicamente possibile per inoltrare ciascun pacchetto verso la destinazione, purché esista almeno un percorso valido nella tabella di routing.
> - Non viene tuttavia fornita **alcuna garanzia deterministica** circa: l'effettiva consegna del pacchetto (che può venire scartato per congestione dei buffer), il tempo di transito (latenza e jitter arbitrari), la conservazione della sequenza temporale (i pacchetti possono arrivare disordinati per instradamento su percorsi asimmetrici o dinamici), e l'assenza di duplicazioni accidentali.
> - L'affidabilità end-to-end, se richiesta dal servizio utente, non è demandata alla rete ma è interamente delegata agli estremi terminali (*end systems*) mediante il protocollo di trasporto (`TCP`).

### 2.7.1 Autonomia del Datagramma e Assenza di Delimitatori di Coda

Ciascun pacchetto IP prende il nome di **datagramma** ed è intrinsecamente **autonomo**: viaggia nella rete trasportando con sé tutte le informazioni di controllo (*Protocol Control Information*, PCI) indispensabili affinché qualsiasi router intermedio possa decidere in modo indipendente come e dove inoltrarlo.

A differenza dei protocolli di collegamento (Layer 2) che delimitano le trame tramite sequenze fisse di sincronismo (*flag* o pattern di *framing*), il datagramma IP **non possiede alcun campo di fine pacchetto**: l'header definisce unicamente l'inizio del datagramma. Per stabilire univocamente dove termini il pacchetto e separarlo dai flussi successivi estratti dal buffer, il ricevitore deve necessariamente decodificare il campo `TOTAL LENGTH` presente nell'intestazione.

### 2.7.2 Struttura e Campi dell'Header IPv4

L'intestazione di un datagramma IPv4 presenta una lunghezza variabile da un minimo di 20 byte (in assenza di opzioni) fino a un massimo di 60 byte (con opzioni). I campi sono organizzati su parole di 32 bit (4 byte), come illustrato in Figura 2.11.

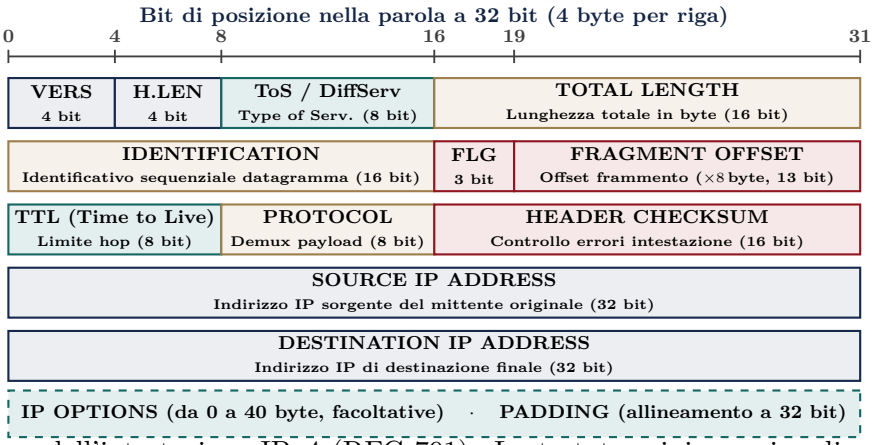



Figura 2.11: Struttura dell'intestazione IPv4 (RFC 791). La testata minima priva di opzioni misura esattamente 20 byte (5 parole da 32 bit).

L'analisi ingegneristica puntuale dei campi evidenzia le seguenti proprietà operative:

**VERS (Version) — 4 bit**
: Versione del protocollo (`0100` per IPv4). Consente ai router dual-stack di discriminare istantaneamente datagrammi IPv4 da IPv6 (`0110`).

**H.LEN (Header Length / IHL) — 4 bit**
: Lunghezza dell'header in parole da 32 bit (4 byte). Valore tipico 5 (20 byte senza opzioni), massimo 15 (60 byte con opzioni).

**Type of Service (ToS / DiffServ) — 8 bit**
: Qualità del servizio: 6 bit DSCP (classi di priorità del traffico) e 2 bit ECN (segnalazione preventiva di congestione senza scarto pacchetti).

**TOTAL LENGTH — 16 bit**
: Dimensione complessiva del datagramma (header + payload) in byte. Limite teorico 65 535 byte; MTU minima di link: 576 byte.

**Campi di Frammentazione (32 bit totali: IDENTIFICATION, FLAGS, FRAGMENT OFFSET)**
: Gestiscono la frammentazione nei router intermedi: **ID** (16 bit) correla i frammenti originati dallo stesso pacchetto; **Flags** (3 bit: `DF`=Don't Fragment impone lo scarto con ICMP Tipo 3 Codice 4 per PMTUD, `MF`=More Fragments indica frammenti successivi); **Fragment Offset** (13 bit) specifica la posizione del carico utile in unità di 8 byte (64 bit).

**TIME TO LIVE (TTL) — 8 bit (Parametro Cardine)**
: Inizializzato dal mittente (es. 64 o 128) e decrementato di 1 a ogni router (TTL $\leftarrow$ TTL $-$ 1). A 0, il pacchetto viene scartato con errore ICMP *Time Exceeded* (Tipo 11). **Ruolo cardine**: previene il looping indefinito dei pacchetti sui cicli di routing.

**PROTOCOL — 8 bit (Parametro Cardine)**
: Demultiplexing verso L4: 1 per `ICMP`, 6 per `TCP`, 17 per `UDP`. **Abilitatore del tunneling**: 4 per IP-in-IP, 41 per `IPv6` (6in4).

**HEADER CHECKSUM — 16 bit (Parametro Cardine)**
: Controllo integrità dell'header (somma in complemento a uno a 16 bit). Poiché il TTL varia a ogni hop, **ogni router deve ricalcolarlo**. Discordanze causano lo scarto silente del datagramma.

**SOURCE IP e DESTINATION IP — 32 bit ciascuno (Parametro Cardine)**
: Indirizzi logici globali di origine e terminazione finale. Restano invariati lungo l'intero cammino attraverso router ordinari.

**OPTIONS e PADDING — da 0 a 40 byte**
: Opzioni diagnostiche facoltative (*Timestamp*, *Record Route*). Il padding garantisce l'allineamento della testata a multipli di 32 bit.

### 2.7.3 Tecniche di Tunneling

Il campo `PROTOCOL` consente la realizzazione del **tunneling**: una tecnica architetturale mediante cui un intero pacchetto di un determinato protocollo (ad esempio IPv6 o una trama Ethernet) viene incapsulato all'interno del payload di un normale datagramma IPv4.

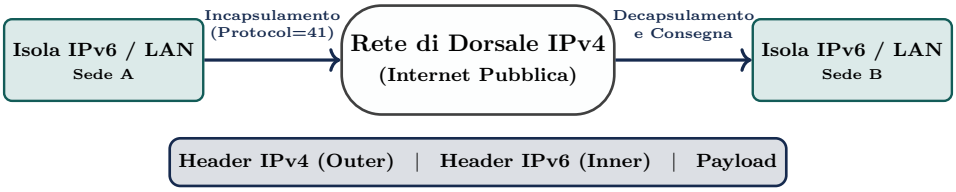



Figura 2.12: Architettura di tunneling IPv6-in-IPv4 (Protocollo 41): le sedi remote comunicano in modo trasparente attraverso la rete IPv4.

Il tunneling consente a isole IPv6 o reti private (VPN) di comunicare in modo trasparente attraverso una dorsale pubblica. Il costo risiede nell'overhead di imbustamento (20 B di header aggiuntivo) e nel potenziale rischio di frammentazione qualora il datagramma incapsulato ecceda la MTU del collegamento di transito.

### 2.7.4 MTU e Meccanica Analitica della Frammentazione

Ogni tecnologia fisica di Livello 2 impone un vincolo superiore alla dimensione massima del frame trasferibile, noto come **Maximum Transmission Unit** (MTU). Qualora la lunghezza complessiva di un datagramma superi la MTU del link sul quale deve essere trasmesso, il router intermedio è obbligato a suddividere il datagramma originale in più frammenti indipendenti (se DF = 0).

| Tecnologia di Livello 2 | Tipologia / Descrizione | Valore MTU Tipico |
|---|---|---|
| Ethernet (IEEE 802.3) | Standard cablato per reti locali | 1500 byte |
| Wi-Fi (IEEE 802.11) | Reti locali wireless (WLAN) | 2304 byte |
| PPPoE | Accesso residenziale broadband | 1492 byte |
| Minimo garantito IPv4 | Soglia minima per nodi IPv4 | 576 byte |

Tabella 2.3: Valori rappresentativi della MTU per le principali tecnologie di collegamento dati.

**Principio / Fondamento: Regole di Frammentazione e Riassemblaggio**

1. **Frammentazione Intermedia**: la frammentazione può avvenire sia alla sorgente sia presso qualsiasi router intermedio lungo il cammino.
2. **Copia Autonoma dell'Header**: ciascun frammento generato costituisce a tutti gli effetti un datagramma IP a sé stante, munito di una copia dell'header originale (con medesimo `IDENTIFICATION`), ma con valori opportunamente modificati di `TOTAL LENGTH`, `FLAGS` e `FRAGMENT OFFSET`.
3. **Riassemblaggio Esclusivo alla Destinazione**: i router intermedi **non riassemblano mai** i frammenti. Il riassemblaggio viene compiuto tassativamente ed unicamente dall'host destinatario finale prima di passare il blocco dati al livello trasporto.
4. **Vulnerabilità alla Perdita**: se anche un solo frammento va disperso nel tragitto, l'host di destinazione non può ricostruire il pacchetto originale. Allo scadere di un timer di riassemblaggio, **l'intero datagramma viene distrutto** e viene notificato un errore ICMP *Time Exceeded*. La frammentazione degrada pertanto esponenzialmente l'affidabilità della connessione.

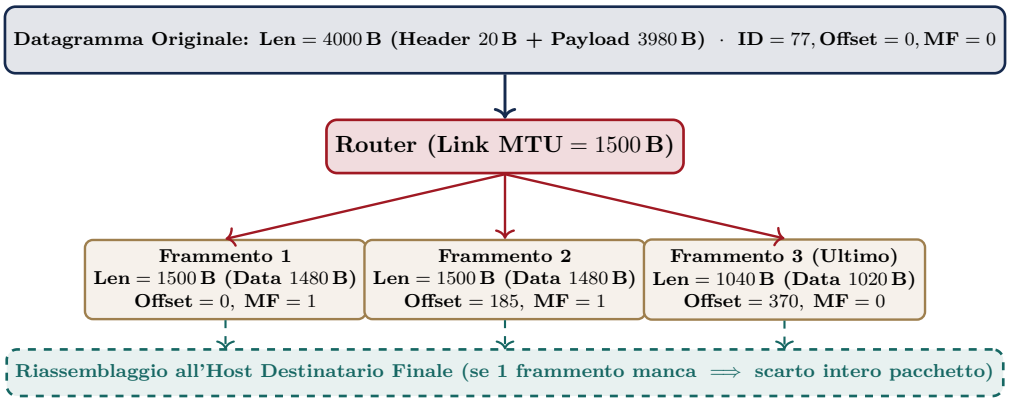



Figura 2.13: Processo di frammentazione IP: calcolo degli offset multipli di 8 byte e gestione dei flag MF.

## 2.8 Indirizzamento IPv4, Subnetting e Indirizzi Speciali

### 2.8.1 L'Indirizzo IP Identifica una Scheda di Rete (NIC), Non un Host

**Principio / Fondamento: Proprietà Fondamentale dell'Identità di Livello 3**

Un indirizzo IP **non identifica un elaboratore fisico o un utente**, bensì identifica in modo univoco una specifica **scheda di interfaccia di rete** (*Network Interface Card*, NIC).

Se un computer possiede due interfacce fisiche attive (ad esempio una porta cablata Ethernet e un modulo radio Wi-Fi), esso possiede **due indirizzi IP distinti e indipendenti**, ciascuno attestato sulla rispettiva sottorete. Un terminale collegato simultaneamente a più sottoreti prende il nome di host **multi-homed**. In assenza di configurazioni speciali di instradamento, il sistema operativo instrada il traffico uscente prediligendo tipicamente l'interfaccia a banda superiore (es. Ethernet rispetto a Wi-Fi), ma **non inoltra pacchetti tra le sottoreti**, compito esclusivo dei router.

### 2.8.2 Notazione Dotted Decimal e Struttura Gerarchica

Un indirizzo IPv4 è costituito da un numero binario di 32 bit. Per facilitarne la lettura e la manipolazione da parte degli operatori umani, i 32 bit vengono convenzionalmente raggruppati in 4 ottetti (byte) e trascritti in **notazione decimale puntata** (*dotted decimal*), dove ciascun byte è interpretato come intero privo di segno compreso tra 0 e 255:

$$\underbrace{10000010}_{130} \,.\, \underbrace{11000000}_{192} \,.\, \underbrace{00001111}_{15} \,.\, \underbrace{00000001}_{1} \implies \texttt{130.192.15.1}$$

Ogni indirizzo IP è strutturato gerarchicamente in due sezioni distinte:

1. **Prefisso di Rete (NetID)**: identifica univocamente la specifica sottorete in tutta la rete Internet. L'assegnazione dei NetID pubblici è coordinata a livello planetario dall'`IANA` (*Internet Assigned Numbers Authority*) e dai registri regionali, `RIR`.
2. **Suffisso di Host (HostID)**: identifica la singola interfaccia di rete collegata a quella determinata sottorete. Viene allocato autonomamente dall'amministratore locale della rete senza necessità di coordinamento globale.

### 2.8.3 Subnet Mask e Notazione CIDR

La demarcazione tra NetID e HostID non è rigidamente prefissata, ma è definita mediante una maschera a 32 bit denominata **Subnet Mask** (o maschera di sottorete). La subnet mask è composta da una serie contigua di $N$ bit posti a 1 (in corrispondenza del NetID), seguiti da $32 - N$ bit posti a 0 (in corrispondenza dell'HostID).

Nella notazione standard `CIDR` (*Classless Inter-Domain Routing*) la maschera viene sintetizzata aggiungendo la barra (slash) seguita dal numero $N$ di bit di rete:

$$\text{Notazione CIDR: } x.y.z.w \,/\, N$$

**Principio / Fondamento: Calcolo delle Dimensioni della Sottorete**

Dato un prefisso $/N$, la sottorete ospita un quantitativo complessivo di indirizzi pari a una potenza intera di due:

$$\text{Numero totale di indirizzi} = 2^{32-N} \tag{2.2}$$

All'interno di ciascuna sottorete, **due indirizzi sono tassativamente riservati** e non possono essere assegnati alle interfacce:

- Il primo indirizzo (HostID = tutti 0): identifica la sottorete stessa (*Network Address*);
- L'ultimo indirizzo (HostID = tutti 1): rappresenta l'indirizzo di *Directed Broadcast*.

Il numero di indirizzi IP effettivamente assegnabili agli host (interfacce utili) risulta pertanto rigorosamente pari a:

$$N_{\text{host utili}} = 2^{32-N} - 2 \tag{2.3}$$

**La Sottorete Minima Operativa: /30**

La sottorete minima utilizzabile per ospitare host è la `/30`, poiché dispone di $2^{32-30} = 2^2 = 4$ indirizzi totali, dei quali $4 - 2 = 2$ sono assegnabili alle interfacce. Tale configurazione è tipicamente riservata ai **collegamenti punto-punto tra due router**, dove sono necessarie esattamente due interfacce comunicanti. Una maschera `/31` ($2^1 = 2$ indirizzi) nel modello classico lascerebbe $2 - 2 = 0$ indirizzi utili, risultando inutilizzabile.

### 2.8.4 Esercitazione d'Aula: Assegnazione Uniforme di Indirizzi IP e Rete di Campus

**Esercizio d'Esame: Assegnazione Uniforme di Indirizzi di Rete (Esercitazione 1)**

**Testo del Problema:** Una rete di campus (Università / Azienda) possiede la topologia rappresentata in Figura 2.14, articolata su tre reti locali distinte designate come **LAN1**, **LAN2** e **LAN3**, interconnesse mediante due router: $R_A$ ed $R_B$. Il router $R_A$ fornisce il gateway di accesso verso Internet. All'amministratore della rete viene assegnato il blocco di indirizzi pubblici:

Pool Assegnato: `16.5.1.0/24` ($2^8 = 256$ indirizzi totali)

Il blocco deve essere utilizzato per dimensionare e configurare le tre sottoreti IP **SN1**, **SN2** e **SN3** da attestare rispettivamente su LAN1, LAN2 e LAN3.

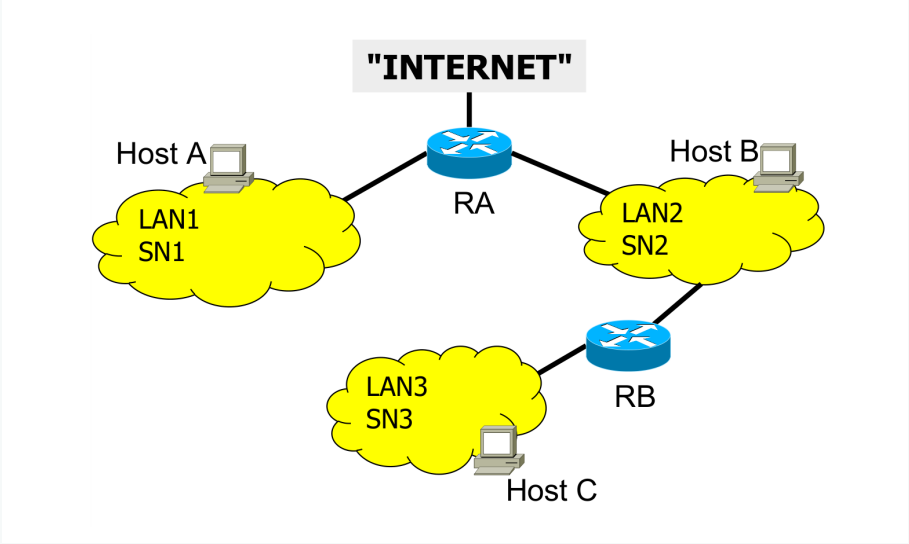



Figura 2.14: Topologia ufficiale della rete di campus: LAN1, LAN2 e LAN3 separate dai router $R_A$ ed $R_B$, con accesso Internet da $R_A$ e posizionamento degli Host A, B e C.

Si richiede di risolvere analiticamente i seguenti quesiti:

a) Assegnare gli indirizzi di rete alle tre subnet (**SN1**, **SN2** e **SN3**) partizionando il blocco in modo che ciascuna subnet possa contenere **lo stesso numero di stazioni**, e **massimizzando** tale numero.
b) Specificare l'indirizzo di **broadcast** per ogni subnet, spiegando rigorosamente come si ottiene a livello binario.
c) Assegnare il primo (o i primi) indirizzo/i disponibile/i per ogni subnet all'interfaccia/e del/dei router ad essa collegato/i.
d) Calcolare l'**efficienza** di utilizzo degli indirizzi usando questo schema di indirizzamento rispetto al blocco `16.5.1.0/24` (rapporto tra indirizzi effettivamente utilizzabili e indirizzi unicast disponibili nel blocco padre).
e) Trovare un metodo architetturale per impiegare un maggior numero di indirizzi rispettando il vincolo inderogabile di assegnare esattamente lo stesso numero di indirizzi a ciascuna delle tre subnet.

**Svolgimento e Risoluzione Analitica Passo-Passo:**

**Quesito a) Partizionamento Uniforme Ottimo del Blocco `16.5.1.0/24`:** Il blocco genitore `16.5.1.0/24` dispone di un suffisso host a $32 - 24 = 8$ bit, per un totale di $2^8 = 256$ indirizzi IP

complessivi. Dovendo generare almeno tre sottoreti di dimensioni identiche sfruttando i prefissi standard CIDR basati su potenze di 2, è necessario individuare la minima potenza di due maggiore o uguale a 3:

$$K = \lceil \log_2(3) \rceil = 2 \implies 2^2 = 4 \text{ sottoreti disgiunte}$$

Prendendo a prestito 2 bit dalla porzione HostID per estendere la lunghezza del prefisso NetID, la nuova subnet mask risulta:

$$N = 24 + 2 = /\mathbf{26} \quad \text{(Subnet Mask: } \texttt{255.255.255.192}\text{)}$$

Ciascun blocco /26 dispone di $2^{32-26} = 2^6 = 64$ indirizzi IP totali, dei quali:

$$N_{\text{host utili}} = 2^6 - 2 = 64 - 2 = \mathbf{62} \text{ indirizzi unicast per stazioni}$$

Tale scelta massimizza la capienza per singola subnet (una divisione in 8 blocchi /27 fornirebbe soltanto 30 host utili a subnet). Le quattro partizioni risultanti nello spazio `16.5.1.0/24` sono:

- **SN1 (LAN1)**: `16.5.1.0/26` (intervallo: `16.5.1.0` – `16.5.1.63`, 62 host utili);
- **SN2 (LAN2)**: `16.5.1.64/26` (intervallo: `16.5.1.64` – `16.5.1.127`, 62 host utili);
- **SN3 (LAN3)**: `16.5.1.128/26` (intervallo: `16.5.1.128` – `16.5.1.191`, 62 host utili);
- **Subnet 4 (Riserva non assegnata)**: `16.5.1.192/26` (intervallo: `16.5.1.192` – `16.5.1.255`, 64 indirizzi disponibili per futuri sviluppi).

**Quesito b) Indirizzi di Broadcast per Ciascuna Sottorete:** L'indirizzo di *Directed Broadcast* di una sottorete si ottiene mantenendo inalterati i 26 bit di NetID e ponendo **tutti i 6 bit dell'HostID al valore logico 1**:

- Per **SN1**: gli ultimi 8 bit sono $\mathbf{00}\,\mathbf{111111}_2 = 63_{10} \implies$ Broadcast: `16.5.1.63`
- Per **SN2**: gli ultimi 8 bit sono $\mathbf{01}\,\mathbf{111111}_2 = 64 + 63 = 127_{10} \implies$ Broadcast: `16.5.1.127`
- Per **SN3**: gli ultimi 8 bit sono $\mathbf{10}\,\mathbf{111111}_2 = 128 + 63 = 191_{10} \implies$ Broadcast: `16.5.1.191`

**Quesito c) Assegnazione Indirizzi alle Interfacce dei Router e Configurazione Host:** Si adotta la prescrizione di assegnare il primo (o i primi) indirizzo unicast utile all'interfaccia/e dei router connessi a ciascuna LAN:

- **LAN1 (SN1: `16.5.1.0/26`)**: collegata esclusivamente al router $R_A$.
  - Interfaccia $R_A$ (`eth1`): primo IP utile $\implies$ `16.5.1.1/26`.
  - Host A e le altre stazioni di LAN1 utilizzano indirizzi nell'intervallo `16.5.1.2` – `16.5.1.62`, configurando come *Default Gateway* `16.5.1.1`.
- **LAN2 (SN2: `16.5.1.64/26`)**: vede attestati **entrambi i router** ($R_A$ ed $R_B$). Si assegnano i primi due indirizzi utili consecutivi:
  - Interfaccia $R_A$ (`eth2`): primo IP utile $\implies$ `16.5.1.65/26`.
  - Interfaccia $R_B$ (`eth2`): secondo IP utile $\implies$ `16.5.1.66/26` (essendo `.65` già impegnato da $R_A$).
  - Host B e le altre stazioni di LAN2 utilizzano indirizzi nell'intervallo `16.5.1.67` – `16.5.1.126`, con Default Gateway `16.5.1.65` per raggiungere Internet o LAN1, e rotta locale su `16.5.1.66` per raggiungere LAN3.
- **LAN3 (SN3: `16.5.1.128/26`)**: collegata esclusivamente al router $R_B$.
  - Interfaccia $R_B$ (`eth3`): primo IP utile $\implies$ `16.5.1.129/26`.
  - Host C e le altre stazioni di LAN3 utilizzano indirizzi nell'intervallo `16.5.1.130` – `16.5.1.190`, configurando come Default Gateway `16.5.1.129`.

| Subnet | Prefisso | Network IP | IP Router | Broadcast | Intervallo Host Utili |
|---|---|---|---|---|---|
| **SN1 (LAN1)** | /26 | `16.5.1.0` | $R_A$: `16.5.1.1` | `16.5.1.63` | `16.5.1.2` – `16.5.1.62` (61 per host) |
| **SN2 (LAN2)** | /26 | `16.5.1.64` | $R_A$: `16.5.1.65`, $R_B$: `16.5.1.66` | `16.5.1.127` | `16.5.1.67` – `16.5.1.126` (60 per host) |
| **SN3 (LAN3)** | /26 | `16.5.1.128` | $R_B$: `16.5.1.129` | `16.5.1.191` | `16.5.1.130` – `16.5.1.190` (61 per host) |
| **Riserva** | /26 | `16.5.1.192` | — | `16.5.1.255` | `16.5.1.193` – `16.5.1.254` (62 disponibili) |

Tabella 2.4: Riepilogo analitico dell'allocazione uniforme: indirizzi di rete, router, broadcast e pool host per le tre sottoreti.

**Quesito d) Calcolo dell'Efficienza di Utilizzo dello Spazio di Indirizzamento:** L'efficienza $\eta$ è definita formalmente dal rapporto tra gli indirizzi effettivamente utilizzabili dalle stazioni e tutti gli indirizzi unicast disponibili nel blocco assegnato.

- Gli indirizzi unicast assegnabili nelle tre subnet operative sono:

$$N_{\text{utilizzabili}} = 3 \times (2^6 - 2) = 3 \times 62 = \mathbf{186} \text{ indirizzi}$$

  (Di questi 186 indirizzi, 4 sono impegnati dalle interfacce dei router e 182 sono disponibili per gli host finali).
- Nel blocco complessivo `16.5.1.0/24`, il numero totale di indirizzi unicast disponibili è pari a $2^8 - 2 = 254$ (escludendo `16.5.1.0` come Network Address globale e `16.5.1.255` come Broadcast Address globale).

Pertanto, l'efficienza reale risulta:

$$\eta = \frac{186}{254} \approx \mathbf{73.23}\% \tag{2.4}$$

*Nota metodologica sul denominatore:* Se il calcolo viene condotto considerando la totalità algebrica dei 256 indirizzi grezzi del blocco /24 (comprensivi dei due indirizzi riservati estremi), il rapporto diviene:

$$\eta' = \frac{186}{256} \approx \mathbf{72.66}\%$$

Entrambe le convenzioni sono considerate corrette se opportunamente motivate; il valore del 73.23% è quello che riflette rigorosamente la definizione di indirizzi unicast utilizzabili.

**Quesito e) Partizionamento Avanzato e Massimizzazione degli Indirizzi (Recupero con /28):** Il vincolo impone di incrementare il numero di indirizzi utilizzati garantendo che **ciascuna delle tre subnet contenga esattamente lo stesso numero di indirizzi**. Attualmente, il quarto blocco `16.5.1.192/26` (64 indirizzi) giace completamente inutilizzato. Per ridistribuire equamente tale capacità sulle tre reti locali senza alterare il partizionamento primario, si procede alla **suddivisione dicotomica del blocco residuo in quattro blocchi con maschera /28** ($2^{32-28} = 16$ indirizzi ciascuno, con $16 - 2 = 14$ host utili):

- **SN1.2** : `16.5.1.192/28` (`16.5.1.192` – `16.5.1.207`, 14 host utili) $\implies$ assegnata a **LAN1**;
- **SN2.2** : `16.5.1.208/28` (`16.5.1.208` – `16.5.1.223`, 14 host utili) $\implies$ assegnata a **LAN2**;
- **SN3.2** : `16.5.1.224/28` (`16.5.1.224` – `16.5.1.239`, 14 host utili) $\implies$ assegnata a **LAN3**;
- **Riserva** : `16.5.1.240/28` (`16.5.1.240` – `16.5.1.255`, 16 indirizzi) $\implies$ residuo libero.

In questo modo, su ciascuna LAN fisica coesistono due sottoreti logiche (multi-netting L3). Ciascuna rete locale guadagna esattamente 14 stazioni utili aggiuntive, portando la capienza utile di ogni zona da 62 a $62 + 14 = \mathbf{76}$ host. Il numero complessivo di indirizzi utilizzabili sale a:

$$N^{\text{ott}}_{\text{utilizzabili}} = 3 \times 76 = 186 + (3 \times 14) = \mathbf{228} \text{ indirizzi}$$

L'efficienza complessiva di impiego dello spazio /24 compie un salto qualitativo notevole:

$$\eta_{\text{ott}} = \frac{228}{254} \approx \mathbf{89.76\%} \quad \left(\text{oppure } \frac{228}{256} \approx \mathbf{89.06\%}\right) \tag{2.5}$$

Tale approccio evidenzia la flessibilità del CIDR nel consentire estensioni modulari e convivenza logica sulla medesima infrastruttura di livello 2.

### 2.8.5 Catalogo Sistematico degli Indirizzi IP Speciali

Non tutti gli indirizzi IP esprimibili nello spazio a 32 bit identificano una normale interfaccia unicast instradabile. Esistono precisi intervalli con semantica dedicata, sintetizzati in Tabella 2.5.

| Tipologia Indirizzo | Sintassi / Blocco | Significato e Ruolo Architetturale |
|---|---|---|
| **Indirizzo di Rete** | HostID = tutti 0 | Identifica l'entità logica della sottorete. Non compare mai come indirizzo sorgente o destinazione in un pacchetto. |
| **Directed Broadcast** | HostID = tutti 1 (es. `130.192.15.255/24`) | Indirizzato a tutte le interfacce della specifica sottorete remota. Instradato dai router fino all'accesso locale. |
| **Limited Broadcast** | `255.255.255.255` (tutti 32 bit a 1) | Broadcast limitato al link locale. **I router non lo inoltrano mai all'esterno**. Compare solo come destinazione. |
| **This Host (Questo Host)** | `0.0.0.0` (tutti 32 bit a 0) | Usato come indirizzo sorgente transitorio all'avvio da un host non ancora configurato (es. fase di DHCP Discovery). |
| **Loopback** | `127.0.0.0/8` (tipic. `127.0.0.1`) | Reindirizzato internamente allo stack IP locale senza transitare sulla scheda di rete. Usato per debug e comunicazioni IPC. |
| **Multicast (Classe D)** | `224.0.0.0/4` (`224.0.0.0` ∇· `239.255.255.255`) | Indirizzi di gruppo per recapitare un datagramma a molti riceventi con duplicazione efficiente a livello di rete. |
| **Link-Local (APIPA)** | `169.254.0.0/16` | Autoconfigurazione locale automatica in assenza di server DHCP (usata per link isolati o domotica). |

Tabella 2.5: Catalogo formale degli indirizzi IPv4 speciali e riservati.

**Multicast Nativo vs Multiple Unicast e Complessità Computazionale**

La trasmissione contemporanea della stessa informazione a $M$ destinatari può avvenire secondo due filosofie:

- **Multiple Unicast (Host-based)**: la sorgente genera e trasmette $M$ datagrammi unicast distinti sul proprio link di accesso (è il modello adottato da applicazioni web, canali Telegram o liste broadcast di messaggistica). L'overhead per la sorgente scala come $O(M)$.
- **Multicast Nativo (Network-based)**: la sorgente invia un **singolo pacchetto** destinato all'indirizzo di gruppo (ad es. `224.0.0.1` per tutti gli host o `224.0.0.2` per tutti i router). Sono i router della rete che si incaricano di calcolare un albero di copertura ottimo e duplicare il pacchetto solo nei nodi di diramazione.

### 2.8.6 Indirizzi Privati e Consegna Diretta vs Indiretta

Per mitigare l'esaurimento globale dello spazio di indirizzamento a 32 bit, l'RFC 1918 ha riservato tre blocchi di indirizzi designati come **indirizzi privati**:

- **Blocco 10**: `10.0.0.0/8` (`10.0.0.0` ∇· `10.255.255.255`, oltre 16 milioni di indirizzi), ideale per grandi campus universitari e reti aziendali complesse suddivise in molteplici sottoreti interne.
- **Blocco 172**: `172.16.0.0/12` (`172.16.0.0` ∇· `172.31.255.255`, 16 blocchi /16), utilizzato in reti aziendali intermedie, ambienti di virtualizzazione e cluster industriali.
- **Blocco 192**: `192.168.0.0/16` (`192.168.0.0` ∇· `192.168.255.255`, 256 blocchi /24), standard de facto universale per le reti locali domestiche e piccoli uffici (*SOHO*).

Gli indirizzi privati **non sono instradabili su Internet**: i router di frontiera scartano qualsiasi pacchetto che rechi un IP privato diretto verso l'esterno. Per consentire a tali nodi di accedere ai servizi della rete pubblica, è indispensabile l'interposizione di un apparato di traslazione NAT.

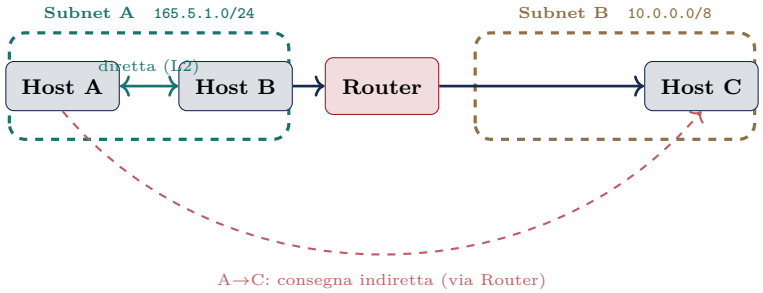



Figura 2.15: Consegna diretta (stessa subnet, livello 2) vs consegna indiretta (subnet diverse, attraverso router).

Il meccanismo di consegna discrimina due casistiche fondamentali (Figura 2.15):

- **Consegna Diretta**: si verifica quando la sorgente e la destinazione appartengono alla **medesima sottorete logica**. L'host mittente esegue l'operazione di AND logico tra l'indirizzo IP di destinazione e la propria subnet mask: se il risultato coincide con il proprio NetID, il pacchetto viene inviato direttamente a livello data link (Layer 2) tramite risoluzione dell'indirizzo MAC locale, senza transitare da alcun router.
- **Consegna Indiretta**: si verifica quando sorgente e destinazione risiedono su sottoreti distinte. L'host mittente constata la non coincidenza del NetID e incapsula il datagramma indirizzandolo fisicamente al proprio **Default Gateway** (il router locale). Il router riceve il datagramma, ispeziona la propria tabella di routing e provvede a inoltrarlo lungo la catena di hop successivi.

## 2.9 Protocolli di Supporto ad IP: ICMP e DHCP

La robustezza del protocollo IP deriva dalla sua essenzialità: non implementa controlli di flusso, segnalazioni diagnostiche o meccanismi di configurazione automatica dei parametri di rete. Per sopperire a tali lacune senza appesantire il motore di commutazione, l'architettura di Internet affianca ad IP due protocolli ausiliari di fondamentale rilevanza: `ICMP` e `DHCP`.

### 2.9.1 ICMP: Internet Control Message Protocol

`ICMP` è un protocollo di segnalazione ed errore operante a livello di rete. Non trasporta dati utente, ma veicola notifiche relative a guasti, irraggiungibilità o misurazioni diagnostiche dello stato dei nodi.

**Principio / Fondamento: Interdipendenza e Collocazione Architetturale**

Tra IP e ICMP sussiste una relazione simbiotica peculiare:

- Il modulo IP fa affidamento su ICMP per gestire e segnalare gli eventi anomali;
- I messaggi ICMP sono **incapsulati direttamente nel payload di un datagramma IP** (Protocol = 1).
- ICMP **non usa protocolli di livello trasporto** (quali TCP o UDP): fa strettamente parte del livello di rete. Questa scelta è indispensabile affinché un router possa emettere messaggi di controllo senza dover implementare le complessità del livello trasporto.

| Tipo (Type) | Codice (Code) | Denominazione e Scopo Operativo |
|---|---|---|
| **0** | 0 | **Echo Reply**: risposta emessa in seguito alla ricezione di un Echo Request (base di `ping`). |
| **3** | $0\nabla \cdot 15$ | **Destination Unreachable**: notifica che una rete, un host, un protocollo o una porta specifica sono irraggiungibili. Il codice 4 indica: *Fragmentation needed and DF set* (base di PMTUD). |
| **5** | $0\nabla \cdot 3$ | **Redirect**: un router informa un host locale che esiste un router alternativo più efficiente sulla stessa subnet per quella specifica destinazione. |
| **8** | 0 | **Echo Request**: richiesta di eco inviata per saggiare la raggiungibilità di un nodo remoto. |
| **9 / 10** | 0 | **Router Discovery**: messaggi di sollecitazione e annuncio dei router attivi sulla subnet locale. |
| **11** | 0 / 1 | **Time Exceeded**: codice 0 notifica che il TTL è decrementato a zero durante l'inoltro; codice 1 notifica il timeout durante il riassemblaggio dei frammenti. |
| **12** | 0 / 1 | **Parameter Problem**: intestazione IP malformata (checksum errato o puntatore su opzione corrotta). |

Tabella 2.6: Messaggi ICMP fondamentali analizzati nel corso (RFC 792).

**Principio / Fondamento: La Regola Aurea Anti-Tempesta (Anti-Storm Rule)**

Per prevenire reazioni a catena distruttive, lo standard stabilisce rigorosamente che:
**Un messaggio di errore ICMP non deve in nessun caso generare un ulteriore messaggio ICMP.**
Se un pacchetto contenente un messaggio ICMP di errore riscontra a sua volta un errore lungo la rete (ad esempio timeout o TTL nullo), esso viene scartato silenziosamente. In caso contrario, una serie di guasti a catena genererebbe una tempesta (*storm*) di segnalazioni in loop che saturerebbe l'intera infrastruttura telematica.

#### Applicazioni Diagnostiche Operative Basate su ICMP

- **Il Comando `ping`**: trasmette pacchetti IP contenenti un messaggio ICMP *Echo Request* (Tipo 8) verso l'indirizzo bersaglio e attende la restituzione dell'*Echo Reply* (Tipo 0). Consente di verificare la raggiungibilità logica del nodo e calcolare statistiche sul **Round Trip Time** (RTT) e sulla percentuale di pacchetti persi. Permette inoltre di discriminare guasti di rete da anomalie applicative (se il web server non risponde ma il `ping` ha successo, la rete IP è integra ed è collassato il demone software HTTP).
- **Il Comando `traceroute`**: permette di scoprire e mappare la sequenza esatta dei router intermedi attraversati per raggiungere una destinazione. Il mittente trasmette una sequenza di pacchetti con TTL progressivo:
  1. Invia pacchetto con TTL = 1: il primo router decrementa il TTL a 0, scarta il pacchetto e restituisce alla sorgente un ICMP *Time Exceeded* (Tipo 11), rivelando il proprio indirizzo IP e permettendo di misurare l'RTT del primo hop.
  2. Invia pacchetto con TTL = 2: supera il primo router ed è scartato con TTL = 0 dal secondo router, che restituisce il secondo messaggio *Time Exceeded*.
  3. La procedura itera incrementando il TTL fino a raggiungere l'host di destinazione finale, che risponde con un messaggio applicativo o un ICMP *Echo Reply / Port Unreachable*.

### 2.9.2 DHCP: Dynamic Host Configuration Protocol

Affinché un terminale possa operare in rete, il relativo software IP deve essere inizializzato con una quaterna fondamentale di parametri:

1. Un indirizzo IP unicast valido e univoco sulla sottorete;
2. La relativa Subnet Mask (per discriminare le consegne dirette da quelle indirette);
3. L'indirizzo IP del router di uscita (*Default Gateway*);
4. L'indirizzo IP di almeno un server `DNS` per la risoluzione dei nomi simbolici.

La configurazione manuale di tali parametri genera frequenti errori umani, duplicazioni e blocchi operativi catastrofici. Il protocollo `DHCP` risolve tale criticità implementando l'**autoconfigurazione dinamica centralizzata**.

### Architettura e Trasporto UDP

DHCP opera secondo un modello **Client-Server**: il client risiede sull'host che richiede la configurazione, mentre il server gestisce un pool di indirizzi allocabili. Contrariamente a ICMP, DHCP non è incapsulato direttamente in IP ma si appoggia al livello di trasporto tramite `UDP` (sfruttando la porta server 67 e la porta client 68).

L'adozione di UDP anziché TCP è un vincolo tecnologico invalicabile: un host privo di indirizzo IP valido non può in alcun modo instaurare il three-way handshake di TCP (che richiede indirizzi sorgente e destinazione unicast già operativi).

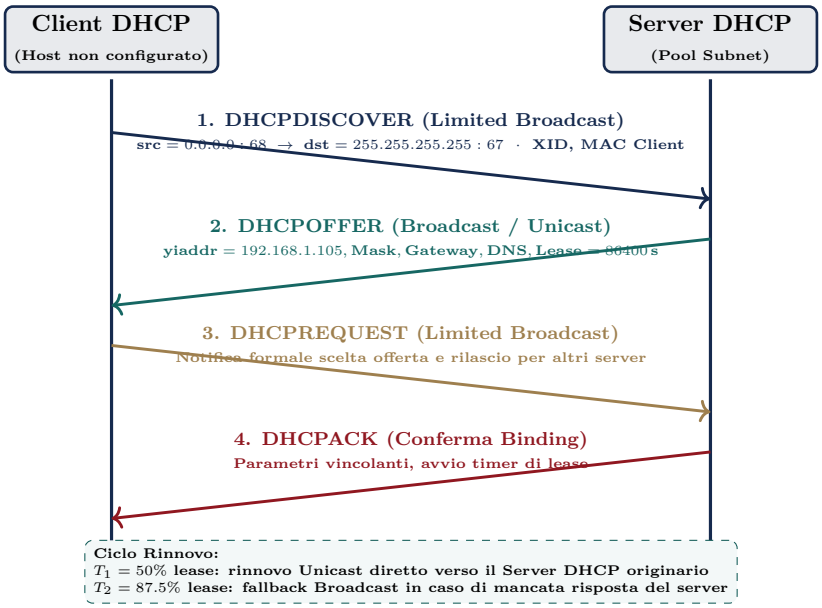



Figura 2.16: Ladder diagram del processo di configurazione a quattro vie (*4-Way Handshake DORA*) in DHCP.

Il colloquio si articola nella sequenza a quattro vie (**DORA**, Figura 2.16):

1. **DHCPDISCOVER**: l'host appena acceso non possiede un IP, per cui impiega come sorgente `0.0.0.0` e trasmette in limited broadcast verso `255.255.255.255` sulla porta UDP 67. Il messaggio include l'indirizzo fisico MAC della scheda (campo CHADDR) e un identificativo di transazione casuale (*Transaction ID*, XID).
2. **DHCPOFFER**: uno o più server DHCP attivi sulla sottorete estraggono un indirizzo libero dal proprio pool e formulano una proposta inviata all'host (tramite broadcast o indirizzamento MAC diretto). Il messaggio valorizza il campo `yiaddr` (*Your IP Address*) con l'IP proposto, accompagnato da subnet mask, default gateway, DNS e durata del contratto (*lease*).
3. **DHCPREQUEST**: il client raccoglie le offerte ricevute e ne seleziona una (tipicamente la prima pervenuta). Invia quindi un messaggio di Request in **limited broadcast**. L'impiego del broadcast in questa terza fase è cruciale: permette di confermare l'accettazione al server prescelto (identificato dal campo *Server Identifier*) e contemporaneamente informa implicitamente tutti gli altri server DHCP che le loro proposte sono state rifiutate, consentendo loro di rimettere gli indirizzi offerti a disposizione del pool.
4. **DHCPACK**: il server selezionato riscontra la richiesta e sigilla il binding (*IP assegnato* $\leftrightarrow$ *MAC host*), chiudendo formalmente la transazione.

### Gestione del Lease Time nel DHCP

Ogni assegnazione DHCP è soggetta a una scadenza temporale (**lease time**), comunemente compresa tra 30 minuti e 24 ore:

- Al tempo $T_1 = 0.5 \times$ lease, il client invia una `DHCPREQUEST` in **unicast diretto** al server per prorogare il contratto.
- Se il server non risponde, al tempo $T_2 = 0.875 \times$ lease il client ritrasmette la richiesta in broadcast verso qualsiasi server disponibile.
- Se il lease spira senza rinnovo, l'host deve cessare istantaneamente l'impiego dell'indirizzo e riavviare la fase di Discover.

Oltre all'assegnazione dinamica temporanea, DHCP supporta l'assegnazione **statica o permanente** (binding rigido configurato a monte tra un determinato indirizzo MAC e uno specifico indirizzo IP riservato).

## 2.10 Network Address Translation (NAT) e NAPT

### 2.10.1 Genesi, Ruolo e Architettura Stateful

La proliferazione planetaria di nodi interconnessi a fronte della disponibilità limitata di circa 4.3 miliardi di indirizzi IPv4 ha reso indispensabile l'adozione su scala universale del **NAT** (*Network Address Translation*, RFC 3022). Il NAT opera come apparato intermedio (**middlebox**), tipicamente integrato nel router di frontiera, che separa la rete privata interna (costruita con indirizzi RFC 1918) dalla rete Internet pubblica.

**Principio / Fondamento: Contrasto Architetturale: Router Stateless vs NAT Stateful**

- Un **router IP convenzionale** opera in modo puramente **stateless**: elabora ciascun pacchetto in modo atomico, senza conservare alcuna memoria dei pacchetti transitati in precedenza.
- Al contrario, un dispositivo **NAT** è un elemento fortemente **stateful**: per consentire ai pacchetti di risposta provenienti dall'esterno di raggiungere l'host privato sorgente, il NAT deve memorizzare e aggiornare continuamente in memoria RAM una **tabella di traslazione di stato**.

### 2.10.2 Basic NAT vs NAPT (Network Address and Port Translation)

Si distinguono due metodologie operative:

1. **Basic NAT**: associa un indirizzo IP privato a un indirizzo IP pubblico estratto da un pool condiviso, operando una traduzione uno-a-uno pura a Livello 3. Tale approccio non risolve la carenza di indirizzi, in quanto se $K$ nodi interni sono simultaneamente attivi, sono richiesti $K$ indirizzi IP pubblici registrati.
2. **NAPT (*Network Address and Port Translation* o IP Masquerading)**: costituisce la tecnologia adottata universalmente nei router domestici, aziendali e nelle reti mobili. Permette a **centinaia o migliaia di host privati di condividere un unico indirizzo IP pubblico** (o un ristretto insieme di IP pubblici), sfruttando le porte di trasporto per discriminare i singoli flussi.

### 2.10.3 Il Meccanismo di Multiplexing basato sulla Quintupla

Nel modello Internet, una sessione di comunicazione bidirezionale end-to-end è identificata in modo univoco da una **quintupla**:

$$\text{Flusso} = \left\{ \text{IP}_{\text{src}},\ \text{IP}_{\text{dst}},\ \text{Protocollo (TCP/UDP)},\ \text{Port}_{\text{src}},\ \text{Port}_{\text{dst}} \right\} \tag{2.6}$$

Quando un host privato genera una connessione uscente (*outbound*):

1. Il client seleziona una **porta sorgente effimera** (*ephemeral port*, numero a 16 bit compreso solitamente tra 49152 e 65535, generato casualmente o sequenzialmente dallo stack operativo) e si indirizza verso la porta fissa del servizio remoto (es. porta 80 per HTTP o 443 per HTTPS).
2. Quando il datagramma attraversa il NAPT, il dispositivo:
   - Sostituisce l'indirizzo sorgente privato con il proprio **indirizzo IP pubblico di uscita**;
   - Sostituisce la porta sorgente originaria con una **nuova porta effimera pubblica** libera allocata al momento;
   - **Non modifica mai** l'indirizzo di destinazione né la porta di destinazione (altrimenti il server remoto non riconoscerebbe il servizio richiesto);
   - Inserisce una nuova riga nella tabella di stato del NAT, registrando l'associazione bidirezionale;
   - **Ricalcola sia l'Header Checksum IP sia il Checksum di Livello Trasporto (TCP/UDP)**.
3. Quando la risposta perviene da Internet, il NAPT legge la porta di destinazione (che coincide con la porta pubblica effimera assegnata prima), effettua il lookup nella tabella, ripristina l'IP e la porta privata originari e inoltra il pacchetto all'interno della LAN.

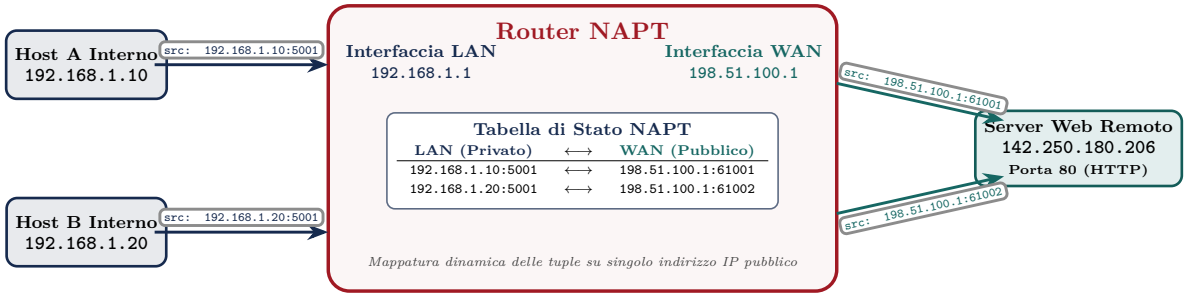



Figura 2.17: Architettura NAPT: due host con medesima porta sorgente interna (5001) vengono multiplexati su porte pubbliche distinte (61001 e 61002) verso il medesimo server HTTP.

### 2.10.4 Esercitazione d'Aula: Tabella NAT a Quintupla, Flussi Uscenti e Filtraggio Inbound

**Principi Fondamentali del NAT negli Esercizi d'Esame**

1. **Invarianza della Destinazione**: D-IP e DP rimangono rigorosamente inalterati per non dirottare il traffico su altri host o servizi.
2. **Invarianza del Protocollo ($P$)**: il tipo di trasporto ($P = 6$ per TCP, $P = 17$ per UDP) non muta mai durante la traslazione.
3. **Porta Sorgente (SP)**: il NAT può riallocare SP per evitare collisioni; se non vi sono conflitti, negli esercizi è preferibile conservare la porta originaria del client.
4. **Filtraggio Stateful Inbound**: pacchetti provenienti dall'esterno sono inoltrati solo se corrispondono a una sessione già attiva in tabella (quintupla invertita); il traffico non sollecitato viene scartato (*drop*).

**Esercizio d'Esame: Tabella di NAT a Quintupla e Ispezione dei Pacchetti (Esercizio 3)**

**Testo del Problema:** Si consideri la sottorete privata illustrata in Figura 2.18, configurata con lo spazio di indirizzamento privato `192.168.1.0/24`. L'interfaccia verso la rete Internet del **Router-NAT** possiede l'indirizzo pubblico assegnato:

IP Pubblico Router-NAT: `130.190.27.44/28`

Tale indirizzo viene impiegato anche come indirizzo IP pubblico globale per la funzione di traduzione NAPT. Definiamo **interno** l'ambito della sottorete privata ed **esterno** la rete pubblica Internet.

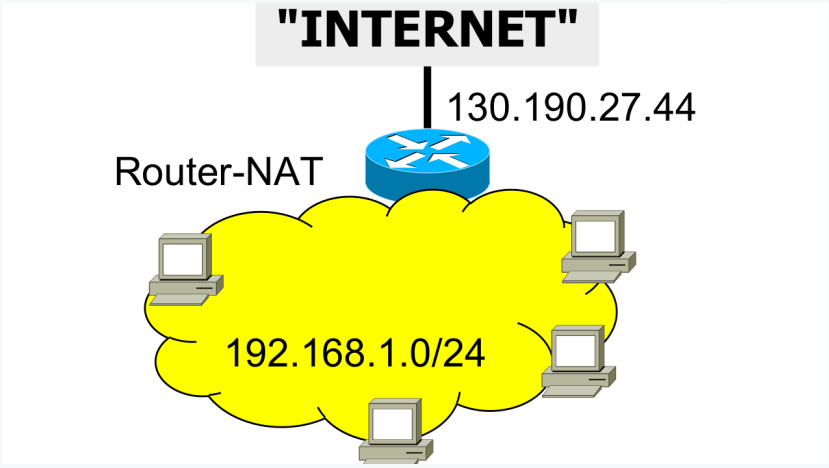



Figura 2.18: Topologia ufficiale della subnet privata attestata a Internet tramite il Router-NAT con interfaccia esterna `130.190.27.44/28`.

In un dato istante temporale, la tabella di traslazione di stato del NAT memorizza le seguenti associazioni bidirezionali basate sulla quintupla:

$$\text{interno} \longleftrightarrow \text{esterno}$$

$$\langle \text{S-IP},\ \text{D-IP},\ P,\ \text{SP},\ \text{DP} \rangle \longleftrightarrow \langle \text{S-IP},\ \text{D-IP},\ P,\ \text{SP},\ \text{DP} \rangle$$

| **Flusso Interno** | | **Flusso Esterno (Tradotto)** |
|---|---|---|
| ⟨192.168.1.17, 8.22.44.1, 6, 1456, 443⟩ | ⟷ | ⟨130.190.27.44, 8.22.44.1, 6, 6453, 443⟩ |
| ⟨192.168.1.17, 8.22.44.1, 6, 11223, 443⟩ | ⟷ | ⟨130.190.27.44, 8.22.44.1, 6, 44331, 443⟩ |
| ⟨192.168.1.131, 121.221.25.5, 17, 11223, 25⟩ | ⟷ | ⟨130.190.27.44, 121.221.25.5, 17, 3456, 25⟩ |

Tabella 2.7: Tabella di stato NAT iniziale a tre flussi (notazione standard con campi identificati da: S-IP = Source IP, D-IP = Destination IP, $P$ = Protocol (6=TCP, 17=UDP), SP = Source Port, DP = Destination Port).

Successivamente giungono al Router-NAT dall'interno della LAN tre nuovi pacchetti con i seguenti campi di intestazione:

- $P_{\text{in},1}$: ⟨192.168.1.28, 8.22.44.1, 6, 22334, 443⟩
- $P_{\text{in},2}$: ⟨192.168.1.66, 8.22.44.1, 6, 5134, 443⟩
- $P_{\text{in},3}$: ⟨192.168.1.131, 121.221.25.5, 17, 33113, 25⟩

Si richiede di:

a) Inserire nella tabella di stato del NAT le righe plausibili corrispondenti ai flussi originati dai tre nuovi pacchetti in arrivo.

b) Data la tabella completata al punto a), definire rigorosamente come si comporta il Router-NAT quando riceve dall'**esterno** (Internet) pacchetti con i seguenti campi, fornendo la motivazione analitica:

- $P_{\text{out},1}$: ⟨88.22.123.144, 130.190.27.44, 6, 443, 6453⟩
- $P_{\text{out},2}$: ⟨8.22.44.1, 130.190.27.44, 6, 443, 44331⟩

**Svolgimento e Risoluzione Analitica Passo-Passo:**

**Quesito a) Costruzione delle Nuove Righe di Traduzione nella Tabella NAT:** Applicando le regole fondamentali di traslazione NAPT:

1. L'indirizzo sorgente privato S-IP viene obbligatoriamente sostituito con l'indirizzo pubblico del NAT: 130.190.27.44.
2. L'indirizzo di destinazione D-IP e la porta di destinazione DP rimangono identici.
3. Il campo protocollo $P$ rimane invariato (6 per TCP, 17 per UDP).
4. La porta sorgente SP: se la porta originaria è libera sul lato pubblico per la coppia $(\text{D-IP}, \text{DP}, P)$, il NAT può mantenerla inalterata; in alternativa alloca una porta effimera libera. Seguendo la convenzione d'aula adottata dal docente per rapidità di calcolo, conserviamo la porta sorgente se non genera collisioni nella quintupla esterna.

Analisi dettagliata dei tre flussi:

- **Flusso 1 ($P_{\text{in},1}$)**: pacchetto TCP ($P = 6$) da 192.168.1.28 con porta effimera 22334 verso il server web 8.22.44.1:443. La quintupla esterna tradotta diviene:

$$\begin{aligned} &\text{Interno: } \langle \texttt{192.168.1.28}, \texttt{8.22.44.1}, 6, 22334, 443 \rangle \\ \longleftrightarrow\ &\text{Esterno: } \langle \textbf{130.190.27.44}, \texttt{8.22.44.1}, 6, \textbf{22334}, 443 \rangle \end{aligned}$$

  (La porta 22334 non è utilizzata da nessun'altra entry esterna, pertanto è perfettamente valida e univoca).

- **Flusso 2 ($P_{\text{in},2}$)**: pacchetto TCP ($P = 6$) da 192.168.1.66 con porta 5134 verso 8.22.44.1:443. La quintupla esterna tradotta risulta:

$$\begin{aligned} &\text{Interno: } \langle \texttt{192.168.1.66}, \texttt{8.22.44.1}, 6, 5134, 443 \rangle \\ \longleftrightarrow\ &\text{Esterno: } \langle \textbf{130.190.27.44}, \texttt{8.22.44.1}, 6, \textbf{5134}, 443 \rangle \end{aligned}$$

- **Flusso 3 ($P_{\text{in},3}$)**: pacchetto UDP ($P = 17$) originato da 192.168.1.131 con porta 33113 diretto

verso il server di posta `121.221.25.5:25`. La quintupla tradotta diviene:

$$\text{Interno: } \langle \texttt{192.168.1.131},\ \texttt{121.221.25.5},\ 17,\ 33113,\ 25\rangle$$
$$\longleftrightarrow \text{Esterno: } \langle \texttt{130.190.27.44},\ \texttt{121.221.25.5},\ 17,\ \mathbf{33113},\ 25\rangle$$

| **Flusso Interno** | | **Flusso Esterno (Tradotto)** |
|---|---|---|
| ***Sessioni Preesistenti:*** | | |
| ⟨192.168.1.17, 8.22.44.1, 6, 1456, 443⟩ | ⟷ | ⟨130.190.27.44, 8.22.44.1, 6, 6453, 443⟩ |
| ⟨192.168.1.17, 8.22.44.1, 6, 11223, 443⟩ | ⟷ | ⟨130.190.27.44, 8.22.44.1, 6, 44331, 443⟩ |
| ⟨192.168.1.131, 121.221.25.5, 17, 11223, 25⟩ | ⟷ | ⟨130.190.27.44, 121.221.25.5, 17, 3456, 25⟩ |
| ***Nuove Registrazioni (Punto a):*** | | |
| ⟨192.168.1.28, 8.22.44.1, 6, 22334, 443⟩ | ⟷ | ⟨**130.190.27.44**, 8.22.44.1, 6, **22334**, 443⟩ |
| ⟨192.168.1.66, 8.22.44.1, 6, 5134, 443⟩ | ⟷ | ⟨**130.190.27.44**, 8.22.44.1, 6, **5134**, 443⟩ |
| ⟨192.168.1.131, 121.221.25.5, 17, 33113, 25⟩ | ⟷ | ⟨**130.190.27.44**, 121.221.25.5, 17, **33113**, 25⟩ |

Tabella 2.8: Tabella di stato NAT completa a sei flussi attivi.

**Quesito b) Analisi del Comportamento del Router-NAT sui Pacchetti in Arrivo dall'Esterno:** Quando un pacchetto giunge dall'esterno verso l'interfaccia WAN del NAT, il router effettua un'operazione di *lookup* verificando se la tupla del pacchetto in ingresso coincide esattamente con la configurazione di ritorno di una riga attiva:

$$\text{Condizione di Risposta: } \begin{cases} \text{S-IP}_{\text{pkt}} = \text{D-IP}_{\text{entry}}, & \text{SP}_{\text{pkt}} = \text{DP}_{\text{entry}} \\ \text{D-IP}_{\text{pkt}} = \text{S-IP}_{\text{entry}}, & \text{DP}_{\text{pkt}} = \text{SP}_{\text{entry}} \\ P_{\text{pkt}} = P_{\text{entry}} \end{cases}$$

1. **Esame del Pacchetto $P_{\mathbf{out},1}$**: ⟨88.22.123.144, 130.190.27.44, 6, 443, 6453⟩
   - *Destinazione*: 130.190.27.44, porta 6453.
   - *Ispezione Tabella*: la porta WAN 6453 compare nella riga 1 della tabella.
   - *Controllo della Sorgente*: la riga 1 della tabella attende traffico esclusivamente dal server web 8.22.44.1, con cui l'host interno 192.168.1.17 ha instaurato la connessione. Il pacchetto in esame giunge invece da un indirizzo completamente estraneo: 88.22.123.144.
   - *Verdetto*: la quintupla non corrisponde. Si tratta di un pacchetto spurio, un tentativo di penetrazione casuale da parte di un hacker o una scansione non sollecitata.
   - **Azione del NAT**: **IL PACCHETTO VIENE SCARTATO (*PACKET DROPPED*)**. Il NAT non inoltra il pacchetto all'interno, preservando l'integrità della sottorete privata.
2. **Esame del Pacchetto $P_{\mathbf{out},2}$**: ⟨8.22.44.1, 130.190.27.44, 6, 443, 44331⟩
   - *Destinazione*: 130.190.27.44, porta 44331, protocollo TCP ($P = 6$).
   - *Sorgente*: 8.22.44.1, porta 443.
   - *Ispezione Tabella*: coincide in modo perfetto ed esatto con la riga 2 della tabella:

     $$\text{Riga 2 (Esterno): } \langle \texttt{130.190.27.44},\ \texttt{8.22.44.1},\ 6,\ 44331,\ 443\rangle$$

     Invertendo mittente e destinatario, il pacchetto in ingresso è esattamente la risposta attesa dal server 8.22.44.1.
   - **Azione del NAT**: **IL PACCHETTO VIENE TRADOTTO E INOLTRATO SULLA LAN PRIVATA**. Il Router-NAT compie la traduzione inversa:
     - Sostituisce D-IP ← 192.168.1.17;
     - Sostituisce DP ← **11223**;
     - Mantiene inalterati S-IP = 8.22.44.1, SP = 443 e $P = 6$;
     - Ricalcola i checksum IP e TCP;
     - Inoltra la trama sul link interno verso l'host 192.168.1.17.

### 2.10.5 Esercizio d'Esame: Flussi Paralleli, Blind Forwarding e Riuso di Porte nel NAT (Esercizio 4)

**Esercizio d'Esame: Flussi Multidirezionali, Blind Forwarding e Riuso Porta WAN (Esercizio 4)**

**Testo del Problema:** Si consideri l'infrastruttura di rete illustrata in Figura 2.19, in cui una sottorete privata `192.168.1.0/24` è interconnessa alla rete Internet tramite il router $R_A$, il quale svolge anche le funzioni di Network Address and Port Translation (NAPT). L'interfaccia esterna di $R_A$ verso Internet possiede l'indirizzo pubblico assegnato:

$$\text{IP Pubblico Esterno Router-NAT } R_A : \texttt{130.190.1.1}$$

mentre l'interfaccia interna attestata sulla LAN privata possiede l'indirizzo `130.190.1.2` (o convenzionalmente `192.168.1.1`). L'host interno Host A, con indirizzo `192.168.1.127`, avvia un flusso web verso il server remoto $S_1$ attestato su Internet all'indirizzo pubblico `150.1.1.3`, selezionando la porta sorgente effimera locale 5331. Lungo la dorsale Internet transitano i router intermedi $R_B$, $R_C$ e $R_D$, mentre su una rete pubblica remota è attestato anche un secondo server web $S_2$ con indirizzo pubblico `90.9.1.3`.

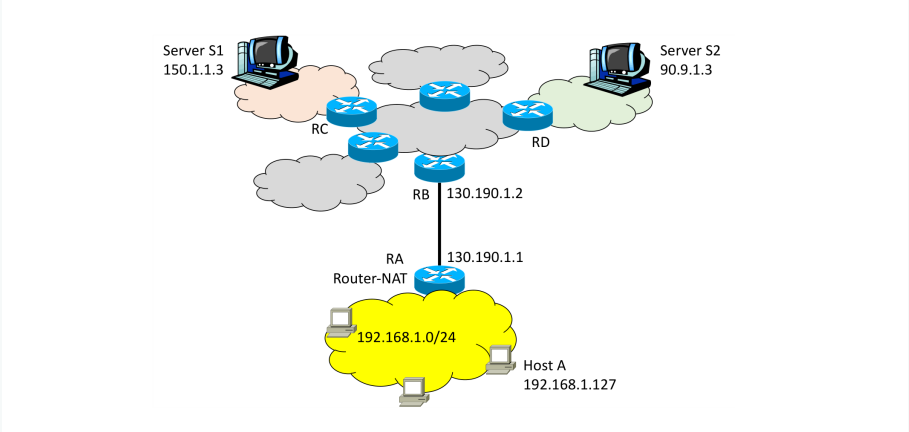



Figura 2.19: Topologia ufficiale della subnet privata attestata a Internet con Router-NAT $R_A$, router di transito $R_B, R_C, R_D$ e server remoti $S_1, S_2$ (Esercizio 4).

Si richiede di rispondere analiticamente ai seguenti quesiti:

a) Sapendo che il NAT impiega la porta pubblica 16001 per tradurre la quintupla della connessione verso l'esterno, indicare i valori esatti della 5-tupla $\langle \text{S-IP}, \text{D-IP}, P, \text{SP}, \text{DP} \rangle$ che identifica il flusso quando i pacchetti vengono:
   - generati da Host A;
   - trasmessi dal NAT ($R_A$, interfaccia verso l'esterno);
   - inoltrati dai router intermedi $R_B$ ed $R_C$.

b) Indicare i valori della quintupla che identifica il flusso sul **percorso inverso** (risposta applicativa), quando il pacchetto viene:
   - generato dal server $S_1$;
   - trasmesso attraverso i router intermedi $R_C$ ed $R_B$;
   - infine tradotto e consegnato dal NAT ($R_A$, interfaccia verso l'interno sulla LAN privata).

c) Si consideri ora il caso in cui Host A inizializzi contemporaneamente o successivamente un ulteriore flusso (sempre web) verso il server remoto $S_2$ (`90.9.1.3`). Il Router-NAT può impiegare la **stessa porta WAN** 16001 per tradurre la 5-tupla di questa nuova connessione? Motivare rigorosamente la risposta dal punto di vista protocollare.

d) Se entrambi i flussi verso $S_1$ ed $S_2$ risultano **attivi contemporaneamente**, la risposta al quesito precedente cambia? Valutare la fattibilità alla luce dell'architettura dei sistemi operativi e della gestione dei socket di rete.

**Svolgimento e Risoluzione Analitica Passo-Passo:**

**Quesito a) Flusso nel Percorso di Andata (LAN Privata $\longrightarrow$ Server $S_1$):**

1. **Pacchetto generato da Host A:**
   - S-IP = `192.168.1.127` (indirizzo IP privato del client mittente);
   - D-IP = `150.1.1.3` (indirizzo IP pubblico del server web $S_1$);
   - $P = 6$ (protocollo TCP): il testo specifica che il client contatta un server web. Il protocollo HTTP/HTTPS richiede un trasporto rigorosamente affidabile, privo di perdite e sequenziato, garantito unicamente da TCP ($P = 6$). Qualora il server erogasse servizi multimediali real-time a tolleranza di perdita (es. streaming RTP/UDP), si specificherebbe UDP ($P = 17$);
   - SP = 5331 (porta effimera locale scelta dal sistema operativo di Host A, compresa nell'intervallo standard [1024, 65535]);
   - DP = 80 (porta nota assegnata da IANA al servizio HTTP non cifrato; è altrettanto pienamente corretta la scelta di DP = 443 qualora si assuma una connessione cifrata HTTPS).

   Quintupla generata da Host A: $\langle$`192.168.1.127`, `150.1.1.3`, 6, 5331, 80$\rangle$
2. **Pacchetto trasmesso dal Router-NAT $R_A$ verso l'esterno:** Il dispositivo NAPT applica la traslazione mascherando l'indirizzo interno:
   - Sostituisce S-IP con il proprio IP pubblico WAN: `130.190.1.1`;
   - Sostituisce la porta sorgente SP con la porta pubblica assegnata dal testo: **16001**;
   - Mantiene inalterati D-IP = `150.1.1.3`, $P = 6$ e DP = 80 (il NAT è un intermediario fedele e non dirotta mai la destinazione).

   Quintupla trasmessa da $R_A$ verso l'esterno: $\langle$`130.190.1.1`, `150.1.1.3`, 6, **16001**, 80$\rangle$
3. **Pacchetto inoltrato dai router intermedi $R_B$ ed $R_C$:** I router di transito Internet operano secondo il principio cardinale del **Blind Forwarding**:
   - I router IP convenzionali operano al solo Livello 3 e non effettuano *Deep Packet Inspection* (DPI) sui normali flussi di transito;
   - Consultano unicamente il campo D-IP dell'header di rete per decidere l'interfaccia di uscita (lookup nella tabella di inoltro);
   - Non modificano mai gli indirizzi IP sorgente o destinazione, né alterano i campi di livello 4 (porte TCP/UDP) o il carico utile applicativo;
   - Si limitano a decrementare il campo TTL (Hop Limit) e a ricalcolare il checksum dell'header IP.

   Pertanto, la quintupla che transita attraverso $R_B$ ed $R_C$ è **perfettamente identica** a quella emessa da $R_A$:

   Quintupla inoltrata da $R_B$ ed $R_C$ : $\langle$`130.190.1.1`, `150.1.1.3`, 6, 16001, 80$\rangle$

**Quesito b) Flusso nel Percorso Inverso (Server $S_1 \longrightarrow$ LAN Privata):**

1. **Pacchetto di risposta generato dal Server $S_1$:** Essendo una risposta applicativa (*response*) a un pacchetto di richiesta (*request*), mittente e destinatario risultano specularmente invertiti:
   - S-IP = `150.1.1.3`, SP = 80 (il server risponde dal proprio processo web);
   - D-IP = `130.190.1.1`, DP = 16001 ($S_1$ indirizza la risposta unicamente all'IP pubblico del NAT $R_A$, ignorando totalmente l'esistenza e l'indirizzo privato di Host A);
   - $P = 6$ (invarianza del protocollo di trasporto TCP).

   Quintupla generata da $S_1$ : $\langle$`150.1.1.3`, `130.190.1.1`, 6, 80, 16001$\rangle$
2. **Pacchetto inoltrato dai router intermedi $R_C$ ed $R_B$:** Anche sul tragitto di ritorno, i router intermedi operano mediante blind forwarding, recapitano il pacchetto invariato all'interfaccia esterna di $R_A$:

   Quintupla inoltrata da $R_C$ ed $R_B$ : $\langle$`150.1.1.3`, `130.190.1.1`, 6, 80, 16001$\rangle$

3. **Pacchetto tradotto da $R_A$ e consegnato ad Host A:** Il Router-NAT ispeziona la tabella di stato interna. Rilevando che la porta WAN 16001 con protocollo TCP e sorgente remota `150.1.1.3:80` è legata alla sessione aperta da `192.168.1.127:5331`, compie la traduzione inversa:
   - Sostituisce D-IP ← **`192.168.1.127`**;
   - Sostituisce DP ← **5331**;
   - Mantiene inalterati S-IP = `150.1.1.3`, SP = 80 e $P = 6$.

     Quintupla consegnata da $R_A$ ad Host A: ⟨`150.1.1.3`, **`192.168.1.127`**, 6, 80, **5331**⟩

**Quesito c) Secondo Flusso verso il Server $S_2$ e Fattibilità del Riuso della Porta** 16001**: Risposta: SÌ, a livello di protocollo e di demultiplexing di rete, il NAT può impiegare la stessa porta WAN** 16001**.**
**Dimostrazione e Motivazione Formale:** Si esamini la configurazione della tabella di stato del NAT qualora venga riutilizzata la porta WAN 16001:

| Flusso Interno | | Flusso Esterno (Tradotto) |
|---|---|---|
| ⟨`192.168.1.127`, `150.1.1.3`, 6, 5331, 80⟩ | ⟷ | ⟨`130.190.1.1`, **`150.1.1.3`**, 6, **16001**, 80⟩ |
| ⟨`192.168.1.127`, `90.9.1.3`, 6, 5332, 80⟩ | ⟷ | ⟨`130.190.1.1`, **`90.9.1.3`**, 6, **16001**, 80⟩ |

Tabella 2.9: Tabella di stato NAT con riuso della medesima porta pubblica 16001 su destinazioni remote distinte.

Sul versante esterno, le due entry differiscono univocamente per l'indirizzo IP di destinazione (`150.1.1.3` contro `90.9.1.3`). Quando un pacchetto di risposta giunge dall'esterno sull'interfaccia WAN con porta di destinazione 16001:

- se proviene da S-IP = `150.1.1.3`, il lookup coincide univocamente con la prima riga e viene recapitato a `192.168.1.127:5331`;
- se proviene da S-IP = `90.9.1.3`, il lookup coincide univocamente con la seconda riga e viene recapitato a `192.168.1.127:5332`.

Non sussiste alcuna ambiguità logica nella 5-tupla, poiché la chiave di demultiplexing per il router NAPT è l'intera quintupla ⟨S-IP, D-IP, $P$, SP, DP⟩ e non la sola porta sorgente esterna.

**Quesito d) Flussi Attivi Contemporaneamente e Vincoli dei Sistemi Operativi Host: Risposta d'Esame e Analisi Critica:**

- **A Livello Concettuale di Rete / Dispositivi di Rete Dedicati (ASIC/Netfilter): NO, la risposta non cambia**. I due flussi rimangono matematicamente disgiunti e perfettamente distinguibili anche se attivi nello stesso istante. Un apparato router dedicato gestisce tabelle di hashing basate sulla quintupla completa senza alcun degrado o interferenza.
- **A Livello di Sistema Operativo Host Standard (Stack Socket POSIX): SÌ, la risposta cambia per vincoli software dello stack**. In un sistema operativo general-purpose convenzionale, l'apertura di un socket TCP e l'assegnazione locale di una porta effimera tramite la chiamata di sistema `bind()` impegna la porta locale in modo esclusivo. Se due processi distinti o due thread tentano di agganciarsi simultaneamente alla medesima porta locale effimera **16001** per aprire due connessioni in uscita concorrenti, il kernel solleva un'eccezione di collisione di risorsa (`EADDRINUSE`), a meno che:
  1. i due flussi siano separati nel tempo (il primo flusso ha completato il teardown TCP ed è scaduto il relativo timeout di stato);
  2. il sistema operativo supporti flag di riuso avanzato dei socket (`SO_REUSEADDR` / `SO_REUSEPORT`) configurati a livello applicativo;
  3. il NAT sia implementato mediante un modulo kernel dedicato che agisce a livello di packet filtering (come `iptables/conntrack` in Linux) prima che i pacchetti raggiungano il livello socket dell'applicazione.

### 2.10.6 Gestione dei Timeout e Rilascio delle Mappature

Poiché lo spazio delle porte a 16 bit ammette fino a 65536 valori distinti e alla disambiguazione concorrono anche l'IP e la porta di destinazione, un singolo IP pubblico consente teoricamente di gestire contemporaneamente decine di migliaia di flussi attivi. Tuttavia, le entry della tabella non possono rimanere memorizzate all'infinito:

- **Flussi TCP**: il protocollo implementa una terminazione esplicita della sessione tramite i pacchetti `FIN` ed `RST`. Il NAPT intercetta tale segnalazione e, dopo un breve tempo di sicurezza (*TIME_WAIT*), elimina la riga dalla tabella rilasciando la porta.
- **Flussi UDP**: essendo privo di connessione e di chiusura formalizzata, il NAPT non può sapere quando l'applicazione cessa la trasmissione. Si affida obbligatoriamente a **timer di inattività euristici** (tipicamente compresi tra 30 e 120 secondi). Se nessun pacchetto transita prima dello scadere del timer, l'associazione viene cancellata.

### 2.10.7 Costi Computazionali, Criticità Architetturali e Tecniche di Traversal

Nonostante l'enorme successo commerciale, il NAT introduce profonde alterazioni nell'architettura originale di Internet:

1. **Violazione dell'Invarianza End-to-End e del Layering**: un dispositivo di rete intermedio ispeziona e manipola campi di livello 4 (porte di trasporto), violando il principio di trasparenza del canale logico.
2. **Onere Computazionale del Ricalcolo dei Checksum**: poiché i campi IP e porta vengono modificati, il processore del NAT deve ricalcolare l'Header Checksum IP e, cosa ancora più onerosa, il Checksum del livello trasporto TCP/UDP (che copre l'intero segmento, payload incluso).
3. **Asimmetria delle Connessioni (Inbound Blocked)**: un client esterno su Internet non può aprire alcuna connessione verso un host privato interno, poiché la tabella del NAT non contiene alcuna mappatura associata e scarta immediatamente il pacchetto in ingresso.

Tale asimmetria penalizza fortemente le architetture **Peer-to-Peer** (P2P), la telefonia **VoIP** (protocollo SIP) e il gaming cooperativo, in cui ciascun nodo funge simultaneamente da client e server. Per scavalcare il NAT (**NAT Traversal**) si ricorre a:

- **Port Forwarding Statico**: configurazione manuale sul router che mappa permanentemente una porta pubblica verso un IP/porta privato;
- **Protocolli STUN / TURN**: `STUN` (*Session Traversal Utilities for NAT*) permette a un host interno di contattare un server pubblico per scoprire quale IP pubblico e quale porta il NAT gli abbia assegnato; se il NAT è simmetrico e restittivo, si ripiega su `TURN` (*Traversal Using Relays around NAT*), instradando l'intero flusso dati attraverso un server di inoltro terzo.

### 2.10.8 Confronto Architetturale: NAT vs Proxy Applicativi

Prima della diffusione capillare del NAT, la condivisione dell'accesso e il filtraggio del traffico erano frequentemente affidati ai **Proxy** di Livello 7 (applicativo). A differenza del NAT, che opera a Livello 3/4 manipolando unicamente gli header esterni (IP e porte TCP/UDP) in modo del tutto trasparente alle applicazioni, un Proxy richiede supporto esplicito e termina la connessione a livello utente.

Il declino dei proxy applicativi tradizionali è stato decretato dalla transizione universale a `HTTPS / TLS` e `QUIC`: non potendo ispezionare il payload cifrato end-to-end senza attacchi man-in-the-middle invasivi, i proxy applicativi diventano inoperanti, mentre il NAT preserva la totale compatibilità con il traffico crittografato.

## 2.11 Architettura Interna del Router e Forwarding Hardware

La funzione di inoltro dei pacchetti (**forwarding**) deve avvenire nei tempi rigidamente dettati dalla frequenza di cifra dei link. Su una moderna interfaccia di dorsale a 10 Gbit/s, un pacchetto medio di 1000 byte (8000 bit) impegna il canale per:

$$d_{\text{trans}} = \frac{8 \cdot 10^3 \text{ bit}}{10 \cdot 10^9 \text{ bit/s}} = 0.8\,\mu\text{s} = 800\,\text{ns}$$

Il router dispone dunque di una frazione di microsecondo per accogliere il datagramma, ispezionare l'indirizzo IP di destinazione, consultare la tabella di inoltro, decrementare il TTL, aggiornare il checksum e instradare il pacchetto verso la porta corretta. La memoria RAM convenzionale gestita da CPU general-purpose è inadeguata: si adotta un'architettura hardware dedicata che separa nettamente il **piano di controllo** dal **piano dei dati**.

### 2.11.1 Disaccoppiamento dei Piani e i Quattro Blocchi Funzionali

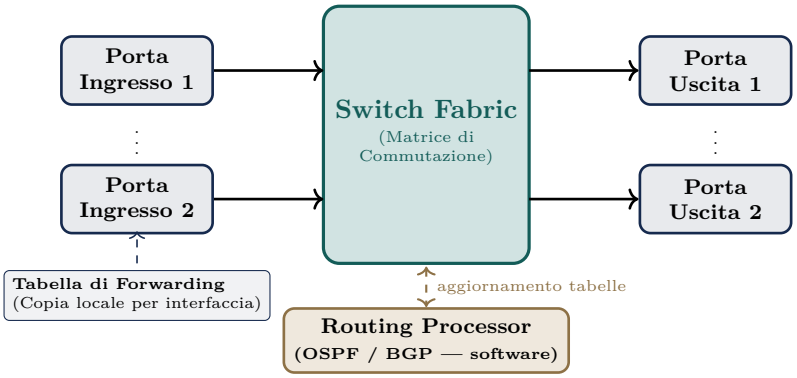



Figura 2.20: Architettura interna di un router: separazione tra il piano dati ad alta velocità (line card di ingresso/uscita e matrice di commutazione) e il piano di controllo software (Routing Processor).

L'architettura interna (Figura 2.20) è suddivisa in quattro blocchi:

1. **Porte di Ingresso (*Input Ports*)**: terminano la linea fisica, decapsulano la trama di Livello 2 ed eseguono il lookup della destinazione mediante copie locali della tabella di forwarding.
2. **Switch Fabric (Matrice di Commutazione)**: struttura hardware interna che connette fisicamente i buffer di ingresso con quelli di uscita.
3. **Porte di Uscita (*Output Ports*)**: accolgono i pacchetti commutati dalla fabric, li accodano nei rispettivi buffer di uscita e provvedono all'imbustamento nella nuova trama L2 e alla trasmissione sul link.
4. **Routing Processor (Piano di Controllo)**: microprocessore general-purpose che esegue in software gli algoritmi di routing (`OSPF`, `BGP`, `RIP`). Il suo compito consiste nel calcolare le rotte ottime e installare le tabelle di forwarding precompilate direttamente sulle line card di ingresso. Opera su tempi di secondi o minuti, lasciando il transito dei dati completamente svincolato dalla CPU.

## 2.12 Progettazione di Sottoreti ed Esercizio Pratico di Tabelle di Routing

### 2.12.1 Principi Metodologici: Relazione tra LAN, Subnet e Link Punto-Punto

Nella progettazione delle reti è fondamentale distinguere il costrutto fisico da quello logico:

- Una **LAN** (*Local Area Network*) è una porzione di infrastruttura fisica di Livello 2 (switch Ethernet, cavi, access point Wi-Fi) che condivide lo stesso dominio di broadcast L2.
- Una **Subnet** è un insieme logico di indirizzi IP (Livello 3) accomunati dal medesimo prefisso di rete (NetID).
- Sulla **stessa LAN fisica possono coesistere più subnet distinte** (sia pubbliche sia private): la convivenza è resa possibile dal fatto che un'interfaccia può ospitare più indirizzi IP logici. Host attestati su subnet logiche differenti che condividono il medesimo switch fisico non possono scambiarsi dati direttamente a Livello 2: la comunicazione deve necessariamente transitare da un router (Livello 3).
- Un **collegamento punto-punto tra due router** costituisce a tutti gli effetti una sottorete IP dedicata: si impiega tipicamente una subnet minima con prefisso `/30` (oppure indirizzi autoconfigurati link-local `169.254.0.0/16`).

### 2.12.2 Esercizio d'Esame: Tabelle di Inoltro (Forwarding), Subnetting /20 e /24 e Collegamento Punto-Punto /30 con ISP (Esercizio 5)

**Esercizio d'Esame: Tabelle di Inoltro (Forwarding), Rete di Campus e Tratta Punto-Punto con ISP (Esercizio 5)**

**Testo del Problema:** Una rete aziendale/universitaria di campus possiede la struttura rappresentata in Figura 2.21, composta da tre reti locali fisiche (LAN1, LAN2, LAN3) interconnesse da due router: $R_A$ ed $R_B$. Il router $R_A$ fornisce accesso alla rete globale Internet e possiede sull'interfaccia verso l'Internet Service Provider (ISP) l'indirizzo pubblico assegnato:

IP interfaccia ISP di $R_A$ : `88.121.11.133/30`

La sottorete SN1, attestata su LAN1, è una sottorete pubblica mediante la quale l'azienda eroga servizi pubblici accessibili da Internet (server web, posta, DNS), configurata con il blocco CIDR assegnato:

Network Address SN1: `131.127.144.32/27`

Le sottoreti SN2 (attestata su LAN2) ed SN3 (attestata su LAN3) sono reti private ricavate dallo spazio di indirizzamento privato standard `192.168.0.0/16`, con vincoli di dimensione prefissati rispettivamente pari a una subnet di tipo /**24** per SN2 e di tipo /**20** per SN3.

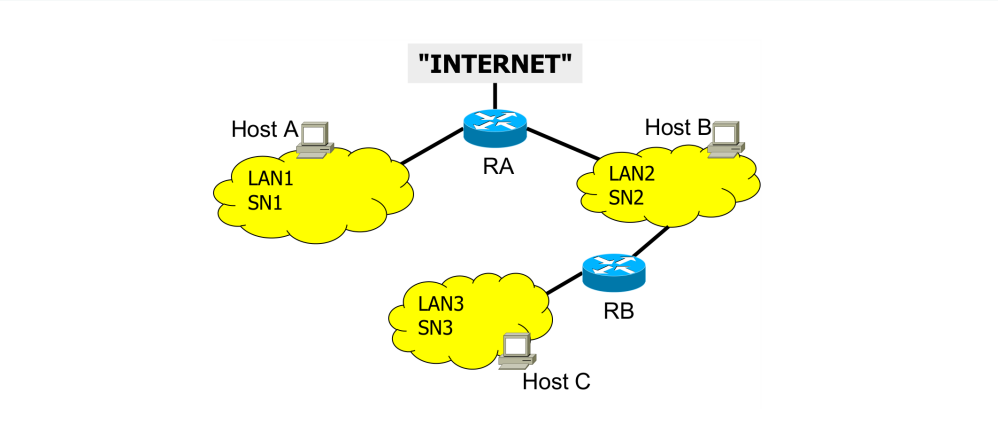



Figura 2.21: Topologia ufficiale della rete di campus con tre zone (LAN1, LAN2, LAN3), router $R_A$ ed $R_B$, collegamento ISP /30 e host $A, B, C$ (Esercizio 5).

Si richiede di svolgere in dettaglio i seguenti punti:

a) **Assegnazione Subnet Private:** Assegnare gli indirizzi di rete (Network Address) alle sottoreti SN2 e SN3 attingendo dallo spazio privato `192.168.0.0/16`, garantendo che i due blocchi siano perfettamente disgiunti e che la subnet /20 non inglobi la subnet /24. Specificare per ciascuna subnet l'intervallo completo di indirizzi, il Broadcast Address e il numero di host utili.

b) **Assegnazione Indirizzi IP alle Interfacce dei Router:** Assegnare gli indirizzi IP a tutte le interfacce fisiche di tutti i router ($R_A$ ed $R_B$) e ricavare i parametri della connessione punto-punto /30 verso l'ISP (Network Address della tratta, IP del gateway ISP e Broadcast Address). Rispettare rigorosamente la convenzione d'esame sulla nomenclatura delle interfacce.

c) **Compilazione Formale delle Tabelle di Inoltro (Forwarding Tables):** Compilare le tabelle di inoltro per:
   - gli host Host A (su LAN1), Host B (su LAN2) ed Host C (su LAN3);
   - il router $R_A$;
   - il router $R_B$.

   Tutte le tabelle devono riportare le colonne canoniche Prefix/Plen, Interface (IFI), Next-Hop (NH), ed essere tassativamente ordinate secondo il principio del **Longest Prefix Match** (LPM decrescente). Discutere infine la possibilità di aggregazione delle rotte (*Route Aggregation / Supernetting*) in $R_B$.

---

**Svolgimento e Risoluzione Analitica Passo-Passo:**

**Quesito a) Partizionamento e Assegnazione degli Indirizzi di Rete a SN2 ed SN3:** Dobbiamo ricavare una subnet /20 (SN3) e una subnet /24 (SN2) dallo spazio privato `192.168.0.0/16`.

> **Principio / Fondamento: Regola Aurea di Allocazione Decrescente (VLSM)**
>
> Quando si partiziona uno spazio di indirizzi per creare sottoreti di dimensioni eterogenee, **è tassativo allocare per prima la sottorete con prefisso più breve (cardinalità maggiore)**, e successivamente le reti più piccole a partire dallo spazio contiguo residuo. Allocando prima la rete piccola, si rischia di frammentare lo spazio o di far sì che la successiva subnet grande inglobi per errore quella già allocata.

1. **Allocazione di SN3 (Subnet /20):** Una maschera /20 riserva $32 - 20 = 12$ bit per l'HostID, consentendo:

$$N_{\text{ind, SN3}} = 2^{12} = 4096 \text{ indirizzi totali} \implies N_{\text{host, SN3}} = 2^{12} - 2 = 4094 \text{ host unicast utili}$$

I primi due byte (`192.168.`) occupano 16 bit. I successivi 4 bit del terzo byte appartengono al NetID, mentre gli ultimi 4 bit del terzo byte e l'intero quarto byte appartengono all'HostID:

$$\texttt{192.168.}\underbrace{0000}_{\text{NetID}}\ \underbrace{0000.00000000}_{\text{HostID}} \implies \text{Network Address: } \texttt{192.168.0.0/20}$$

Ponendo a 1 logico tutti i 12 bit di HostID, si ricava il Broadcast Address:

$$\texttt{192.168.}\underbrace{0000}_{\text{NetID}}\ \underbrace{1111.11111111}_{\text{HostID}} \implies \text{Broadcast Address: } \texttt{192.168.15.255}$$

Intervallo totale di SN3: da `192.168.0.0` a `192.168.15.255` (host utili: [`192.168.0.1`, `192.168.15.254`]).

2. **Allocazione di SN2 (Subnet /24):** Per garantire la totale disgiunzione, la subnet successiva deve iniziare dal primo indirizzo libero immediatamente consecutivo al Broadcast di SN3:

$$\text{Network Address SN2} = \text{BA}_{\text{SN3}} + 1 = \texttt{192.168.15.255} + 1 = \texttt{192.168.16.0/24}$$

Una maschera /24 riserva 8 bit per l'HostID ($2^8 - 2 = 254$ host utili):

$$\text{Broadcast Address SN2} = \texttt{192.168.16.255}$$

Intervallo totale di SN2: da `192.168.16.0` a `192.168.16.255` (host utili: [`192.168.16.1`, `192.168.16.254`]).

| Subnet | CIDR | Network IP | Primo Host | Ultimo Host | Broadcast |
|---|---|---|---|---|---|
| **SN1 (Pubblica)** | /27 | `131.127.144.32` | `131.127.144.33` | `131.127.144.62` | `131.127.144.63` |
| **SN2 (Privata)** | /24 | `192.168.16.0` | `192.168.16.1` | `192.168.16.254` | `192.168.16.255` |
| **SN3 (Privata)** | /20 | `192.168.0.0` | `192.168.0.1` | `192.168.15.254` | `192.168.15.255` |

Tabella 2.10: Riepilogo delle tre sottoreti della rete di campus dopo l'allocazione.

**Quesito b) Assegnazione Indirizzi alle Interfacce dei Router e Tratta ISP /30:**

1. **Analisi della Tratta Punto-Punto /30 verso l'ISP:** Il router $R_A$ possiede l'IP verso Internet `88.121.11.133/30`. Una subnet /30 dispone di $2^{32-30} = 2^2 = 4$ indirizzi totali (2 host utili), ideali per collegamenti seriali o punto-punto:
   - L'ultimo byte in binario vale $133_{10} = 10000101_2$. I 2 bit meno significativi costituiscono l'HostID ($01_2 = 1$).
   - Ponendo i bit di HostID a $00_2$: Network Address Tratta = `88.121.11.132/30`.
   - Indirizzo assegnato ad $R_A$: `88.121.11.133/30` (HostID $01_2$).
   - **Indirizzo Peer del Router ISP (Next-Hop per Default Route)**: `88.121.11.134/30` (HostID $10_2$).
   - Ponendo i bit di HostID a $11_2$: Broadcast Address Tratta = `88.121.11.135/30`.
2. **Nomenclatura Canonica delle Interfacce dei Router:** Seguendo la regola d'esame ribadita dal docente, le interfacce cablate si denominano `ethX`, dove $X$ corrisponde al numero della LAN a cui il router è fisicamente attestato. Non si devono mai inventare indici arbitrari:
   - **Interfacce Router $R_A$**:
     - $R_A$ `eth1`: interfaccia su LAN1 (SN1). Assegniamo il primo IP utile di SN1: `131.127.144.33/27`.
     - $R_A$ `eth2`: interfaccia su LAN2 (SN2). Assegniamo il primo IP utile di SN2: `192.168.16.1/24`.

  - $R_A$ `ethInternet`: interfaccia verso l'ISP: `88.121.11.133/30`.
- **Interfacce Router $R_B$**:
  - $R_B$ `eth2`: interfaccia su LAN2 (SN2). Assegniamo il secondo IP utile di SN2: `192.168.16.2/24`.
  - $R_B$ `eth3`: interfaccia su LAN3 (SN3). Assegniamo il primo IP utile di SN3: `192.168.0.1/20`.

**Quesito c) Compilazione Formale delle Tabelle di Inoltro (Forwarding Tables):**

**Principio / Fondamento: Struttura Canonica e Longest Prefix Match (LPM)**

Una tabella di inoltro (forwarding table) è composta da tre colonne formali:

1. **Prefix / Plen**: il prefisso di rete con la lunghezza della maschera;
2. **IFI (Interface Identifier)**: l'interfaccia locale di uscita (es. `eth0`, `eth1`);
3. **Next-Hop (NH)**: l'indirizzo IP del dispositivo a cui affidare il pacchetto, oppure l'indicazione esplicita di **Consegna Diretta (CD)** se la destinazione appartiene a una sottorete a cui il nodo è direttamente attestato.

**Regola d'Oro d'Esame:** Le righe della tabella devono essere tassativamente ordinate per **lunghezza di prefisso decrescente** (Plen maggiore in alto, default route `0.0.0.0/0` in fondo). Ciò riflette l'algoritmo hardware di Longest Prefix Match, garantendo che le regole più specifiche prevalgano su quelle più generiche.

**1. Tabelle di Inoltro degli Host (Host A, Host B, Host C):** Gli host sono sistemi terminali privi di capacità di calcolo di instradamento (*routing*). Dispongono di un'unica interfaccia di rete (`eth0`) e necessitano di sole **due regole di inoltro**:

1. La regola per la propria sottorete locale (consegna diretta tramite risoluzione ARP a livello 2);
2. La **Default Route** (`0.0.0.0/0`) verso il proprio Default Gateway per qualsiasi altra destinazione.

| Prefix / Plen | IFI | Next-Hop (NH) | Note Metodologiche |
|---|---|---|---|
| ***Host A (attestato su LAN1 / SN1 con IP 131.127.144.x/27):*** | | | |
| `131.127.144.32/27` | `eth0` | **Consegna Diretta** | Subnet locale SN1 (risoluzione ARP L2) |
| `0.0.0.0/0` | `eth0` | `131.127.144.33` | Default Route verso Gateway $R_A$ (`eth1`) |
| ***Host B (attestato su LAN2 / SN2 con IP 192.168.16.x/24):*** | | | |
| `192.168.16.0/24` | `eth0` | **Consegna Diretta** | Subnet locale SN2 |
| `0.0.0.0/0` | `eth0` | `192.168.16.1` | Default Route verso Gateway $R_A$ (`eth2`) |
| ***Host C (attestato su LAN3 / SN3 con IP 192.168.x.x/20):*** | | | |
| `192.168.0.0/20` | `eth0` | **Consegna Diretta** | Subnet locale SN3 |
| `0.0.0.0/0` | `eth0` | `192.168.0.1` | Default Route verso Gateway $R_B$ (`eth3`) |

Tabella 2.11: Tabelle di inoltro essenziali per i tre host della rete di campus.

**Nota d'Esame: Il Traffico tra Host B ed SN3**

In Host B non è presente alcuna rotta specifica verso SN3, sebbene il router $R_B$ sia direttamente collegato su LAN2 all'indirizzo `192.168.16.2`. Negli host, per semplicità di gestione, si delega tutto il traffico non locale al Default Gateway ($R_A$); sarà poi $R_A$ a re-inoltrare il pacchetto a $R_B$. Inserire in Host B una rotta statica esplicita (`192.168.0.0/20` → `eth0` → `192.168.16.2`) è didatticamente lecita e ottimizzerèbbe gli hop, ma non è richiesta nella prassi d'esame.

**2. Tabella di Inoltro del Router $R_A$:** Il router $R_A$ è collegato a Internet (ISP), a LAN1 e a LAN2, e deve inoltrare verso $R_B$ il traffico destinato ad SN3. Le 5 regole, ordinate per Plen decrescente ($30 \to 27 \to 24 \to 20 \to 0$), risultano:

| Prefix / Plen | IFI | Next-Hop (NH) | Descrizione Operativa |
|---|---|---|---|
| `88.121.11.132/30` | `ethInternet` | **Consegna Diretta** | Link punto-punto ISP a cui $R_A$ è direttamente connesso |
| `131.127.144.32/27` | `eth1` | **Consegna Diretta** | Sottorete locale pubblica SN1 attestata su LAN1 |
| `192.168.16.0/24` | `eth2` | **Consegna Diretta** | Sottorete locale privata SN2 attestata su LAN2 |
| `192.168.0.0/20` | `eth2` | `192.168.16.2` | Inoltro verso $R_B$ (interfaccia su LAN2) per raggiungere SN3 |
| `0.0.0.0/0` | `ethInternet` | `88.121.11.134` | Default Route verso il gateway dell'ISP per tutto il traffico Internet |

Tabella 2.12: Tabella di inoltro canonica del Router $R_A$ (ordinamento decrescente LPM).

**3. Tabella di Inoltro del Router $R_B$ e Route Aggregation:** Il router $R_B$ è direttamente attestato su LAN2 (SN2) e su LAN3 (SN3). Per raggiungere SN1 e la rete Internet deve affidare il traffico a $R_A$ attraverso la sua interfaccia `eth2`:

| Prefix / Plen | IFI | Next-Hop (NH) | Descrizione Operativa |
|---|---|---|---|
| `192.168.16.0/24` | `eth2` | **Consegna Diretta** | Sottorete locale SN2 attestata su LAN2 |
| `192.168.0.0/20` | `eth3` | **Consegna Diretta** | Sottorete locale SN3 attestata su LAN3 |
| `131.127.144.32/27` | `eth2` | `192.168.16.1` | Inoltro a $R_A$ per raggiungere la sottorete pubblica SN1 |
| `0.0.0.0/0` | `eth2` | `192.168.16.1` | Default Route verso $R_A$ per tutto il resto del traffico |

Tabella 2.13: Tabella di inoltro estesa del Router $R_B$.

**Route Aggregation / Supernetting in $R_B$ (Focus d'Esame)**

Si osservi che in $R_B$ la riga per SN1 (`131.127.144.32/27`) e la Default Route (`0.0.0.0/0`) condividono **la medesima interfaccia di uscita** (`eth2`) e **il medesimo Next-Hop** (`192.168.16.1`). Pertanto, il router $R_B$ può operare una **aggregazione di rotta**: cancellando la voce specifica per SN1, qualsiasi pacchetto per SN1 verrà comunque catturato dalla Default Route e instradato correttamente a $R_A$. La tabella aggregata di $R_B$ si riduce a sole 3 righe:

| Prefix / Plen | IFI | Next-Hop (NH) |
|---|---|---|
| `192.168.16.0/24` | `eth2` | Consegna Diretta |
| `192.168.0.0/20` | `eth3` | Consegna Diretta |
| `0.0.0.0/0` | `eth2` | `192.168.16.1` (Gateway $R_A$) |

Nelle prove d'esame entrambe le soluzioni (estesa con 4 righe o aggregata con 3 righe) sono ritenute pienamente corrette, ma l'illustrazione della semplificazione per aggregazione denota una padronanza superiore dell'architettura di rete.

## 2.13 Il Protocollo IPv6: Architettura, Header, Indirizzamento ed Estensioni

### 2.13.1 Crisi di IPv4 e Genesi di IPv6: Motivazioni Architetturali

Il protocollo `IPv4`, formalizzato nel 1981 con la `RFC 791`, ha costituito l'infrastruttura fondamentale della rete Internet per oltre quattro decenni. Tuttavia, la sua architettura riflette i vincoli computazionali e le esigenze operative tipiche dell'era originaria di ARPANET, fondata su un numero ridotto di calcolatori stazionari e su router a commutazione prevalentemente software.

**Principio / Fondamento: I Limiti Strutturali dell'Header IPv4**

Oltre all'ormai conclamata saturazione del numero di indirizzi, la necessità di ridefinire il protocollo di rete (`IPv6`) è scaturita da inefficienze architetturali intrinseche nell'header `IPv4`:

1. **Lunghezza Variabile dell'Header e Decodifica Software**: la presenza del campo `IHL` (Internet Header Length) e delle opzioni facoltative (20–60 B) impedisce l'allineamento deterministico a 64 bit nei circuiti hardware, costringendo i router ad analizzare l'header sequenzialmente e rallentando la commutazione a livello 3.

2. **Campi a Basso Utilizzo nel Percorso Critico**: campi quali `Identification`, `Flags` e `Fragment Offset` occupano 4 B (il 20% dell'header base) in ogni singolo datagramma, benché la frammentazione interessi una quota del tutto marginale del traffico Internet moderno.
3. **Ricalcolo Obbligatorio del Checksum ad Ogni Hop**: poiché il campo `TTL` viene decrementato da ciascun router di transito, il checksum a 16 bit deve essere interamente ricalcolato ad ogni salto, introducendo un carico computazionale parassita nella pipeline di instradamento.

**L'Esaurimento degli Indirizzi IPv4 e il Ruolo del NAT:** Lo spazio di indirizzamento a 32 bit di IPv4 dispone di $2^{32} \approx 4{,}29 \times 10^9$ indirizzi teorici. Nei primi mesi del 2011, la IANA (Internet Assigned Numbers Authority) ha formalmente esaurito la disponibilità degli ultimi blocchi /8 centrali, trasferendoli ai RIR (Regional Internet Registries), i quali hanno progressivamente esaurito le riserve ordinarie.

### 2.13.2 L'Header Base a 40 Byte (RFC 8200)

In IPv6, l'header base ha una dimensione rigorosamente fissa pari a **40 byte** (320 bit), esattamente il doppio rispetto alla dimensione minima dell'header IPv4 (20 B). Tale incremento è interamente giustificato dalla presenza dei due indirizzi a 128 bit (16 B ciascuno). Tutti i campi sono allineati a 64 bit per massimizzare la velocità di elaborazione delle pipeline hardware ASIC dei router di core.

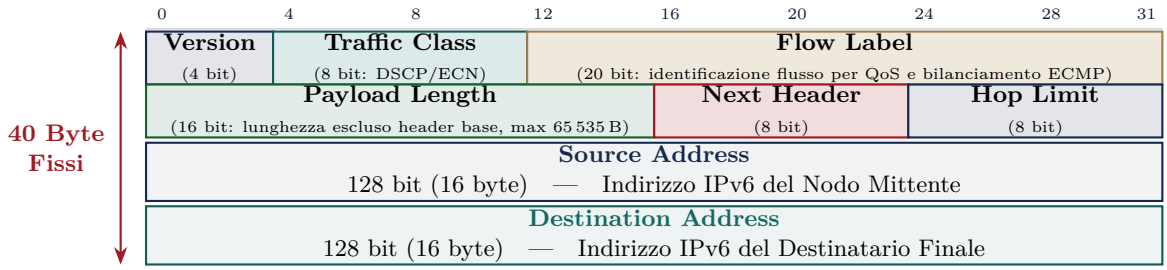



Figura 2.22: Struttura e campi dell'Header Base IPv6 a 40 byte fissi (RFC 8200).

**Definizione: Campi Chiave dell'Header Base IPv6**

A fronte della totale autoesplicatività dei campi **Version** (4 bit fissati a 6), **Source Address** e **Destination Address** (128 bit ciascuno, Figura 2.22), l'architettura introduce cinque campi funzionali chiave:

- **Traffic Class (8 bit)**: equivalente del campo ToS/DiffServ di IPv4. Include 6 bit per il codice DSCP e 2 bit per la notifica esplicita di congestione (ECN).
- **Flow Label (20 bit)**: identifica una sequenza coerente di pacchetti tra la medesima sorgente e destinazione per QoS e bilanciamento multi-path (ECMP). **Proprietà fondamentale**: consente ai router di discriminare i flussi applicativi **senza ispezionare le porte L4** (TCP/UDP), proprietà essenziale per flussi cifrati end-to-end.
- **Payload Length (16 bit)**: dimensione in byte di tutto ciò che segue l'header base di 40 B (inclusi eventuali Extension Headers e payload L4). Valore massimo standard 65 535 B; se nullo (0), segnala la presenza dell'opzione *Jumbo Payload*.
- **Next Header (8 bit)**: specifica il tipo del blocco immediatamente successivo all'header base: può puntare a un **Extension Header** o al protocollo di trasporto (TCP= 6, UDP= 17, ICMPv6= 58).
- **Hop Limit (8 bit)**: contatore decrementato di un'unità ad ogni salto dal router. A zero, il pacchetto viene scartato con `ICMPv6 Time Exceeded`. Rimpiazza il `TTL` di IPv4 rimuovendo l'ambiguità sui "secondi".

**Principio / Fondamento: Abolizione del Checksum a Livello 3 e Interdipendenza tra Livelli**

In IPv6 il checksum d'intestazione a livello 3 è stato **del tutto abolito**:

1. **Efficienza di Inoltro**: i router intermedi non devono più ricalcolare il checksum ad ogni salto quando decrementano l'`Hop Limit`, azzerando il ritardo di elaborazione per-hop.
2. **Affidabilità Delegata ai Livelli Adiacenti**: a livello 2, il controllo di ridondanza ciclica (CRC-32) dei frame fisici elimina i frame corrotti sul canale; a livello 4, il checksum di TCP e UDP diventa

**rigorosamente obbligatorio** in IPv6. Tale checksum include uno *pseudo-header* con gli indirizzi IPv6 sorgente e destinazione a 128 bit.

*Nota critica del Prof. Lo Cigno*: l'obbligatorietà del checksum L4 con pseudo-header rappresenta un'evidente violazione del principio di rigida separazione tra i livelli della pila ISO/OSI: il livello di trasporto è costretto ad accedere e manipolare campi appartenenti all'header di livello di rete.

### 2.13.3 Architettura dello Spazio di Indirizzamento a 128 Bit

Lo spazio di indirizzamento di IPv6 estende la lunghezza degli indirizzi da 32 a **128 bit**, mettendo a disposizione:

$$2^{128} \approx 3{,}4028 \times 10^{38} \text{ indirizzi} \tag{2.7}$$

Un numero astronomico che supera di gran lunga qualsiasi esigenza di allocazione planetaria futura.

**Notazione Esadecimale e Regole di Compressione (RFC 5952):** Un indirizzo IPv6 è rappresentato come una sequenza di 8 quartetti esadecimali da 16 bit, separati da due punti (`:`):

`2001:0db8:0000:0000:0008:0800:200c:417a`

Per agevolare la lettura e standardizzare la scrittura, si applicano due regole formali:

1. **Omissione degli Zeri Iniziali**: all'interno di ciascun gruppo da 16 bit, gli zeri non significativi a sinistra vengono omessi (es. `:0008:` diventa `:8:`).
2. **Compressione con Doppio Due Punti (`::`)**: la più lunga sequenza continua di gruppi composti unicamente da zeri può essere sostituita dal simbolo `::`. Tale contrazione può essere applicata **una sola volta** all'interno dello stesso indirizzo, per evitare ambiguità nell'espansione dei campi.

L'indirizzo di esempio assume quindi la forma canonica: `2001:db8::8:800:200c:417a`. L'indirizzo di loopback locale (`0:0:0:0:0:0:0:1`) viene contratto semplicemente in `::1`.

**Struttura Gerarchica dell'Indirizzo Unicast:** Gli indirizzi IPv6 Unicast sono convenzionalmente strutturati secondo uno schema fisso a $64 + 64$ bit:

$$\underbrace{\text{Routing Prefix (48 bit)} + \text{Subnet ID (16 bit)}}_{\textbf{Network Prefix } (64 \text{ bit})} \quad + \quad \underbrace{\text{Interface ID (64 bit)}}_{\textbf{Host Identifier } (64 \text{ bit})} \tag{2.8}$$

- **Global Routing Prefix (48 bit)**: prefisso pubblico assegnato dai registri Internet (RIR) e dagli ISP per l'instradamento globale.
- **Subnet ID (16 bit)**: identificatore che consente a una singola organizzazione o utente di creare fino a $2^{16} = 65\,536$ sottoreti indipendenti.
- **Interface ID / Host ID (64 bit)**: identificatore univoco dell'interfaccia all'interno della subnet, derivabile direttamente dall'indirizzo MAC fisico (48 bit) tramite conversione standard a 64 bit.

**Principio / Fondamento: SLAAC e il Superamento del Protocollo ARP**

La derivazione deterministica dell'Host ID dall'hardware dell'interfaccia comporta due conseguenze architetturali fondamentali:

1. **Autoconfigurazione Stateless (SLAAC)**: un host IPv6 appena connesso alla rete non necessita obbligatoriamente di un server DHCP per ottenere un indirizzo IP. È sufficiente che ascolti l'annuncio periodico del router locale (*Router Advertisement*) per apprendere il NetID a 64 bit della sottorete; l'host concatena quindi il proprio Host ID a 64 bit, auto-generando il proprio indirizzo IPv6 globale valido. Il DHCPv6 rimane utilizzato prevalentemente per distribuire parametri accessori (come gli indirizzi dei server DNS).
2. **Superamento di ARP**: conoscendo l'indirizzo IPv6 di un nodo sulla stessa LAN, il MAC address dell'interfaccia è ricavabile direttamente dal suffisso a 64 bit, o comunque risolto tramite il `Neighbor Discovery Protocol` (NDP) su ICMPv6, eliminando la necessità di trasmissioni ARP via broadcast.

### 2.13.4 Classificazione degli Indirizzi e Bando al Broadcast

L'architettura IPv6 definisce tre tipologie di indirizzi:

- **Unicast**: identificano una singola interfaccia di rete. Un pacchetto inviato a tale indirizzo viene recapitato esclusivamente a quell'interfaccia. Si distinguono in:
  - *Global Unicast*: indirizzi pubblici instradabili su Internet (prefisso `2000::/3`).
  - *Link-Local* (`fe80::/10`): indirizzi validi unicamente all'interno del segmento di rete locale fisico (non oltrepassano mai il primo router), impiegati per la configurazione iniziale e la scoperta dei vicini.
  - *Loopback* (`::1/128`): per la comunicazione interna all'host.
- **Multicast** (`ff00::/8`): identifica un gruppo di interfacce iscritte al servizio. In `IPv6` il concetto di broadcast è **formalmente abolito**: la diffusione locale sul link avviene tramite l'indirizzo speciale **Multicast All-Nodes** (`ff02::1`). Tale scelta elimina il consumo energetico parassita del broadcast: le schede di rete (NIC) filtrano il multicast direttamente nei circuiti hardware L2, scartando i frame irrilevanti senza sollevare interruzioni (IRQ) alla CPU e senza risvegliare il sistema operativo da stati di risparmio energetico (*sleep mode*).
- **Anycast**: indirizzo sintatticamente indistinguibile da un Unicast, ma assegnato a più interfacce appartenenti a nodi geograficamente distinti. I router instradano il pacchetto verso la prima interfaccia raggiungibile secondo le metriche di instradamento (il nodo topologicamente più vicino).

### 2.13.5 Regole di Frammentazione e Path MTU Discovery

Una delle innovazioni operative più rilevanti di IPv6 concerne la gestione della dimensione dei datagrammi e della frammentazione lungo il cammino di rete.

> **Principio / Fondamento: Divieto di Frammentazione nei Router di Transito**
>
> Nei router intermedi che instradano traffico IPv6, la frammentazione è **tassativamente vietata**:
>
> 1. Se un router riceve un pacchetto la cui dimensione supera la MTU del collegamento di uscita, il router **scarta immediatamente il datagramma**.
> 2. Contestualmente allo scarto, il router genera e invia alla sorgente un messaggio di controllo `ICMPv6 Packet Too Big` (PTB, Type 2, Code 0), specificando nel corpo del messaggio il valore esatto della MTU supportata sul proprio link di uscita.

**Path MTU Discovery (PMTUD, RFC 8201):** Grazie alla risposta deterministica dei router tramite messaggi PTB, il nodo sorgente è in grado di determinare la dimensione massima trasmissibile lungo l'intero cammino prima di avviare la trasmissione dei dati. L'host invia datagrammi con dimensione standard (ad esempio 1500 B); se riceve un messaggio PTB, riduce la dimensione dei pacchetti successivi al valore indicato, convergendo alla minima MTU presente lungo il percorso (*Path MTU*).

**Frammentazione alla Sorgente (Next Header = 44):** Qualora un pacchetto superi la Path MTU, la frammentazione può avvenire **esclusivamente alla sorgente** tramite l'inserimento del **Fragment Header** (Next Header = 44). Nella pratica ingegneristica, è largamente preferibile far intervenire il livello di trasporto (L4), dimensionando i segmenti TCP conformemente alla PMTU per evitare l'overhead della frammentazione a livello di rete.

La **MTU minima obbligatoria** per qualsiasi collegamento fisico abilitato a IPv6 è pari a **1280 byte** (rispetto ai soli 68 byte previsti da IPv4). Qualsiasi interfaccia di rete che non supporti una MTU di almeno 1280 B non può supportare nativamente IPv6.

### 2.13.6 Architettura degli Extension Headers (RFC 8200)

La modularità e l'estensibilità di IPv6 si basano sul principio del **Daisy-Chaining**: le funzionalità accessorie non risiedono nell'header base ma sono incapsulate in **Extension Headers** posizionati in sequenza concatenata tra l'header base di 40 byte e il payload di livello superiore.

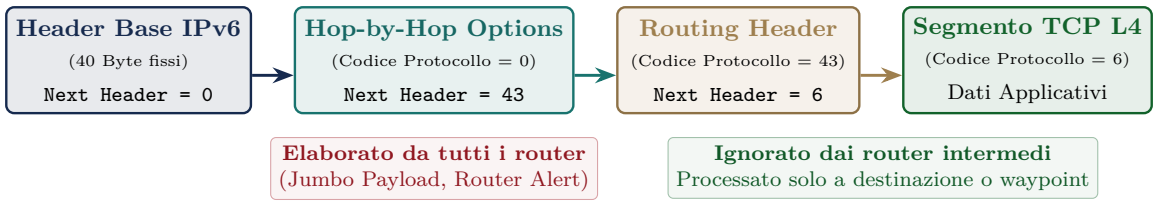



Figura 2.23: Principio del Daisy-Chaining degli Extension Headers con puntatori `Next Header`.

**Principio / Fondamento: Efficienza del Forwarding Hardware con gli Extension Headers**

I router intermedi lungo il percorso di rete esaminano ed elaborano **soltanto** l'header `Hop-by-Hop` (qualora presente immediatamente dopo l'header base con codice 0). Tutti gli altri extension header (Routing, Frammentazione, Sicurezza, Destination Options) vengono completamente **ignorati dai router di transito** e analizzati unicamente dal nodo destinatario finale. Ciò consente ai router di core di operare a velocità di linea (*wire speed*) senza subire degradazioni prestazionali.

**Panoramica dei Principali Extension Headers:**

1. **Hop-by-Hop Options (Next Header = 0)**: posizionato subito dopo l'header base ed esaminato da ogni router lungo il cammino. Abilita la **Jumbo Payload Option** per specificare lunghezze fino a 4 GB su canali ad altissima capacità.
2. **Fragment Header (Next Header = 44)**: inserito unicamente dalla sorgente per trasmettere frammenti generati localmente.
3. **Routing (Next Header = 43) e Sicurezza IPsec (AH = 51, ESP = 50)**: header per source routing o sicurezza crittografica end-to-end (autenticazione e cifratura), elaborati unicamente dal nodo destinatario finale.

### 2.13.7 Tecniche di Transizione e Coesistenza con IPv4

Data l'immensa dimensione della rete Internet e il gigantesco parco di dispositivi legacy basati su IPv4, il passaggio a IPv6 è stato impostato come un processo di transizione a lunghissimo termine. Le due strategie fondamentali introdotte per consentire la coesistenza sono:

1. **Dual Stack**: la soluzione principale, in cui host e router eseguono simultaneamente entrambi gli stack di protocollo (IPv4 e IPv6) sulle medesime interfacce di rete fisiche. Il sistema operativo sceglie quale protocollo utilizzare in base al tipo di indirizzo (record A per IPv4, record AAAA per IPv6) restituito dalla risoluzione DNS per il nome di dominio richiesto.
2. **Tunneling (IPv6 over IPv4)**: consente a due nodi o sottoreti IPv6 di comunicare attraverso una dorsale di transito che supporta esclusivamente IPv4. Il datagramma IPv6 completo viene incapsulato all'interno del campo payload di un normale pacchetto IPv4, instradato attraverso i router della rete IPv4 come un normale flusso di dati, e infine decapsulato dal router IPv6 ricevente all'uscita dal tunnel.

### 2.13.8 Tabella Sinottica Comparativa Sintetica: IPv4 vs IPv6

A completamento dell'analisi dell'architettura di rete, la Tabella 2.14 sintetizza organicamente i dieci parametri architetturali cardinali che differenziano i due protocolli, focalizzandosi sugli aspetti affrontati a lezione:

| Parametro Architetturale | Protocollo IPv4 (RFC 791) | Protocollo IPv6 (RFC 8200) |
|---|---|---|
| **Dimensione Indirizzo** | 32 bit (4 byte) | 128 bit (16 byte) |
| **Dimensione Header Base** | Variabile: 20–60 B (presenza campo `IHL`) | Fissa: **40 byte** (320 bit, allineamento 64 bit) |
| **Checksum a Livello 3** | Presente; ricalcolato ad ogni salto dal router | **Eliminato**; affidato a L2 (CRC) e L4 (obbligatorio) |
| **Frammentazione** | Eseguita sia dai router sia dagli host sorgente | **Vietata nei router**; gestita solo da sorgente via PMTUD |
| **MTU Minima di Link** | 68 B (raccomandata 576 B) | 1280 **byte** (obbligatoria su qualsiasi collegamento) |
| **Risoluzione Indirizzi L2** | Protocollo `ARP` su broadcast puro L2 | MAC integrato nell'Host ID o `NDP` su Multicast |
| **Supporto al Broadcast** | Nativo (indirizzi `255.255.255.255` o subnet) | **Totalmente abolito**; sostituito da Multicast All-Nodes |
| **Autoconfigurazione** | Tramite server centralizzato `DHCP` | `SLAAC` (stateless nativo) e/o `DHCPv6` |
| **Riconoscimento Flussi** | Non nativo (richiede ispezione porte L4) | Campo dedicato **Flow Label** (20 bit nativo) |
| **Gestione Opzioni** | Nel corpo dell'header (rallenta i router) | **Extension Headers** modulari in cascata (*daisy-chain*) |

Tabella 2.14: Confronto sinottico tra le architetture dei protocolli IPv4 e IPv6.

# Capitolo 3

# Livello di Collegamento Dati (*Data Link Layer*)

Il **livello di collegamento dati** (o *Data Link Layer*, Livello 2 nell'architettura ISO/OSI e nello stack TCP/IP) rappresenta il primo gradino dell'astrazione logica costruita al di sopra del canale fisico di trasmissione. Mentre il livello fisico si occupa esclusivamente della traslazione elettromagnetica, ottica o radiofonica di simboli grezzi, il livello di collegamento dati governa la comunicazione tra due o più nodi *direttamente adiacenti*, fisicamente attestati sul medesimo mezzo o appartenenti alla medesima sottorete locale (*broadcast domain*).

## 3.1 Inquadramento, Obiettivi e Modello Architetturale

**Definizione: Livello di Collegamento Dati (Data Link Layer)**

Il livello di collegamento dati è l'insieme di protocolli, algoritmi e circuiti hardware preposti a trasformare un canale fisico grezzo, rumoroso e soggetto a distorsioni, in un **collegamento logico ragionevolmente affidabile** a uso del livello di rete (`IP`), consentendo la consegna corretta, ordinata e priva di ambiguità delle trame (*frame*) tra nodi reciprocamente adiacenti.

### 3.1.1 L'Astrazione del "Tubo di Unità Dati"

A differenza dei livelli superiori dello stack (come il livello di trasporto con `TCP`), che devono gestire pacchetti fuori ordine causati da cammini divergenti tra router, il livello di collegamento opera lungo un singolo arco del grafo di rete. Di conseguenza, il canale visto dal livello 2 si comporta idealmente come un **tubo di unità dati**: le trame numerate $1, 2, 3, \ldots, n$ immesse dal trasmettitore vengono consegnate al ricevitore nello **stesso identico ordine di emissione**. Il livello data link non altera mai la sequenzialità temporale delle trame. Tuttavia, tale "tubo" è imperfetto: le unità dati possono essere corrotte dal rumore termico, disperse per attenuazione o temporaneamente accumulate in buffer di ricezione saturi.

### 3.1.2 Visibilità della Rete: Ambito Locale vs. Globale

Un principio cardine nella progettazione delle reti risiede nella **differente visibilità topologica**:

- **Visibilità del Livello 2 (Locale)**: limitata strettamente al *singolo link* o alla *sottorete locale* (*broadcast domain*). Il livello 2 ignora la topologia complessiva della rete mondiale, non sa quanti router separino la sorgente dalla destinazione e non conosce gli indirizzi IP remoti. Il suo compito è circoscritto alla **consegna diretta**: tra host e *default gateway*, tra due router adiacenti di dorsale (*point-to-point link*), o tra l'ultimo router e la destinazione finale.
- **Visibilità del Livello 3 (Globale)**: estesa all'intera rete mondiale (*end-to-end*). Il protocollo `IP` indirizza il datagramma dalla sorgente originaria alla destinazione finale attraverso molteplici link fisici eterogenei.

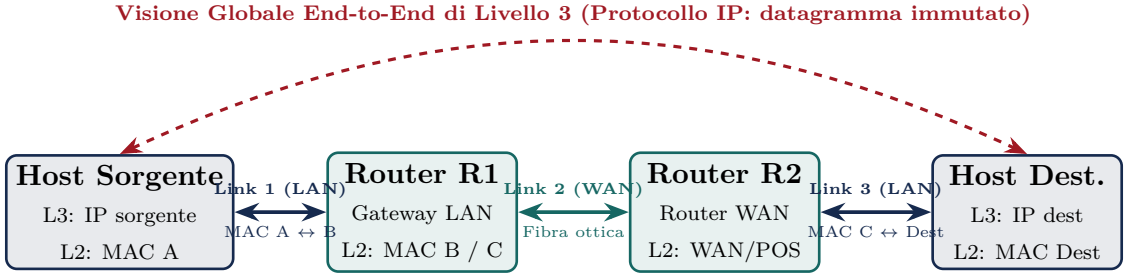



Figura 3.1: Confronto tra la visibilità locale dei protocolli di Livello 2 (ristretta al singolo link fisico adiacente) e la visione globale end-to-end del protocollo `IP` a Livello 3.

### 3.1.3 I Limiti Fisici del Canale da Compensare

Per garantire il corretto funzionamento del sovrastante livello di rete, il livello data link compensa tre limitazioni del mezzo fisico:

1. **Errori di trasmissione**: rumore termico gaussiano, interferenze radio, diafonia tra doppini in rame e dispersione su fibra ottica.
2. **Disparità di velocità trasmissiva ed elaborativa**: la velocità di trasmissione sul canale può eccedere la capacità di ricezione della scheda o della CPU destinataria.
3. **Ritardi di propagazione non nulli**: il segnale viaggia a velocità finita ($\approx 200.000\,\text{km/s}$ su rame/fibra, $300.000\,\text{km/s}$ nell'etere), introducendo latenze che condizionano la sincronizzazione e la contesa.

**Principio / Fondamento: Requisito di Affidabilità del Protocollo IP**

Il protocollo `IP` opera secondo il paradigma *connectionless* e *best-effort*: non richiede connessioni a L2, ma esige una consegna **ragionevolmente affidabile**. Indicativamente, la probabilità di perdita complessiva di trame lungo il cammino end-to-end deve risultare $P_{\text{loss}} < 10^{-2}$ ($< 1\%$). Se il tasso di perdita superasse tale soglia, il sovrastante protocollo di trasporto affidabile (`TCP`) collasserebbe a causa di continue ritrasmissioni e del drastico ridimensionamento della finestra di congestione.

## 3.2 Interfaccia con il Livello Fisico e Architettura Hardware (NIC)

### 3.2.1 Cosa Offre il Livello Fisico al Data Link

Il livello fisico (Livello 1) mette a disposizione del livello 2 un **flusso indifferenziato di bit** (*stream of binary information units*):

- **Assenza di semantica**: non esistono bit "colorati", bit di payload o bit di intestazione. Un bit vale puramente 0 o 1.
- **Assenza di metadati di correttezza**: se un bit viene corrotto dal rumore e il ricevitore sbaglia il campionamento, il bit errato viene consegnato senza alcuna marcatura.
- **Contiguità logica**: sul canale i bit scorrono ininterrottamente, senza barriere fisiche o cesure temporali a separare l'header dal payload o una trama dalla successiva.

**Codici di Correzione a Livello Fisico (FEC)**

I moderni livelli fisici (in particolare comunicazioni wireless 802.11, reti cellulari 4G/5G e fibre sottomarine) integrano algoritmi di **Forward Error Correction** (FEC), quali codici convoluzionali o LDPC (*Low-Density Parity-Check*). Tali circuiti correggono parte del rumore, ma la quota residua di errori non corretti viene immessa nel flusso di bit consegnato al livello 2, che ha il compito inderogabile di rilevarli.

### 3.2.2 Collocazione Architetturale: Kernel Space vs. Scheda di Rete (NIC)

Nell'architettura interna di qualsiasi elaboratore moderno si osserva una netta separazione tra software e hardware:

- **Spazio Utente (User Space)**: ospita i processi applicativi (`HTTP`, browser, client).
- **Spazio Kernel (Kernel Space)**: ospita lo stack di rete e trasporto (`IP`, `ICMP`, `TCP`, `UDP`), eseguiti dalla **CPU principale**.
- **Scheda di Rete (NIC – Network Interface Card)**: ospita **interamente il Livello Fisico e il Livello Data Link**. La NIC è un sottosistema a microprocessore autonomo dotato di memoria FIFO interna, circuiti logici specializzati (ASIC/FPGA) e ricetrasmettitore fisico (*PHY transceiver*), connesso alla scheda madre mediante bus ad alta velocità (`PCI Express`).

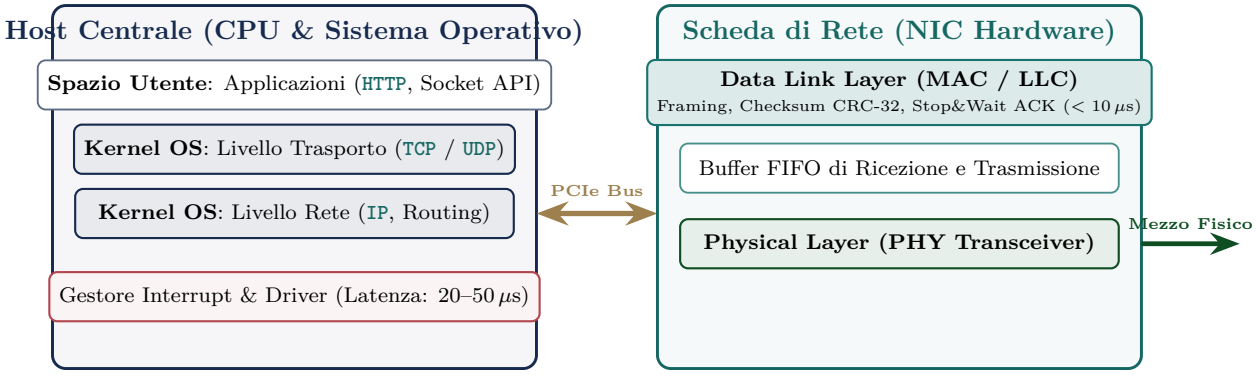



Figura 3.2: Ripartizione architetturale tra le funzioni software gestite dalla CPU nel kernel del sistema operativo (Livelli 3 e 4) e le funzioni in tempo reale eseguite in hardware a bordo della scheda di rete (NIC, Livelli 1 e 2).

### 3.2.3 Analisi Quantitativa: Vincoli di Latenza e Carico CPU a 10 Gbit/s

L'implementazione hardware a bordo della NIC risponde a due requisiti inderogabili:

**Principio / Fondamento: Latenza degli Interrupt vs. Tempi di Reazione Protocollo**

L'interazione tra periferiche e CPU è governata da **interrupt hardware**. La gestione di un interrupt impone un overhead non comprimibile: salvataggio dei registri sullo stack, cambio di contesto utente-kernel, esecuzione della *Interrupt Service Routine* (ISR) e ripristino dei registri, con una latenza di **10–50 microsecondi** (10–50 $\mu$s).
Nelle reti wireless (`IEEE 802.11`), la ricezione corretta di una trama impone la generazione dell'`ACK` entro l'intervallo **SIFS** (*Short Interframe Space*), pari a circa **10–16 microsecondi**:

$$T_{\text{SIFS}} \approx 10\,\mu\text{s} < T_{\text{interrupt}} \approx 20\text{–}50\,\mu\text{s} \tag{3.1}$$

Se il controllo del CRC e l'invio dell'ACK fossero gestiti dal software sulla CPU, la risposta arriverebbe in ritardo, facendo fallire il protocollo. Validazione CRC e ACK devono quindi avvenire istantaneamente in hardware sulla NIC.

**Stima del Carico della CPU a 10 Gbit/s**

Su un link in fibra a $R = 10\,\text{Gbit/s} = 10^{10}\,\text{bit/s}$, una trama Ethernet standard con MTU da 1500 byte ha lunghezza totale $L \approx 1518\,\text{byte} \approx 12.144\,\text{bit}$. Il tempo di trasmissione sul canale vale:

$$T_{\text{tx}} = \frac{12.144\,\text{bit}}{10 \times 10^9\,\text{bit/s}} \approx 1{,}214\,\mu\text{s} \implies \text{Frequenza di arrivo} \approx 823.000\,\text{trame/s} \tag{3.2}$$

Se ogni trama generasse due interrupt software (L2 $\rightarrow$ IP e IP $\rightarrow$ TCP), la CPU dovrebbe gestire oltre $1{,}65 \times 10^6$ interrupt/s, saturandosi al 100% unicamente per il cambio di contesto. Nelle NIC ad alte prestazioni si ricorre perciò a **Interrupt Moderation** (accumulo di trame in buffer DMA circolari ad anello con un solo interrupt periodico) e a circuiti **TCP Offload Engine** (TOE).

## 3.3 Le Tre Classi di Servizio Offerte al Livello di Rete

A seconda del mezzo fisico e dell'ambiente operativo, il livello data link offre tre classi di servizio:

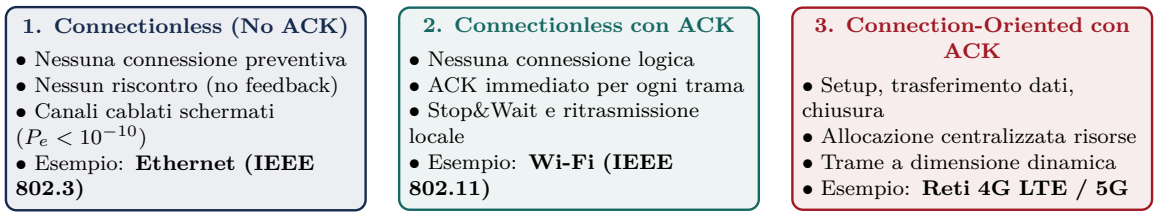



Figura 3.3: Tassonomia delle tre classi di servizio offerte dal Livello 2 al protocollo di rete `IP`.

- **Connectionless senza Riscontro (No ACK)**: non richiede connessione né feedback. Se una trama è corrotta, viene scartata; il recupero è demandato a `TCP`. È la soluzione ottimale per le LAN cablate

(Ethernet), dove il mezzo confinato garantisce tassi d'errore trascurabili ($P_e < 10^{-10}$).

- **Connectionless con Riscontro (ACK)**: non richiede connessione, ma ogni trama esige un ACK immediato. Se l'ACK non giunge entro un timeout, la trama viene ritrasmessa localmente (*Stop-and-Wait*). Tipico del Wi-Fi, dove l'etere rumoroso ($P_e \approx 10^{-3}$–$10^{-2}$) impone di assorbire le perdite localmente per evitare il degrado del throughput TCP.
- **Connection-Oriented con Riscontro**: prevede setup formale della connessione logica, riscontro e abbattimento finale. È impiegato nelle reti cellulari (GSM, UMTS, LTE, 5G) non perché serva a IP, ma per consentire l'allocazione centralizzata delle risorse radio (frequenze, time slot e codici) da parte della stazione radiobase.

**Principio / Fondamento: Perché il Riscontro Negativo (NACK Puro) Non Funziona nel Mondo Reale**

L'impiego del solo NACK per risparmiare banda è impraticabile per due ragioni strutturali:

1. *Perdita totale della trama*: se la trama viene distrutta o il preambolo non agganciato, il ricevitore non sa che una trama era attesa, quindi non potrà mai emettere un NACK.
2. *Corruzione dell'header*: se i bit dell'indirizzo sorgente sono alterati, il ricevitore invierebbe il NACK all'host sbagliato.

I protocolli reali adottano perciò esclusivamente il **riscontro positivo (ACK)** abbinato a un **timer di timeout** lato trasmettitore.

| Classe di Servizio | Gestione Connessione | Meccanismo Riscontro | Tecnologie di Riferimento |
|---|---|---|---|
| **Connectionless (No ACK)** | Nessun setup; invio in sequenza. | Nessun riscontro; recupero a L4 (TCP). | Ethernet IEEE 802.3, fibra ottica point-to-point. Ambienti guidati con $P_e < 10^{-10}$. |
| **Connectionless con ACK** | Nessuna connessione preliminare. | ACK per trama; Stop&Wait con timeout e ritrasmissione locale. | Wi-Fi IEEE 802.11, reti wireless industriali. Canali rumorosi ($P_e \approx 10^{-3}$–$10^{-2}$). |
| **Connection-Oriented con ACK** | Setup esplicito, trasferimento, chiusura. | ACK e tracciamento rigoroso con riordino trame. | Reti cellulari (GSM, LTE, 5G). Gestione e allocazione centralizzata delle risorse radio. |

Tabella 3.1: Quadro sinottico comparativo delle tre classi di servizio offerte dal Livello Data Link.

## 3.4 Tecniche di Delimitazione delle Trame (Framing)

### 3.4.1 Definizione e Dipendenza dal Livello Fisico

Il datagramma IP di dimensione variabile viene incapsulato dal livello 2 in una **trama** provvista di **header** e **trailer**. Il **framing** definisce le convenzioni codificate per individuare inizio e fine di ogni trama nel flusso continuo di bit di Livello 1.

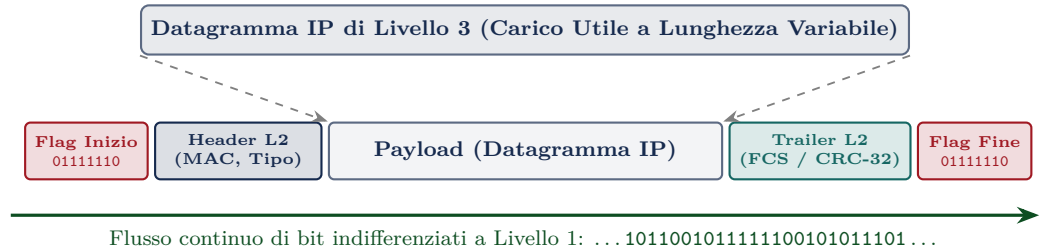



Figura 3.4: Incapsulamento del datagramma IP nella trama di Livello 2 e sua proiezione sul flusso continuo di bit di Livello 1.

Le strategie di framing dipendono strettamente dal canale fisico:

- **Canali in Fibra Ottica (Trasmissione Continua e Idle Pattern)**: i laser non vengono mai spenti per preservare la stabilità termica e l'aggancio del clock. In assenza di traffico, il trasmettitore invia

continuamente un **idle pattern** (`10101010...`). Il framing coincide con una deviazione nota da tale sequenza (es. due '1' consecutivi `...101011...`).

- **Canali a Diffusione (Broadcast Cablato e Radio)**: tra le trame vi è silenzio assoluto. All'inizio della trasmissione occorre un **preambolo** (56 bit alternati `10101010`) per consentire al ricevitore di sincronizzare il clock locale, seguito dallo **Start Frame Delimiter** (SFD, byte `10101011`) che delimita l'inizio della trama.

Esistono quattro tecniche fondamentali di framing:

### 3.4.2 1. Character Count (Conteggio dei Caratteri)

Il primo campo dell'header contiene un intero $N$ che indica la lunghezza complessiva della trama in byte. Sebbene concettualmente banale, è affetto da una **vulnerabilità catastrofica**: se un errore corrompe il campo conteggio, il ricevitore leggerà un numero errato di byte, interpretando un byte di dati come conteggio della trama successiva. L'allineamento collassa in modo permanente per tutte le trame a seguire. Per questo motivo, il conteggio caratteri **non viene mai usato da solo**.

### 3.4.3 2. Character Stuffing (Byte Stuffing)

Derivato dalle telescriventi a codice ASCII a 7 bit più parità, delimita la trama mediante caratteri di controllo speciali: `DLE STX` (*Data Link Escape + Start of TeXt*) all'inizio e `DLE ETX` alla fine. Per garantire la trasparenza di carichi utili binari arbitrari (che potrebbero contenere accidentalmente il byte `DLE` = `0x10`):

- **Al Trasmettitore**: ogni byte `DLE` presente nei dati viene raddoppiato (`DLE` → `DLE DLE`).
- **Al Ricevitore**: ogni coppia `DLE DLE` viene ridotta a un singolo `DLE`; la sequenza `DLE ETX` marca invece la fine legittima della trama.

### 3.4.4 3. Bit Stuffing (Inserimento di Bit)

Utilizzato nei protocolli a orientamento di bit (`HDLC`, `PPP`), svincola il framing dalla dimensione fissa del byte. La trama inizia e termina con lo speciale **flag pattern** `01111110` (`0x7E`, uno zero, sei uni, uno zero).

**Definizione: Algoritmo di Bit Stuffing e De-Stuffing**

- **Trasmettitore**: monitora i dati emessi; ogni volta che incontra **cinque bit '1' consecutivi**, inserisce forzatamente un bit **'0'** (*stuffed bit*), a prescindere dal valore del bit successivo.
- **Ricevitore**: quando individua cinque bit '1' consecutivi, ispeziona il 6° bit:
  1. Se è uno **'0'**, lo scarta (era uno zero di stuffing) ripristinando la sequenza corretta;
  2. Se è un **'1'** e il 7° bit è **'0'** (`01111110`), ha rilevato il **flag di fine trama**;
  3. Se sia il 6° sia il 7° bit sono ad **'1'** (`1111111`), segnala una condizione di **frame abort**.

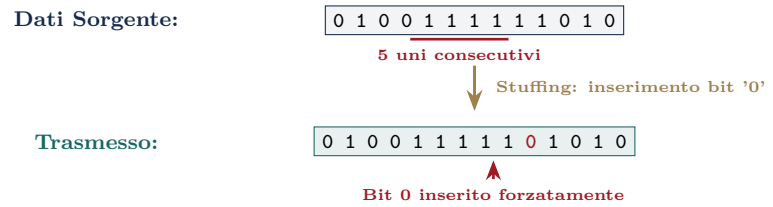



Figura 3.5: Meccanismo del Bit Stuffing: inserimento forzato di uno '0' dopo cinque '1' consecutivi per preservare l'univocità del flag.

### 3.4.5 4. Violazioni di Codifica Fisica (Physical Layer Violations)

Questa tecnica opera a livello intermedio "1.5", sfruttando la ridondanza intrinseca delle codifiche di linea del livello fisico. Nelle trasmissioni digitali su cavo non si usa una codifica banale con potenziale costante alto per '1' e basso per '0': una lunga sequenza ininterrotta di bit identici porterebbe alla saturazione dell'elettronica dei circuiti di ricezione (amplificatori operazionali e stadi d'ingresso). Per funzionare correttamente, l'elettronica richiede che il segnale mantenga un **valore medio di tensione nullo** nel tempo.

Per garantire questo bilanciamento e consentire al ricevitore di mantenere il sincronismo di clock, si adottano codifiche che impongono continue transizioni di livello, come la celebre **codifica Manchester** (usata storicamente in `Ethernet 10BASE-T` e `Token Ring`):

- **Bit '1' logico**: codificato come transizione **Alto** → **Basso** (simbolo composito $AB$).
- **Bit '0' logico**: codificato come transizione **Basso** → **Alto** (simbolo composito $BA$).

In entrambi i casi leciti, il segnale presenta **obbligatoriamente una transizione a centro bit**.

Di conseguenza, i simboli fisici privi di transizione:

- **Simbolo** $AA$: potenziale costantemente **Alto** per l'intero tempo di bit;
- **Simbolo** $BB$: potenziale costantemente **Basso** per l'intero tempo di bit;

sono **simboli invalidi** per i dati ordinari. I protocolli sfruttano questa violazione intenzionale delle regole fisiche utilizzandola per **delimitare l'inizio o la fine della trama** (come in Token Ring e FDDI). Il vantaggio fondamentale è l'assenza totale di overhead di byte stuffing: nessun dato utente potrà mai generare un simbolo $AA$ o $BB$, eliminando il rischio di false delimitazioni.

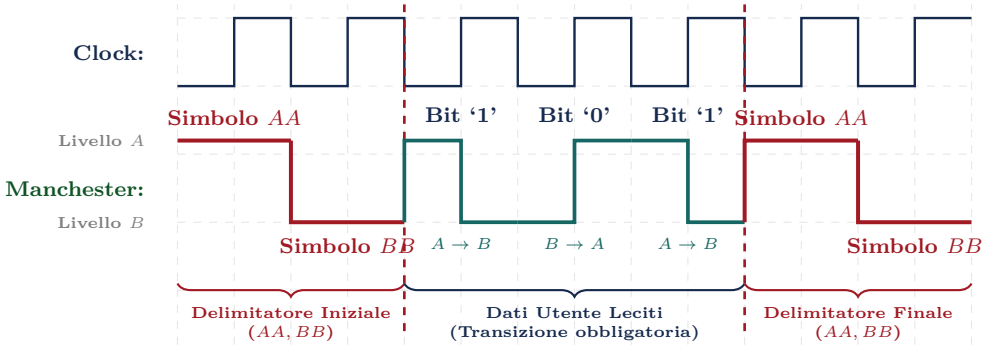



Figura 3.6: Cronogramma delle violazioni di codifica fisica su codifica Manchester: i bit leciti ('0' e '1') presentano sempre una transizione di livello a centro periodo, mentre i simboli costanti senza transizione ($AA$ e $BB$) sono violazioni intenzionali impiegate per delimitare la trama.

### 3.4.6 5. Intervalli Temporali e Preambolo (Canali Broadcast)

Nelle reti locali con canale condiviso o broadcast (come `Ethernet` e `Wi-Fi`), le trame non fluiscono in modo continuo, ma sono separate da periodi di **silenzio** (assenza di trasmissione sul mezzo). Ogni volta che una stazione comincia a trasmettere dopo un silenzio, il ricevitore deve necessariamente ri-sincronizzarsi: deve determinare la frequenza esatta di trasmissione, la fase del segnale e individuare dove iniziano e finiscono i singoli bit.

Questa procedura richiede l'invio preventivo di una sequenza nota a priori:

1. **Preambolo di Sincronizzazione**: il trasmettitore invia una sequenza alternata nota (tipicamente il pattern `101010...10`) ripetuta per decine o centinaia di cicli. Il ricevitore rileva la presenza di energia sul canale e aggancia i propri oscillatori interni.
2. **Violazione della Sequenza Nota (Start Frame Delimiter)**: la sincronizzazione termina quando la sequenza alternata si interrompe con un pattern differente (ad esempio due bit '1' consecutivi, `...10101011`, noto come *Start Frame Delimiter* o SFD). Quella rottura indica inequivocabilmente che il bit successivo è l'inizio della trama vera e propria.
3. **Delimitazione della Fine tramite Conteggio**: per sapere quando la trama termina, l'header include un campo indicante la **lunghezza della trama** (*Length/Character Count*). Il ricevitore conta i byte ricevuti e chiude la trama, eliminando la necessità di cercare un flag di chiusura e prevenendo del tutto il problema del byte stuffing.

## 3.5 Rilevazione e Gestione degli Errori

### 3.5.1 Tipologie di Guasto e Filosofia di Rilevazione a Livello 2

I disturbi fisici si traducono in:

1. **Errore di singolo bit** (*single-bit error*): un bit isolato viene invertito ($0 \leftrightarrow 1$).
2. **Burst di errore** (*burst error*): un gruppo contiguo di $k$ bit subisce alterazioni da disturbi impulsivi.
3. **Perdita o inserimento di bit**: scivolamento del clock (*cycle slip*) nei circuiti di campionamento.

> **Principio / Fondamento: Filosofia a Livello 2: Scarto senza Correzione**
>
> Mentre a Livello 1 i codici FEC tentano attivamente di ricostruire i bit corrotti, a Livello 2 vige la regola dello **scarto immediato senza correzione**: se il checksum ricalcolato al ricevitore differisce da quello trasmesso, la trama viene scartata. Correggere un bit in una posizione arbitraria non identificata con

certezza assoluta produrrebbe danni devastanti ai protocolli superiori.

### 3.5.2 Codici a Ripetizione e Parità Semplice

- **Codici a Ripetizione**: ripetere ciascun bit 2 volte ($1 \to 11$, solo rilevazione) comporta il 100% di overhead; ripeterlo 3 volte ($1 \to 111$, correzione a maggioranza) comporta il 200% di overhead. Sono totalmente impraticabili.
- **Codici di Parità Semplice**: si aggiunge un bit di parità $b_{n+1} = \sum_{i=1}^{n} b_i \pmod 2 = b_1 \oplus \cdots \oplus b_n$. Rileva solo un **numero dispari di errori** (1, 3, 5...): qualsiasi coppia o numero pari di errori si compensa reciprocamente eludendo il controllo.

### 3.5.3 Codici di Parità Bidimensionale (Riga/Colonna)

Disponendo i dati in una matrice di $r$ righe e $c$ colonne, si calcola la parità su ogni riga e su ogni colonna:

- **Rilevazione**: rileva con certezza qualsiasi errore di 1, 2 e 3 bit, nonché qualsiasi combinazione dispari;
- **Correzione dell'Errore Singolo**: un errore isolato inverte la parità della sua riga e della sua colonna; l'intersezione univoca individua la coordinata del bit errato, consentendone la correzione logica;
- **Pattern Non Rilevabile**: una combinazione di **4 errori ai quattro vertici di un rettangolo** nella matrice inverte due volte la parità di ciascuna riga e colonna interessata: tutte le parità rimangono inalterate e il quadruplo errore elude la rilevazione.

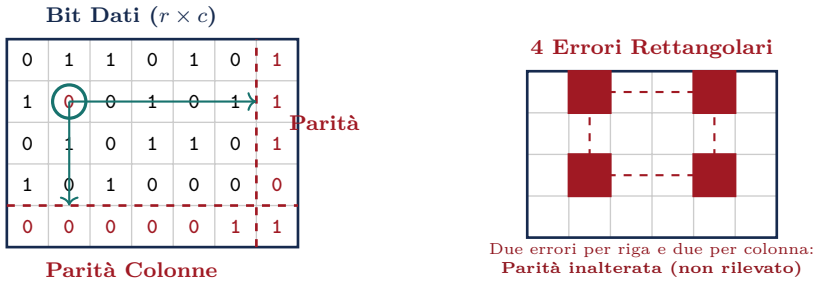



Figura 3.7: A sinistra: correzione hardware dell'errore singolo all'intersezione riga-colonna. A destra: il pattern di 4 errori ai vertici di un rettangolo che elude completamente il controllo.

## 3.6 Codici di Ridondanza Ciclica (CRC)

I **Codici di Ridondanza Ciclica** (CRC) rappresentano la tecnica universalmente adottata a Livello 2 (`Ethernet`, `Wi-Fi`, bus `SATA`) per rilevare errori di trasmissione con efficacia prossima alla certezza e ridottissimo costo computazionale.

**Principio di Funzionamento e Rilevazione** A Livello 2 gli errori si rilevano ma **non si correggono**: tentare la correzione mediante codici fragili rischierebbe di propagare corruzioni ulteriori. Se la verifica del codice di controllo (alloggiato nel trailer) fallisce, la trama viene **immediatamente scartata** (*drop*); l'eventuale recupero affidabile è demandato a timeout e ritrasmissioni o ai livelli superiori (`TCP`). Il meccanismo logico si articola in due fasi:

1. **In trasmissione (TX)**: il payload e l'header passano in un'unità di calcolo che ricava il codice $CRC$, inserito nel trailer della trama;
2. **In ricezione (RX)**: la destinazione ricalcola $CRC'$ sui bit ricevuti e lo confronta con il $CRC$ estratto dal trailer: se $CRC' = CRC$ la trama viene inoltrata a Livello 3, altrimenti viene scartata.

**Rappresentazione Polinomiale e Modulo 2** Una sequenza di $n$ bit viene interpretata come i coefficienti di un polinomio di grado $n-1$ (es. $\texttt{01011001}_2 \iff x^6 + x^4 + x^3 + 1$). I coefficienti appartengono al campo finito $\mathrm{GF}(2) = \{0, 1\}$ (campo di Galois), dove addizioni e sottrazioni coincidono entrambe con l'operazione logica di **XOR** bit a bit ($\oplus$) senza riporti né prestiti ($A \oplus B = 1$ se $A \neq B$, $A \oplus A = 0$, con $A, B \in \mathrm{GF}(2)$).

**Costruzione del Checksum e Polinomio Generatore** Sia $M(x)$ il polinomio dei dati da trasmettere. Per riservare $k$ bit di controllo in coda, si trasla $M(x)$ di $k$ posizioni moltiplicandolo per $x^k$ (equivalente ad accodare $k$ zeri). I bit di ridondanza cercati sono il **resto** $R(x)$ della divisione in $\mathrm{GF}(2)$ per un prefissato

**polinomio generatore** $G(x)$ di grado $k$ con termine noto unitario ($x^0 = 1$):

$$M(x) \cdot x^k = Q(x) \cdot G(x) \oplus R(x) \quad \Longrightarrow \quad R(x) = (M(x) \cdot x^k) \bmod G(x) \tag{3.3}$$

Il quoziente $Q(x)$ viene ignorato. Il trasmettitore sostituisce i $k$ zeri finali con il resto $R(x)$, componendo la trama trasmessa $T(x) = M(x) \cdot x^k \oplus R(x)$. Poiché sommare in modulo 2 equivale ad annullare ($R \oplus R = 0$), l'intera trama trasmessa risulta un multiplo esatto del generatore: $T(x) \bmod G(x) \equiv 0$.

A seconda del grado di protezione desiderato, codici elementari si arrestano a gradi inferiori (es. $k = 1$ con $G(x) = x+1$ per la semplice parità, o $k = 8$ per Bluetooth); nelle reti locali e geografiche lo standard industriale è l'`IEEE CRC-32` ($k = 32$, 4 byte di checksum), implementabile ad altissima velocità hardware tramite registri a scorrimento (LFSR) e porte XOR:

$$G(x) = x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1 \tag{3.4}$$

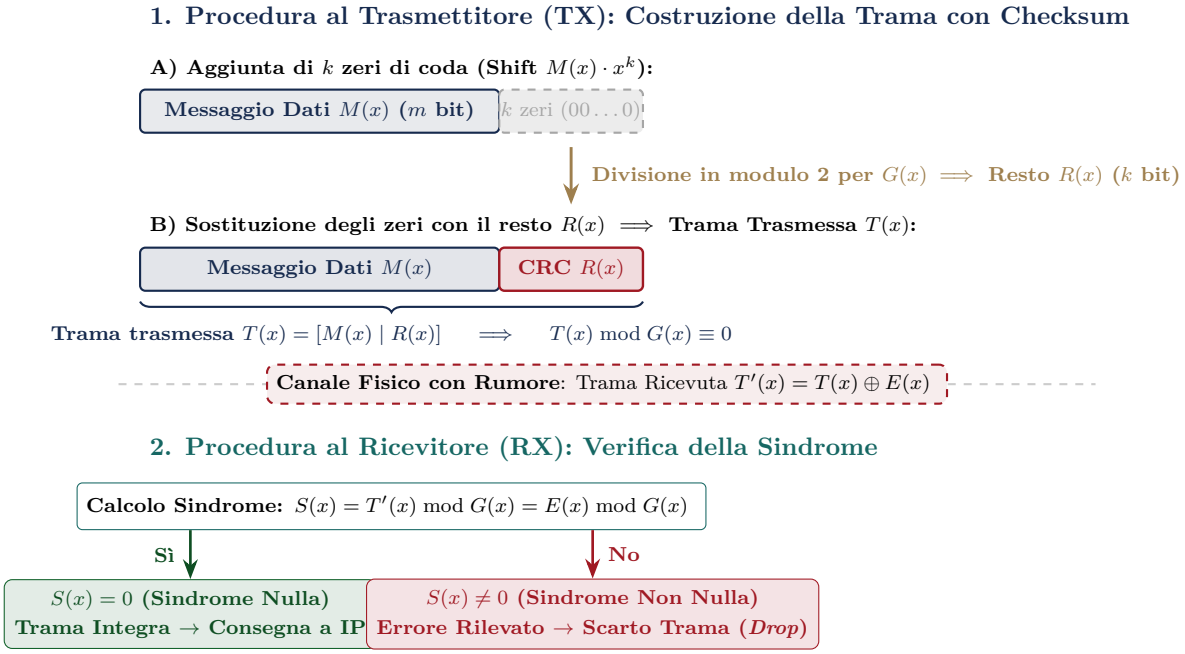



Figura 3.8: Diagramma logico a blocchi del controllo di ridondanza ciclica (CRC): il trasmettitore sposta i dati di $k$ posizioni e calcola il resto $R(x)$ tramite divisione modulo 2 per $G(x)$, accodandolo ai dati; il ricevitore calcola direttamente la sindrome $S(x) = T'(x) \bmod G(x)$ sull'intera trama ricevuta, isolando l'eventuale rumore $E(x)$ per decidere l'inoltro o lo scarto.

**Garanzie di Rilevazione dell'IEEE CRC-32** La struttura algebrica di $G(x)$ assicura proprietà matematiche formali di eccezionale robustezza:

1. Rileva al **100% tutti gli errori a singolo bit** e ogni combinazione con un **numero dispari di bit errati**;
2. Rileva al **100% tutti i burst di errore di lunghezza** $\leq 32$ **bit** (salvo il caso patologico in cui il burst coincida esattamente con $G(x)$);
3. Rileva al **100% tutti i pattern di 2 errori** purché la lunghezza della trama rispetti $|M(x)| < 2^{31} - 1$ bit (oltre 268 MB, valore immensamente superiore al payload massimo Ethernet di 1500 byte o ai 64 KB di un pacchetto IP);
4. Rileva i residui errori casuali con probabilità di mancata individuazione inferiore a $2^{-32} \approx 2{,}3 \times 10^{-10}$ (meno di una su quattro miliardi).

## 3.7 Controllo di Flusso e Negoziazione della Velocità

Una tematica cardine della trasmissione affidabile a livello di collegamento è la gestione della velocità di emissione e il controllo di flusso.

### 3.7.1 Distinzione Concettuale: Negoziazione della Velocità vs. Controllo di Flusso

Un errore comune consiste nel confondere la negoziazione della velocità trasmissiva con il controllo di flusso:

- **Negoziazione della Velocità (Autonegotiation)**: è un processo **statico e preliminare**, effettuato una tantum all'atto dell'allacciamento fisico del cavo o all'aggancio della portante radio. Due interfacce di rete devono concordare la velocità trasmissiva comune (ad esempio 10, 100 o 1000 Mbit/s). Tale accordo avviene mediante un protocollo di handshake che può risiedere a livelli differenti:
  - *A Livello 1 (Fisico)*: apparati tradizionali come vecchi MoDem telefonici, linee `ADSL` ed `Ethernet` su doppino (tramite *Fast Link Pulses*) negoziano la velocità direttamente a livello elettrico/fisico mediante toni e grandezze circuitali.
  - *A Livello 2 (Data Link)*: tecnologie più moderne utilizzano speciali trame di segnalazione a Livello 2 per negoziare la configurazione ottimale partendo da parametri di default.
- **Controllo di Flusso (Flow Control)**: è un processo **dinamico e continuo a runtime**. Non serve a variare la velocità di trasmissione sul cavo (che rimane fissa a 1 Gbit/s o 10 Gbit/s), bensì a regolare la **frequenza temporale di emissione delle trame** da parte della sorgente, per impedire che il ricevitore venga sommerso di dati.

### 3.7.2 Origine Fisica del Problema: La Saturazione dei Buffer del Ricevitore

Il controllo di flusso **dipende esclusivamente dalle condizioni del ricevitore locale, NON dallo stato della rete**:

- Se i nodi intermedi della rete si congestionano, si parla di **controllo di congestione** (funzione globale end-to-end svolta a Livello 4 da `TCP`);
- Il **controllo di flusso** a Livello 2 è invece strettamente **locale (hop-by-hop)**: riguarda unicamente il trasmettitore e il ricevitore direttamente affacciati sul medesimo link.

Quando il trasmettitore invia trame consecutive a raffica (*back-to-back*) alla massima velocità di linea, il ricevitore deve campionare i bit, verificare il CRC, estrarre il payload e passarlo al Livello 3 (IP). Se la CPU del ricevitore o il bus interno sono occupati o lenti, i dati in arrivo si accumulano nella memoria di transito (buffer FIFO) della scheda di rete (NIC). Se il buffer della NIC si satura completamente, ogni ulteriore trama in arrivo andrà irrimediabilmente perduta per overflow. Il ricevitore deve quindi poter inviare un segnale al trasmettitore per intimargli: *«Fermati temporaneamente, non riesco a smaltire i dati!»*.

### 3.7.3 Meccanismi di Realizzazione a Livello 2

Nelle architetture di rete il controllo di flusso a Livello 2 viene implementato mediante due approcci:

1. **Protocolli a Finestra (Stop & Wait)**: a Livello 2 la finestra è tipicamente unitaria (una singola trama). La sorgente trasmette una trama e attende un segnale prima di emettere la successiva.
   - *Nei servizi con riscontro (es. Wi-Fi)*: il feedback è costituito dagli stessi pacchetti di riscontro positivo (**ACK**). Se l'ACK non giunge, la sorgente non trasmette la trama successiva.
   - *Nei servizi senza riscontro (es. Ethernet)*: non essendovi ACK ordinari, il controllo richiede canali dedicati o trame speciali di inibizione.
2. **Segnali Hardware di Inibizione**: su alcuni cablaggi fisici punto-punto storici, il trasmettitore veniva arrestato mediante fili fisici di controllo di flusso (coppie elettriche dedicate che portavano un potenziale di blocco).

### 3.7.4 Ethernet Full-Duplex Flow Control (Standard IEEE 802.3x) e il PAUSE Frame

Nelle moderne reti `Ethernet` su link punto-punto commutati in modalità **full duplex** (cavo che collega direttamente un host a una porta di uno switch), non vi sono collisioni, ma le code d'uscita o d'ingresso dello switch possono riempirsi se più host inviano traffico verso la medesima destinazione. Per impedire la perdita di pacchetti, lo standard `IEEE 802.3x` ha introdotto una speciale trama di controllo detta **PAUSE Frame**:

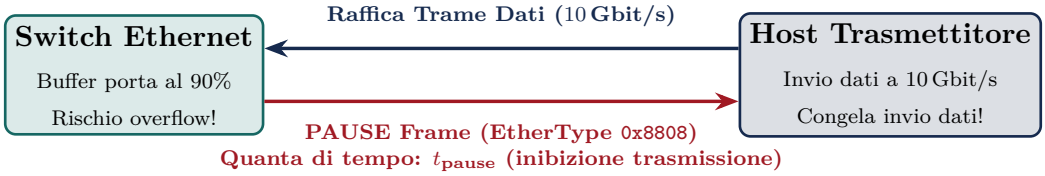



Figura 3.9: Controllo di flusso Ethernet Full-Duplex (IEEE 802.3x): lo switch invia un PAUSE Frame per arrestare temporaneamente il trasmettitore ed evitare l'overflow dei buffer interni.

Il PAUSE Frame possiede caratteristiche standardizzate rigorose:

- **Indirizzo MAC Destinazione**: indirizzo multicast riservato `01-80-C2-00-00-01`, progettato per essere intercettato ed elaborato esclusivamente dalla porta adiacente: **gli switch non inoltrano mai questo frame su altre porte**.
- **EtherType**: valore speciale `0x8808` (*MAC Control*).
- **Opcode**: `0x0001` (comando esplicito di PAUSA).
- **Parametro del Tempo di Pausa (Quanta)**: intero a 16 bit (0–65535) che specifica la durata del blocco espressa in **quanta**, dove ogni quantum equivale convenzionalmente a:

$$1\,\text{Quantum} = 512\,\text{bit-time} \tag{3.5}$$

Ricevuto il frame, l'adattatore dell'host congela immediatamente la trasmissione di dati verso lo switch per il numero di quanti specificato, consentendo allo switch di svuotare le code interne. L'host può comunque continuare a ricevere traffico in senso opposto. Se lo switch smaltisce i buffer prima della scadenza del timer, può inviare un secondo PAUSE frame con tempo di pausa nullo (quanta = 0), ripristinando all'istante la trasmissione.

## 3.8 Controllo di Accesso al Mezzo (MAC) — Inquadramento e Tassonomia Generale

### 3.8.1 Sottolivello MAC nel Modello Architetturale IEEE 802

Il livello di collegamento dati è direttamente a contatto con il mezzo fisico di trasmissione. Il mezzo può essere:

- **Dedicato (reti punto-punto)**: router collegati direttamente tramite fibra ottica o connessioni seriali; non vi è competizione per il mezzo.
- **Condiviso (reti broadcast)**: molteplici stazioni trasmettono su un unico canale comune (cavo coassiale, doppino condiviso tramite hub, o le onde radio dell'etere per laptop e smartphone Wi-Fi).

Quando il mezzo è condiviso, sorge il problema primario della competizione: se due o più sorgenti trasmettono contemporaneamente sul canale, i loro segnali si sovrappongono distruggendosi a vicenda. Questo evento prende il nome di **collisione**. L'informazione viene irrimediabilmente corrotta e le trame vanno perdute.

Per disciplinare l'accesso al mezzo condiviso, l'architettura standard `IEEE 802` spezza il Livello 2 in due sottolivelli gerarchici distinti:

1. **LLC (Logical Link Control, IEEE 802.2)**: sottolivello superiore, comune a tutte le tecnologie di rete locale. Gestisce il controllo di flusso, i riscontri logici e l'interfacciamento verso il Livello 3 (IP).
2. **MAC (Medium Access Control, sottolivello inferiore)**: preposto a governare le regole di accesso al mezzo fisico, la gestione della contesa, la risoluzione delle collisioni e l'indirizzamento fisico delle interfacce.

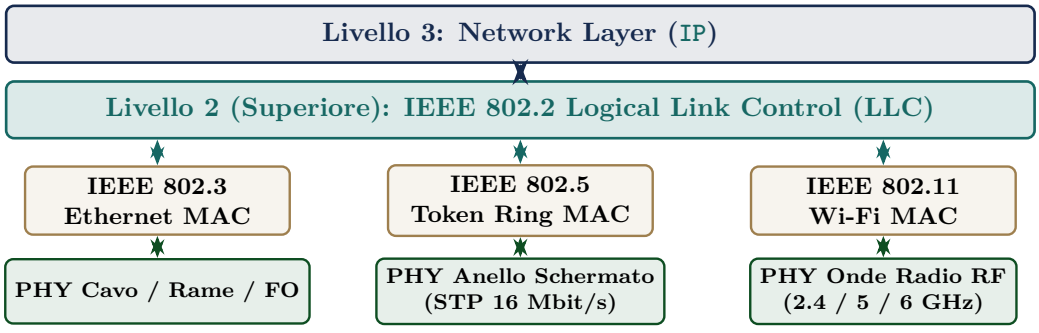



Figura 3.10: Architettura standard IEEE 802: disaccoppiamento tra il sottolivello superiore LLC (comune e indipendente dal mezzo) e il sottolivello inferiore MAC (specifico per ciascuna tecnologia di canale condiviso).

### 3.8.2 Tassonomia delle Tecniche di Allocazione del Canale

Le tecniche per assegnare la capacità di un canale condiviso a molteplici sorgenti si suddividono in due grandi famiglie:

1. **Allocazione Statica**: il canale viene partizionato a priori in sottocanali dedicati permanenti;
2. **Allocazione Dinamica**: il canale viene assegnato di volta in volta solo a chi ne fa esplicita richiesta.

### 3.8.3 Allocazione Statica del Mezzo (Partizionamento Fisico)

Nel partizionamento statico, la risorsa fisica viene rigidamente frazionata tra $N$ utenti:

- **Frequency Division Multiple Access (FDMA)**: lo spettro frequenziale è diviso in bande non sovrapposte assegnate a ciascun utente.
- **Time Division Multiple Access (TDMA)**: il tempo è suddiviso in intervalli ciclici periodici (*frame*), a loro volta ripartiti in *time-slot*; ogni utente trasmette esclusivamente nel proprio slot periodico (telefonia 2G GSM).
- **Code Division Multiple Access (CDMA)**: tutte le stazioni trasmettono contemporaneamente sull'intero spettro, moltiplicando i bit per codici numerici pseudo-ortogonali (telefonia 3G UMTS).
- **Orthogonal Frequency Division Multiple Access (OFDMA)**: allocazione dinamico-statica a griglia bidimensionale tempo-frequenza su migliaia di sottoportanti ortogonali (4G LTE e 5G).

**Inefficienza dell'Allocazione Statica con Traffico Discontinuo**

L'allocazione statica è semplice e garantisce ottime prestazioni quando il numero di utenti è piccolo e il traffico è continuo nel tempo (come una telefonata a 64 kbit/s). Tuttavia, applicata al traffico Internet, risulta **catastroficamente inefficiente**: il traffico generato dai calcolatori è per sua natura a raffica (*bursty*), con lunghi silenzi tra una richiesta web e l'altra. In un sistema TDMA con $N$ utenti, se un utente è momentaneamente inattivo, la frazione di canale $1/N$ a lui riservata **rimane desolatamente vuota e va irrimediabilmente perduta**, mentre altri nodi con code piene restano forzatamente bloccati. L'efficienza di utilizzazione del canale crolla drammaticamente.

## 3.9 Allocazione Dinamica del Mezzo: Protocolli a Turno vs. a Contesa

Per supportare efficientemente il traffico dati a pacchetto, le reti di calcolatori ricorrono all'**allocazione dinamica**, assegnando il mezzo trasmissivo esclusivamente alle stazioni che possiedono attivamente trame da emettere. L'assegnazione dinamica può essere strutturata secondo due filosofie contrapposte:

- **Protocolli a Turno (Coordinati)**: il diritto di trasmettere viene disciplinato e trasferito ordinatamente tra le stazioni;
- **Protocolli a Contesa (Random Access)**: ciascuna sorgente trasmette in modo indipendente, affrontando le eventuali collisioni.

### 3.9.1 Protocolli a Turno (Token Passing e Token Ring)

Nei protocolli a turno, l'accesso al mezzo condiviso è regolato dal possesso di un "gettone" logico (**token**), costituito da una sequenza di bit con un pattern rigidamente definito:

- Il gettone circola continuamente lungo un anello logico tra tutte le stazioni collegate ($S_1 \to S_2 \to \cdots \to S_N \to S_1$).
- Solo la stazione che si trova momentaneamente in possesso del gettone ha il diritto insindacabile di trasmettere trame sul canale.
- La trasmissione è vincolata da un tempo limite superiore (*Token Holding Time*), trascorso il quale la stazione ha l'obbligo di terminare la propria trasmissione e cedere il gettone alla stazione logicamente adiacente.

I sistemi a turno garantiscono due vantaggi teorici evidenti: elevata efficienza ad alto carico (le collisioni sono fisicamente impossibili) e un ritardo massimo di accesso garantito a priori (*bounded delay*), proprietà ideale per applicazioni industriali in tempo reale. Tuttavia, essi sono afflitti da gravissimi **limiti strutturali**:

1. **Complessità di gestione e guasti del gettone**: se un disturbo sul canale corrompe il pattern del token, il gettone va perduto e la rete si blocca istantaneamente. Occorrono complessi timer di watchdog e algoritmi distribuiti di elezione: se nessuna stazione vede passare il token per un tempo limite $Y$, tutte le stazioni estraggono un numero casuale e il vincitore viene eletto come stazione master incaricata di rigenerare il gettone. Se la stazione master si spegne improvvisamente, la procedura di elezione deve ripartire da capo.
2. **Inefficienza a canale scarico**: se in una rete di $N = 100$ stazioni una sola macchina desidera trasmettere, essa non può utilizzare il canale a pieno ritmo: dopo aver trasmesso una trama deve rilasciare il token e attendere che esso compia a vuoto il giro di tutti gli altri 99 nodi silenziosi prima di poter trasmettere la trama successiva.

### 3.9.2 Protocolli a Contesa (Random Access): Perché Hanno Vinto la Competizione Storica

Nei protocolli a contesa non esiste alcun gettone e non vi è alcuna gerarchia tra le stazioni:

- **Indipendenza assoluta delle stazioni**: ogni nodo decide autonomamente quando e come accedere al canale utilizzando esclusivamente informazioni rilevate a livello locale (senza conoscere lo stato delle altre stazioni).
- **Assenza di coordinamento centrale**: non esiste un coordinatore né una stazione master, eliminando qualsiasi singolo punto di fallimento (*single point of failure*).
- **Gestione probabilistica**: l'accesso avviene senza la certezza matematica di non collidere. Il protocollo integra algoritmi per rilevare la collisione e ritrasmettere la trama dopo un tempo casuale, prevenendo collisioni deterministiche a catena.

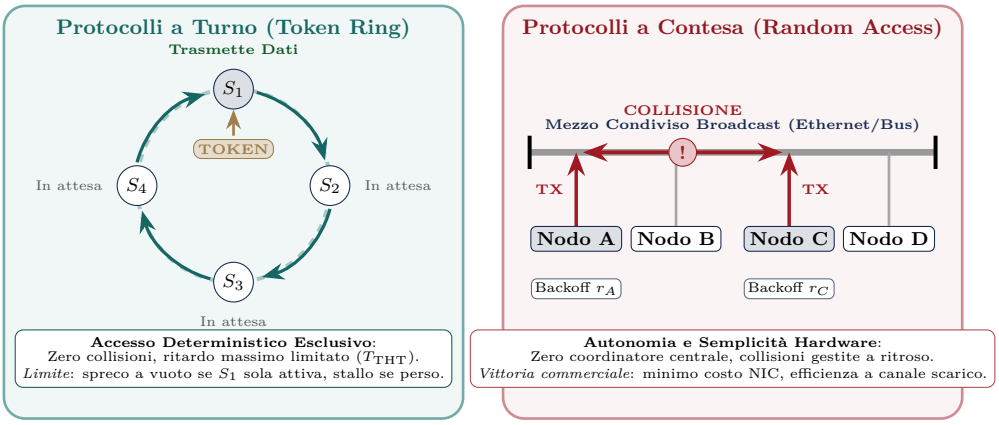



Figura 3.11: Confronto concettuale tra le due filosofie di allocazione dinamica del canale (§3.9.1 vs §3.9.2): a sinistra, i **Protocolli a Turno (Token Ring)**, in cui l'accesso è rigidamente coordinato dalla circolazione deterministica di un gettone logico che previene alla radice le collisioni al prezzo di complessità e latenza; a destra, i **Protocolli a Contesa (Random Access)**, fondati su stazioni autonome che accedono spontaneamente al mezzo condiviso e risolvono le inevitabili collisioni tramite algoritmi probabilistici di backoff.

**Approfondimento: La Scomparsa dei Protocolli a Turno e la Supremazia della Contesa**

Sia Ethernet che i protocolli a turno erano stati sviluppati negli anni '70, ma a partire dalla metà degli anni '80 i protocolli a turno sono stati progressivamente spazzati via dal mercato e sono completamente scomparsi dalle reti commerciali a vantaggio di Ethernet e dei protocolli a contesa. La ragione fondamentale risiede in una ferrea legge economica e prestazionale:

- **Costo e semplicità hardware**: un'interfaccia a contesa richiede circuiti di controllo elementari, riducendo drasticamente il costo di produzione delle schede di rete (NIC) a una frazione rispetto alle complessità di un chip Token Ring.
- **Scalabilità tecnologica delle velocità**: per far funzionare bene un protocollo a contesa è sufficiente mantenere il canale mediamente "scarico". Con il progresso tecnologico che consentiva di passare a velocità 10 o 100 volte superiori a parità di costo (da 10 Mbit/s a 100 Mbit/s e 1 Gbit/s), il canale si è trovato fisiologicamente scarico, annullando all'istante ogni vantaggio dei complessi protocolli coordinati.

Oggi, come ribadito dal docente a lezione, la contesa è il pilastro indiscusso di tutte le reti ad accesso radio non cablate (il `Wi-Fi IEEE 802.11` è intrinsecamente e interamente a contesa, poiché non esiste un Wi-Fi commutato).

## 3.10 Modello Stocastico per l'Analisi dei Protocolli a Contesa

Poiché nei protocolli a contesa ciascuna stazione opera in modo autonomo, il comportamento del canale condiviso non può essere descritto deterministicamente, ma deve essere analizzato rigorosamente mediante la **teoria della probabilità e i processi stocastici**.

### 3.10.1 Ipotesi Fondamentali del Modello e "Regole del Gioco"

La letteratura scientifica adotta una serie di assunzioni canoniche, introdotte originariamente per consentire l'analisi matematica comparativa delle prestazioni:

1. **Canale Singolo Condiviso (*Single Channel Assumption*)**: un unico canale di comunicazione è condiviso da tutte le stazioni; il tempo di propagazione del segnale sul mezzo è trascurabile rispetto alla durata della trama (ipotesi che rilasseremo in CSMA ed Ethernet).
2. **Modello delle Stazioni (*Station Model*)**: un insieme di $N$ stazioni indipendenti, ciascuna delle quali genera nuove trame. Si assume che una stazione, una volta generata una trama, sia **bloccata** fino al completamento della trasmissione con successo (assenza di buffer di memorizzazione infiniti per stazione).
3. **Assunzione di Collisione Distruttiva (*Collision Assumption*)**: se due o più trame si sovrappongono sul canale anche per un singolo bit, si verifica una collisione e l'informazione di tutte le trame coinvolte viene totalmente distrutta. Si assume l'assenza dell'effetto cattura (*no capture effect*), ossia nessuna stazione prevale sull'altra per potenza superiore.
4. **Assenza di Errori Fisici Residui**: le uniche perdite di trame sono dovute alle collisioni sul mezzo condiviso; il canale è ideale rispetto al rumore termico.
5. **Modello Temporale**:
   - *Tempo Continuo*: la trasmissione di una trama può iniziare in qualsiasi istante temporale $t \in \mathbb{R}$.
   - *Tempo Discretizzato (*Slotted*)*: l'asse temporale è suddiviso in intervalli discreti (slot) di durata fissa; la trasmissione può iniziare solo all'inizio di ciascuno slot.

### 3.10.2 Il Processo di Generazione di Poisson e Sovrapposizione degli Arrivi

Il traffico immesso sul canale deriva dalla somma dei comportamenti indipendenti di una moltitudine di stazioni. Dati empirici e fondamenti teorici dimostrano che il processo di arrivo delle trame è fedelmente descritto da un **processo di Poisson**:

- Il tempo di interarrivo $t$ tra la generazione di una trama e la successiva su ciascuna stazione è una variabile casuale indipendente con distribuzione esponenziale negativa a tasso $\lambda$:

$$f(t) = \lambda e^{-\lambda t}, \quad t \geq 0 \tag{3.6}$$

- Gode della proprietà di **assenza di memoria** (*memoryless*): il tempo trascorso dall'ultima trama non altera la probabilità che una nuova trama sia generata nell'istante successivo.
- **Teorema di Sovrapposizione di Processi di Poisson**: la somma di $N$ processi di Poisson indipendenti con tassi $\lambda_1, \lambda_2, \ldots, \lambda_N$ genera un **unico processo di Poisson complessivo** avente tasso pari alla somma algebrica dei singoli tassi:

$$\lambda_{\text{tot}} = \sum_{i=1}^{N} \lambda_i \tag{3.7}$$

**Fondamento nel Teorema Centrale del Limite**

L'adozione del processo di Poisson e l'assunzione di trame a lunghezza fissa possono essere motivate dal **Teorema Centrale del Limite**: sommando una molteplicità di variabili casuali indipendenti generate da utenti eterogenei, la distribuzione complessiva converge verso forme canoniche gaussiane e poissoniane. Ciò legittima l'approssimazione del sistema reale con un unico processo stocastico complessivo.

### 3.10.3 Carico Offerto Normalizzato ($G$) e Throughput ($S$)

Sia $T$ la durata di trasmissione di una singola trama (tempo di trama, assunto costante). Definiamo due parametri adimensionali fondamentali:

**Definizione: Carico Offerto $G$ e Throughput $S$**

- **Carico Offerto al Canale ($G$)**: è il numero medio di trame complessivamente generate nell'intervallo temporale pari alla durata di una trama $T$:

$$G = \lambda_{\text{tot}} \cdot T \tag{3.8}$$

  Il valore $G$ ingloba sia le **nuove trame** generate dagli host, sia le **trame ritrasmesse** a seguito

di precedenti collisioni. Poiché il tempo di attesa prima della ritrasmissione è scelto casualmente $t_c \gg T$, le ritrasmissioni risultano stocasticamente decorrelate e indistinguibili da nuovi arrivi.
- **Throughput ($S$)**: è la frazione media del tempo di canale impiegata per trasmissioni che hanno avuto **successo** (ossia prive di collisioni), normalizzata alla capacità massima trasmissiva $C$ del canale ($0 \leq S \leq 1$). In un canale perfetto senza alcuna contesa, $S = 1$.

Dalla distribuzione di Poisson, la probabilità che vengano generate esattamente $k$ trame all'interno di un intervallo di tempo pari a un tempo di trama $T$ vale:

$$P[k] = \frac{G^k e^{-G}}{k!} \tag{3.9}$$

In particolare, la probabilità che durante un tempo di trama $T$ non venga generata **alcuna trama** ($k = 0$) vale:

$$P[0] = \frac{G^0 e^{-G}}{0!} = e^{-G} \tag{3.10}$$

## 3.11 Protocollo ALOHA Puro (Abramson, 1970)

### 3.11.1 Origine Storica e Principio di Funzionamento

Sviluppato nel 1970 da Norman Abramson presso l'Università delle Hawaii, il protocollo **ALOHA puro** (*Pure ALOHA*) nacque per interconnettere terminali distribuiti su diverse isole dell'arcipelago mediante onde radio bidirezionali condivise.

Il principio dell'ALOHA puro è l'estrema semplicità:

1. Il tempo è continuo: ogni stazione che ha una trama pronta per la trasmissione la invia **immediatamente**, senza compiere alcuna verifica preventiva sul canale.
2. Se la trama non entra in collisione con altre emissioni, viene ricevuta correttamente.
3. Se due o più trame collidono, i ricevitori non riscontrano la ricezione; le stazioni sorgente rilevano l'assenza di riscontro e programmano la ritrasmissione.
4. **Regola anti-blocco**: la ritrasmissione non può avvenire dopo un tempo prefissato fisso, altrimenti le stesse stazioni ri-colliderebbero all'infinito. La stazione attende perciò un **tempo casuale di backoff** $t_c \gg T$, decorrelando i tentativi successivi.

### 3.11.2 La Finestra di Vulnerabilità di ALOHA Puro ($2T$)

Si consideri una stazione che inizia a trasmettere una trama di durata $T$ all'istante temporale $t_0$. Affinché tale trama non subisca collisioni:

- Nessun'altra stazione deve aver iniziato a trasmettere nell'intervallo $[t_0 - T, t_0]$, altrimenti la coda di quella trasmissione colliderebbe con l'inizio della nostra trama.
- Nessun'altra stazione deve iniziare a trasmettere nell'intervallo $[t_0, t_0 + T]$, altrimenti la testa di quella trasmissione colliderebbe con la coda della nostra trama.

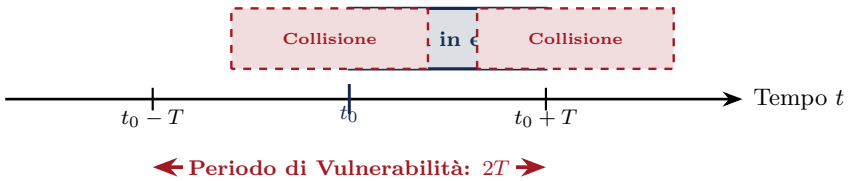



Figura 3.12: Periodo di vulnerabilità del protocollo ALOHA puro: affinché la trama trasmessa nell'intervallo $[t_0, t_0 + T]$ abbia successo, nessun'altra trama deve essere emessa nell'intervallo $[t_0 - T, t_0 + T]$, di estensione complessiva pari a $2T$.

Il **periodo di vulnerabilità** per ALOHA puro si estende perciò per una durata pari a **due tempi di trama**:

$$T_{\text{vulnerabilità}} = 2T \tag{3.11}$$

### 3.11.3 Derivazione Analitica del Throughput e Limite Teorico del 18.4%

Il throughput $S$ è dato dal prodotto tra il carico medio offerto $G$ e la probabilità che non si verifichino altre emissioni per l'intero periodo di vulnerabilità di durata $2T$. Poiché gli arrivi seguono una distribuzione di

Poisson con media $G$ per tempo $T$, nell'intervallo $2T$ la media degli eventi è $2G$:

$$P[0 \text{ trasmissioni in } 2T] = e^{-2G} \tag{3.12}$$

Si ottiene così la celebre legge del Throughput di Abramson per ALOHA puro:

$$S = G \cdot e^{-2G} \tag{3.13}$$

Per determinare il punto di massimo throughput, deriviamo $S$ rispetto a $G$ e poniamo la derivata uguale a zero:

$$\frac{dS}{dG} = \frac{d}{dG}\left(Ge^{-2G}\right) = 1 \cdot e^{-2G} + G \cdot (-2)e^{-2G} = (1-2G)e^{-2G} = 0 \tag{3.14}$$

Poiché l'esponenziale $e^{-2G}$ è strettamente positivo, l'annullamento impone:

$$1 - 2G = 0 \implies G^* = \frac{1}{2} = 0{,}5 \tag{3.15}$$

Il massimo valore di throughput ottenibile in ALOHA puro vale:

$$S_{\max} = S(0{,}5) = 0{,}5 \cdot e^{-2 \cdot 0{,}5} = \frac{1}{2}e^{-1} = \frac{1}{2e} \approx 0{,}1839 \implies \mathbf{18{,}4}\% \tag{3.16}$$

**Principio / Fondamento: Significato del Limite del 18.4% e Instabilità di ALOHA**

Il protocollo ALOHA puro è in grado di sfruttare al massimo il **18.4% della capacità nominale del canale**. Il picco di efficienza si registra quando il traffico offerto è pari alla metà della capacità ($G = 0{,}5$). La probabilità che una generica trama subisca una collisione vale:

$$P_{\text{collisione}} = 1 - P[0 \text{ in } 2T] = 1 - e^{-2G} \tag{3.17}$$

Al picco di throughput ($G = 0{,}5$), $P_{\text{collisione}} = 1 - e^{-1} \approx 63{,}2\%$: oltre sei trame su dieci collidono!
Se il carico generato dagli utenti supera la soglia critica ($G > 0{,}5$), il throughput comincia a decrescere rapidamente. Tuttavia, le stazioni le cui trame collidono tentano continuamente di ritrasmetterle, provocando un aumento esponenziale del carico totale $G$. Questo incremento induce ulteriori collisioni, innescando un circolo vizioso a valanga:

$$G \to \infty \implies S = Ge^{-2G} \to 0 \tag{3.18}$$

Il canale collassa e non eroga più traffico utile. Il protocollo ALOHA puro è intrinsecamente **instabile ad alto carico**.

## 3.12 Protocollo Slotted ALOHA (Roberts, 1972)

### 3.12.1 Sincronizzazione Temporale e Dimezzamento del Periodo di Vulnerabilità

Nel 1972, Lawrence Roberts propose una modifica architetturale fondamentale per raddoppiare l'efficienza di ALOHA: distribuire alle stazioni un **riferimento temporale comune** (sincronismo di clock).

In **Slotted ALOHA**:

- Il tempo continuo viene partizionato in intervalli discreti (*time slot*) di durata fissa pari a $T$ (il tempo di trasmissione di una trama).
- Le stazioni hanno il vincolo tassativo di poter iniziare a trasmettere **solamente in corrispondenza dell'inizio di ciascuno slot**.
- Se una stazione genera una trama a metà di uno slot, è costretta ad accodarla e attenderne la fine, iniziando la trasmissione al fronte del primo slot utile successivo.

Grazie a questo vincolo di allineamento, le trame non possono più scontrarsi parzialmente per sovrapposizioni sfasate nel tempo. Una collisione si verifica **se e solo se due o più stazioni generano una trama**

**all'interno del medesimo slot precedente**. Il periodo di vulnerabilità si dimezza:

$$T_{\text{vulnerabilità, Slotted}} = T \tag{3.19}$$

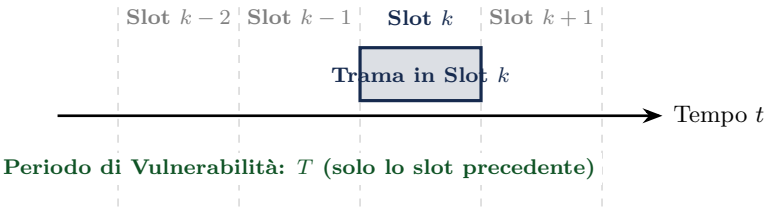



Figura 3.13: Periodo di vulnerabilità di Slotted ALOHA: la discretizzazione temporale vincola le trasmissioni all'inizio degli slot, dimezzando la vulnerabilità a un singolo tempo di trama $T$.

### 3.12.2 Derivazione del Throughput e Raddoppio al 36.8%

La probabilità di successo per uno slot corrisponde alla probabilità che nello slot precedente sia stata generata esattamente una sola trama (oppure che nell'intervallo $T$ arrivino zero ulteriori trame a collidere):

$$P[0 \text{ trasmissioni in } T] = e^{-G} \tag{3.20}$$

Il Throughput di Slotted ALOHA vale:

$$S_{\text{slotted}} = G \cdot e^{-G} \tag{3.21}$$

Calcolando la derivata rispetto a $G$:

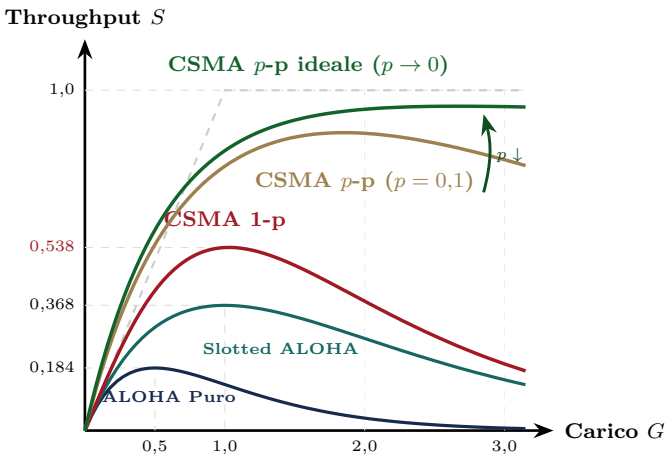
$$\frac{dS_{\text{slotted}}}{dG} = (1-G)e^{-G} = 0 \implies G^* = 1 \tag{3.22}$$

Il massimo valore del throughput per Slotted ALOHA vale:

$$S_{\text{max,slotted}} = S(1) = 1 \cdot e^{-1} = \frac{1}{e} \approx 0{,}3679 \implies \mathbf{36{,}8}\% \tag{3.23}$$

| **Protocollo** | **Vulnerabilità** | **Carico $G^*$** | **Throughput $S_{\max}$** | **Requisito Tecnologico** |
|---|---|---|---|---|
| **ALOHA Puro** | $2T$ | 0,5 | $\frac{1}{2e} \approx 18{,}4\%$ | Nessun coordinamento; tempo continuo. |
| **Slotted ALOHA** | $T$ | 1,0 | $\frac{1}{e} \approx 36{,}8\%$ | Riferimento di clock (*slotted time*). |

Tabella 3.2: Confronto analitico tra ALOHA puro e Slotted ALOHA.




**Dinamica della Contesa e Collisioni**

**CSMA 1-Persistente ($a = 0$):**
I pacchetti che giungono con canale occupato attendono la liberazione. All'istante del rilascio, collidono se ne è arrivato almeno uno:

$$\mathbf{P_c = 1 - P[0] = 1 - e^{-G}}$$

- Per $G = 1 \implies P_c \approx 63{,}2\%$
- Per $G \geq 2 \implies P_c > 86{,}5\%$ (collasso rapido a $S \to 0$)

**CSMA $p$-Persistente ($a = 0$):**
Ciascuna stazione trasmette con probabilità $p$ e differisce con $1 - p$. Distribuendo i tentativi su micro-slot:

$$\mathbf{p \downarrow \implies S \to 1} \quad \text{(per } \mathbf{a = 0}\text{)}$$

Figura 3.14: Confronto analitico del Throughput normalizzato $S$ in funzione del carico offerto $G$ per protocolli ad accesso casuale (ALOHA puro, Slotted ALOHA, CSMA 1-persistente e CSMA $p$-persistente ideale con $a = 0$). Si evidenzia il limite di CSMA 1-p ($S_{\max} \approx 53{,}8\%$) e il suo collasso dovuto all'accumulo di stazioni durante la trasmissione ($P_c = 1 - P[0] = 1 - e^{-G}$), superato dal CSMA $p$-persistente la cui efficienza converge a 1 al diminuire della persistenza $p$.

### 3.12.3 Ambiti di Utilizzo Moderni di Slotted ALOHA

Sebbene Slotted ALOHA sia un protocollo antico e instabile ad alti carichi, esso non è affatto defunto: costituisce un componente architetturale indispensabile nelle moderne reti cellulari e satellitari.

**1. Accesso Iniziale Radio nelle Reti Cellulari (PRACH in 4G LTE e 5G):** Quando uno smartphone si accende o esce dallo stato di idle, non dispone di risorse trasmissive allocate. La stazione radiobase (*eNodeB* o *gNodeB*) emette in continuo un segnale di sincronismo e broadcast in downlink, definendo una periodica tramatura di slot temporali per il canale ad accesso casuale `PRACH` (*Physical Random Access Channel*). Lo smartphone:

1. Si sincronizza sui frame della cella;
2. Estrae un codice numerico pseudocasuale tra 64 preamboli disponibili;
3. Trasmette il preambolo in uplink sfruttando uno slot **Slotted ALOHA**;
4. La stazione radiobase riceve il codice, riconosce la presenza del nuovo host e risponde con una *Random Access Response*, assegnandogli canali dedicati e sicuri per autenticazione e scambio dati.

**2. Canali Satellitari e Canali ad Alto Ritardo di Propagazione:** Nei collegamenti via satellite geostazionario, il ritardo di propagazione bidirezionale vale $\tau \approx 250\,\text{ms}$. Se una trama ha durata $T = 5\,\text{ms}$, il tempo di propagazione è cinquanta volte superiore alla durata del pacchetto ($\tau \gg T$). In tali contesti, ascoltare il canale prima di trasmettere non ha alcun valore (quando una stazione sente il canale libero, la trasmissione potrebbe essere già in volo altrove da 200 ms). I protocolli di sensing collassano e l'accesso su slot temporali (Slotted ALOHA) rimane l'unica architettura robusta praticabile.

### 3.12.4 Valutazione delle Prestazioni e Dimensionamento di ALOHA e Slotted ALOHA

Nel passaggio dallo studio qualitativo dei protocolli di rete alla loro ingegnerizzazione rigorosa, assume un ruolo centrale la **performance evaluation** (valutazione analitica delle prestazioni). Attraverso precisi *Key Performance Indicators* (KPI) — in particolare la **probabilità di collisione** $P_C$, il **throughput normalizzato** $S$ e la **capacità di canale dimensionata** $C$ — l'ingegnere quantifica l'efficienza dei meccanismi di arbitraggio e seleziona la tecnologia idonea a soddisfare i requisiti applicativi.

La seguente trattazione unifica l'analisi del controllo d'accesso al mezzo (MAC) ad accesso casuale privo di carrier sense, esplorando l'evoluzione del canale da una popolazione infinita di stazioni fino a sistemi finiti a stazioni simmetriche e asimmetriche.

**Esercizio d'Esame: Analisi Prestazionale Unificata e Dimensionamento di Protocolli ad Accesso Casuale (ALOHA e Slotted ALOHA)**

**Enunciato del Problema Unificato:**
Si consideri un canale broadcast condiviso in cui le stazioni trasmettono trame di durata normalizzata $T = 1\,\text{s}$ su un mezzo privo di carrier sense, adottando alternativamente **ALOHA Puro** e **Slotted ALOHA**. Non essendo implementato alcun buffer di ritrasmissione a livello data link, le trame che subiscono collisione vengono scartate. Si richiede di analizzare in modo organico e quantitativo i seguenti scenari:

**(a) Popolazione Infinita e Carico Aggregato:** Si assuma un numero molto elevato di stazioni ($N \to \infty$) con processo complessivo di generazione delle trame di tipo Poisson di intensità aggregata $\lambda \in \{0{,}1;\ 0{,}2;\ 0{,}5\}$ trame/s. Si formalizzi la Probability Mass Function (PMF) di Poisson, si definiscano i periodi di vulnerabilità $T_v$ e si calcoli la probabilità di collisione $P_C$ per ALOHA Puro e per Slotted ALOHA.

**(b) Curve Analitiche e Caratteristiche di Prestazione:** Si estenda il calcolo al variare continuo del carico offerto $G = \lambda T$ e si traccino le curve caratteristiche della probabilità di collisione $P_C(G)$ e del throughput utile $S(G)$, identificando i punti di massimo teorico e commentando i limiti di stabilità.

**(c) Popolazione Finita Simmetrica (2 Stazioni):** Si consideri il medesimo canale conteso da due sole stazioni gemelle $S_1$ ed $S_2$, ciascuna con processo di generazione di Poisson di intensità $\lambda_1 = \lambda_2 = \lambda \in \{0{,}1;\ 0{,}2;\ 0{,}5\}$ trame/s. Si dimostri analiticamente perché la probabilità di collisione dipenda unicamente dal traffico generato dalla stazione rivale, calcolando $P_{C,1}$, $P_{C,2}$ e il throughput $S_i$ e aggregato $S_{\text{tot}}$.

**(d) Asimmetria di Traffico e Paradosso di Contesa (3 Stazioni):** Si introducano tre stazioni $S_1, S_2, S_3$ caratterizzate da tassi eterogenei: $S_1$ ("timida") con $\lambda_1 = 0{,}05$ trame/s, $S_2$ ("media") con $\lambda_2 = 0{,}10$ trame/s e $S_3$ ("loquace") con $\lambda_3 = 0{,}20$ trame/s. Si determinino le probabilità di collisione individuali $P_{C,i}$ in ALOHA Puro e Slotted ALOHA, fornendo la spiegazione fisica dell'effetto per cui la stazione a più bassa intensità subisce la penalizzazione più severa.

**(e) Throughput Individuale e di Canale:** Si calcoli il throughput normalizzato utile $S_i = \lambda_i(1 - P_{C,i})$ per ciascuna stazione negli scenari (c) e (d), valutando l'efficienza aggregata $S_{\text{tot}} = \sum_i S_i$.

**(f) Dimensionamento della Capacità Fisica per 5 Mbit/s:** Per garantire a **ciascuna** stazione un tasso utile effettivo $R_{\text{eff}} \geq 5\,\text{Mbit/s}$, si calcoli la capacità fisica minima di canale $C$, confrontando i casi simmetrici e asimmetrici in ALOHA Puro e Slotted ALOHA.

---

**Svolgimento Dettagliato e Metodologia Risolutiva:**

**Quesito (a) — Popolazione Infinita e Vulnerabilità:** In un processo di arrivo di Poisson con intensità media $\lambda$ (tasso di generazione o frequenza di eventi nell'unità di tempo), il numero discreto di trame generate $K$ in un generico intervallo temporale $\Delta t$ segue la Probability Mass Function (PMF):

$$P[K = k \text{ in } \Delta t] = \frac{(\lambda \Delta t)^k}{k!}\, e^{-\lambda \Delta t}, \quad k \in \{0, 1, 2, \dots\} \tag{3.24}$$

La condizione necessaria e sufficiente affinché una trama generata venga trasmessa con successo (evento $S$) è che **nessun'altra stazione** generi trame durante la finestra temporale in cui una sovrapposizione produrrebbe interferenza distruttiva, definita come **tempo di vulnerabilità** $T_v$:

$$P_S = P[K = 0 \text{ in } T_v] = \frac{(\lambda T_v)^0}{0!}\, e^{-\lambda T_v} = e^{-\lambda T_v} \tag{3.25}$$

Poiché gli esiti possibili per una trama sono mutuamente esclusivi (successo oppure collisione), la probabilità di collisione $P_C$ è data dalla probabilità complementare:

$$P_C = 1 - P_S = 1 - e^{-\lambda T_v} \tag{3.26}$$

Confrontiamo la dimensione della finestra di vulnerabilità nei due protocolli:

- **ALOHA Puro:** Le stazioni trasmettono non appena la trama viene generata. Una trasmissione

iniziata all'istante $t_0$ collide se un'altra stazione trasmette nell'intervallo $[t_0 - T, t_0]$ (l'ultimo bit della trama precedente sovrascrive il primo bit della nostra) oppure nell'intervallo $[t_0, t_0 + T]$ (il primo bit della nuova trama sovrascrive l'ultimo della nostra). L'intervallo di vulnerabilità ha estensione:

$$T_v^{\text{pure}} = 2T \implies P_C^{\text{pure}} = 1 - e^{-2\lambda T} \tag{3.27}$$

- **Slotted ALOHA:** Il tempo è suddiviso in slot discreti sincronizzati di durata $T$. Le trame generate durante lo slot corrente differiscono la trasmissione all'inizio del successivo. Di conseguenza, collidono solo le trame generate all'interno del medesimo slot:

$$T_v^{\text{slotted}} = T \implies P_C^{\text{slotted}} = 1 - e^{-\lambda T} \tag{3.28}$$

Essendo normalizzato il tempo di trama a $T = 1\,\text{s}$, il prodotto $\lambda T$ coincide numericamente con l'intensità $\lambda$. Valutiamo numericamente per $\lambda \in \{0{,}1;\ 0{,}2;\ 0{,}5\}$:

- Per $\lambda = 0{,}1$:

$$P_C^{\text{pure}} = 1 - e^{-2\cdot 0{,}1\cdot 1} = 1 - e^{-0{,}2} \approx 1 - 0{,}81873 = \mathbf{18{,}13}\%$$
$$P_C^{\text{slotted}} = 1 - e^{-1\cdot 0{,}1\cdot 1} = 1 - e^{-0{,}1} \approx 1 - 0{,}90484 = \mathbf{9{,}52}\%$$

- Per $\lambda = 0{,}2$:

$$P_C^{\text{pure}} = 1 - e^{-2\cdot 0{,}2\cdot 1} = 1 - e^{-0{,}4} \approx 1 - 0{,}67032 = \mathbf{32{,}97}\%$$
$$P_C^{\text{slotted}} = 1 - e^{-1\cdot 0{,}2\cdot 1} = 1 - e^{-0{,}2} \approx 1 - 0{,}81873 = \mathbf{18{,}13}\%$$

- Per $\lambda = 0{,}5$:

$$P_C^{\text{pure}} = 1 - e^{-2\cdot 0{,}5\cdot 1} = 1 - e^{-1{,}0} \approx 1 - 0{,}36788 = \mathbf{63{,}21}\%$$
$$P_C^{\text{slotted}} = 1 - e^{-1\cdot 0{,}5\cdot 1} = 1 - e^{-0{,}5} \approx 1 - 0{,}60653 = \mathbf{39{,}35}\%$$

*Rilievo Ingegneristico:* Si noti la straordinaria corrispondenza: la probabilità di collisione di Slotted ALOHA per $\lambda = 0{,}2$ (18,13%) coincide esattamente con quella di ALOHA Puro per $\lambda = 0{,}1$ (18,13%). La slottizzazione temporale **dimezza la vulnerabilità**, permettendo al sistema di assorbire il doppio del traffico a parità di tasso di collisione.

**Quesito (b) — Curve Analitiche e Caratteristiche di Prestazione:** Definito il carico offerto adimensionale $G = \lambda T$ (numero medio di tentativi di trasmissione per durata di trama), il throughput normalizzato $S(G)$ rappresenta la frazione di tempo in cui il canale trasporta con successo trame utili ($S \leq 1$):

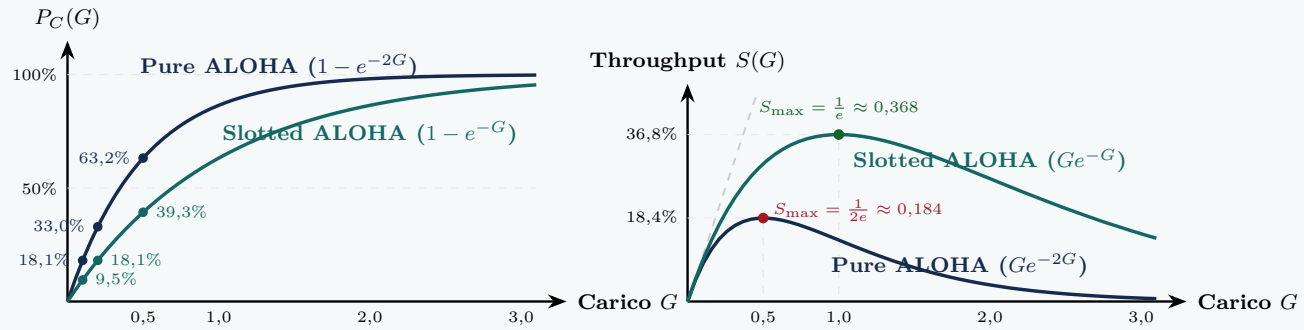
$$S(G) = G \cdot P_S = \begin{cases} G\,e^{-2G} & \text{per ALOHA Puro} \\ G\,e^{-G} & \text{per Slotted ALOHA} \end{cases} \tag{3.29}$$

Determiniamo i massimi teorici di capacità differenziando rispetto a $G$:

$$\frac{dS_{\text{pure}}}{dG} = e^{-2G}(1 - 2G) = 0 \implies G^* = \frac{1}{2} = 0{,}5 \implies S_{\max}^{\text{pure}} = \frac{1}{2e} \approx \mathbf{0{,}1839} \quad (\mathbf{18{,}4}\%) \tag{3.30}$$

$$\frac{dS_{\text{slotted}}}{dG} = e^{-G}(1 - G) = 0 \implies G^* = 1{,}0 \implies S_{\max}^{\text{slotted}} = \frac{1}{e} \approx \mathbf{0{,}3679} \quad (\mathbf{36{,}8}\%) \tag{3.31}$$

La Figura 3.15 illustra l'andamento a confronto della probabilità di collisione $P_C(G)$ e del throughput $S(G)$.




**(a) Probabilità di Collisione $P_C$** **(b) Throughput Effettivo $S$**

Figura 3.15: Confronto analitico tra ALOHA Puro e Slotted ALOHA in funzione del carico offerto $G$: (a) Probabilità di collisione $P_C(G)$ con evidenza dei valori tabulati per $\lambda \in \{0{,}1;\ 0{,}2;\ 0{,}5\}$; (b) Throughput normalizzato $S(G)$, con raddoppio del picco di efficienza da 18,4% a 36,8%.

**Quesito (c) — Popolazione Finita Simmetrica (2 Stazioni Gemelle):** Consideriamo ora il canale conteso esclusivamente da due stazioni distinte, $S_1$ ed $S_2$. Dobbiamo formalizzare la probabilità condizionata di successo per una specifica stazione. Quando la stazione $S_1$ ha generato una trama e tenta di trasmetterla, la sua trasmissione ha successo se e solo se la stazione rivale $S_2$ non genera alcuna trama all'interno della finestra di vulnerabilità $T_v$:

$$P_{S,1} = P[K_2 = 0 \text{ in } T_v \mid S_1 \text{ trasmette}] \tag{3.32}$$

Poiché i processi di generazione delle due stazioni sono **stocasticamente indipendenti**, la condizione che $S_1$ stia trasmettendo non altera il processo degli arrivi di $S_2$. Pertanto:

$$P_{S,1} = P[K_2 = 0 \text{ in } T_v] = e^{-\lambda_2 T_v} \tag{3.33}$$

Questo dimostra il **principio fondamentale della contesa**: *la probabilità di collisione subita da una stazione non dipende dal proprio tasso di trasmissione, bensì unicamente dal traffico generato dalla controparte che le contende il canale.*
Essendo le stazioni identiche ($\lambda_1 = \lambda_2 = \lambda$), si ha per simmetria $P_{C,1} = P_{C,2} = 1 - e^{-\lambda T_v}$. I valori di $P_C$ coincidono numericamente con quelli ottenuti al punto (a):

- ALOHA Puro ($T_v = 2T$): $P_C = 18{,}13\%$ per $\lambda = 0{,}1$; $32{,}97\%$ per $\lambda = 0{,}2$; $63{,}21\%$ per $\lambda = 0{,}5$.
- Slotted ALOHA ($T_v = T$): $P_C = 9{,}52\%$ per $\lambda = 0{,}1$; $18{,}13\%$ per $\lambda = 0{,}2$; $39{,}35\%$ per $\lambda = 0{,}5$.

Tuttavia, la distinzione architetturale è profonda: al punto (a) $\lambda$ rappresentava il carico totale generato da un'infinità di stazioni aggregate, mentre in questo scenario ciascuna stazione genera singolarmente $\lambda$ e il canale sostiene un carico totale offerto $G_{\text{tot}} = \lambda_1 T + \lambda_2 T = 2\lambda$.

**Quesito (d) — Asimmetria di Traffico e Paradosso di Contesa (3 Stazioni):** Introduciamo tre stazioni con tassi di generazione fortemente eterogenei:

$$\lambda_1 = 0{,}05\,\text{trame/s} \quad \text{(timida)}, \quad \lambda_2 = 0{,}10\,\text{trame/s} \quad \text{(media)}, \quad \lambda_3 = 0{,}20\,\text{trame/s} \quad \text{(loquace)} \tag{3.34}$$

Per il **teorema di sovrapposizione dei processi di Poisson**, la somma di processi di Poisson indipendenti con parametri $\lambda_a$ e $\lambda_b$ è un processo di Poisson di parametro $\lambda_{a+b} = \lambda_a + \lambda_b$. Quando una stazione $S_i$ trasmette, il traffico aggregato delle stazioni rivali che possono produrre collisione è $\lambda_{\text{rival},i} = \sum_{j \neq i} \lambda_j$. Di conseguenza:

$$P_{S,i} = e^{-\left(\sum_{j\neq i}\lambda_j\right)T_v} \implies P_{C,i} = 1 - e^{-\left(\sum_{j\neq i}\lambda_j\right)T_v} \tag{3.35}$$

Sviluppiamo analiticamente per le tre stazioni:
1. **Stazione $S_1$ ($\lambda_1 = 0{,}05$):** Compete contro $S_2$ ed $S_3$, il cui tasso congiunto vale $\lambda_{2+3} = \lambda_2 + \lambda_3 =$

$0{,}10 + 0{,}20 = 0{,}30$ trame/s.

$$\text{Pure ALOHA: } P_{S,1} = e^{-2\cdot 0{,}30\cdot 1} = e^{-0{,}60} \approx 0{,}54881 \implies P_{C,1} = 1 - 0{,}54881 = \mathbf{45{,}12}\%$$
$$\text{Slotted ALOHA: } P_{S,1} = e^{-1\cdot 0{,}30\cdot 1} = e^{-0{,}30} \approx 0{,}74082 \implies P_{C,1} = 1 - 0{,}74082 = \mathbf{25{,}92}\%$$

2. **Stazione $S_2$ ($\lambda_2 = 0{,}10$):** Compete contro $S_1$ ed $S_3$, con tasso $\lambda_{1+3} = \lambda_1 + \lambda_3 = 0{,}05 + 0{,}20 = 0{,}25$ trame/s.

$$\text{Pure ALOHA: } P_{S,2} = e^{-2\cdot 0{,}25\cdot 1} = e^{-0{,}50} \approx 0{,}60653 \implies P_{C,2} = 1 - 0{,}60653 = \mathbf{39{,}35}\%$$
$$\text{Slotted ALOHA: } P_{S,2} = e^{-1\cdot 0{,}25\cdot 1} = e^{-0{,}25} \approx 0{,}77880 \implies P_{C,2} = 1 - 0{,}77880 = \mathbf{22{,}12}\%$$

3. **Stazione $S_3$ ($\lambda_3 = 0{,}20$):** Compete contro $S_1$ ed $S_2$, con tasso $\lambda_{1+2} = \lambda_1 + \lambda_2 = 0{,}05 + 0{,}10 = 0{,}15$ trame/s.

$$\text{Pure ALOHA: } P_{S,3} = e^{-2\cdot 0{,}15\cdot 1} = e^{-0{,}30} \approx 0{,}74082 \implies P_{C,3} = 1 - 0{,}74082 = \mathbf{25{,}92}\%$$
$$\text{Slotted ALOHA: } P_{S,3} = e^{-1\cdot 0{,}15\cdot 1} = e^{-0{,}15} \approx 0{,}86071 \implies P_{C,3} = 1 - 0{,}86071 = \mathbf{13{,}93}\%$$

*Interpretazione Fisica e Paradosso di Contesa:*
Emerge un risultato solo in apparenza contro-intuitivo: **la stazione "timida" $S_1$, che emette meno traffico in assoluto, subisce la probabilità di collisione più alta (45,12% in Pure ALOHA), mentre la stazione "loquace" $S_3$ sperimenta la collisione più bassa (25,92%)**. La ragione fisica risiede nella natura non coordinata dell'accesso casuale: quando la stazione timida $S_1$ decide di parlare, si affaccia su un canale costantemente occupato dalle intense trasmissioni altrui ($\lambda_2 + \lambda_3 = 0{,}30$). Al contrario, quando trasmette $S_3$, le restanti stazioni sono per gran parte del tempo in silenzio ($\lambda_1 + \lambda_2 = 0{,}15$). Nei protocolli ALOHA, **le stazioni aggressive proteggono il proprio canale monopolizzando il mezzo a danno delle stazioni sporadiche**, introducendo una severa iniquità di servizio (*unfairness*) che rende indispensabili protocolli con rilevazione della portante (CSMA) o meccanismi di coordinamento dinamico.

**Quesito (e) — Throughput per Singola Stazione e Aggregato:** Il throughput normalizzato utile $S_i$ esprime la frazione di tempo in cui la stazione $i$ trasmette trame che giungono integre a destinazione:

$$S_i = \lambda_i \cdot P_{S,i} = \lambda_i(1 - P_{C,i}) \tag{3.36}$$

- **Scenario (c) — 2 Stazioni Gemelle ($\lambda_1 = \lambda_2 = \lambda$):**

$$\lambda = 0{,}1: \quad S_{\text{each}}^{\text{pure}} = 0{,}1 \cdot 0{,}81873 = 0{,}0819 \implies S_{\text{tot}}^{\text{pure}} = 2 \cdot 0{,}0819 = \mathbf{0{,}1637}$$
$$S_{\text{each}}^{\text{slotted}} = 0{,}1 \cdot 0{,}90484 = 0{,}0905 \implies S_{\text{tot}}^{\text{slotted}} = 2 \cdot 0{,}0905 = \mathbf{0{,}1810}$$
$$\lambda = 0{,}2: \quad S_{\text{each}}^{\text{pure}} = 0{,}2 \cdot 0{,}67032 = 0{,}1341 \implies S_{\text{tot}}^{\text{pure}} = 2 \cdot 0{,}1341 = \mathbf{0{,}2681}$$
$$S_{\text{each}}^{\text{slotted}} = 0{,}2 \cdot 0{,}81873 = 0{,}1637 \implies S_{\text{tot}}^{\text{slotted}} = 2 \cdot 0{,}1637 = \mathbf{0{,}3275}$$
$$\lambda = 0{,}5: \quad S_{\text{each}}^{\text{pure}} = 0{,}5 \cdot 0{,}36788 = 0{,}1839 \implies S_{\text{tot}}^{\text{pure}} = 2 \cdot 0{,}1839 = \mathbf{0{,}3679}$$
$$S_{\text{each}}^{\text{slotted}} = 0{,}5 \cdot 0{,}60653 = 0{,}3033 \implies S_{\text{tot}}^{\text{slotted}} = 2 \cdot 0{,}3033 = \mathbf{0{,}6065}$$

  Si noti come per $\lambda = 0{,}5$ in Slotted ALOHA il throughput aggregato raggiunga il 60,65%, ben superiore al massimo asintotico a popolazione infinita ($1/e \approx 36{,}79\%$), proprio perché con due sole stazioni la contesa è limitata.
- **Scenario (d) — 3 Stazioni Asimmetriche:**

– In **ALOHA Puro**:

$$S_1 = 0{,}05 \cdot 0{,}54881 = \mathbf{0{,}02744} \quad (2{,}74\% \text{ del canale})$$
$$S_2 = 0{,}10 \cdot 0{,}60653 = \mathbf{0{,}06065} \quad (6{,}07\% \text{ del canale})$$
$$S_3 = 0{,}20 \cdot 0{,}74082 = \mathbf{0{,}14816} \quad (14{,}82\% \text{ del canale})$$
$$S_{\text{tot}}^{\text{pure}} = S_1 + S_2 + S_3 = 0{,}02744 + 0{,}06065 + 0{,}14816 = \mathbf{0{,}23626} \quad (\mathbf{23{,}63}\%)$$

– In **Slotted ALOHA**:

$$S_1 = 0{,}05 \cdot 0{,}74082 = \mathbf{0{,}03704} \quad (3{,}70\% \text{ del canale})$$
$$S_2 = 0{,}10 \cdot 0{,}77880 = \mathbf{0{,}07788} \quad (7{,}79\% \text{ del canale})$$
$$S_3 = 0{,}20 \cdot 0{,}86071 = \mathbf{0{,}17214} \quad (17{,}21\% \text{ del canale})$$
$$S_{\text{tot}}^{\text{slotted}} = S_1 + S_2 + S_3 = 0{,}03704 + 0{,}07788 + 0{,}17214 = \mathbf{0{,}28706} \quad (\mathbf{28{,}71}\%)$$

**Quesito (f) — Dimensionamento della Capacità di Canale per 5 Mbit/s:** Se il canale fisico broadcast possiede una capacità trasmissiva lorda pari a $C$ [Mbit/s], la velocità utile effettiva goduta dalla generica stazione $i$ vale:

$$R_{\text{eff},i} = C \cdot S_i \quad [\text{Mbit/s}] \tag{3.37}$$

La specifica di progetto impone che **ciascuna** stazione debba disporre di almeno 5 Mbit/s utili:

$$\forall i, \quad R_{\text{eff},i} \geq 5\,\text{Mbit/s} \iff C \cdot S_i \geq 5\,\text{Mbit/s} \iff C \geq \frac{5\,\text{Mbit/s}}{S_i} \tag{3.38}$$

Per garantire il vincolo a tutte le stazioni contemporaneamente, la capacità fisica del mezzo deve essere dimensionata sul caso peggiore, ossia sulla stazione che ottiene il throughput normalizzato **minimo** (il cosiddetto "collo di bottiglia" del sistema):

$$C_{\min} = \frac{5\,\text{Mbit/s}}{\min_i(S_i)} \tag{3.39}$$

Calcoliamo la capacità richiesta nei vari scenari:

- **2 Stazioni Gemelle (Caso Simmetrico):** Essendo $S_1 = S_2 = S_{\text{each}}$, la capacità minima vale:
  – Per $\lambda = 0{,}1$:

$$\text{Pure ALOHA: } C \geq \frac{5}{0{,}081873} \approx \mathbf{61{,}07}\,\text{Mbit/s}$$
$$\text{Slotted ALOHA: } C \geq \frac{5}{0{,}090484} \approx \mathbf{55{,}26}\,\text{Mbit/s}$$

  – Per $\lambda = 0{,}2$:

$$\text{Pure ALOHA: } C \geq \frac{5}{0{,}134064} \approx \mathbf{37{,}30}\,\text{Mbit/s}$$
$$\text{Slotted ALOHA: } C \geq \frac{5}{0{,}163746} \approx \mathbf{30{,}54}\,\text{Mbit/s}$$

  – Per $\lambda = 0{,}5$:

$$\text{Pure ALOHA: } C \geq \frac{5}{0{,}183940} \approx \mathbf{27{,}18}\,\text{Mbit/s}$$
$$\text{Slotted ALOHA: } C \geq \frac{5}{0{,}303265} \approx \mathbf{16{,}49}\,\text{Mbit/s}$$

- **3 Stazioni Asimmetriche:** La stazione penalizzata è la stazione "timida" $S_1$, che presenta il

throughput utile normalizzato minimo ($\min_i S_i = S_1$):

$$\text{Pure ALOHA: } \min_i(S_i) = S_1 = 0{,}027441 \implies C \geq \frac{5\,\text{Mbit/s}}{0{,}027441} \approx \mathbf{182{,}21}\,\text{Mbit/s}$$

$$\text{Slotted ALOHA: } \min_i(S_i) = S_1 = 0{,}037041 \implies C \geq \frac{5\,\text{Mbit/s}}{0{,}037041} \approx \mathbf{134{,}99}\,\text{Mbit/s}$$

Se si valutassero le stazioni individualmente:

- Per $S_2$: in Pure ALOHA basterebbero 5/0,06065 = 82,44 Mbit/s; in Slotted 5/0,07788 = 64,20 Mbit/s.
- Per $S_3$: in Pure ALOHA basterebbero 5/0,14816 = 33,75 Mbit/s; in Slotted 5/0,17214 = 29,05 Mbit/s.

**Quadro Sinottico Comparativo dei Risultati:** La Tabella 3.3 riassume e pone a diretto confronto tutti i parametri prestazionali calcolati nel presente esercizio unificato.

| **Scenario / Stazione** | **Carico Rivali $\lambda_r$** | | **Prob. Coll. $P_C$ [%]** | | **Throughput $S_i$** | | **Capacità Min. $C$ [Mbit/s]** | |
|---|---|---|---|---|---|---|---|---|
| | **Pure** | **Slotted** | **Pure** | **Slotted** | **Pure** | **Slotted** | **Pure** | **Slotted** |
| *Popolazione Infinita ($N \to \infty$) o 2 Stazioni Gemelle Simmetriche:* | | | | | | | | |
| $\lambda = 0{,}10$ trame/s | 0,10 | 0,10 | 18,13% | 9,52% | 0,0819 | 0,0905 | 61,07 | 55,26 |
| $\lambda = 0{,}20$ trame/s | 0,20 | 0,20 | 32,97% | 18,13% | 0,1341 | 0,1637 | 37,30 | 30,54 |
| $\lambda = 0{,}50$ trame/s | 0,50 | 0,50 | 63,21% | 39,35% | 0,1839 | 0,3033 | 27,18 | 16,49 |
| *3 Stazioni Asimmetriche ($\lambda_1 = 0{,}05$, $\lambda_2 = 0{,}10$, $\lambda_3 = 0{,}20$ trame/s):* | | | | | | | | |
| $S_1$ ("timida", $\lambda_1 = 0{,}05$) | 0,30 | 0,30 | **45,12%** | **25,92%** | 0,0274 | 0,0370 | **182,21** | **134,99** |
| $S_2$ ("media", $\lambda_2 = 0{,}10$) | 0,25 | 0,25 | 39,35% | 22,12% | 0,0607 | 0,0779 | 82,44 | 64,20 |
| $S_3$ ("loquace", $\lambda_3 = 0{,}20$) | 0,15 | 0,15 | 25,92% | 13,93% | 0,1482 | 0,1721 | 33,75 | 29,05 |
| **Aggregato Asimmetrico** | $G_{\text{tot}} = 0{,}35$ | | — | | **0,2363** | **0,2871** | **Dimensionato su $S_1$** | |

Tabella 3.3: Quadro sinottico comparativo delle prestazioni e dimensionamento di ALOHA Puro e Slotted ALOHA.

**Insegnamenti Progettuali sui Protocolli ad Accesso Casuale**

Dallo svolgimento analitico e dal dimensionamento di capacità emergono tre principi architetturali fondamentali:

1. **Il beneficio della sincronizzazione temporale:** La suddivisione del canale in slot (Slotted ALOHA) dimezza la finestra di vulnerabilità ($2T \to T$), raddoppiando l'efficienza massima teorica dal 18,4% al 36,8%. Nei sistemi a capacità dimensionata, ciò si traduce in un risparmio immediato di decine di Mbit/s di banda fisica.
2. **Il costo occulto dell'asimmetria (Overprovisioning Penalty):** In assenza di rilevazione della portante o di controllo centralizzato, garantire una qualità di servizio garantita (5 Mbit/s) a tutte le stazioni impone di sovradimensionare il canale fino a oltre 182 Mbit/s esclusivamente per compensare l'inefficienza e le collisioni subite dalla stazione meno attiva.
3. **La necessità del Carrier Sense in ambito locale:** Nei sistemi in cui il ritardo di propagazione del mezzo è inferiore alla durata della trama ($\tau \ll T$), è inaccettabile trasmettere ciecamente. Ascoltare il canale prima di trasmettere (*listen before talk*) consente di eliminare la gran parte delle collisioni provocate dalle stazioni rivali: questa è la motivazione architetturale che porta all'introduzione di **CSMA**.

## 3.13 Carrier Sense Multiple Access (CSMA)

Nei sistemi in cui il ritardo di propagazione è inferiore alla durata della trama ($\tau < T$), le stazioni possono migliorare radicalmente l'efficienza introducendo il principio dell'**ascolto prima di parlare** (*listen before talk*): il **Carrier Sense Multiple Access** (CSMA).

### 3.13.1 Principio Operativo del Carrier Sense

Le stazioni dotate di interfaccia CSMA sono in grado di "ascoltare" il mezzo trasmissivo prima di emettere bit:

- Misurano la potenza e l'energia presente sul canale: per rilevare la portante occorre compiere l'integrale nel tempo della potenza, operazione che richiede un tempo fisico non nullo $T_a$.
- Se il canale risulta **libero**, la stazione inizia a trasmettere.
- Se il canale risulta **occupato**, la stazione deve rinviare la trasmissione.

### 3.13.2 Varianti di Persistenza del CSMA

In funzione di come una stazione si comporta una volta rilevato il canale occupato, si distinguono tre varianti di persistenza:

**1. CSMA Non-Persistente (0-Persistent):** Se il canale è occupato, la stazione rinuncia immediatamente e programma una nuova verifica dopo un intervallo di tempo casuale $t \gg T$.

- *Vantaggio*: riduce drasticamente le collisioni tra stazioni che attendono la liberazione del canale;
- *Svantaggio*: introduce ritardi di accesso anche quando il canale si libera rapidamente, determinando tempi morti di inattività.

**2. CSMA 1-Persistente (1-Persistent):** La stazione rimane continuamente in ascolto del canale occupato. Nel momento esatto in cui percepisce che il canale torna libero, **trasmette immediatamente con probabilità pari a** 1 ($p = 1$).

- *Vantaggio*: latenza minima a carico basso; non spreca tempo di canale libero.
- *Vulnerabilità Catastrofica (Collisione Deterministica)*: se due o più stazioni generano una trama durante la trasmissione di una terza stazione, entrambe rimangono in attesa del rilascio; appena il canale diventa libero, entrambe trasmettono all'istante, **collidendo deterministicamente con probabilità del 100%**!

**3. CSMA $p$-Persistente ($p$-Persistent):** Costituisce una generalizzazione per sistemi a slot di contesa, analizzata nel 1975 da Kleinrock e Tobagi. Quando il canale diventa libero:

- La stazione trasmette con probabilità $p$;
- Con probabilità $1 - p$, rinvia la trasmissione allo slot di contesa successivo (di durata micro-slot $\ll T$).

Abbassando il valore di $p$, si riducono le collisioni ma cresce il ritardo medio di accesso al canale.

### 3.13.3 Il Ritardo di Vulnerabilità e il Parametro Adimensionale $a$

Il sensing non azzera completamente le collisioni a causa del **tempo finito di propagazione del segnale** $\tau$. Se una stazione $A$ inizia a trasmettere all'istante $t_0$, il segnale impiega un tempo $\tau$ per raggiungere la stazione $B$ all'altro estremo del cavo, e a $B$ serve un tempo $T_a$ per integrare la potenza. Se $B$ verifica il canale nell'intervallo $[t_0, t_0 + \tau + T_a]$, troverà il mezzo apparentemente libero e inizierà a trasmettere, innescando la collisione.

Il periodo di vulnerabilità in CSMA vale:

$$T_{\text{vulnerabilità, CSMA}} = \tau + T_a \tag{3.40}$$

Si definisce il **parametro adimensionale di efficienza** $a$:

$$a = \frac{\tau + T_a}{T} \approx \frac{\tau}{T} = \frac{T_{\text{propagazione}}}{T_{\text{trasmissione}}} \tag{3.41}$$

**Principio / Fondamento: Ruolo del Parametro $a$ e Natura della Collisione al Ricevitore**

Un concetto cardine è che **la collisione è un evento che si consuma esclusivamente al ricevitore, non nel mezzo fisico**: due onde elettromagnetiche che coesistono nello spazio non si scontrano come corpi rigidi, ma si sovrappongono linearmente; è sull'antenna e nei circuiti d'ingresso del ricevitore che la somma dei segnali satura l'elettronica e rende inintelligibili i bit.
L'efficacia del carrier sense locale al trasmettitore dipende criticamente dal parametro $a$:

- **Se** $a \to 0$ ($T_{\text{prop}} \ll T_{\text{tx}}$): il segnale si propaga quasi istantaneamente rispetto alla durata della trama. Il periodo di vulnerabilità è trascurabile e l'efficienza teorica del CSMA sfiora il 100% (reti fisicamente compatte o pacchetti lunghi a bit-rate contenuto).
- **Se** $a \geq 1$ ($T_{\text{prop}} \geq T_{\text{tx}}$): una trama termina di essere emessa prima ancora che il fronte d'onda raggiunga il destinatario. L'ascolto locale non fornisce alcuna informazione su eventuali trame già in volo emesse da stazioni remote (scenario tipico delle grandi distanze, come i collegamenti storici di Abramson tra le isole Hawaii o i canali satellitari); analogamente, in presenza di ostacoli o antenne direzionali (problema del *nodo nascosto*), stazioni che non si avvertono a vicenda collidono sul ricevitore comune. In tali condizioni il sensing è cieco e il CSMA decade verso ALOHA puro.

### 3.13.4 Il Limite Fisico del Sensing: Perché Collision Detection è Impossibile su Wi-Fi

Mentre sui cavi coassiali il segnale è confinato e la potenza subisce un'attenuazione modesta, sulle trasmissioni radio wireless opera la legge di attenuazione con la distanza:

$$P_{\text{rx}} \propto \frac{P_{\text{tx}}}{d^n}, \quad n \geq 2 \tag{3.42}$$

La potenza del segnale emesso dall'antenna della stazione locale supera di decine di migliaia di volte la potenza ricevuta da una stazione distante sul medesimo canale radio (rapporti di potenza spesso superiori a 10.000 : 1, ossia oltre 40–60 dB).

Per rilevare una collisione durante la trasmissione via radio, la scheda dovrebbe essere in grado di rilevare una variazione energetica infinitesima dello 0,01% sovrapposta al segnale gigantesco emesso dal proprio trasmettitore. Nemmeno i moderni prototipi di radio full-duplex con cancellazione analogica e numerica del segnale consentono oggi di realizzare un Collision Detection affidabile ed economico a livello industriale.

Per questa ragione insuperabile della fisica elettromagnetica:

- **Sulle reti cablate (Ethernet)**: è possibile implementare il **CSMA/CD** (*Collision Detection*);
- **Sulle reti wireless (Wi-Fi 802.11)**: è categoricamente impossibile usare Collision Detection. Il Wi-Fi adotta perciò il **CSMA/CA** (*Collision Avoidance*), che evita le collisioni a priori mediante intertempi casuali (DIFS, SIFS), contesa a slot temporali e riscontro esplicito (ACK).

### 3.13.5 Modellazione Analitica Unificata e Prestazioni di CSMA (0-p, 1-p, p-p)

Nel panorama dei protocolli ad accesso multiplo casuale, l'ascolto preventivo della portante (*Carrier Sense*) segna il passaggio concettuale e prestazionale dai meccanismi "ciechi" privi di coordinamento (ALOHA puro e Slotted ALOHA) a protocolli cooperativi capaci di conseguire efficienze prossime al 100% su canali con basso ritardo di propagazione relativo.

La seguente trattazione unifica l'analisi del controllo d'accesso al mezzo con rilevazione di portante, formalizzando in un unico quadro analitico e computazionale le tre varianti operative: **CSMA 0-persistente**, **CSMA 1-persistente** e **CSMA $p$-persistente**. Vengono derivati con rigore i parametri fisici e temporali, la finestra di vulnerabilità $T_v$, il parametro di Kleinrock $a$, la partizione dello stato del canale in cicli di rigenerazione, la dimostrazione mediante serie di potenze della sequenza di trame occupate, le curve di collisione e throughput, fino alla risoluzione formale del noto paradosso asintotico a carico infinito.

**Esercizio d'Esame: Analisi Prestazionale Completa e Modellazione Analitica dei Protocolli CSMA (0-Persistente, 1-Persistente e $p$-Persistente)**

**Enunciato del Problema Unificato:**
Si consideri un mezzo trasmissivo broadcast condiviso ad accesso casuale operante con protocollo **CSMA** (*Carrier Sense Multiple Access*). Si intende studiare e confrontare analiticamente le prestazioni delle configurazioni a persistenza nulla (0-p), persistenza unitaria (1-p) e persistenza frazionaria ($p$-p), affrontando

in modo organico i seguenti quesiti:

**(a) Parametri Fisici, Tempi Caratteristici e Finestra di Vulnerabilità (CSMA 0-p):** Si consideri una prima rete con capacità trasmissiva di canale $C_1 = 10\,\text{Mbit/s}$, trame informative a lunghezza fissa $L_1 = 1000\,\text{byte}$, lunghezza del cavo coassiale $d_1 = 1500\,\text{m}$ e velocità di propagazione del segnale elettromagnetico nel mezzo $V_p = 0{,}5\,c = 1{,}5 \cdot 10^8\,\text{m/s}$. Si calcoli la durata di emissione della trama $T$, il ritardo di propagazione massimo unidirezionale $\tau_p$, il tempo di integrazione per il rilevamento della portante $t_a = \tau_p$, la finestra temporale di vulnerabilità $T_v$ e il parametro adimensionale di Kleinrock $a = T_v/T$.

**(b) Probabilità di Collisione in CSMA 0-Persistente al Variare del Carico:** Assumendo un processo stocastico aggregato di generazione delle trame di tipo Poisson di intensità media $\lambda \in \{10, 100, 1000\}\,\text{trame/s}$, si formalizzi analiticamente la probabilità di collisione $P_c(\lambda)$ per il protocollo CSMA 0-persistente e se ne calcolino i valori numerici esatti. Si esprima quindi $P_c$ in funzione del carico normalizzato offerto al canale $G = \lambda T$ e del parametro $a$.

**(c) Modello a Ciclo di Rigenerazione e Throughput di CSMA 0-Persistente:** Sotto l'ipotesi semplificativa di sensing ideale ($\tau_p \simeq 0$, $t_a \simeq 0 \implies T_v \simeq 0$), si modelli il canale come un processo rigenerativo ad alternanza di stati: periodi di trasmissione utile ($T_O = T$) e periodi di canale libero (*idle*, $T_L$). Sfruttando la proprietà di assenza di memoria della distribuzione esponenziale degli intertempi di Poisson, si calcoli la durata media del periodo libero $E[T_L]$, la durata media del ciclo $E[T_{\text{ciclo}}]$ e si deduca la formula chiusa del throughput normalizzato $S_{0p}(G)$, discutendone il comportamento al limite per $G \to \infty$.

**(d) Parametri Fisici e Ipotesi Semplificative per CSMA 1-Persistente:** Si consideri ora un secondo canale a capacità raddoppiata $C_2 = 20\,\text{Mbit/s}$, lunghezza di tratta ridotta $d_2 = 1000\,\text{m}$ ($V_p = 1{,}5 \cdot 10^8\,\text{m/s}$) e trame di dimensione $L_2 = 1500\,\text{byte}$. Si calcoli la nuova durata di trama $T_2$ e il ritardo di propagazione $\tau_{p,2}$. Si enuncino le ipotesi ideali di sensing perfetto e di rilevazione istantanea delle collisioni (*ideal Collision Detection*) adottate nel modello CSMA 1-persistente.

**(e) Scomposizione Concettuale a Due Stati del Canale:** Si partizioni l'istante di arrivo di una generica trama nei due stati mutualmente esclusivi: canale libero ($L$) con frazione temporale $f_L$, e canale occupato ($O$) con frazione temporale $f_O = 1 - f_L$. Applicando il teorema della probabilità totale, si dimostri la relazione di principio $P_c = f_O \cdot (1 - e^{-G})$.

**(f) Valutazione di Collisione in CSMA 1-Persistente e Risultati Numerici:** Senza impantanarsi nei tediosi passaggi della serie geometrica d'occupazione (riservati all'approfondimento opzionale per i curiosi), si applichi l'espressione chiusa della frazione di canale occupato $f_O(G) = \frac{G}{G+(1-Ge^{-G})^2}$ per valutare la probabilità di collisione $P_c(G)$ a $\lambda \in \{10, 100, 1000\}\,\text{trame/s}$.

**(g) Throughput di CSMA 1-Persistente e Risoluzione del Paradosso Asintotico (Domanda d'Esame):** Si esprima il throughput utile normalizzato $S_{1p}(G) = G(1 - P_c)$ e si analizzi il comportamento asintotico per $G \to \infty$. Si evidenzi l'apparente paradosso matematico ($S_{1p} \to 1$ nonostante $P_c \to 1$) e si illustri come rispondere formalmente all'orale d'esame.

**(h) CSMA $p$-Persistente come Sintesi Convessa e Trade-off Operativo:** Si formalizzi il throughput del CSMA $p$-persistente come combinazione convessa delle varianti estreme, $S_p = pS_{1p} + (1-p)S_{0p}$. Si commenti il bilanciamento architetturale tra l'aggressività della variante 1-persistente a basso carico e la stabilità conservativa della variante 0-persistente ad alto carico.

---

**Svolgimento Dettagliato e Metodologia Risolutiva:**

**Quesito (a) — Parametri Fisici, Tempi Caratteristici e Finestra di Vulnerabilità:** I parametri dimensionali del primo collegamento sono:

$$C_1 = 10\,\text{Mbit/s} = 10 \cdot 10^6\,\text{bit/s} \tag{3.43}$$

$$L_1 = 1000\,\text{byte} = 1000 \cdot 8\,\text{bit} = 8000\,\text{bit} \tag{3.44}$$

$$d_1 = 1500\,\text{m} \tag{3.45}$$

$$V_p = 0{,}5\,c = 0{,}5 \cdot 3 \cdot 10^8\,\text{m/s} = 1{,}5 \cdot 10^8\,\text{m/s} \tag{3.46}$$

Il **tempo di trasmissione della trama** (tempo necessario al trasmettitore per iniettare l'intera sequenza di bit nel canale alla velocità di cifra di picco) vale:

$$T_1 = \frac{L_1}{C_1} = \frac{8000\,\text{bit}}{10 \cdot 10^6\,\text{bit/s}} = 8 \cdot 10^{-4}\,\text{s} = 0{,}8\,\text{ms} = 800\,\mu\text{s} \tag{3.47}$$

Il **ritardo di propagazione fisico** tra i nodi posti alle estremità del cavo coassiale ($d_1 = 1500\,\text{m}$) è:

$$\tau_p = \frac{d_1}{V_p} = \frac{1500\,\text{m}}{1{,}5 \cdot 10^8\,\text{m/s}} = 10^{-5}\,\text{s} = 10\,\mu\text{s} \tag{3.48}$$

L'ascolto della portante (*carrier sensing*) non è istantaneo: per distinguere in modo deterministico la presenza di un segnale modulato rispetto al rumore termico gaussiano di fondo, la circuiteria analogica di ricezione necessita di un tempo finito di integrazione e filtraggio $t_a$. Nel dimensionamento standard di progetto si assume:

$$t_a = \tau_p = 10\,\mu\text{s} \tag{3.49}$$

La **finestra temporale di vulnerabilità** $T_v$ rappresenta l'intervallo temporale a partire dall'inizio della trasmissione di una stazione durante il quale una seconda stazione remota, ascoltando il canale, non percepisce ancora il fronte d'onda del segnale in arrivo (a causa del tempo di propagazione $\tau_p$) oppure non ha ancora completato l'integrazione del rilevatore ($t_a$). Tale finestra corrisponde esattamente alla somma dei due ritardi:

$$T_v = \tau_p + t_a = 10\,\mu\text{s} + 10\,\mu\text{s} = 20\,\mu\text{s} = 2 \cdot 10^{-5}\,\text{s} = 2\tau_p \tag{3.50}$$

Il **parametro adimensionale di Kleinrock** $a$, rapporto fondamentale di efficienza per ogni rete ad accesso multiplo, misura la frazione del tempo di trama durante la quale il sistema rimane esposto a collisione:

$$a = \frac{T_v}{T_1} = \frac{20 \cdot 10^{-6}\,\text{s}}{800 \cdot 10^{-6}\,\text{s}} = \frac{20}{800} = \frac{1}{40} = 0{,}025 \tag{3.51}$$

Il valore $a = 0{,}025 = 2{,}5\%$ indica che la finestra di vulnerabilità costituisce soltanto una frazione minuscola della durata del pacchetto, condizione ideale che garantisce al CSMA un'efficacia di arbitraggio nettamente superiore rispetto ad ALOHA.

**Quesito (b) — Probabilità di Collisione in CSMA 0-Persistente:** Nel protocollo **CSMA 0-persistente**, una stazione che ha una trama da trasmettere ascolta preventivamente il mezzo:

- Se il canale è rilevato **libero**, inizia immediatamente la trasmissione;
- Se il canale è rilevato **occupato**, la stazione non insiste: rinuncia immediatamente alla trasmissione ed estrae un intervallo di ritardo casuale (*backoff*) prima di ripetere il carrier sensing.

Assumendo che il processo di generazione aggregato delle trame segua un processo di Poisson di intensità media $\lambda$ [trame/s], il numero di arrivi $K$ in una finestra temporale generica $\Delta t$ rispetta la distribuzione:

$$P[K = k \text{ in } \Delta t] = \frac{(\lambda \Delta t)^k}{k!} e^{-\lambda \Delta t} \tag{3.52}$$

Una stazione che trova il canale libero e comincia a trasmettere subisce una collisione se e solo se almeno un'altra stazione inizia a trasmettere all'interno della finestra di vulnerabilità $T_v$. Pertanto, la probabilità di successo è la probabilità di registrare 0 arrivi rivali in $T_v$, e la **probabilità di collisione** vale:

$$P_c(\lambda) = 1 - P[K = 0 \text{ in } T_v] = 1 - e^{-\lambda T_v} \tag{3.53}$$

Sostituendo i tre valori di intensità aggregata $\lambda \in \{10, 100, 1000\}$ trame/s con $T_v = 2 \cdot 10^{-5}$ s:

1. **Per $\lambda = 10$ trame/s:**

$$\lambda T_v = 10 \cdot 2 \cdot 10^{-5} = 2 \cdot 10^{-4} = 0{,}0002 \tag{3.54}$$

$$P_c = 1 - e^{-0{,}0002} \approx 1 - (1 - 0{,}0002) = 0{,}0002 = \mathbf{0{,}020}\% \quad (1{,}9998 \cdot 10^{-4}) \tag{3.55}$$

2. **Per $\lambda = 100$ trame/s:**

$$\lambda T_v = 100 \cdot 2 \cdot 10^{-5} = 2 \cdot 10^{-3} = 0{,}002 \tag{3.56}$$
$$P_c = 1 - e^{-0{,}002} = 1 - 0{,}998002 = 1{,}998 \cdot 10^{-3} = \mathbf{0{,}1998}\% \tag{3.57}$$

3. **Per $\lambda = 1000$ trame/s:**

$$\lambda T_v = 1000 \cdot 2 \cdot 10^{-5} = 2 \cdot 10^{-2} = 0{,}02 \tag{3.58}$$
$$P_c = 1 - e^{-0{,}02} = 1 - 0{,}980199 = 0{,}01980 = \mathbf{1{,}980}\% \tag{3.59}$$

Esprimendo la relazione in termini del **carico normalizzato offerto** $G = \lambda T_1$ (da cui $\lambda = G/T_1$):

$$\lambda T_v = \frac{G}{T_1} T_v = G\left(\frac{T_v}{T_1}\right) = a \cdot G = 0{,}025\, G \tag{3.60}$$

Otteniamo la formulazione canonica adimensionale della probabilità di collisione per il CSMA 0-persistente:

$$P_c(G) = 1 - e^{-aG} = 1 - e^{-0{,}025\, G} \tag{3.61}$$

Si noti come, anche sotto un carico normalizzato pari a $G = 1$ (dove in ALOHA Puro le collisioni superano il 86% e in Slotted ALOHA il 63%), nel CSMA 0-persistente la probabilità di collisione rimane contenuta ad appena $P_c(1) = 1 - e^{-0{,}025} \approx 2{,}47\%$.

**Quesito (c) — Modello a Ciclo di Rigenerazione e Throughput di CSMA 0-Persistente:** Sotto l'ipotesi teorica di **sensing ideale** ($\tau_p \to 0, t_a \to 0$, ossia $T_v \to 0$), il canale non sperimenta collisioni ($P_c \approx 0$) poiché qualunque stazione accende la portante in modo istantaneamente visibile a tutti i nodi.
Il comportamento dinamico del mezzo è descritto da un **processo di rigenerazione alternato**:

- Un **periodo di occupazione** (*busy period*, $T_O$), in cui una stazione trasmette la propria trama con successo per una durata fissa $T_O = T$;
- Un **periodo di canale libero** (*idle period*, $T_L$), tempo che intercorre tra la fine di una trasmissione e la generazione del successivo pacchetto da parte di qualsiasi stazione della rete.

Per la fondamentale **proprietà di assenza di memoria** (*memoryless*) della variabile aleatoria esponenziale che governa gli intertempi di un processo di Poisson a tasso aggregato $\lambda$, l'intertempo residuo tra la fine della trama precedente e il nuovo arrivo è anch'esso distribuito esponenzialmente con media:

$$E[T_L] = \frac{1}{\lambda} \tag{3.62}$$

Esprimendo $\lambda$ in funzione del carico normalizzato $G = \lambda T \iff \lambda = \frac{G}{T}$, la durata media dell'intertempo di silenzio diventa:

$$E[T_L] = \frac{T}{G} \tag{3.63}$$

La durata media di un intero ciclo di canale (da un inizio trasmissione al successivo) è la somma algebrica:

$$E[T_{\text{ciclo}}] = E[T_O] + E[T_L] = T + \frac{T}{G} = T\left(1 + \frac{1}{G}\right) = T\left(\frac{G+1}{G}\right) \tag{3.64}$$

Il **throughput utile normalizzato** $S_{0p}$ definisce la frazione media di tempo in cui il canale è utilizzato per trasportare informazione utile senza collisioni:

$$S_{0p}(G) = \frac{E[\text{tempo utile nel ciclo}]}{E[T_{\text{ciclo}}]} = \frac{T}{T\left(\frac{G+1}{G}\right)} = \frac{G}{G+1} \tag{3.65}$$

**Comportamento Asintotico al Limite per $G \to \infty$:**

$$\lim_{G\to\infty} S_{0p}(G) = \lim_{G\to\infty} \frac{G}{G+1} = \lim_{G\to\infty} \frac{1}{1+\frac{1}{G}} = \mathbf{1} \quad (\mathbf{100\%}) \tag{3.66}$$

*Spiegazione fisica del risultato:*
Nel CSMA 0-persistente a sensing ideale, una stazione che ascolta il canale occupato si astiene immediatamente dal trasmettere e attende un tempo di backoff casuale. Quando il carico $G$ cresce indefinitamente ($G \to \infty$), il tempo medio di canale inattivo crolla a zero ($E[T_L] = T/G \to 0$): non appena una trasmissione si conclude, c'è sempre almeno una stazione che ha completato il backoff pronta a trasmettere. Poiché l'ascolto istantaneo previene ogni sovrapposizione iniziale, il canale si trasforma in una sequenza continua e ininterrotta di trasmissioni utili senza alcun secondo sprecato per collisione o silenzio, raggiungendo l'efficienza asintotica unitaria (100%).

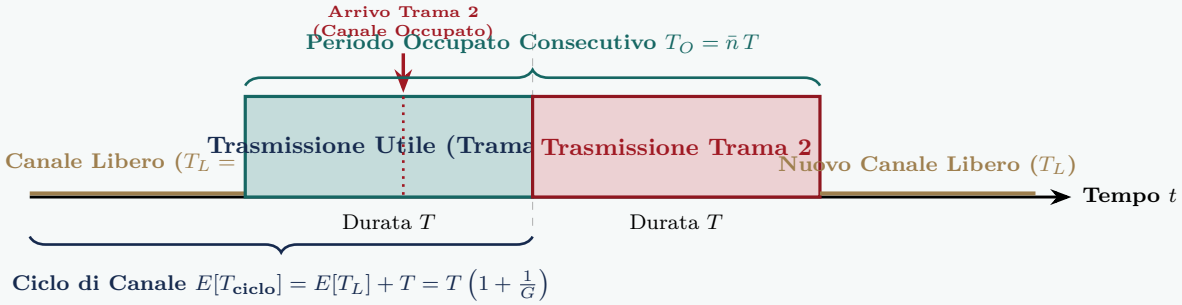



Figura 3.16: Scomposizione temporale del canale CSMA: alternanza tra periodo di silenzio stocastico $T_L$ e periodo di occupazione concatenato $T_O = \bar{n}T$.

**Quesito (d) — Parametri Fisici e Ipotesi Semplificative per CSMA 1-Persistente:** Per il secondo canale a velocità e taglia di trama superiori:

$$C_2 = 20\,\text{Mbit/s} = 20 \cdot 10^6\,\text{bit/s} \tag{3.67}$$
$$L_2 = 1500\,\text{byte} = 1500 \cdot 8\,\text{bit} = 12000\,\text{bit} \tag{3.68}$$
$$d_2 = 1000\,\text{m} \tag{3.69}$$
$$V_p = 1{,}5 \cdot 10^8\,\text{m/s} \tag{3.70}$$

La durata di emissione della nuova trama vale:

$$T_2 = \frac{L_2}{C_2} = \frac{12000\,\text{bit}}{20 \cdot 10^6\,\text{bit/s}} = 6 \cdot 10^{-4}\,\text{s} = 0{,}6\,\text{ms} = 600\,\mu\text{s} \tag{3.71}$$

Il ritardo di propagazione unidirezionale massimo è:

$$\tau_{p,2} = \frac{d_2}{V_p} = \frac{1000\,\text{m}}{1{,}5 \cdot 10^8\,\text{m/s}} = 6{,}67 \cdot 10^{-6}\,\text{s} = 6{,}67\,\mu\text{s} \tag{3.72}$$

Nel modello CSMA 1-persistente si assumono due **ipotesi ideali fondamentali**:

1. **Sensing Ideale ($T_v \to 0$):** Il ritardo di propagazione e il tempo di integrazione sono trascurabili rispetto alla durata della trama ($\tau_p \ll T$, qui $6{,}67\,\mu\text{s} \ll 600\,\mu\text{s}$). Pertanto, una trama generata quando il canale è libero inizia la trasmissione e tutte le stazioni della rete ne rilevano istantaneamente l'occupazione: la probabilità di collisione a canale libero è identicamente nulla ($P_{c,L} = 0$).
2. **Collision Detection Ideale / Durata di Contesa Vincolata:** Qualora si verifichi una collisione tra stazioni che hanno trovato il canale occupato e trasmettono simultaneamente al liberarsi del mezzo, il canale si libera esattamente dopo una durata pari al tempo di trama $T$ (oppure la contesa viene risolta in un slot di durata $T$).

**Quesito (e) — Scomposizione a Due Stati del Canale e Collisione in CSMA 1-Persistente:** Nel protocollo **CSMA 1-persistente**, una stazione che ascolta il canale procede come segue:

- Se il canale è **libero**, trasmette immediatamente con probabilità $p = 1$;
- Se il canale è **occupato**, la stazione non rinuncia né effettua backoff (differenza cardine rispetto al 0-persistente!): rimane in ascolto continuo della portante e, **nell'istante esatto in cui il canale si libera, trasmette immediatamente con probabilità** $p = 1$.

Partizionando l'istante di arrivo di una generica trama nei due stati mutualmente esclusivi ed esaustivi del canale:

- Stato $L$ (Libero / *Idle*), con frazione temporale $f_L$;
- Stato $O$ (Occupato / *Busy*), con frazione temporale $f_O = 1 - f_L$;

la probabilità complessiva di collisione $P_c$ si esprime tramite il **Teorema della Probabilità Totale**:

$$P_c = P_c(L) \cdot f_L + P_c(O) \cdot f_O \tag{3.73}$$

Poiché per l'ipotesi di sensing ideale non si verificano collisioni a canale libero, abbiamo:

$$P_c(L) = 0 \tag{3.74}$$

Valutiamo ora la probabilità condizionata di collisione $P_c(O)$ per una trama che arriva durante lo stato occupato $O$. Tale trama deferisce la propria trasmissione, mettendosi in attesa della fine della trama attualmente in transito di durata $T$. Quando il canale si libera, trasmette immediatamente. Si verificherà una collisione se e solo se, durante il medesimo intervallo di tempo $T$ della trasmissione in corso, è stata generata **almeno un'altra trama** da parte di qualunque altra stazione rivale, poiché tutte le stazioni accumulate trasmetteranno all'unisono al tempo $t = T$.
La probabilità che nessun'altra stazione si sia accodata durante $T$ è data dalla probabilità di 0 arrivi di Poisson:

$$P[\text{0 arrivi rivali in } T] = e^{-\lambda T} = e^{-G} \tag{3.75}$$

Di conseguenza, la probabilità condizionata che si verifichi una collisione allo sblocco del canale vale:

$$P_c(O) = 1 - P[\text{0 arrivi in } T] = 1 - e^{-G} \tag{3.76}$$

Sostituendo nella legge della probabilità totale, otteniamo la formulazione fondamentale:

$$P_c = 0 \cdot f_L + (1 - e^{-G}) \cdot f_O = \mathbf{f_O \cdot (1 - e^{-G})} \tag{3.77}$$

Questo elegante risultato dimostra che in CSMA 1-persistente la collisione non nasce dal sensing imperfetto, ma dall'**accumulo simultaneo di trasmettitori concorrenti durante i periodi di canale occupato**.

**Quesito (f) — Valutazione di Collisione in CSMA 1-Persistente e Risultati Numerici:** A differenza della variante 0-persistente (in cui ogni ciclo è composto da una singola trasmissione utile $T$ seguita dall'intertempo di riposo), nel CSMA 1-persistente la trasmissione al termine di una trama può prolungarsi continuamente a causa delle stazioni accumulate che trasmettono istantaneamente al liberarsi del canale.
Come formalizzato nella soluzione analitica ufficiale, la frazione stazionaria media di tempo in cui il canale risulta occupato ($f_O$) dipende dal carico normalizzato $G$ secondo la relazione:

$$f_O(G) = \frac{G}{G + (1 - Ge^{-G})^2} \tag{3.78}$$

Sostituendo tale frazione nella formula di probabilità totale ricavata nel quesito (e) ($P_c = f_O \cdot (1 - e^{-G})$), si ottiene la **formula chiusa definitiva di $P_c(G)$ per CSMA 1-persistente**:

$$P_c(G) = \frac{G\,(1 - e^{-G})}{G + (1 - Ge^{-G})^2} \tag{3.79}$$

**Valutazione Numerica per il Canale 2 ($C_2 = 20\,\mathbf{Mbit/s}$, $L_2 = 1500\,\mathbf{byte} \implies T_2 = 0{,}6\,\mathbf{ms}$):**

1. **Per $\lambda = 10\,\textbf{trame/s} \implies G = 10 \cdot 6 \cdot 10^{-4} = 0{,}006$:**

$$e^{-G} \approx 0{,}994018, \quad P[1] = Ge^{-G} \approx 0{,}005964 \implies P_c \approx \frac{3{,}589 \cdot 10^{-5}}{0{,}994107} \approx \mathbf{0{,}004}\% \tag{3.80}$$

2. **Per $\lambda = 100\,\textbf{trame/s} \implies G = 100 \cdot 6 \cdot 10^{-4} = 0{,}06$:**

$$e^{-G} \approx 0{,}941765, \quad P[1] = Ge^{-G} \approx 0{,}056506 \implies P_c \approx \frac{3{,}4941 \cdot 10^{-3}}{0{,}950181} \approx \mathbf{0{,}368}\% \tag{3.81}$$

3. **Per $\lambda = 1000\,\textbf{trame/s} \implies G = 1000 \cdot 6 \cdot 10^{-4} = 0{,}6$:**

$$e^{-G} \approx 0{,}548812, \quad P[1] = Ge^{-G} \approx 0{,}329287 \implies P_c \approx \frac{0{,}270713}{1{,}049856} \approx \mathbf{25{,}786}\% \tag{3.82}$$

**Approfondimento: Dimostrazione della Catena di Contesa e Serie Geometrica**

La derivazione della serie geometrica non è necessaria per applicare il modello: è sufficiente padroneggiare la logica a due stati ($P_c = f_O(1 - e^{-G})$) e la formula finale di $f_O$. Per completezza, si riporta comunque la derivazione formale.

Quando una trasmissione si conclude, il canale resta occupato se almeno una stazione è arrivata durante $T$. La probabilità che durante $T$ sia arrivata esattamente una trama pronta è data dal termine di Poisson per $K = 1$:

$$P[1] = P[K = 1 \text{ in } T] = (\lambda T)e^{-\lambda T} = Ge^{-G} \tag{3.83}$$

Il numero di trasmissioni consecutive $N \in \{1, 2, 3, \dots\}$ che formano la catena ininterrotta segue una legge geometrica: $P[N = i] = (1 - P[1])(P[1])^{i-1}$.
**Dimostrazione analitica tramite differenziazione della serie geometrica:**
Si consideri la serie di potenze fondamentale per $|x| < 1$:

$$\sum_{i=0}^{\infty} x^i = \frac{1}{1-x} \implies \frac{d}{dx}\left(\sum_{i=0}^{\infty} x^i\right) = \sum_{i=1}^{\infty} i\, x^{i-1} = \frac{1}{(1-x)^2} \tag{3.84}$$

Ponendo $x = P[1] = Ge^{-G}$, la lunghezza media della sequenza occupata nel modello semplificato è data da:

$$\bar{n} = \sum_{i=1}^{\infty} i\,(P[1])^{i-1} = \frac{1}{(1 - P[1])^2} = \frac{1}{(1 - Ge^{-G})^2} \tag{3.85}$$

La durata media complessiva del periodo occupato vale $T_O = \bar{n}T = \frac{T}{(1-Ge^{-G})^2}$, mentre la durata media del periodo di silenzio (dalla proprietà memoryless di Poisson) è $T_L = \frac{1}{\lambda} = \frac{T}{G}$. La frazione temporale di canale occupato risulta dunque:

$$f_O = \frac{T_O}{T_O + T_L} = \frac{\frac{T}{(1-Ge^{-G})^2}}{\frac{T}{(1-Ge^{-G})^2} + \frac{T}{G}} = \frac{G}{G + (1 - Ge^{-G})^2} \tag{3.86}$$

**Quesito (g) — Throughput di CSMA 1-Persistente e Risoluzione del Paradosso Asintotico:**
Il throughput utile normalizzato $S_{1p}$ rappresenta la frazione del carico offerto $G$ che viene trasmessa con successo senza incorrere in collisioni:

$$S_{1p}(G) = G \cdot (1 - P_c(G)) \tag{3.87}$$

$$= G\left[1 - \frac{G\,(1 - e^{-G})}{G + (1 - Ge^{-G})^2}\right] = G - \frac{G^2\,(1 - e^{-G})}{G + (1 - Ge^{-G})^2} \tag{3.88}$$

**Analisi del Comportamento Asintotico al Limite per $G \to \infty$:**
Poiché $\lim_{G\to\infty} e^{-G} = 0$ e $\lim_{G\to\infty} Ge^{-G} = 0$, il denominatore tende a $G+1$. Di conseguenza:

$$\lim_{G\to\infty} P_c(G) = \lim_{G\to\infty} \frac{G}{G+1} = \mathbf{1} \quad (\mathbf{100}\% \text{ collisioni}) \tag{3.89}$$

$$\lim_{G\to\infty} S_{1p}(G) = \lim_{G\to\infty} G\left(1 - \frac{G}{G+1}\right) = \lim_{G\to\infty} \frac{G}{G+1} = \mathbf{1} \quad (\mathbf{100}\%) \tag{3.90}$$

**Principio / Fondamento: Risoluzione del Paradosso Asintotico: Matematico vs Fisico e Consiglio per l'Orale**

Si manifesta qui un **apparente paradosso concettuale**: com'è possibile che un sistema in cui la probabilità di collisione tende al 100% ($P_c \to 1$) esibisca contemporaneamente un throughput utile del 100% ($S \to 1$)?

1. **L'Origine Matematica dell'Indecisione ($0 \cdot \infty$):** Nel prodotto $S = G(1 - P_c)$, il primo fattore $G$ diverge a $+\infty$ mentre la probabilità di successo $(1 - P_c) = \frac{1}{G+1}$ tende a 0. Poiché il decadimento della probabilità di successo è esattamente proporzionale a $1/G$, la forma indeterminata $\infty \cdot 0$ si risolve nel valore finito unitario $S \sim G \cdot \frac{1}{G} = 1$.
2. **L'Ipotesi Fisica Irrealistica Sottostante:** Questo risultato è la conseguenza diretta delle ipotesi di **sensing istantaneo** ($\tau_p = 0$) e di **ideal collision detection** ($T_{\text{coll}} = T$). Nel modello semplificato, le trame che collidono si raggruppano istantaneamente in singoli slot di durata $T$ senza generare interferenza residua nel tempo. Inoltre, per quanto grande sia il numero di stazioni che collidono simultaneamente allo sblocco del canale, il modello assume che il canale non perda tempo extra per riallinearsi.
3. **Il Comportamento Fisico Reale (Modello di Kleinrock-Tobagi):** In un canale reale con ritardo di propagazione finito ($a = \tau_p/T > 0$), la realtà fisica è radicalmente opposta: per $G > 1$, decine di stazioni che hanno deferito trasmettono all'unisono al termine della trama; le collisioni richiedono tempo di jam e propagazione per essere rilevate, e le ritrasmissioni immediate (dovute alla persistenza 1) provocano ulteriori collisioni a cascata. Il canale entra in uno stato di collasso permanente per contesa (*congestion collapse*), e il throughput reale decade catastroficamente a zero:
$$\lim_{G\to\infty} S_{1p}^{\text{reale}}(G) = \mathbf{0} \tag{3.91}$$
4. **Limiti del modello ideale**: un calcolo reale ed esatto su reti fisiche richiede strumenti matematici avanzati (modello di Kleinrock–Tobagi). In prima approssimazione si usano $S_{0p} = \frac{G}{G+1}$ e $S_{1p}$; tali formule sono efficaci soprattutto per bassi carichi ($G \ll 1$), mentre per carichi elevati divergono dalla fisica reale per l'assenza del collasso da contesa.

**Quesito (h) — CSMA $p$-Persistente come Sintesi Convessa e Trade-off Operativo:** Il protocollo **CSMA $p$-persistente** opera su canali suddivisi in micro-slot temporali di durata pari al massimo ritardo di propagazione $\tau_p$. Una stazione con pacchetto pronto che rileva il canale occupato deferisce mettendosi in ascolto; non appena il canale torna libero:

- con probabilità $p$, inizia la trasmissione immediata;
- con probabilità $1 - p$, attende un micro-slot temporale $\tau_p$ e ripete il sensing.

Il comportamento prestazionale del CSMA $p$-persistente può essere espresso in prima approssimazione come la **combinazione convessa** dei due protocolli estremi:

$$S_p(G) = p \cdot S_{1p}(G) + (1 - p) \cdot S_{0p}(G) \tag{3.92}$$

dove:

$$S_{1p}(G) = G - \frac{G^2(1-e^{-G})}{G + (1-Ge^{-G})^2} \tag{3.93}$$

$$S_{0p}(G) = \frac{G}{G+1} \tag{3.94}$$

**Il Trade-off Ingegneristico Fondamentale:**

- **Per $p \to 1$ (Aggressività / Bassa Latenza):** Le stazioni massimizzano la prontezza di risposta. A carichi leggeri ($G \ll 1$), il ritardo medio di accesso è minimo poiché nessuno spreca tempo differendo. Tuttavia, all'aumentare del carico, il protocollo eredita l'instabilità del 1-persistente, moltiplicando le collisioni.
- **Per $p \to 0$ (Prudenza / Massima Capacità):** Le stazioni evitano le collisioni di massa distribuendo le trasmissioni su slot successivi. A carichi pesanti ($G \gg 1$), il sistema preserva l'alta efficienza del 0-persistente, scongiurando il collasso del throughput. Tuttavia, a carichi leggeri, introduce un'inutile penalizzazione di latenza causata dai continui rinvii immotivati a canale deserto.
- **Regola di Ottimalità:** Per massimizzare il throughput senza instabilità in una popolazione di $N$ nodi attivi in contesa, il parametro deve essere dimensionato dinamicamente secondo la regola:

$$p^* \approx \frac{1}{N} \tag{3.95}$$

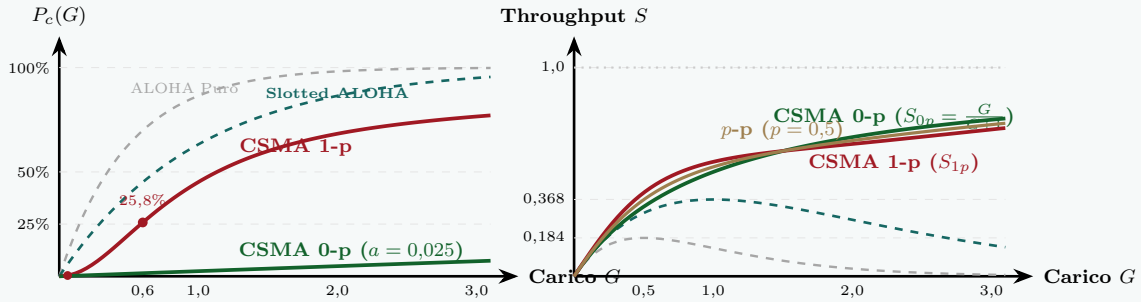



Figura 3.17: Confronto quantitativo delle curve caratteristiche di prestazione: a sinistra la probabilità di collisione $P_c(G)$, a destra il throughput normalizzato $S(G)$ al variare del carico offerto.

**Sintesi Comparativa dei Risultati:** La Tabella 3.4 raccoglie il quadro comparativo completo di tutti i parametri fisici, delle formule chiuse derivate e dei valori numerici determinati per le tre configurazioni considerate.

| **Protocollo MAC** | **Finestra $T_v$** | **Espressione $P_c(G)$** | **$P_c$ ($\lambda = 10$)** | **$P_c$ ($\lambda = 100$)** | **$P_c$ ($\lambda = 1000$)** |
|---|---|---|---|---|---|
| **ALOHA Puro** | $2T$ | $1 - e^{-2G}$ | 1,59% | 14,79% | 79,81% |
| **Slotted ALOHA** | $T$ | $1 - e^{-G}$ | 0,80% | 7,69% | 55,07% |
| **CSMA 0-Persistente** | $2\tau_p$ | $1 - e^{-aG} = 1 - e^{-0{,}025G}$ | **0,020%** | **0,1998%** | **1,980%** |
| **CSMA 1-Persistente** | 0 (Ideale) | $\frac{G(1-e^{-G})}{G+(1-Ge^{-G})^2}$ | **0,004%** | **0,368%** | **25,786%** |
| **CSMA $p$-Persistente** | Slot $\tau_p$ | Combinazione $p$-pesata | Intermedio | Intermedio | Intermedio |

**Riepilogo Throughput Utile Normalizzato $S(G)$ e Comportamento Asintotico:**

| | | |
|---|---|---|
| **CSMA 0-p Ideale:** | $S_{0p}(G) = \frac{G}{G+1}$ | $\lim_{G\to\infty} S_{0p}(G) = 1{,}0 \quad (100\%)$ |
| **CSMA 1-p Ideale:** | $S_{1p}(G) = G(1 - P_c)$ | $\lim_{G\to\infty} S_{1p}(G) = 1{,}0 \quad$ (Paradosso $0 \cdot \infty$) |
| **CSMA $p$-p Ideale:** | $S_p(G) = pS_{1p} + (1-p)S_{0p}$ | Compreso tra le curve limite |

Tabella 3.4: Quadro sinottico comparativo delle prestazioni, probabilità di collisione e limiti di throughput dei protocolli CSMA.

## Insegnamenti Progettuali e Domande Frequenti d'Esame sui Protocolli CSMA

Dall'analisi rigorosa dell'esercitazione emergono quattro cardini concettuali ricorrenti nelle prove d'esame:

1. **La gerarchia prestazionale e l'abbattimento della vulnerabilità:** Il passaggio da ALOHA a CSMA riduce la finestra di vulnerabilità da ordini di grandezza del tempo di trama ($2T$ e $T$)

al solo tempo di propagazione e sensing ($2\tau_p$). In termini percentuali, per $\lambda = 1000$ trame/s, la probabilità di collisione passa dal 79,8% di ALOHA puro ad appena l'1,98% del CSMA 0-persistente, consentendo risparmi di capacità nell'ordine delle decine di Mbit/s.

2. **Perché il CSMA 0-persistente è più stabile del 1-persistente ad alti carichi:** Nel 1-persistente, tutte le stazioni che arrivano a canale occupato deferiscono in ascolto continuo e scaricano simultaneamente la trama all'istante esatto dello sblocco, provocando un picco di collisione del 25,8% già a $\lambda = 1000$ trame/s. Nel 0-persistente, la rinuncia immediata e l'estrazione di un backoff casuale disperdono i tentativi nel tempo, rendendo il canale immune da collisioni sincronizzate di massa.
3. **Il significato dell'apparente paradosso per $G \to \infty$:** All'orale viene spesso chiesto: *"Se in CSMA 1-persistente $P_c \to 1$, come può il throughput teorico tendere a 1?"*. La risposta corretta risiede nell'indecisione matematica $0 \cdot \infty$: il numero di tentativi diverge con la stessa velocità con cui la probabilità di successo si annulla. Nella realtà fisica, a causa del ritardo $a > 0$ e della durata non nulla delle contese, il throughput reale collassa invece a zero.
4. **La necessità insostituibile del Collision Detection (Ethernet):** Per superare il collasso ad alto carico del CSMA 1-persistente senza sacrificare la bassa latenza a basso carico, l'ingegneria di rete introduce il meccanismo di ascolto *durante* la trasmissione: il **CSMA/CD**, su cui si fonda lo standard Ethernet analizzato nella sezione successiva.

## 3.14 Ethernet Classica (IEEE 802.3) e CSMA/CD su Cavo Coassiale

### 3.14.1 Il Progetto Originale di Metcalfe (1973–1976)

Concepita nel 1973 da Robert Metcalfe presso lo Xerox PARC e successivamente standardizzata dal consorzio DIX (Digital, Intel, Xerox) e dall'IEEE come `802.3`, Ethernet nacque con un'idea rivoluzionaria: trasferire i principi di accesso casuale di ALOHA (l'"etere" radio) all'interno di un **cavo coassiale schermato** guidato, garantendo bit-rate enormemente superiori e tassi d'errore minimi.

La prima implementazione commerciale (nota come `10BASE5` o *Thick Ethernet*) presentava le seguenti specifiche fisiche:

- **Cavo Coassiale Spesso Giallo**: un cavo rigido ad alta schermatura con impedenza di 50 Ω, con diametro di circa 1 cm.
- **Topologia a Bus Condiviso**: il cavo coassiale correva lungo i soffitti degli edifici per una lunghezza massima di **500 metri** per singolo segmento, chiuso alle estremità da resistori di terminazione da 50 Ω per evitare riflessioni dell'onda.
- **Limite delle 100 Stazioni per Segmento**: su ciascun segmento potevano essere collegate al massimo 100 stazioni, distanziate a multipli di 2.5 metri. Il limite non era protocollare, ma dipendeva dal carico elettrico dei circuiti di ricetrasmissione innestati a perforazione d'isolante (*vampire taps*).
- **Transceiver Cable (Cavo di Discesa)**: la scheda interna al PC era collegata al ricetrasmettitore sul cavo mediante un cavo flessibile (*drop cable* con interfaccia AUI su connettore DB15) lungo fino a **50 metri**, sul quale transitavano quattro linee funzionali ben distinte: trasmissione dati (Tx), ricezione dati (Rx), notifica di collisione rilevata (Collision Presence, dal transceiver alla scheda) e alimentazione elettrica per i circuiti attivi del transceiver.
- **Diametro di Collisione Massimo di 2800 Metri**: per estendere la rete era consentito concatenare un massimo di 5 segmenti di cavo coassiale mediante 4 ripetitori attivi a livello fisico (*repeater*). La somma dei 2500 m di cavo coassiale, dei ritardi di attraversamento dei 4 ripetitori e dei cavi di discesa corrispondeva a una **dimensione equivalente massima della rete di** 2800 **metri**.

### 3.14.2 Round Trip Delay (RTD) e la Genesi Fisica della Trama Minima di 64 Byte

La dimensione equivalente massima di 2800 m è il parametro cardine che determina tutti i vincoli temporali del protocollo Ethernet originale:

- La velocità di propagazione del segnale sul cavo coassiale è $v \approx 200.000\,\text{km/s} = 2 \times 10^8\,\text{m/s}$ ($\approx 5\,\text{ns/m}$).
- Il tempo necessario per percorrere nel caso peggiore l'intera rete (2800 m) sommato ai ritardi logici dei ripetitori definisce il tempo di propagazione massimo $\tau \approx 25{,}6\,\mu\text{s}$.

- Il tempo di andata e ritorno nel caso peggiore prende il nome di **Round Trip Delay** (RTD):

$$\text{RTD} = 2\tau \approx 51{,}2\,\mu\text{s} \tag{3.96}$$

**Principio / Fondamento: Condizione di Collision Detection e Trama Minima**

Affinché una stazione trasmettitrice possa rilevare con certezza che la propria trama ha colliso con quella di un'altra stazione situata all'estremo opposto del cavo, **la trasmissione deve durare almeno quanto il Round Trip Delay ($2\tau$)**:

$$T_{\text{tx}} \geq 2\tau = 51{,}2\,\mu\text{s} \tag{3.97}$$

Se infatti una stazione trasmettesse una trama brevissima (ad esempio da 10 byte, della durata di appena $8\,\mu$s a 10 Mbit/s), essa terminerebbe l'emissione e libererebbe il trasmettitore molto prima che l'onda riflessa della collisione torni indietro lungo il cavo. La stazione si convincerebbe falsamente che la trama sia stata ricevuta correttamente, mentre in realtà è andata distrutta.
Alla velocità nominale di $R = 10\,\text{Mbit/s} = 10^7\,\text{bit/s}$, la durata di $51{,}2\,\mu$s impone tassativamente una **dimensione minima della trama**:

$$L_{\text{min}} = R \cdot 2\tau = (10 \times 10^6\,\text{bit/s}) \times (51{,}2 \times 10^{-6}\,\text{s}) = \mathbf{512}\,\text{bit} = \mathbf{64}\,\text{byte} \tag{3.98}$$

Tale valore di **512 bit-time** costituisce esattamente la dimensione della trama minima di Ethernet e l'unità di misura dello slot di contesa. Se un payload IP è troppo corto, la scheda inserisce forzatamente dei byte di riempimento (*padding*) per raggiungere sempre i 64 byte.

**Efficienza di CSMA/CD e Trade-Off del Limite $a \to 0$**

L'efficienza $\eta$ del protocollo CSMA/CD è strettamente governata dal rapporto tra tempo di propagazione e tempo di trasmissione, risultando proporzionale a:

$$\eta \propto (1 - a) \quad \text{con} \quad a = \frac{\tau}{T_{\text{tx}}} = \frac{T_{\text{propagazione}}}{T_{\text{trasmissione}}} \tag{3.99}$$

Le prestazioni risultano dunque ottimali su reti piccole ($\tau$ contenuto) e con trame lunghe ($T_{\text{tx}} \gg \tau$). Tuttavia, emerge un paradosso fisico fondamentale: pretendere $a \to 0$ per avere efficienza ideale del 100% equivarrebbe ad avere una rete inutile, ossia o di dimensione spaziale nulla ($\tau = 0$) oppure con trame di lunghezza infinita ($T_{\text{tx}} \to \infty$). Inoltre, all'aumentare del bit-rate (Fast Ethernet a 100 Mbit/s e Gigabit Ethernet a 1 Gbit/s), 512 bit-time scendono rispettivamente a $5{,}12\,\mu$s e a soli 512 ns: a tali velocità il diametro fisico del cavo dovrebbe ridursi a poche decine di metri, rendendo la collision detection su rame insostenibile e imponendo il passaggio alle reti commutate su switch.

### 3.14.3 Procedura di Collision Detection e la Sequenza di Jamming

Sul cavo coassiale, la collision detection opera per via energetica: ogni transceiver in trasmissione ascolta il livello di tensione sul cavo mediante un circuito comparatore. Se sul cavo è presente un'energia superiore a quella emessa dal singolo trasmettitore, significa che un'altra stazione sta trasmettendo: si è verificata una collisione.

Tuttavia, a causa del ritardo finito di propagazione, due stazioni $B$ e $D$ possono rilevare la collisione in istanti temporali sfasati. Se una stazione interrompesse la trasmissione nell'istante esatto in cui percepisce la collisione, la quantità di energia sul cavo potrebbe non essere sufficiente affinché la stazione remota all'altro capo se ne accorga.

Per garantire che **tutte le stazioni collegate sul canale rilevino inequivocabilmente la collisione**, la stazione che avverte per prima il conflitto emette immediatamente una speciale **sequenza di disturbo** (**Jamming Sequence**):

- Consiste in una sequenza continua di **32–48 bit** formata da un pattern alternato `10101010...`;
- Questa sequenza massimizza la potenza media immessa sul cavo coassiale e la sua durata «complementa» la trasmissione della trama minima, garantendo che la sovrapposizione energetica raggiunga l'intero diametro della rete prima che le stazioni desistano;

- Tutte le stazioni rilevano la collisione, interrompono l'invio delle trame corrotte e passano alla fase di backoff.

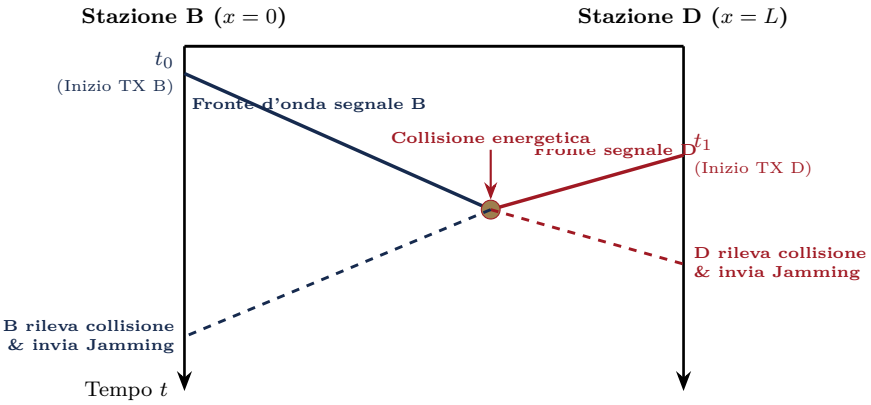



Figura 3.18: Diagramma spazio-temporale della collisione tra due stazioni Ethernet ($B$ e $D$) su bus condiviso: $D$ inizia a trasmettere a $t_1$ prima che il segnale di $B$ la raggiunga. La collisione viene rilevata in istanti differenti; l'emissione della Jamming Sequence garantisce che la sovrapposizione energetica raggiunga entrambe le stazioni.

### 3.14.4 L'Algoritmo di Binary Exponential Backoff

Ethernet opera ordinariamente secondo il paradigma **CSMA/CD 1-persistente**. Tuttavia, per risolvere le collisioni deterministiche a catena, lo standard definisce l'algoritmo di **Binary Exponential Backoff** (ritiro esponenziale binario):

**Definizione: Algoritmo di Binary Exponential Backoff in IEEE 802.3**

Quando una trama subisce una collisione:

1. La contesa successiva è discretizzata in **slot temporali di contesa**, ciascuno avente durata rigorosamente pari a:

$$T_{\text{slot}} = 512\,\text{bit-time} \quad (51{,}2\,\mu\text{s a } 10\,\text{Mbit/s}, \quad 5{,}12\,\mu\text{s a } 100\,\text{Mbit/s}) \tag{3.100}$$

2. Alla collisione consecutiva numero $i$ (per la medesima trama):
   - Si calcola l'esponente di backoff: $k = \min(i, 10)$;
   - La stazione estrae un numero intero $r$ con distribuzione uniforme nell'intervallo:

$$r \in \{0, 1, 2, \ldots, 2^k - 1\} \tag{3.101}$$

   - La stazione attende un tempo di backoff pari a $r \times T_{\text{slot}}$ prima di riprovare ad accedere al canale in modalità 1-persistente.
3. **Crescita della Finestra di Contesa**:
   - $i = 0$ (nuova trama): $r \in [0, 0] \implies$ trasmissione immediata.
   - $i = 1$ ($1^{\text{a}}$ collisione): $k = 1 \implies r \in [0, 1]$ (2 slot; probabilità di ricolidere $= 0{,}5$).
   - $i = 2$ ($2^{\text{a}}$ collisione): $k = 2 \implies r \in [0, 3]$ (4 slot; probabilità di ricolidere $= 0{,}25$).
   - $i = 3$ ($3^{\text{a}}$ collisione): $k = 3 \implies r \in [0, 7]$ (8 slot).
   - ...
   - Da $i = 10$ a $i = 15$: la finestra si congela al valore massimo di **1024 slot** ($r \in [0, 1023]$).
4. **Condizione di Aborto Irreversibile**: se la medesima trama subisce **16 collisioni consecutive**, la scheda di rete desiste, scarta definitivamente la trama e segnala una condizione di **failure hardware** al sistema operativo (presumendo la rottura fisica del cavo o la disconnessione del terminatore).
5. Non appena una trama viene trasmessa con successo, il contatore delle collisioni viene azzerato istantaneamente ($i \to 0$).

**Perché l'Algoritmo è Adattativo e Resiliente**

L'algoritmo di backoff binario esponenziale adatta dinamicamente la propria aggressività al carico istantaneo della rete: a canale poco trafficato la finestra è strettissima ($[0, 1]$), garantendo ripartenze pressoché immediate; se la congestione sale, la finestra si allarga esponenzialmente fino a 1024 slot, disperdendo nel tempo i tentativi delle stazioni e ripristinando la stabilità del canale.

## 3.15 Formato della Trama Ethernet ed Evoluzione del Cablaggio Fisico

### 3.15.1 Struttura della Trama Ethernet II e IEEE 802.3

La trama Ethernet costituisce l'unità fondamentale di trasferimento dei dati (PDU di Livello 2) sulle reti locali cablate. Nel corso dei decenni si sono affermati due formati principali, successivamente armonizzati: lo standard **Ethernet II** (noto anche come formato DIX: Digital, Intel, Xerox) e lo standard **IEEE 802.3**.

La struttura dettagliata dei campi che compongono il frame è illustrata nella Tabella 3.5.

| Campo | Lunghezza | Valore / Formato | Funzione Architetturale |
|---|---|---|---|
| **Preamble** | 7 byte | Alternanza `10101010` | Sincronizzazione fisica del clock del ricevitore (onda quadra a 5 MHz a 10 Mbit/s). |
| **SFD** | 1 byte | Sequenza `10101011` | *Start of Frame Delimiter*: la coppia finale di bit `11` segnala l'inizio dei dati della trama. |
| **Dest. MAC** | 6 byte | Indirizzo a 48 bit | Indirizzo fisico della scheda di rete destinataria (unicast, multicast o broadcast). |
| **Source MAC** | 6 byte | Indirizzo a 48 bit | Indirizzo fisico della scheda di rete trasmettitrice (usato per l'auto-apprendimento degli switch). |
| **EtherType / Length** | 2 byte | $\geq$ `0x0600` (Type) oppure $\leq$ 1500 (Length) | Identifica il protocollo di Livello 3 incapsulato (`0x0800` per IPv4, `0x0806` per ARP, `0x86DD` per IPv6). |
| **Payload Dati** | 46–1500 B | Pacchetto di Livello 3 | Dati utente. Se $<$ 46 byte, viene inserito un *padding* per raggiungere la trama minima di 64 byte. |
| **FCS (CRC-32)** | 4 byte | Polinomio standard 802.3 | Codice di ridondanza ciclica a 32 bit per rilevare errori di trasmissione su tutta la trama. |

Tabella 3.5: Struttura dettagliata dei campi della trama Ethernet (Ethernet II / IEEE 802.3).

**Principio / Fondamento: Dimensionamento della Trama e Maximum Transmission Unit (MTU)**

L'architettura della trama Ethernet impone due vincoli dimensionali rigorosi:

1. **Dimensione Minima di 64 Byte**: come dimostrato dall'equazione della collision detection ($L_{\min} = 2\tau \cdot R$), l'intera trama (esclusi Preambolo ed SFD, che operano a livello fisico) deve misurare almeno 64 byte (512 bit). Poiché l'intestazione occupa $6 + 6 + 2 = 14$ byte e il checksum FCS occupa 4 byte (totale overhead fisso = 18 byte), il payload utile deve contenere almeno:

$$L_{\text{payload,min}} = 64\,\text{byte} - 18\,\text{byte} = 46\,\text{byte} \tag{3.102}$$

   Se un pacchetto IP è più corto di 46 byte, la scheda di rete accoda forzatamente byte di riempimento (*padding*).
2. **Dimensione Massima (MTU = 1500 Byte)**: il carico utile massimo trasferibile in una singola trama senza frammentazione è fissato a **1500 byte**. Tale valore costituisce la **Maximum Transmission Unit (MTU)** standard di Internet e rappresenta un compromesso storico tra overhead d'intestazione e probabilità di errore sul canale.
3. **Inter-Packet Gap (IPG)**: tra la fine di una trama e l'inizio del preambolo della successiva deve intercorrere un periodo minimo di silenzio di **96 bit-time** (9,6 $\mu$s a 10 Mbit/s, 96 ns a 1 Gbit/s) per consentire ai circuiti del ricevitore di resettare lo stato interno.

### 3.15.2 Evoluzione Fisica dei Mezzi Trasmissivi

La tecnologia Ethernet si è evoluta dal bus coassiale condiviso al cablaggio strutturato a stella:

- **10BASE5 / 10BASE2**: cavo coassiale a bus condiviso a 10 Mbit/s (500 m per il cavo spesso con ricetrasmettitore esterno; 185 m per il coassiale flessibile con connettori BNC a T).
- **10BASE-T (1990)**: cablaggio a stella su doppino ritorto UTP (Cat 3/5, connettore RJ-45) con tratte dedicate verso apparato centrale (Hub/Switch, max 100 m). Introduce isolamento dai guasti e supporto PoE.
- **Fast Ethernet (100BASE-TX)**: 100 Mbit/s su doppino Cat 5. A parità di trama minima (64 B), per rispettare $T_{\text{tx}} \geq 2\tau$ il diametro massimo del dominio di collisione scende a circa 200 m.
- **Gigabit Ethernet (1000BASE-T)**: 1 Gbit/s su 4 coppie Cat 5e/6. Poiché il diametro teorico di collisione CSMA/CD scenderebbe ad appena 20 m, opera quasi esclusivamente su switch in modalità **full-duplex** (senza collisioni).

## 3.16 Commutazione di Livello 2: Hub vs. Switch e Domini di Rete

### 3.16.1 Dispositivi di Livello 1: Hub e Repeater

All'introduzione della topologia a stella, l'apparato centrale originario era l'**Hub** (o ripetitore multiporta):

- **Operatività puramente Fisica (Livello 1)**: l'Hub non interpreta i bit, non conosce la struttura della trama Ethernet, non calcola il CRC e non dispone di memorie di accodamento.
- **Replicazione Energetica Cieca**: qualunque segnale elettrico ricevuto su una determinata porta viene rigenerato e ritrasmesso simultaneamente su tutte le altre porte attive.
- **Condivisione Totale della Capacità**: se due stazioni collegate all'Hub trasmettono nello stesso istante, i loro segnali si sovrappongono all'interno dell'apparato generando una collisione. L'intera rete costituisce un **unico dominio di collisione** e un **unico dominio di broadcast**. La capacità del mezzo (es. 10 Mbit/s) è spartita tra tutti gli host collegati.

### 3.16.2 Dispositivi di Livello 2: Bridge e Switch

La rivoluzione delle reti locali si compie con l'introduzione dei **Bridge** e, successivamente, degli **Switch** (bridge multiporta basati su matrici di commutazione hardware ad alta velocità, *ASIC*):

- **Operatività a Trame (Livello 2)**: lo switch riceve le trame per intero (*Store-and-Forward*), controlla l'integrità del checksum FCS (scartando immediatamente le trame corrotte) e legge gli indirizzi MAC nell'intestazione.
- **Microsegmentazione e Full-Duplex**: collegando un singolo host a ciascuna porta fisica mediante cavo UTP operante in modalità full-duplex (linee di trasmissione e ricezione disaccoppiate), la contesa per il canale scompare. Ogni host dispone della banda nominale piena (es. 1 Gbit/s bidirezionale) senza alcuna possibilità di collisione.

### 3.16.3 Algoritmo di Auto-Apprendimento (Backward Learning)

Gli switch Ethernet sono apparati *Plug & Play*: non richiedono alcuna configurazione manuale iniziale. Lo switch costruisce e aggiorna dinamicamente la propria tabella di inoltro (**Forwarding Table** o memoria associativa **CAM**, *Content Addressable Memory*) mediante l'algoritmo di **Backward Learning** (apprendimento retrogrado):

**Definizione: Algoritmo di Backward Learning dello Switch**

La tabella di commutazione memorizza tuple della forma:

$$\langle \text{Indirizzo MAC}, \text{Porta Fisica d'Interfaccia}, \text{Timer di Aging} \rangle \tag{3.103}$$

Quando una trama transita sulla porta $P_{\text{in}}$ con indirizzo MAC sorgente $MAC_{\text{src}}$:

1. Lo switch ispeziona il campo $MAC_{\text{src}}$ e registra: *"Il nodo con indirizzo $MAC_{src}$ si trova sul segmento attestato alla porta $P_{in}$"*;
2. Se la voce è nuova, viene creata; se è già presente, il rispettivo timer di scadenza (*aging timer*, tipicamente impostato a **300 secondi**) viene riavviato;
3. Se una stazione non trasmette alcuna trama prima della scadenza del timer, la voce viene rimossa per fare spazio nella tabella CAM e consentire il riconoscimento automatico di eventuali rilocazioni fisiche dei terminali.

### 3.16.4 Algoritmo di Inoltro e Filtraggio (Filtering and Forwarding)

All'arrivo di una trama sulla porta $P_{\text{in}}$ diretta al MAC di destinazione $MAC_{\text{dst}}$:

1. **Caso Broadcast / Multicast**: se $MAC_{\text{dst}}$ = `FF:FF:FF:FF:FF:FF`, la trama deve raggiungere ogni terminale; lo switch la inoltra su tutte le porte attive tranne quella di ingresso (**Flooding**);
2. **Caso Destinazione Sconosciuta**: se $MAC_{\text{dst}}$ non è ancora presente nella tabella CAM, lo switch non sa dove si trovi il destinatario ed esegue il **Flooding** su tutte le porte; non appena il destinatario emetterà una risposta, l'auto-apprendimento registrerà la sua posizione;
3. **Caso Destinazione Nota (Filtraggio vs. Inoltro)**: se $MAC_{\text{dst}}$ è associato alla porta $P_{\text{out}}$ nella tabella:
   - Se $P_{\text{out}} = P_{\text{in}}$: il destinatario si trova sullo stesso ramo fisico dal quale è arrivata la trama. Lo switch **filtra (scarta)** la trama, evitando di occupare inutilmente le altre porte;
   - Se $P_{\text{out}} \neq P_{\text{in}}$: lo switch commuta e trasmette la trama **selettivamente in unicast** solo sulla porta $P_{\text{out}}$.

### 3.16.5 Isolamento dei Domini di Rete e Criticità delle Reti Commutate

La transizione a reti commutate definisce una netta separazione tra due concetti fondamentali:

**Principio / Fondamento: Dominio di Collisione vs. Dominio di Broadcast**

- **Dominio di Collisione**: porzione di rete in cui due trasmissioni contemporanee si sovrappongono e collidono. Lo switch **isola i domini di collisione**: ciascuna porta fisica costituisce un dominio di collisione indipendente. In modalità full-duplex le collisioni sono azzerate.
- **Dominio di Broadcast**: insieme dei dispositivi raggiunti da una trama avente indirizzo MAC di destinazione impostato a broadcast. Gli switch **non isolano i domini di broadcast**: una trama broadcast attraversa indistintamente tutti gli switch e raggiunge ogni terminale della medesima LAN.

La delimitazione dei domini di broadcast richiede tassativamente un dispositivo di **Livello 3 (Router)**.

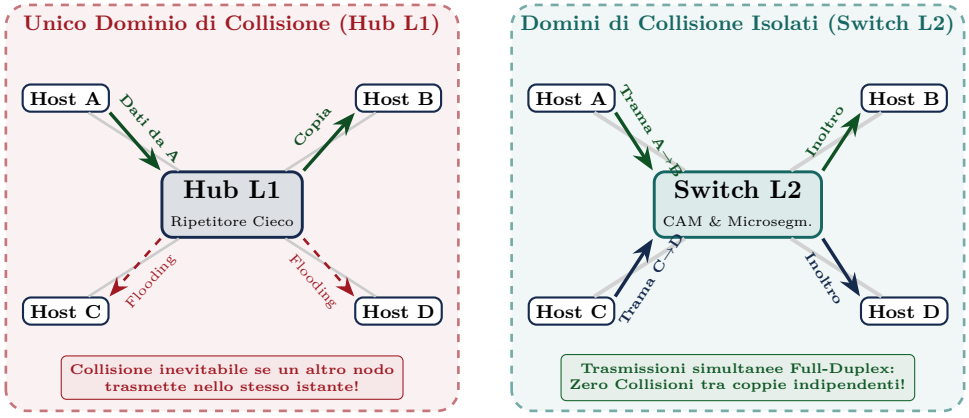



Figura 3.19: Confronto architetturale tra Hub (Livello 1) e Switch (Livello 2): l'Hub replica ciecamente i segnali generando un unico dominio di collisione condiviso; lo switch isola ciascuna porta in un dominio di collisione indipendente permettendo trasmissioni parallele e full-duplex privo di contesa.

**Buffer Overflow, Backpressure e Mismatch Temporale con TCP**

Nelle reti commutate, le collisioni fisiche sono sostituite dall'accodamento nei buffer di memoria dello switch: se più porte ricevono contemporaneamente trame destinate alla medesima porta di uscita, le trame vengono accodate. Se il traffico persiste, il buffer satura e le trame eccedenti vengono scartate (*tail drop*).
In ambito half-duplex, alcuni switch implementavano meccanismi di **backpressure**, simulando una collisione fittizia per costringere la sorgente a fermarsi. Tuttavia, quando le perdite di trame si verificano su larga scala, si manifesta un **mismatch temporale con il protocollo TCP** (Livello 4): mentre il livello Data Link gestisce recuperi nell'ordine dei microsecondi o millisecondi, TCP opera con timer RTO dell'ordine dei centinaia di millisecondi. Le perdite a raffica causate da buffer pieni innescano scadenze di timeout in TCP, provocando il crollo della finestra di congestione e facendo procedere la trasmissione

"a singhiozzo".

### 3.16.6 Esercizio d'Esame: Probabilità di Collisione, Backoff e Throughput in Ethernet con Hub

Per consolidare i modelli quantitativi del controllo d'accesso al mezzo in ambiente cablato condiviso a commutazione di Livello 1, si riporta di seguito la trattazione analitica integrale del primo problema dell'**Esercitazione Ufficiale MAC 2** (*A.A. 2026-2027, Prof. Renato Lo Cigno e Dott. Lorenzo Ghiro*). L'esercizio analizza la contesa e il rendimento di una rete Ethernet attestata a un Hub condiviso, ponendo le basi per il confronto prestazionale con il paradigma wireless Wi-Fi (svolto nella Sezione 3.19.7).

**Esercizio d'Esame: Analisi Prestazionale e Throughput in Ethernet con Hub (Esercitazione MAC 2 — Parte 1)**

**Testo del Problema Ufficiale:**
Si consideri un semplice Hub Ethernet a $N$ porte come rappresentato schematicamente nella Figura 3.20. Una porta è costantemente dedicata al collegamento verso un router di sottorete, mentre alle restanti porte vengono connesse fino a $N-1$ stazioni terminali. I parametri operativi del sistema sono:

- Velocità nominale di cifra del canale: $C = 100\,\text{Mbit/s}$;
- Dimensione del carico utile (*payload*): variabile tra 46 e 1500 byte;
- Intestazione di Livello 2 (Header Ethernet): 18 byte complessivi (14 byte di campi header MAC standard e 4 byte di Frame Check Sequence / CRC);
- Preambolo di Livello Fisico: 6 byte (incluso il byte di delimitazione *Start-of-Frame Delimiter*, SFD);
- Spazio minimo inter-trama (*Inter-Packet Gap*, IPG): 12 byte di silenzio sul mezzo;
- Sequenza di *Jamming* (in caso di collisione rilevata): durata equivalente alla trasmissione completa di una trama minima da 64 byte più il preambolo fisico di 6 byte;
- Ripresa post-collisione: la trasmissione può ripartire dopo un intervallo pari a 1 IPG dal termine della sequenza di jamming;
- Durata dello slot di contesa (*Slot Time*): $T_{\text{slot}} = 64\,\text{byte}$;
- Ritardo di propagazione del segnale: assunto trascurabile rispetto ai tempi di trasmissione ($\tau_p \simeq 0$).

Si richiede di risolvere analiticamente i seguenti quesiti:

**(a)** Nel caso $N = 2$ con **1 sola stazione satura** (dispone perennemente di trame pronte all'invio, mentre il router non genera mai traffico), calcolare la probabilità di collisione $P_C$ e il throughput massimo conseguibile.

**(b)** Nel caso $N = 2$ con **entrambi i nodi saturi** (sia la stazione sia il router hanno sempre trame pronte in coda), calcolare la probabilità che una trama venga definitivamente scartata assumendo che siano ammesse al massimo 6 collisioni consecutive, tutte a finestra crescente (*Binary Exponential Backoff*), prima di dichiarare la perdita irreversibile.

**(c)** Nel caso di $N$ generico, con ciascuna stazione che genera traffico secondo un processo di arrivo poissoniano indipendente con frequenza $\lambda/N$ trame/s (per un carico aggregato complessivo pari a $\lambda$ trame/s):

**c.1)** Calcolare il valore del rate aggregato $\lambda$ che corrisponde a un carico normalizzato $G = 1$ nel caso di trame a dimensione massima (1500 byte di payload).

**c.2)** Fissando $N = 2$, sempre con trame a dimensione massima e ciascuna stazione che offre un carico normalizzato $G = 0{,}2$, calcolare la probabilità di osservare $k = 1, 2, 3$ collisioni consecutive, assumendo che lo stato iniziale del canale sia occupato da una trama in trasmissione.

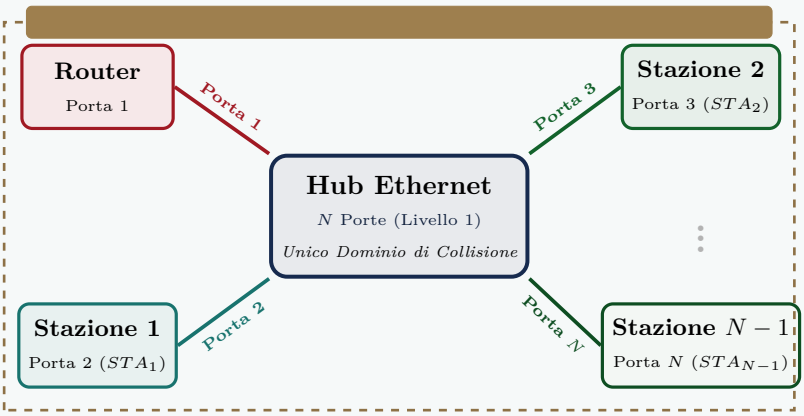



Figura 3.20: Topologia dell'Hub Ethernet a $N$ porte: una porta è dedicata al router e le restanti alle stazioni locali. L'hub opera al Livello Fisico inoltrando ciecamente i bit su tutte le porte e generando un unico dominio di collisione condiviso.

**Risoluzione Analitica Dettagliata:**

**Quesito (a) — Singola Stazione Satura e Throughput di Livello 2:** Nel primo scenario abbiamo $N = 2$ nodi fisici: la Stazione 1 e il Router. Per ipotesi, il router non genera mai traffico ($\lambda_{\text{router}} = 0$), mentre la stazione dispone continuativamente di trame in coda. Poiché la stazione non ha competitori attivi sul mezzo condiviso, non si verifica alcuna interferenza trasmissiva:

$$P_C = 0 \tag{3.104}$$

La stazione trasmette ininterrottamente sul canale, separando ciascuna trama consecutiva dal solo intervallo di silenzio prescritto dal protocollo (IPG = 12 byte).

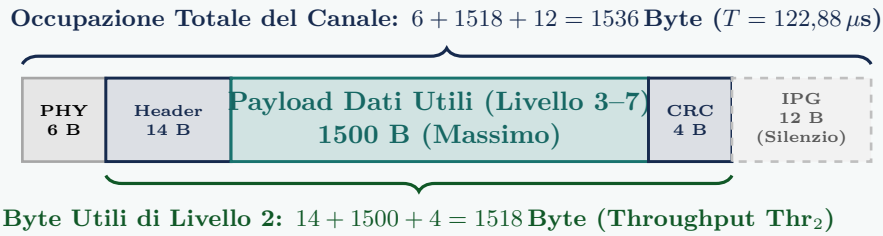



Figura 3.21: Struttura temporale della trasmissione Ethernet: ripartizione tra i byte utili di Livello 2 (Header, Payload e CRC) e i byte di overhead fisico/canale (preambolo e silenzio IPG).

Per calcolare il throughput occorre distinguere a quale livello protocollare si misura il rendimento:

- **Throughput di Livello 2 ($\text{Thr}_2$)**: considera come informazione utile tutti i campi gestiti dal protocollo di collegamento (Header MAC da 14 B, Payload applicativo fino a 1500 B e CRC da 4 B), escludendo l'overhead fisico (preambolo) e l'intervallo silente IPG;
- **Throughput di Livello 3 ($\text{Thr}_3$)**: considera utile unicamente il payload informativo veicolato al livello IP.

Seguendo la convenzione d'esame e la Figura 3.21, calcoliamo il throughput di Livello 2. Poiché il testo richiede il throughput *massimo ottenibile*, consideriamo la condizione più favorevole con trame a dimensione massima (1500 byte di carico utile), che minimizza l'impatto percentuale dell'overhead:

$$L_{\text{L2}} = 14 + 1500 + 4 = 1518\,\text{byte} \tag{3.105}$$

L'occupazione complessiva di canale necessaria a emettere una trama e rispettare la spaziatura minima è:

$$L_{\text{tot}} = L_{\text{preambolo}} + L_{\text{L2}} + L_{\text{IPG}} = 6 + 1518 + 12 = 1536\,\text{byte} \tag{3.106}$$

Il throughput di Livello 2 risulta pertanto:

$$\text{Thr}_2 = \frac{L_{\text{L2}}}{L_{\text{tot}}} \cdot C = \frac{1518}{1536} \cdot 100\,\text{Mbit/s} \approx 0{,}988281 \cdot 100\,\text{Mbit/s} \approx \mathbf{98{,}83}\,\text{Mbit/s} \tag{3.107}$$

L'efficienza di canale sfiora il 98,83%: l'unico fattore di perdita è costituito dai 18 byte di overhead di livello fisico e spaziatura.

**Quesito (b) — Due Stazioni Sature e Probabilità di Scarto a 6 Collisioni:** Consideriamo ora $N = 2$ con entrambi i nodi saturi (Stazione 1 e Router). Avendo entrambe le stazioni pacchetti costantemente pronti, al primo tentativo di trasmissione entrambe avvertono il canale libero (o il termine della trama precedente) e trasmettono simultaneamente dopo l'IPG: la **prima collisione è deterministica**:

$$P_{C,1} = 1 \tag{3.108}$$

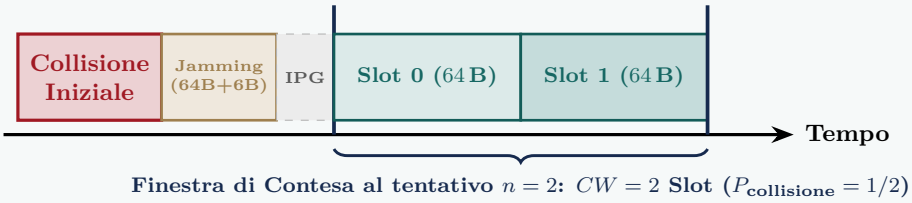



Figura 3.22: Dinamica di risoluzione post-collisione con Binary Exponential Backoff: al secondo tentativo ($n = 2$), la finestra di contesa contiene 2 slot; la probabilità di collisione è la probabilità che entrambi i nodi scelgano lo stesso slot (1/2).

Rilevata la collisione ed emesso il segnale di Jamming, scatta l'algoritmo di **Binary Exponential Backoff**:

- Al tentativo $n = 2$, ciascuna stazione sceglie un ritardo casuale espresso in slot time da 64 byte nell'intervallo $\{0, 1\}$, con $CW_2 = 2^1 = 2$ slot;
- Avendo due stazioni indipendenti che estraggono uniformemente uno slot in $\{0, 1\}$, la probabilità che scelgano lo stesso slot collidendo nuovamente è pari a:

$$P(\text{Collisione al tentativo 2}) = \frac{1}{2} \tag{3.109}$$

- Se collidono al tentativo 2, raddoppiano la finestra a $CW_3 = 2^2 = 4$ slot $\{0, 1, 2, 3\}$: la probabilità di scegliere il medesimo slot si dimezza a 1/4;
- Generalizzando per induzione, al tentativo $n$-esimo ($1 \leq n \leq 6$) la finestra comprende $2^{n-1}$ slot e la probabilità di collisione condizionata vale:

$$P(\text{Collisione al tentativo } n \mid \text{collisioni nei tentativi } 1, \ldots, n-1) = \frac{1}{2^{n-1}} \tag{3.110}$$

Una trama può giungere al tentativo $n$ solo se **tutti i tentativi precedenti hanno generato collisione**. La probabilità di subire esattamente $n$ collisioni consecutive è data dal prodotto delle probabilità di collisione condizionate:

$$P_{C,n} = \prod_{i=1}^{n} \frac{1}{2^{i-1}} = P_{C,n-1} \cdot \frac{1}{2^{n-1}} \tag{3.111}$$

La probabilità di subire 6 collisioni consecutive, fallendo la risoluzione della contesa e causando lo scarto definitivo della trama, risulta:

$$\begin{aligned} P_{C,6} &= \prod_{i=1}^{6} \frac{1}{2^{i-1}} = 1 \cdot \frac{1}{2} \cdot \frac{1}{4} \cdot \frac{1}{8} \cdot \frac{1}{16} \cdot \frac{1}{32} \\ &= 2^0 \cdot 2^{-1} \cdot 2^{-2} \cdot 2^{-3} \cdot 2^{-4} \cdot 2^{-5} = 2^{-(0+1+2+3+4+5)} = 2^{-15} \\ &= \frac{1}{32768} \approx \mathbf{3{,}05 \cdot 10^{-5}} \end{aligned} \tag{3.112}$$

**Robustezza dell'Algoritmo di Backoff in Ethernet**

Il risultato quantitativo $P_{C,6} \approx 3 \cdot 10^{-5}$ evidenzia l'eccezionale affidabilità del meccanismo di backoff esponenziale binario: perfino sotto l'ipotesi penalizzante di soli 6 tentativi ammessi e stazioni permanentemente sature, la probabilità di scartare una trama è di appena 3 su centomila! Nello standard IEEE 802.3 reale la soglia di scarto è fissata a ben 16 tentativi consecutivi (con $CW$ crescente fino al 10° tentativo a quota 1024 slot), rendendo la perdita di trame per mancata risoluzione della contesa un evento pressoché trascurabile.

**Quesito (c.1) — Calcolo del Tasso Poissoniano $\lambda$ per Carico Unitario $G = 1$:** Il carico normalizzato offerto a un canale di comunicazione è definito dalla relazione adimensionale:

$$G = \lambda \cdot T \tag{3.113}$$

dove $\lambda$ è il tasso aggregato di generazione delle trame ( trame/s) e $T$ è la durata temporale di trasmissione di una singola trama. Imponendo la condizione di saturazione nominale $G = 1$, si ottiene:

$$G = 1 \implies \lambda = \frac{1}{T} \tag{3.114}$$

La durata di trasmissione $T$ per trame a dimensione massima (1500 byte di payload) deve comprendere l'occupazione fisica complessiva del canale, inclusi il preambolo e il tempo inter-trama IPG:

$$\begin{aligned} T = \frac{L_{\text{tot}}}{C} &= \frac{[6 + (14 + 1500 + 4) + 12]\,\text{byte} \cdot 8\ \text{bit/byte}}{100 \cdot 10^6\,\text{bit/s}} \\ &= \frac{1536 \cdot 8\,\text{bit}}{10^8\,\text{bit/s}} = \frac{12288\,\text{bit}}{10^8\,\text{bit/s}} = 1{,}2288 \cdot 10^{-4}\,\text{s} = \mathbf{122{,}88}\,\mu\text{s} \end{aligned} \tag{3.115}$$

Ne deriva il tasso medio di arrivo corrispondente a $G = 1$:

$$\lambda = \frac{1}{122{,}88 \cdot 10^{-6}\,\text{s}} \approx \mathbf{8138}\,\text{trame/s} \tag{3.116}$$

**Quesito (c.2) — Probabilità di Collisioni Consecutive con Canale Inizialmente Occupato:** Fissiamo ora $N = 2$ stazioni gemelle simmetriche, ciascuna con carico offerto $G = 0{,}2$. Il canale è inizialmente occupato da una trama in corso di trasmissione. Poiché Ethernet adotta una politica di carrier sensing **1-persistente**, le stazioni che generano una o più trame durante la trasmissione iniziale rimangono in ascolto sul mezzo in attesa che si liberi. Al termine della trama corrente (dopo l'IPG), entrambe tenteranno di trasmettere immediatamente: avremo una prima collisione **se e solo se entrambe le stazioni hanno generato almeno una trama durante l'intervallo di trasmissione della trama iniziale**.
Per un processo di Poisson con tasso medio $G$, la probabilità che una generica stazione generi $k$ trame in un intervallo pari a $T$ è:

$$P[k] = \frac{G^k}{k!} e^{-G} \tag{3.117}$$

La probabilità che una stazione generi almeno una trama ($k \geq 1$) è il complemento della probabilità di generare zero arrivi:

$$P[\geq 1\ \text{trama}] = 1 - P[0] = 1 - e^{-G} \tag{3.118}$$

Essendo i due processi di generazione stocasticamente indipendenti, la probabilità della prima collisione risulta:

$$P_{C,1} = \left(1 - e^{-G}\right)^2 = \left(1 - e^{-0{,}2}\right)^2 \approx (1 - 0{,}818731)^2 \approx (0{,}181269)^2 \approx \mathbf{3{,}29 \cdot 10^{-2}} \tag{3.119}$$

Se la prima collisione si è verificata, entrambe le stazioni attivano il backoff binario esponenziale. Le successive collisioni dipendono dalla scelta casuale degli slot di contesa:

- **Due collisioni consecutive** ($k = 2$): dopo la prima collisione, la finestra è $CW_2 = 2$ slot; collidono con probabilità condizionata 1/2:

$$P_{C,2} = P_{C,1} \cdot \frac{1}{2} \approx \frac{3{,}2858 \cdot 10^{-2}}{2} \approx \mathbf{1{,}64 \cdot 10^{-2}} \tag{3.120}$$

- **Tre collisioni consecutive** ($k = 3$): dopo la seconda collisione, la finestra raddoppia a $CW_3 = 4$ slot; collidono con probabilità condizionata 1/4:

$$P_{C,3} = P_{C,2} \cdot \frac{1}{4} \approx \frac{1{,}6429 \cdot 10^{-2}}{4} \approx \mathbf{0{,}41 \cdot 10^{-2}} = 4{,}11 \cdot 10^{-3} \tag{3.121}$$

## 3.17 Il Protocollo Point-to-Point (PPP)

### 3.17.1 Inquadramento e Assenza di Contesa

Quando due dispositivi sono collegati direttamente attraverso una linea dedicata esclusiva (collegamenti punto-punto come linee seriali sincrone/asincrone, modem telefonici su doppino dial-up V.90 a 56 kbit/s o sessioni `PPPoE` – *PPP over Ethernet* – su linee ADSL), le problematiche di accesso multiplo scompaiono:

- Essendo presenti esattamente due interlocutori, non sussiste alcuna contesa sul canale; protocolli come CSMA/CD o CSMA/CA non hanno ragion d'essere;
- Non è necessario alcun indirizzamento hardware dinamico: qualunque dato emesso da un estremo può essere destinato unicamente all'altro estremo del cavo.

Lo standard di riferimento universale per tali collegamenti è il protocollo `PPP` (*Point-to-Point Protocol*, RFC 1661).

### 3.17.2 Formato della Trama e Meccanismo di Byte Stuffing

Il formato della trama PPP deriva storicamente dal protocollo `HDLC` (*High-Level Data Link Control*) dei mainframe IBM:

- **Flag di Delimitazione (1 byte)**: la trama è racchiusa all'inizio e alla fine dal byte sentinella $\texttt{01111110}_2 = \texttt{0x7E}$;
- **Address (1 byte)**: impostato rigidamente al valore broadcast fittizio `0xFF`;
- **Control (1 byte)**: impostato al valore costante `0x03` (trama non numerata);
- **Protocol (2 byte)**: indica quale protocollo di livello superiore è contenuto nel payload (ad es. `0x0021` per IPv4, `0x8021` per IPCP, `0xC021` per LCP, `0xC223` per CHAP);
- **Payload**: dati utili fino alla MTU negoziata (default 1500 byte);
- **FCS (2 o 4 byte)**: checksum CRC per la convalida d'integrità.

**Definizione: Trasparenza del Canale e Byte Stuffing in PPP**

Per consentire il trasporto arbitrario di qualsiasi sequenza binaria all'interno del payload evitando che un byte di dato casualmente uguale a `0x7E` venga interpretato come fine della trama, PPP adotta la tecnica del **Byte Stuffing** (riempimento a livello di byte), impiegando il byte di escape `0x7D`:

1. Se nei dati compare il byte di delimitazione `0x7E`, esso viene rimosso e sostituito dalla coppia di byte:

$$\texttt{0x7E} \implies \texttt{0x7D}\ \ \texttt{0x5E} \qquad (\text{poiché } \texttt{0x7E} \oplus \texttt{0x20} = \texttt{0x5E}) \tag{3.122}$$

2. Se nei dati compare lo stesso byte di escape `0x7D`, esso viene sostituito dalla coppia:

$$\texttt{0x7D} \implies \texttt{0x7D}\ \ \texttt{0x5D} \qquad (\text{poiché } \texttt{0x7D} \oplus \texttt{0x20} = \texttt{0x5D}) \tag{3.123}$$

3. Il ricevitore, non appena individua il byte di escape `0x7D`, lo scarta e applica lo XOR con `0x20` al byte successivo, ricostruendo l'esatto byte originario prima di consegnare il payload a Livello 3.

### 3.17.3 Architettura Modulare a Tre Livelli di PPP

Il protocollo PPP non si limita alla semplice incapsulazione, ma offre una suite modulare completa:

1. **LCP (Link Control Protocol)**: negozia la creazione, la configurazione e il rilascio del collegamento dati. Determina la Maximum Receive Unit (MRU), negozia l'eventuale compressione delle intestazioni e testa la qualità della linea mediante messaggi periodici di *Echo-Request* ed *Echo-Reply*.
2. **Protocolli di Autenticazione**:
   - **PAP (Password Authentication Protocol)**: procedura elementare a due vie; il client trasmette identificativo e password in chiaro sul canale (scarsamente sicuro);
   - **CHAP (Challenge Handshake Authentication Protocol)**: procedura a tre vie basata su crittografia simmetrica unidirezionale. Il server invia una stringa casuale (*challenge*); il client combina la propria password segreta con il challenge, calcola l'impronta hash MD5 e la invia al server. Il server, conoscendo la medesima password, ripete il calcolo: se i valori combaciano, autorizza la connessione senza che la password sia mai transitata sul canale.
3. **NCP (Network Control Protocol)**: famiglia di protocolli ausiliari specifici per ciascun protocollo di rete da trasportare. Nelle reti IPv4 viene utilizzato **IPCP** (RFC 1332) per negoziare e assegnare dinamicamente l'indirizzo IP all'host client e configurare gli indirizzi dei server DNS.

### 3.17.4 Cenni sulle Linee Asimmetriche xDSL (ADSL)

Nel contesto delle telecomunicazioni residenziali di transizione verso la banda larga, la tecnologia **ADSL** (*Asymmetric Digital Subscriber Line*) ha consentito di trasformare il vecchio doppino telefonico in rame in un canale dati ad alta velocità affiancato alla telefonia analogica tradizionale.

La modulazione suddivide la banda di frequenza del doppino mediante multiplexing FDM:

- 0–4 kHz: canale vocale analogico fonia tradizionale (POTS);
- 25–138 kHz: canale dati di salita (*uplink*), con larghezza di banda contenuta;
- 138 kHz–1,1 MHz: canale dati di discesa (*downlink*), con ampia allocazione di spettro.

L'**asimmetria marcata** (downlink di gran lunga superiore all'uplink) risponde a due requisiti: da un lato la natura sbilanciata del consumo residenziale medio (l'utente scarica flussi multimediali e pagine web pesanti a fronte di richieste brevi); dall'altro una motivazione di gestione commerciale da parte dei provider, volta a impedire che contratti residenziali a basso costo possano essere impiegati per ospitare server pubblici senza sottoscrivere costose linee dedicate simmetriche.

## 3.18 Reti Wireless LAN: Architettura IEEE 802.11 (Wi-Fi)

### 3.18.1 Inquadramento nello Standard IEEE 802 e Caratteristiche del Canale Radio

Lo standard `IEEE 802.11` (commercialmente noto con il marchio certificato **Wi-Fi**) nasce all'interno del comitato IEEE 802 per estendere l'accesso locale alle reti informatiche su mezzo radio non guidato. Pur condividendo con Ethernet (802.3) il modello di indirizzamento a 48 bit e l'interfaccia verso i livelli superiori, il Wi-Fi affronta un canale fisico con proprietà radicalmente differenti:

- **Assenza di Confinamento Guidato**: le onde elettromagnetiche si propagano nello spazio tridimensionale; chiunque all'interno del raggio di copertura può ascoltare e tentare di trasmettere sul medesimo canale radio;
- **Attenuazione Severa e Multipath Fading**: il segnale decade rapidamente con la distanza ($P_{\text{rx}} \propto d^{-n}$ con $n \geq 2$) e subisce riflessioni su pareti e ostacoli, generando interferenza costruttiva e distruttiva e tassi d'errore sui bit (*BER*) enormemente superiori rispetto ai cavi in rame;
- **Bande ISM non Licenziate**: il Wi-Fi opera prevalentemente sulle frequenze a 2,4 GHz e 5 GHz (e recentemente 6 GHz), soggette a forti interferenze esterne generate da forni a microonde, dispositivi Bluetooth e reti concorrenti.

### 3.18.2 Evoluzione Cronologica degli Standard IEEE 802.11

La Tabella 3.6 riassume le principali tappe evolutive della tecnologia Wi-Fi dal 1997 a oggi.

| Standard | Anno | Banda | Bit-rate Max | Innovazione Fisica Principale |
|---|---|---|---|---|
| **802.11 legacy** | 1997 | 2,4 GHz | 2 Mbit/s | Primitivo, scarsa interoperabilità commerciale. |
| **802.11b** | 1999 | 2,4 GHz | 11 Mbit/s | Modulazione DSSS/CCK. Primo vero successo di massa del Wi-Fi. |
| **802.11a** | 1999 | 5 GHz | 54 Mbit/s | Introduzione di **OFDM** su banda libera da microonde. |
| **802.11g** | 2003 | 2,4 GHz | 54 Mbit/s | Estensione di OFDM sulla banda 2,4 GHz, retrocompatibile con 11b. |
| **802.11n (Wi-Fi 4)** | 2009 | 2,4/5 GHz | 600 Mbit/s | Antenne multiple (**MIMO**), canali a 40 MHz e aggregazione trame. |
| **802.11ac (Wi-Fi 5)** | 2013 | 5 GHz | $>1$ Gbit/s | Canali estesi fino a 160 MHz, modulazione densa 256-QAM. |
| **802.11ax (Wi-Fi 6)** | 2020 | 2,4/5/6 GHz | 9,6 Gbit/s | Tecnologia **OFDMA** per canali ultra-densi e multi-utente (MU-MIMO). |

Tabella 3.6: Tappe storiche ed evoluzione tecnologica degli standard IEEE 802.11 (Wi-Fi).

### 3.18.3 Entità Architetturali di una WLAN: BSS, ESS e Access Point

L'architettura logica di una rete locale senza fili si articola sui seguenti componenti:

- **Station (STA)**: qualsiasi dispositivo terminale equipaggiato con una scheda di interfaccia di rete radio conforme a 802.11 (laptop, smartphone, apparati IoT).
- **Access Point (AP)**: dispositivo di coordinamento centrale dotato di interfaccia radio e porta cablata Ethernet. Opera logicamente come un ponte di Livello 2 (*bridge*), commutando le trame tra il mezzo radio condiviso e la dorsale cablata dell'edificio.
- **Basic Service Set (BSS)**: la cella elementare della rete wireless. Nella modalità con infrastruttura (*Infrastructure Mode*), comprende l'Access Point e l'insieme delle stazioni radio associate a esso.
- **Independent BSS (IBSS o Modalità Ad-Hoc)**: configurazione priva di Access Point, in cui le stazioni comunicano direttamente peer-to-peer per scambi temporanei.
- **Extended Service Set (ESS)**: unione di due o più BSS contigui interconnessi attraverso un **Distribution System (DS)** (tipicamente una rete commutata Ethernet). L'ESS consente agli utenti di spostarsi da una cella radio all'altra (**Roaming**) mantenendo inalterata la sessione e l'indirizzo IP di Livello 3.

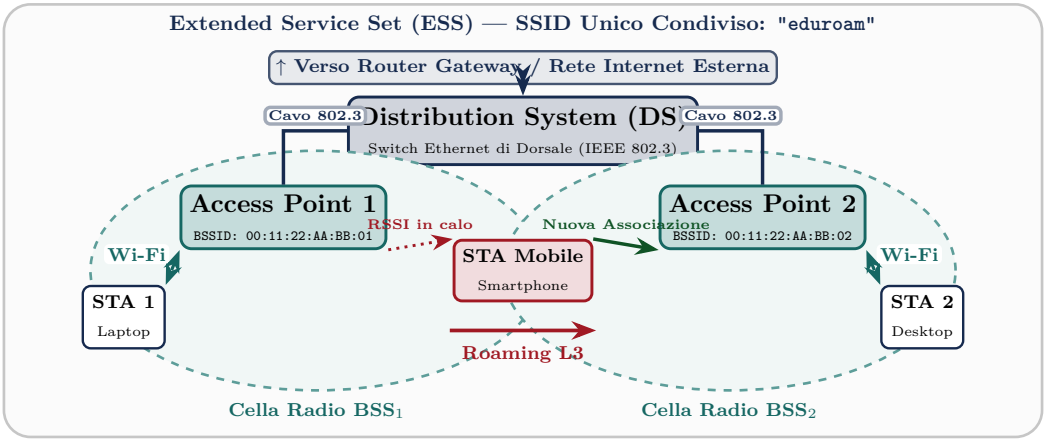



Figura 3.23: Architettura di una rete WLAN IEEE 802.11 in modalità infrastruttura: l'Extended Service Set (ESS) raggruppa celle elementari ($BSS_1$ e $BSS_2$) identificate dallo stesso SSID logico. Gli Access Point commutano il traffico verso la dorsale cablata (Distribution System), garantendo il roaming continuo dei terminali senza interruzione della sessione IP.

### 3.18.4 Identificatori di Rete: SSID vs. BSSID e Trame Beacon

Per gestire l'associazione radio, lo standard definisce due identificatori distinti:

1. **SSID (Service Set Identifier)**: stringa alfanumerica (fino a 32 caratteri) che definisce il nome logico e visibile della rete agli utenti (ad es. `"eduroam"` o `"WiFi-Casa"`). Viene annunciato periodicamente dagli Access Point attraverso trame di gestione speciali trasmesse in broadcast ogni circa 100 ms, denominate

**Beacon Frames**. I beacon sincronizzano il clock interno delle stazioni e trasportano parametri di modulazione, canali supportati e requisiti di sicurezza.

2. **BSSID (Basic Service Set Identifier)**: indirizzo MAC fisico a 48 bit dell'interfaccia radio dell'Access Point specifico che governa la cella. In un grande campus universitario coperto da un unico SSID distribuito (`"eduroam"`), ciascun Access Point possiede un BSSID univoco differente. Quando un utente cammina lungo i corridoi, il dispositivo mobile misura la potenza del segnale (RSSI) dei vari BSSID e seleziona autonomamente l'AP ottimale a cui associarsi.

## 3.19 Protocollo di Accesso Wi-Fi: CSMA/CA e Meccanismi di Risoluzione

### 3.19.1 L'Impossibilità Fisica di Collision Detection su Canale Radio

La differenza fondamentale tra Ethernet cablata e Wi-Fi risiede nell'impossibilità di implementare la Collision Detection (CD) via radio:

$$P_{\text{rx}} \propto \frac{P_{\text{tx}}}{d^n} \tag{3.124}$$

La potenza immessa nel canale dall'antenna del trasmettitore locale ($P_{\text{tx}} \approx 100\,\text{mW} = +20\,\text{dBm}$) supera di un fattore compreso tra $10^4$ e $10^{10}$ (40–100 dB) la debolissima potenza del segnale che giunge da una stazione remota ($P_{\text{rx}} \approx -70 - -90\,\text{dBm}$).

Se una seconda stazione trasmettesse contemporaneamente, l'incremento di potenza rilevabile sull'antenna del trasmettitore locale sarebbe inferiore a una parte su 10.000 (una frazione infinitesima dello 0,01%). L'emissione locale acceca completamente i circuiti di ricezione; nessun apparato economico a radiofrequenza può rilevare una collisione durante la trasmissione.

Se il Collision Detection è impossibile, il protocollo non può interrompere le collisioni in corso; se due trame collidono, l'intera durata della trasmissione andrà sprecata. L'accesso al canale deve quindi basarsi sulla prevenzione preventiva delle collisioni: nasce così il protocollo **CSMA/CA** (*Collision Avoidance*).

### 3.19.2 Spazi Inter-Trama Prioritari (IFS) e Gestione su Canale Libero

La coordinazione temporale dell'accesso si basa su intervalli calibrati di silenzio radio denominati **Inter-Frame Spaces (IFS)**:

- **SIFS (Short Inter-Frame Space)**: il tempo più breve in assoluto ($\sim 10\,\mu$s su 802.11b/g, $16\,\mu$s su 802.11a). Gode della **massima priorità**; è riservato a trasmissioni che completano un dialogo già in corso: riscontri (**ACK**), risposte **CTS** e frammenti successivi di dati. Nessuna stazione terza può iniziare a contendere durante un SIFS.
- **PIFS (Point Coordination IFS)**: pari a SIFS + $T_{\text{slot}}$, riservato all'Access Point per la gestione centralizzata a polling (PCF).
- **DIFS (Distributed IFS)**: pari a SIFS $+ 2 \times T_{\text{slot}}$ ($\approx 28$–$50\,\mu$s). Tempo di attesa obbligatorio su canale libero prima che una stazione possa trasmettere una trama ordinaria di dati o un pacchetto RTS.
- **EIFS (Extended IFS)**: tempo di guardia prolungato impiegato quando il livello fisico riceve una trama con bit errati o CRC non decodificabile, allo scopo di evitare che una nuova trasmissione sovrascriva un eventuale ACK in transito.

**Principio / Fondamento: Funzionamento del Dialogo Base su Canale Libero e Risparmio Energetico (NAV)**

1. La stazione ascolta il canale: se rimane continuativamente libero per almeno un tempo **DIFS**, inizia immediatamente a trasmettere la trama dati.
2. Il ricevitore decodifica la trama, calcola il CRC-32 e, se valida, attende esattamente un intervallo **SIFS** e invia indietro la trama di **ACK**.
3. **Network Allocation Vector (NAV) e Virtual Sensing**: l'intestazione della trama include il campo *Duration/ID*, che quantifica la durata complessiva del dialogo residuo (tempo trasmissione dati + SIFS + ACK). Tutte le stazioni terze leggono tale campo e impostano il proprio contatore interno **NAV**. Durante il periodo indicato dal NAV, le stazioni considerano il canale virtualmente occupato e **disattivano i circuiti di ricezione** (*sleep mode*), preservando l'autonomia della batteria dei dispositivi mobili.

### 3.19.3 Contesa su Canale Occupato e la Regola del Congelamento (Freeze)

Se al momento della generazione del pacchetto la stazione percepisce che il canale è occupato da un'altra trasmissione, essa non trasmette:

1. Attende che la trasmissione corrente termini e che il canale rimanga libero per un tempo consecutivo pari a **DIFS**;
2. Discretizza il tempo in micro-slot ($T_{\text{slot}} \approx 9$–$20\,\mu$s) ed estrae una variabile casuale intera uniforme all'interno della finestra di contesa (**Contention Window, CW**):

$$r \in \{0, 1, 2, \ldots, \text{CW} - 1\} \qquad \text{con } \text{CW}_{\min} = 16 \text{ (oppure 32)} \tag{3.125}$$

3. Per ogni slot temporale in cui il canale rimane libero, il contatore viene decrementato di 1 ($r \leftarrow r - 1$). Quando il contatore tocca lo zero ($r = 0$), la stazione avvia la trasmissione.

**Principio / Fondamento: La Regola del Congelamento (Freeze del Contatore) e l'Equità a Breve Termine**

Un meccanismo cardine del Wi-Fi è la conservazione del contatore di backoff:

- Se mentre una stazione sta decrementando il proprio contatore un'altra stazione arriva a zero e inizia a trasmettere, il canale torna occupato. La stazione in conteggio **congela istantaneamente il proprio contatore residuo** (ad esempio a $r = 3$) e sospende il countdown.
- Quando il canale torna definitivamente libero e trascorre un nuovo intervallo DIFS, la stazione **non estrae un nuovo numero casuale**, ma riprende a contare dal valore precedentemente congelato ($3 \to 2 \to 1 \to 0$).

**Perché il Freeze garantisce equità a breve termine?** I protocolli ad accesso casuale privi di freeze sono puramente *asintoticamente equi* (le risorse convergono alla parità solo sull'orizzonte infinito). L'utente umano ha tuttavia bisogno di equità immediata. Chi ha appena vinto la contesa precedente e ha trasmesso la trama, per inviarne un'altra deve estrarre un nuovo valore casuale sull'intera finestra $[0, \text{CW} - 1]$ (valore medio $\approx 7{,}5$ per $\text{CW} = 16$). Chi invece ha perso la contesa precedente parte già con un contatore ridotto ($r = 2$ o $1$). Di conseguenza, chi ha atteso riceve un vantaggio statistico immediato nelle contese successive, impedendo la monopolizzazione del canale e garantendo un accesso distribuito ed equo su tempi brevissimi.

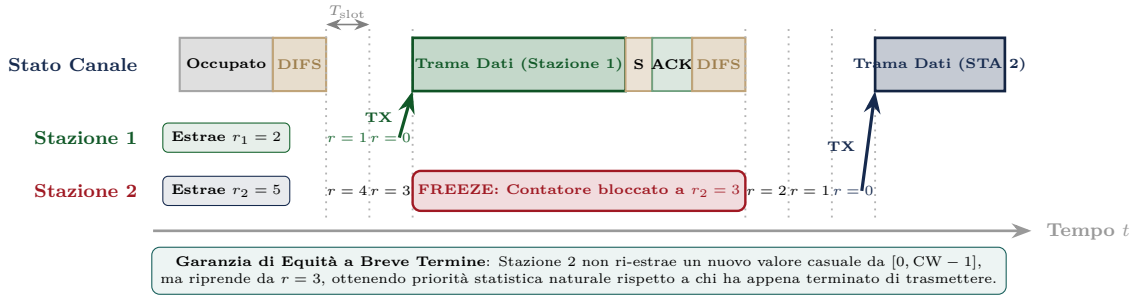



Figura 3.24: Cronogramma del protocollo CSMA/CA con meccanismo di congelamento (*freeze*) del contatore di backoff. La Stazione 1 vince la contesa azzerando per prima il timer; la Stazione 2 congela istantaneamente il proprio contatore residuo a $r = 3$ non appena avverte il canale occupato, riprendendo il countdown dal medesimo valore dopo il successivo DIFS.

### 3.19.4 Riscontro Esplicito (ACK) e Raddoppio della Contention Window

Poiché la Collision Detection è assente, l'unica prova del buon esito della trasmissione è l'arrivo dell'ACK entro una finestra temporale prestabilita.

Se l'ACK non giunge:

1. La stazione presume che si sia verificata una collisione (anche se potrebbe trattarsi di rumore di canale);
2. Raddoppia la finestra di contesa: $\text{CW} \leftarrow \min(2 \cdot \text{CW}, \text{CW}_{\max})$, con $\text{CW}_{\max} = 1024$;
3. Estrae una nuova variabile casuale sulla finestra allargata e ritenta la contesa;
4. Se una trama subisce **7 fallimenti consecutivi**, la stazione desiste, scarta definitivamente la trama e segnala una condizione di fallimento della connessione.

### 3.19.5 Geometrie Critiche del Mezzo Radio: Terminale Nascosto ed Esposto

**1. Il Problema del Terminale Nascosto (Hidden Node Problem):** Si verifica quando due stazioni $A$ e $C$ si trovano ai lati opposti di una stazione o Access Point intermedio $B$. Sia $A$ che $C$ riescono a comunicare efficacemente con $B$, ma a causa della distanza o di ostacoli interposti non si avvertono a vicenda:

- Quando $A$ sta trasmettendo a $B$, la stazione $C$ esegue il carrier sensing; non avvertendo il segnale di $A$, deduce falsamente che il canale sia libero e inizia a trasmettere verso $B$.
- I segnali collidono presso il ricevitore comune $B$, provocando collisioni sistematiche.

**Soluzione con Handshaking a Quattro Vie (RTS/CTS):** Per mitigare il problema, lo standard definisce lo scambio preliminare di micro-trame di controllo:

1. La sorgente $A$ attende per DIFS e invia un breve pacchetto **RTS** (*Request to Send*);
2. Se $B$ riceve l'RTS, attende per SIFS e risponde con un **CTS** (*Clear to Send*);
3. Poiché il CTS è trasmesso da $B$, esso raggiunge sia $A$ sia la stazione nascosta $C$! La stazione $C$ legge il campo Duration nel CTS e imposta il proprio **NAV**, congelandosi per tutta la durata del successivo scambio dati tra $A$ e $B$;
4. Ricevuto il CTS, la stazione $A$ invia la trama dati e riceve l'ACK da $B$, con ogni scambio distanziato dal tempo protetto SIFS.

Poiché l'RTS e il CTS sono trame brevissime, l'eventuale collisione iniziale dura una frazione minima di tempo rispetto a una collisione sull'intero pacchetto dati. Tuttavia, l'handshaking introduce un notevole overhead (2 trame e 2 SIFS in più); per questo motivo gli Access Point definiscono una soglia (*RTS Threshold*): RTS/CTS viene impiegato solo se la dimensione della trama supera tale valore (nella pratica commerciale reale la soglia è spesso fissata al massimo, disattivando di fatto RTS/CTS).

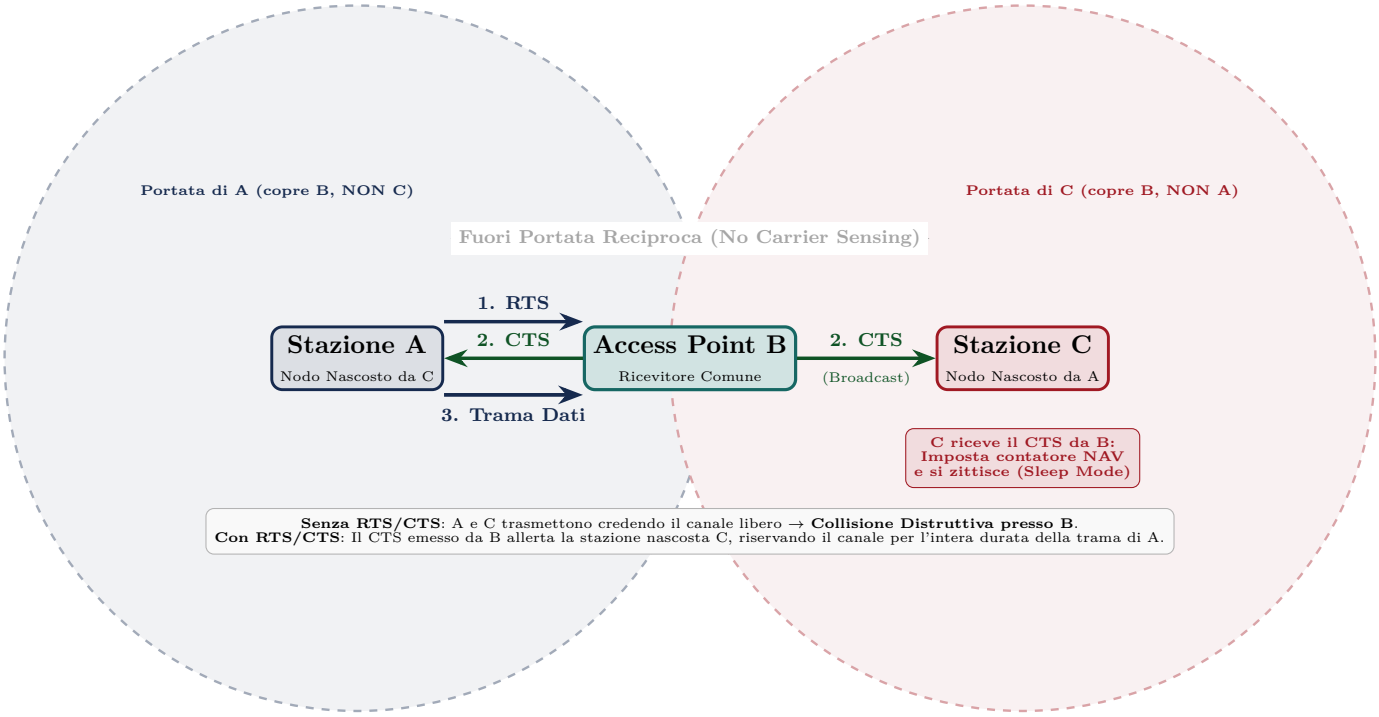



Figura 3.25: Il Problema del Terminale Nascosto (*Hidden Node*) nelle reti Wi-Fi e sua risoluzione tramite handshake a quattro vie (RTS/CTS). Le stazioni A e C non possono avvertirsi reciprocamente; l'Access Point intermedio B trasmette il Clear to Send (CTS) in broadcast, consentendo a C di acquisire la durata dell'occupazione e impostare il proprio Network Allocation Vector (NAV) per congelarsi durante la trasmissione di A.

**2. Il Problema del Terminale Esposto (Exposed Node Problem):** Per standard, la sensibilità del ricevitore per il sensing del segnale sul canale ($R_{\text{CS}}$) è sensibilmente più estesa rispetto alla portata utile di trasmissione e corretta decodifica ($R_{\text{TX}}$):

$$R_{\text{CS}} > R_{\text{TX}} \tag{3.126}$$

Si consideri una topologia a quattro nodi disposti nello spazio: $A \leftarrow B$ e $C \rightarrow D$. La stazione $B$ sta trasmettendo verso il proprio ricevitore $A$. Contemporaneamente, la stazione $C$ intende trasmettere verso il ricevitore $D$, ma si trova all'interno del raggio di carrier sensing di $B$:

- Ogni volta che tenta di accedere, $C$ percepisce l'energia irradiata da $B$, crede falsamente che la propria trasmissione disturberebbe il canale e si blocca;
- Quando il countdown finalmente giunge a zero, $C$ trasmette, ma se $D$ è temporaneamente disturbato o la trama tarda a ricevere ACK, $C$ deduce una collisione e raddoppia la propria finestra CW;
- La probabilità di vincere le contese successive decade ulteriormente. Dopo ripetuti tentativi infruttuosi (7 collisioni), la stazione $C$ si convince che la rete sia indisponibile e si disconnette.

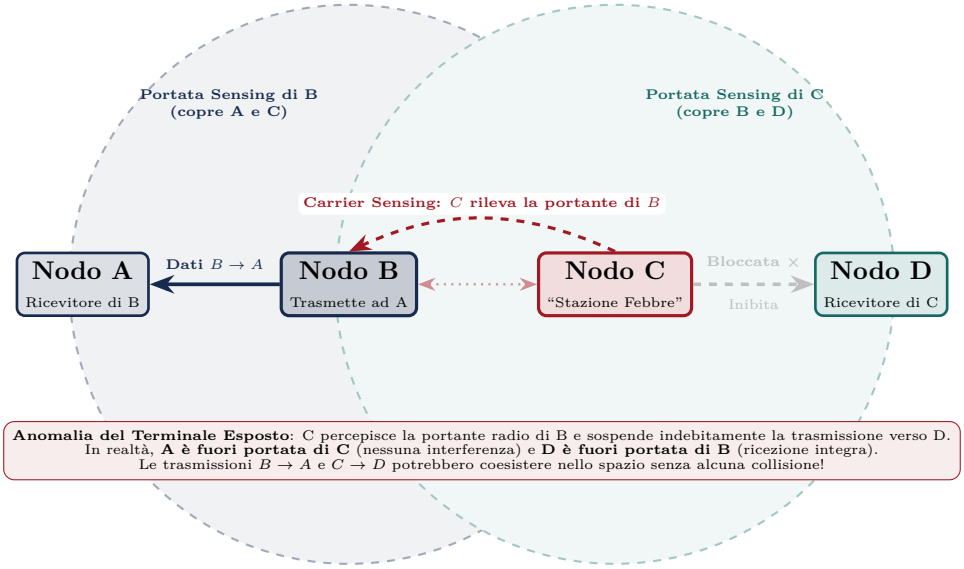



Figura 3.26: Il Problema del Terminale Esposto (*Exposed Node*) nelle reti radio locali. Il nodo C (la "stazione con la febbre") avverte l'energia elettromagnetica della trasmissione $B \rightarrow A$ e sospende indebitamente l'invio verso D, sebbene il ricevitore D sia immune all'interferenza di B e il ricevitore A sia oltre la portata di C, causando uno spreco di capacità spaziale del canale.

**3. Iniquità dell'Adattamento del Bit-Rate (Rate Adaptation):** Nel Wi-Fi, ogni scheda seleziona dinamicamente la propria velocità di trasmissione in funzione della qualità del segnale (SNR). Una stazione vicina all'AP trasmette a bit-rate elevato (es. 54 Mbit/s o oltre), occupando il canale per brevissimi microsecondi. Una stazione lontana adotta velocità minime (es. 1 Mbit/s o 6 Mbit/s) per limitare il BER: la sua trama impiega sul canale un tempo decine di volte superiore, monopolizzando il mezzo e degradando il throughput complessivo di tutti gli altri utenti. Inoltre, le stazioni periferiche incorrono in un maggior numero di errori di canale (scambiati erroneamente per collisioni), subendo raddoppi continui di CW che ne penalizzano ingiustamente l'accesso.

### 3.19.6 Meccanismi ad Alte Prestazioni: TXOP e Aggregazione Trame

Per superare i limiti di efficienza del CSMA/CA su reti Gigabit (802.11n/ac/ax):

- **Transmission Opportunity (TXOP)**: la contesa non avviene più per singolo pacchetto, ma per un intervallo temporale predefinito. La stazione che vince la contesa ha facoltà di trasmettere una sequenza continuativa di trame multiple separate esclusivamente da brevi intervalli SIFS.
- **Aggregazione delle Trame (A-MPDU e A-MSDU)**: per evitare che il tempo occupato da preamboli fisici e spazi DIFS superi la durata dei dati a velocità gigabit, decine di datagrammi IP vengono fusi in un'unica super-trama fisica (A-MPDU), convalidata al termine da un singolo **Block ACK** cumulativo.

### 3.19.7 Esercizio d'Esame: Collision Avoidance, Backoff e Throughput in Reti Wi-Fi

A completamento dell'analisi prestazionale dei protocolli di accesso al mezzo conteso, si riporta la risoluzione analitica integrale del secondo problema dell'**Esercitazione Ufficiale MAC 2** (*A.A. 2026-2027, Prof. Renato Lo Cigno e Dott. Lorenzo Ghiro*). L'esercizio esamina un BSS wireless 802.11 operante in modalità CSMA/CA, confrontandone direttamente i risultati con lo scenario Ethernet cablato svolto nella Sezione 3.16.6.

## Esercizio d'Esame: Analisi Prestazionale, Collision Avoidance e Throughput in Reti Wi-Fi (Esercitazione MAC 2 — Parte 2)

**Testo del Problema Ufficiale:**
Si consideri un Basic Service Set (BSS) Wi-Fi che replica la medesima topologia logica e operativa dell'esercizio precedente (Sezione 3.16.6): il router è attestato a un Access Point (AP) e il canale radio funge da mezzo broadcast condiviso su cui le trasmissioni contemporanee collidono. Sebbene non esista uno standard 802.11 con velocità fisica esattamente pari a 100 Mbit/s, si assume tale valore di capacità nominale per rendere i risultati rigorosamente confrontabili con Ethernet. I parametri operativi del sistema Wi-Fi sono:

- Capacità nominale di canale: $C = 100\,\text{Mbit/s}$;
- Carico utile (*payload*): variabile tra 46 e 1500 byte;
- Intestazione MAC di Livello 2: 18 byte complessivi (14 byte di header e 4 byte di CRC);
- Preambolo di Livello Fisico: 12 byte;
- Spazio inter-trama breve: SIFS $= 10\,\mu$s;
- Durata dello slot di contesa: $T_S = 20\,\mu$s;
- Spazio inter-trama distribuito (*DCF Interframe Space*): DIFS $= 50\,\mu$s;
- Finestra di contesa minima: $CW_{\min} = 32$ slot (con indici casuali estratti uniformemente nell'intervallo $\{0, 1, \ldots, 31\}$);
- Riscontro esplicito (*ACK*): formato unicamente dall'intestazione MAC da 18 byte più il preambolo fisico da 12 byte (totale 30 byte);
- Dinamica di contesa: durante il count-down del backoff le stazioni continuano il carrier sensing; la prima che conclude il conteggio trasmette, costringendo la rivale a congelare il timer per poi riprenderlo appena il mezzo torna libero.

Si richiede di risolvere analiticamente i seguenti quesiti:

**(a)** Nel caso $N = 2$ con **1 sola stazione satura** (ha sempre pacchetti da inviare, mentre l'AP/router non trasmette mai), calcolare la probabilità di collisione $P_C$ e il throughput massimo conseguibile.

**(b)** Nel caso $N = 2$ con **entrambi i nodi saturi** (stazione e AP competono permanentemente per il mezzo), calcolare la probabilità di collisione e di scarto della trama assumendo ammesse al massimo 6 collisioni a finestra crescente prima dell'abbandono.

**(c)** Nel caso di $N$ generico con traffico aggregato poissoniano a tasso complessivo $\lambda$ trame/s:

**c.1)** Calcolare il valore di $\lambda$ corrispondente al carico normalizzato $G = 1$ nel caso di trame a dimensione massima (1500 byte di payload).

**c.2)** Fissando $N = 2$, trame a dimensione massima e carico offerto da ciascuna stazione pari a $G = 0{,}2$, calcolare la probabilità di osservare $k = 1, 2, 3$ collisioni consecutive, assumendo che lo stato iniziale del canale sia occupato.




Figura 3.27: Topologia del BSS Wi-Fi: l'Access Point è collegato al router di rete locale e condivide il canale a radiofrequenza con la stazione STA; le collisioni avvengono nell'etere e richiedono il protocollo CSMA/CA con riscontri espliciti.

**Risoluzione Analitica Dettagliata:**

**Quesito (a) — Singola Stazione Satura e Crollo Prestazionale del Throughput:** Con una sola stazione attiva e l'AP permanentemente in ricezione passiva, non vi è alcun competitore sul mezzo:

$$P_C = 0 \tag{3.127}$$

Tuttavia, diversamente da Ethernet dove la stazione satura trasmette una trama dopo l'altra intervallate dal solo IPG (12 byte $\approx 0{,}96\,\mu$s), nel protocollo **Wi-Fi CSMA/CA la stazione satura è vincolata a rispettare il ciclo completo di contesa e riscontro** (Figure 3.28 e 3.29):

1. **Dialogo con Riscontro (ACK)**: il mezzo wireless è inaffidabile e non supporta la Collision Detection; ogni trasmissione dati deve essere seguita da un silenzio SIFS e dall'emissione di una trama ACK da parte del ricevitore;
2. **Backoff Obbligatorio a Canale Occupato**: se la stazione ha pacchetti sempre pronti, il canale risulta continuamente occupato; per le regole CSMA/CA, prima di ogni nuova trasmissione la stazione deve attendere un tempo DIFS e completare un'intera procedura di **Backoff**, estraendo un numero casuale di slot!

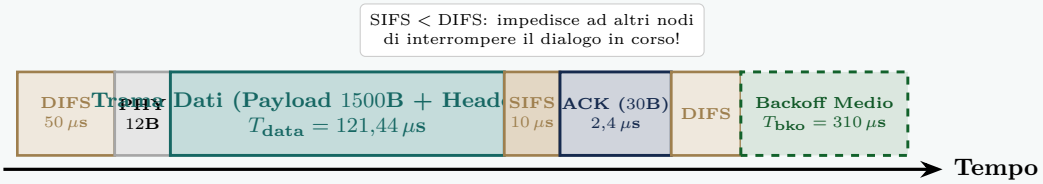



Figura 3.28: Struttura temporale del dialogo completo Wi-Fi CSMA/CA: l'invio della trama dati è protetto dal silenzio SIFS fino alla ricezione dell'ACK; tra dialoghi consecutivi la stazione satura deve attendere un DIFS e scontare un backoff medio.

Calcoliamo i singoli contributi temporali del ciclo a regime:

- **Tempo di trasmissione dati ($T_{\text{data}}$)**: corrisponde ai 1518 byte utili di Livello 2:

$$T_{\text{data}} = \frac{1518\,\text{byte} \cdot 8\ \text{bit/byte}}{100 \cdot 10^6\ \text{bit/s}} = \frac{12144\,\text{bit}}{10^8\ \text{bit/s}} = 121{,}44\,\mu\text{s} \approx \mathbf{121}\,\mu\text{s} \tag{3.128}$$

- **Overhead fisico dati ($T_{\text{ovh}}$)**: preambolo PHY da 12 byte:

$$T_{\text{ovh}} = \frac{12 \cdot 8}{100 \cdot 10^6} = 0{,}96\,\mu\text{s} \approx \mathbf{1}\,\mu\text{s} \tag{3.129}$$

- **Tempo di riscontro ACK ($T_{\text{ack}}$)**: header da 14 B + CRC 4 B + PHY 12 B = 30 byte:

$$T_{\text{ack}} = \frac{30 \cdot 8}{100 \cdot 10^6} = 2{,}4\,\mu\text{s} \approx \mathbf{2}\,\mu\text{s} \tag{3.130}$$

- **Tempo medio di Backoff ($T_{\text{bko}}$)**: in assenza di collisioni la finestra è minima ($CW_{\min} = 32$ slot, da 0 a 31). Essendo la scelta uniforme tra 0 e 31, il numero medio di slot estratti è $31/2 = 15{,}5$:

$$T_{\text{bko}} = \frac{31 \cdot T_S}{2} = \frac{31 \cdot 20\,\mu\text{s}}{2} = 31 \cdot 10\,\mu\text{s} = \mathbf{310}\,\mu\text{s} \tag{3.131}$$

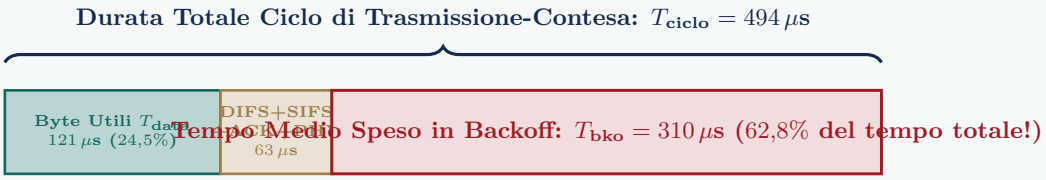



Figura 3.29: Ripartizione temporale del ciclo Wi-Fi: oltre il 62% del tempo di canale è interamente assorbito dal conteggio del backoff, provocando il crollo del throughput rispetto ad Ethernet.

La durata totale del ciclo di trasmissione-contesa risulta:

$$T_{\text{ciclo}} = T_{\text{data}} + T_{\text{ovh}} + \text{SIFS} + T_{\text{ack}} + \text{DIFS} + T_{\text{bko}} = 121 + 1 + 10 + 2 + 50 + 310 = \mathbf{494}\,\mu\text{s} \tag{3.132}$$

Il throughput massimo effettivo di Livello 2 ottenuto dalla stazione satura in Wi-Fi vale:

$$\text{Thr}_{\text{Wi-Fi}} = \frac{T_{\text{data}}}{T_{\text{ciclo}}} \cdot C = \frac{121\,\mu\text{s}}{494\,\mu\text{s}} \cdot 100\,\text{Mbit/s} \approx 0{,}244939 \cdot 100\,\text{Mbit/s} \approx \mathbf{24{,}5}\,\text{Mbit/s} \tag{3.133}$$

**Principio / Fondamento: Il Costo Fisico della Collision Avoidance: Confronto Ethernet vs. Wi-Fi**

Il confronto quantitativo tra i due esercizi evidenzia un risultato architetturale drammatico:

- Su **Ethernet cablata** (Sezione 3.16.6), a parità di banda (100 Mbit/s) e in assenza di collisioni, il throughput raggiunge **98**,**83** Mbit/s;
- Su **Wi-Fi** il throughput crolla a soli **24**,**5** Mbit/s, registrando una perdita netta di oltre un **fattore quattro**!

Questo divario riflette il pesante overhead protocollare imposto dal mezzo radio:

1. L'obbligo di riscontro esplicito con trame ACK e intervalli di silenzio SIFS/DIFS;
2. L'esecuzione obbligatoria del Backoff (310 $\mu$s), che dura **più del doppio dell'emissione dei dati stessi** (121 $\mu$s), sprecando oltre il 62% del tempo di canale perfino in assenza totale di rivali!

È proprio per arginare questo degrado strutturale che nelle moderne evoluzioni di 802.11:

- A 5 GHz lo slot time $T_S$ è stato abbattuto da 20 $\mu$s a soli 9 $\mu$s, riducendo drasticamente la durata del backoff;
- Negli standard IEEE 802.11n/ac/ax sono state introdotte le **TXOP** (*Transmission Opportunity*) e l'**Aggregazione A-MPDU**, che consentono alla stazione che vince la contesa di inviare raffiche di decine di pacchetti consecutivi ammortizzando il costo di un singolo backoff!

**Quesito (b) — Due Nodi Saturi e Probabilità di Scarto a 6 Collisioni:** Consideriamo ora $N = 2$ stazioni permanentemente sature (STA e AP) in competizione. Nel protocollo Wi-Fi CSMA/CA, a differenza di Ethernet, **la prima collisione non è deterministica**: ciascun nodo estrae in modo indipendente uno slot casuale nell'intervallo iniziale $CW_{\min} = 32 = 2^5$. La collisione si manifesta se e solo se entrambi estraggono esattamente il medesimo slot. La probabilità della prima collisione vale pertanto:

$$P_{C,1} = \frac{1}{CW_{\min}} = \frac{1}{2^5} = \frac{1}{32} \tag{3.134}$$

Se si verifica una collisione (mancata ricezione dell'ACK entro il timeout ACK), le stazioni raddoppiano la finestra di contesa esponenzialmente:

$$CW_n = 2^{5+n-1} \qquad \text{per } n = 1, 2, \ldots, 6 \tag{3.135}$$

La successione delle probabilità di collisione condizionate per i sei tentativi risulta:

| **Tentativo ($n$)** | **Finestra $CW_n$** | **Probabilità di Collisione Condizionata** |
|---|---|---|
| $n = 1$ | $2^5 = 32$ | $P(C_1) = 1/2^5 = 1/32$ |
| $n = 2$ | $2^6 = 64$ | $P(C_2 \mid C_1) = 1/2^6 = 1/64$ |
| $n = 3$ | $2^7 = 128$ | $P(C_3 \mid C_1, C_2) = 1/2^7 = 1/128$ |
| $n = 4$ | $2^8 = 256$ | $P(C_4 \mid \ldots) = 1/2^8 = 1/256$ |
| $n = 5$ | $2^9 = 512$ | $P(C_5 \mid \ldots) = 1/2^9 = 1/512$ |
| $n = 6$ | $2^{10} = 1024$ | $P(C_6 \mid \ldots) = 1/2^{10} = 1/1024$ |

Tabella 3.7: Progressione esponenziale delle finestre di contesa e relative probabilità condizionate di collisione nei sei tentativi ammessi in Wi-Fi CSMA/CA.

La probabilità di subire esattamente 6 collisioni consecutive, esaurendo i tentativi ammessi e scartando la trama, è data dal prodotto:

$$\begin{aligned} P_{C,6} &= \prod_{i=1}^{6} \frac{1}{2^{5+i-1}} = \frac{1}{2^5 \cdot 2^6 \cdot 2^7 \cdot 2^8 \cdot 2^9 \cdot 2^{10}} \\ &= 2^{-(5+6+7+8+9+10)} = 2^{-45} \approx \mathbf{2{,}84 \cdot 10^{-14}} \end{aligned} \tag{3.136}$$

**Confronto della Probabilità di Scarto: Ethernet vs. Wi-Fi**

Confrontando questo risultato con quello calcolato per Ethernet nell'Esempio 3.16.6:

- In **Ethernet**: $P_{C,6} = 2^{-15} \approx \mathbf{3{,}05 \cdot 10^{-5}}$;
- In **Wi-Fi**: $P_{C,6} = 2^{-45} \approx \mathbf{2{,}84 \cdot 10^{-14}}$.

Grazie alla Collision Avoidance (partenza con finestra ampia da 32 slot anziché 1 slot deterministico), in Wi-Fi la probabilità di perdita è **nove ordini di grandezza inferiore**! Questo spiega perché lo standard IEEE 802.11 adotti una soglia di scarto molto più conservativa, fissata a soli **7 tentativi** (*dot11ShortRetryLimit*) rispetto ai 16 di Ethernet.

**Quesito (c.1) — Calcolo del Tasso Poissoniano $\lambda$ per Carico Unitario $G = 1$:** Nel contesto Wi-Fi, la nozione di "tempo di trama" $T$ coincide con la durata di un **dialogo completo con successo** su canale privo di interferenze:

$$T_{\text{dialogo}} = T_{\text{ovh}} + \text{DIFS} + T_{\text{data}} + \text{SIFS} + T_{\text{ack}} = (1 + 50 + 122 + 10 + 2)\,\mu\text{s} = \mathbf{185}\,\mu\text{s} \tag{3.137}$$

Imponendo la condizione di carico normalizzato unitario $G = \lambda \cdot T_{\text{dialogo}} = 1$:

$$\lambda = \frac{1}{T_{\text{dialogo}}} = \frac{1}{185 \cdot 10^{-6}\,\text{s}} \approx \mathbf{5405}\,\text{trame/s} \tag{3.138}$$

(In Ethernet il medesimo carico unitario corrispondeva a ben 8138 trame/s, a ulteriore riprova della maggiore dilatazione temporale introdotta dai protocolli wireless).

**Quesito (c.2) — Probabilità di Collisioni Consecutive con Canale Inizialmente Occupato:** Siano $N = 2$ stazioni che offrono ciascuna un carico $G = 0{,}2$, con canale inizialmente occupato. La probabilità che una stazione generi almeno una trama durante il tempo di dialogo è data dalla distribuzione di Poisson:

$$P[\geq 1 \text{ arrivo}] = 1 - e^{-G} = 1 - e^{-0{,}2} \approx 0{,}181269 \tag{3.139}$$

Affinché si verifichi la prima collisione, devono verificarsi congiuntamente due eventi indipendenti:

1. **Entrambe le stazioni generano almeno una trama** durante l'occupazione: probabilità pari a $(1 - e^{-0{,}2})^2$;
2. **Entrambe scelgono casualmente il medesimo slot** di contesa tra i 32 disponibili in $CW_{\min}$: probabilità condizionata pari a $2^{-5} = 1/32$.

La probabilità della prima collisione risulta:

$$P_{C,1} = \left(1 - e^{-0{,}2}\right)^2 \cdot 2^{-5} \approx (0{,}181269)^2 \cdot \frac{1}{32} \approx \frac{0{,}032858}{32} \approx \mathbf{1{,}03 \cdot 10^{-3}} \approx 10^{-3} \tag{3.140}$$

Per le collisioni successive, le stazioni raddoppiano la finestra di contesa:

- **Due collisioni consecutive ($k = 2$)**: dopo la prima collisione la finestra è $CW_2 = 64 = 2^6$; collidono se scelgono lo stesso slot con probabilità $1/2^6$:

$$P_{C,2} = P_{C,1} \cdot \frac{1}{2^6} \approx \frac{1{,}0268 \cdot 10^{-3}}{64} \approx \mathbf{1{,}60 \cdot 10^{-5}} \tag{3.141}$$

- **Tre collisioni consecutive ($k = 3$)**: dopo la seconda collisione la finestra è $CW_3 = 128 = 2^7$:

$$P_{C,3} = P_{C,2} \cdot \frac{1}{2^7} \approx \frac{1{,}6044 \cdot 10^{-5}}{128} \approx \mathbf{1{,}25 \cdot 10^{-7}} \tag{3.142}$$

**Sintesi Comparativa Finale tra Ethernet e Wi-Fi:** Mettendo a diretto confronto le probabilità di collisione a parità di carico offerto ($G = 0{,}2$):

- Prima collisione: in Ethernet vale $P_{C,1} \approx 3{,}29 \cdot 10^{-2}$, mentre in Wi-Fi scende a $P_{C,1} \approx 1{,}03 \cdot 10^{-3}$ (**oltre 30 volte inferiore**);

- Seconda collisione: in Ethernet vale $1{,}64 \cdot 10^{-2}$, mentre in Wi-Fi crolla a $1{,}60 \cdot 10^{-5}$ (**mille volte inferiore**);
- Terza collisione: in Ethernet vale $0{,}41 \cdot 10^{-2}$, mentre in Wi-Fi è appena $1{,}25 \cdot 10^{-7}$ (**oltre trentamila volte inferiore**)!

La Collision Avoidance garantisce un abbattimento straordinario delle collisioni sul mezzo radio; il prezzo ingegneristico pagato è l'inefficienza sul throughput della singola stazione satura, che viene tuttavia ampiamente compensata in scenari multi-utente con traffico bidirezionale e flussi simmetrici dati/ACK.

## 3.20 Risoluzione degli Indirizzi (ARP) e Funzioni di Rete Locale

**NOTA INTEGRATIVA DI AGGIORNAMENTO DIDATTICO (A.A. 2026/2027)**

**Avviso agli Studenti sul Materiale delle Slide Ufficiali e sulle Lezioni Frontali:**
Durante le lezioni frontali dell'anno accademico di riferimento, a causa di una temporanea avaria tecnica ai microfoni d'aula, la spiegazione vocale del docente per le ultime slide del fascicolo didattico 3.3 (relative alla risoluzione degli indirizzi tramite il protocollo ARP, alla differenza tra consegna diretta e indiretta e al meccanismo di Proxy ARP) non è stata registrata nella traccia audio.
Per colmare in modo rigoroso e completo questo vuoto, la presente sezione espone la trattazione tecnica integrale e sistematica ricavata direttamente dalle slide del docente.

### 3.20.1 Incapsulamento e Duplicità dell'Indirizzamento: Indirizzi MAC vs. Indirizzi IP

Nelle reti a commutazione di pacchetto contemporanee si manifesta una fondamentale duplicità dell'indirizzamento tra Livello 3 (Rete) e Livello 2 (Collegamento):

- **Indirizzo Logico di Livello 3 (Indirizzo IP)**: coordinata logica e gerarchica a 32 bit (IPv4) con **valenza globale ed end-to-end**. Identifica l'interfaccia di rete nel contesto planetario dell'Internet. Rimane invariato nel datagramma IP lungo l'intero percorso dall'host sorgente iniziale fino all'host destinatario finale.
- **Indirizzo Fisico di Livello 2 (Indirizzo MAC)**: identificatore hardware a 48 bit cablato nella memoria della scheda di rete (NIC), con **valenza strettamente locale ed hop-by-hop**. Governa unicamente il transito della trama fisica all'interno della singola sottorete locale (dominio di broadcast). A ogni attraversamento di un router, la vecchia intestazione MAC viene rimossa e sostituita da una nuova intestazione contenente i MAC del nuovo collegamento.

Il datagramma IP (contenente a sua volta il segmento di trasporto) viene incapsulato nel payload della trama Ethernet.

### 3.20.2 Modalità di Risoluzione: Consegna Diretta vs. Consegna Indiretta

Quando un host genera un pacchetto IP da trasmettere, il modulo IP confronta la maschera di sottorete (*subnet mask*) dell'indirizzo IP di destinazione con la propria:

1. **Consegna Diretta (Direct Delivery)**: la destinazione appartiene alla medesima sottorete locale dell'host mittente. La trama di Livello 2 deve essere recapitata direttamente alla scheda di rete dell'host finale. Il mittente deve risolvere l'**indirizzo MAC dell'host destinatario**.
2. **Consegna Indiretta (Indirect Delivery)**: l'indirizzo IP di destinazione appartiene a una rete remota (ad esempio un web server oltreoceano). In questa circostanza si applica una regola cardine dell'architettura di rete:

   **Principio / Fondamento: Risoluzione nella Consegna Indiretta**

   Un host **non tenta MAI** di risolvere l'indirizzo MAC di una macchina appartenente a un'altra sottorete. L'host incapsula il datagramma mantenendo l'IP di destinazione finale nel campo `IP Destination`, ma risolve e imposta come `MAC Destination` l'indirizzo fisico del **Default Gateway** (il router locale di uscita, next-hop):

$$\text{Trama L2}: \begin{cases} \text{MAC Sorgente} = \text{MAC Host Locale} \\ \text{MAC Destinazione} = \textbf{MAC Router Default Gateway} \end{cases} \tag{3.143}$$

### 3.20.3 La Filosofia Broadcast-and-Select e il Protocollo ARP

Come mappare dinamicamente un indirizzo IP a 32 bit nel rispettivo indirizzo MAC a 48 bit? L'impiego di un server centralizzato di risoluzione (simile a un DNS locale) comporterebbe gravi criticità: single point of failure, carico concentrato e impossibilità per un database statico di gestire agilmente la mobilità e la consegna indiretta.

L'IETF ha perciò standardizzato il protocollo **ARP** (*Address Resolution Protocol*, RFC 826), fondato sul paradigma **Broadcast-and-Select**:

1. **ARP Request (Broadcast L2)**: quando l'host $A$ deve recapitare una trama all'indirizzo $IP_B$ di cui ignora il MAC, formula una domanda pubblica: *"A chi appartiene l'indirizzo IP $IP_B$? Comunica il tuo indirizzo MAC ad A"*. La richiesta viene emessa all'interno di una trama avente indirizzo MAC di destinazione impostato a `FF:FF:FF:FF:FF:FF` (broadcast di Livello 2). Tutti i nodi della sottorete ricevono ed esaminano il messaggio.
2. **ARP Reply (Unicast L2)**: ciascun nodo confronta il target IP con il proprio: tutti scartano silenziosamente la trama, tranne l'host $B$, legittimo proprietario di $IP_B$, il quale risponde inviando una trama **unicast** direttamente all'indirizzo MAC di $A$: *"L'indirizzo $IP_B$ è mio e questo è il mio MAC $MAC_B$"*.

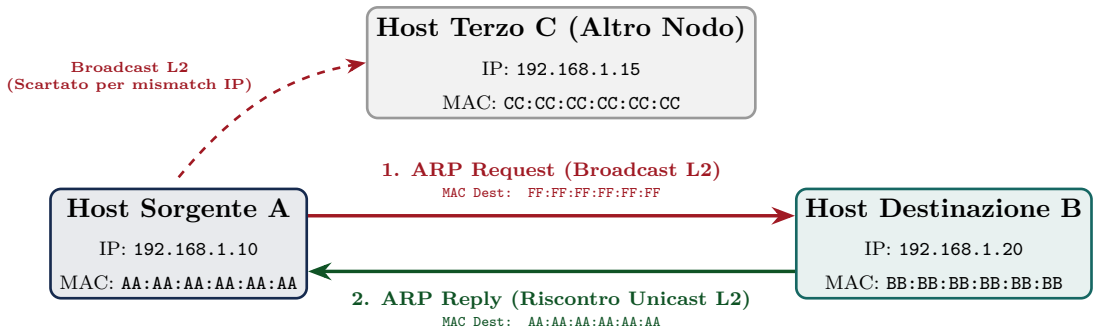



Figura 3.30: Flusso operativo del protocollo ARP: interrogazione broadcast di Livello 2 e riscontro unicast diretto dal nodo destinatario.

### 3.20.4 Formato Analitico del Pacchetto ARP (RFC 826)

Il pacchetto ARP è progettato con campi a lunghezza variabile per essere indipendente dalla tecnologia fisica e di rete sottostante (spesso definito protocollo di Livello 2.5). I suoi campi sono dettagliati nella Tabella 3.8.

| Campo | Lunghezza | Valore (IPv4 / Eth) | Descrizione Funzionale |
|---|---|---|---|
| **Hardware Type** | 16 bit | `1` | Specifica la tecnologia hardware (valore `1` identifica reti Ethernet/IEEE 802). |
| **Protocol Type** | 16 bit | `0x0800` | Specifica il protocollo di rete da risolvere (valore `0x0800` per IPv4). |
| **HADDR LEN** | 8 bit | `6` | Lunghezza in byte dell'indirizzo hardware (6 ottetti per MAC). |
| **PADDR LEN** | 8 bit | `4` | Lunghezza in byte dell'indirizzo di protocollo (4 ottetti per IPv4). |
| **Operation (Opcode)** | 16 bit | `1` (Req) / `2` (Rep) | Tipo di messaggio: `1` indica ARP Request, `2` indica ARP Reply. |
| **Sender HADDR** | 6 byte | MAC Mittente | Indirizzo fisico MAC della stazione che emette il pacchetto ARP. |
| **Sender PADDR** | 4 byte | IP Mittente | Indirizzo logico IPv4 della stazione che emette il pacchetto ARP. |
| **Target HADDR** | 6 byte | `0` (Req) / MAC Dest (Rep) | Indirizzo MAC cercato. Nella richiesta è azzerato (`00:00:00:00:00:00`). |
| **Target PADDR** | 4 byte | IP Destinazione | Indirizzo IPv4 di cui si richiede la risoluzione hardware. |

Tabella 3.8: Struttura analitica del pacchetto ARP RFC 826 su reti Ethernet e IPv4 (totale 28 byte).

Il pacchetto ARP (lungo complessivamente 28 byte) viene incapsulato direttamente nel frame Ethernet impostando il campo EtherType al valore esadecimale `0x0806`.

### 3.20.5 Gestione della Tabella/Cache ARP e Caching Opportunistico

Inviare una richiesta ARP per ciascun pacchetto IP da trasmettere comporterebbe un triplicamento del traffico di rete (ARP Request + ARP Reply + Pacchetto Dati).

Ciascuna stazione mantiene pertanto una **ARP Table (o ARP Cache)** nella propria memoria di sistema:

- **Verifica Preventiva**: prima di emettere una richiesta ARP sul mezzo, il sistema operativo consulta la cache locale. Se la corrispondenza IP $\leftrightarrow$ MAC è presente, la trama viene incapsulata e spedita istantaneamente.
- **Apprendimento Opportunistico del Mittente**: quando una stazione riceve una richiesta ARP broadcast, memorizza o aggiorna nella propria cache il binding del mittente (`Sender IP`, `Sender MAC`), traendo profitto dalla richiesta per evitare future interrogazioni.
- **Timer di Aging ( 30–60 Secondi)**: le voci memorizzate nella cache ARP non sono permanenti, ma decadono dopo un intervallo di tempo limitato. Tale scadenza garantisce la coerenza della rete: se un host sostituisce la propria scheda di rete o riceve un nuovo IP via DHCP, il mapping obsoleto viene rimosso automaticamente senza creare buchi neri di instradamento.

### 3.20.6 Broadcast Storm nelle Reti Commutate e Proxy ARP

Sebbene il broadcast funzionasse a costo nullo sul cavo coassiale condiviso originale, nelle moderne reti commutate esso impone allo switch di replicare la trama su tutte le porte attive. Se una sottorete IP viene configurata con una maschera troppo estesa (ad esempio un'intera classe `/16` con decine di migliaia di terminali), le continue richieste broadcast ARP generano una **Broadcast Storm** (tempesta di broadcast) che satura i canali di commutazione e manda in blocco la rete. Per questo motivo le subnet locali vengono dimensionate ordinariamente a blocchi `/24` (massimo 254 host).

**Il Meccanismo di Proxy ARP:** Per limitare la propagazione del broadcast e consentire la comunicazione tra sottoreti diverse senza riconfigurare gli host, molti router e switch di fascia elevata forniscono la funzione di **Proxy ARP**:

- L'apparato ascolta le richieste broadcast ARP per IP non appartenenti alla subnet locale o situati su un'altra interfaccia;
- Risponde direttamente all'interrogazione fornendo il **proprio indirizzo MAC**, impersonando surrettiziamente la destinazione.
- L'host mittente spedirà le trame allo switch/router, il quale si farà carico di re-instradarle.

**Approfondimento: Violazione della Stratificazione e il Caso del Worm Nimda (2001)**

Il Proxy ARP rappresenta una **palese violazione della gerarchia a livelli**: un apparato di commutazione intercetta e manipola messaggi di protocollo sovrapposti, ingannando le tabelle di routing locali.
I pericoli di tale architettura emersero drammaticamente nel settembre 2001 con la diffusione del worm informatico **Nimda**. Nimda non distruggeva i file dei PC, ma generava milioni di scansioni broadcast casuali per propagarsi: le implementazioni di Proxy ARP sui router e switch della dorsale mondiale andarono in saturazione e crash continuo per buffer overflow. Per ripristinare la funzionalità della rete, intere porzioni della dorsale Internet rimasero spente per oltre una settimana, costringendo i responsabili di rete a riavviare e riattestare manualmente gli Autonomous Systems uno per uno.

# Capitolo 4

# Livello di Trasporto: Servizi, UDP, Trasferimento Affidabile e Protocollo TCP

**NOTA INTEGRATIVA DI AGGIORNAMENTO DIDATTICO**

**Avviso relativo alle Slide Ufficiali e alle Lezioni Frontali:**
Durante le lezioni frontali dell'anno accademico, a causa di una temporanea avaria tecnica ai sistemi di registrazione d'aula, la spiegazione vocale del docente per il fascicolo didattico 4.1 (*Trasporto, introduzione e UDP*) non è presente nella traccia audio registrata.
Per colmare in modo rigoroso, esaustivo e organico questo vuoto, la presente sezione espone la trattazione tecnica integrale e sistematica ricavata direttamente dalle slide ufficiali del Prof. Renato Lo Cigno.

Il livello di trasporto (*Transport Layer*) si colloca in una posizione nevralgica all'interno della pila protocollare di Internet: esso fornisce la comunicazione logica tra processi applicativi (tramite un protocollo di livello applicazione) in esecuzione su host differenti.

Mentre il livello di rete fornisce una comunicazione **host-to-host** tra nodi della rete, il compito del livello di trasporto è estendere tale servizio a una comunicazione logica **process-to-process** (o **end-to-end**) tra istanze di processi applicativi.

## 4.1 Il Livello di Trasporto nell'Architettura Internet

### 4.1.1 Posizionamento Architetturale e Visione End-to-End

Il livello di trasporto risponde a precisi principi architetturali:

- **Assenza di visibilità della rete**: il livello di trasporto, al pari di quello applicativo, non possiede alcuna visibilità dei dettagli interni della rete fisica (topologie, capacità dei canali, code istantanee).
- **Implementazione esclusiva nei sistemi terminali**: viene implementato unicamente nei sistemi terminali (*hosts*), ovvero dove l'informazione viene originariamente generata e fruita.
- **Semantica end-to-end**: la rete intermedia a commutazione di pacchetto viene trattata come una vera e propria scatola nera (*black box*). Lo scopo generale del protocollo è fornire un'astrazione pura di comunicazione ai protocolli di livello applicativo mediante servizi astratti dedicati denominati **SAP** (*Service Access Points*), noti in gergo Internet con il termine di **Socket**.
- **Incapsulamento e flusso dati**:
  - *Trasmettitore*: organizza i messaggi applicativi in segmenti di trasporto e li passa allo strato di rete sottostante;
  - *Ricevitore*: riassembla i segmenti in messaggi e li consegna al livello di applicazione;
  - Consente la **multiplazione di tanti servizi su un unico livello di rete**, senza la necessità di circuiti fisici o logici pre-assegnati.

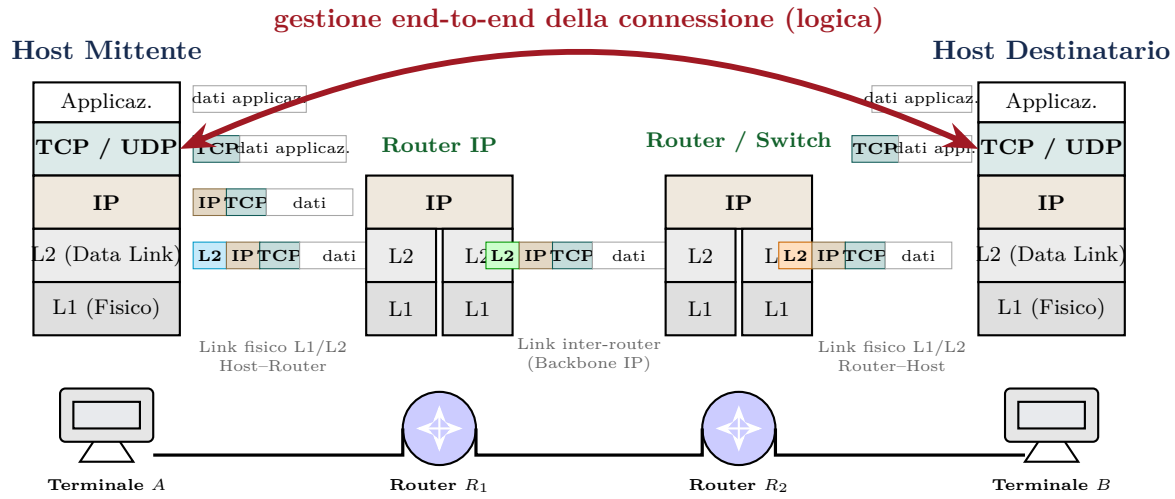



Figura 4.1: Visione d'insieme architetturale del Livello di Trasporto: comunicazione logica process-to-process e gestione end-to-end della connessione sopra nodi intermedi a soli 3 livelli (Router IP) e 2 livelli (Switch L2).

### 4.1.2 Panoramica dei Protocolli di Trasporto in Internet

La suite dei protocolli Internet prevede storicamente due protocolli di base alternativi, con modelli di servizio profondamente diversi:

1. **UDP (User Datagram Protocol)**: non connesso (*connectionless*), non affidabile, privo di stato (*stateless*), senza controllo di flusso e senza controllo di congestione.
2. **TCP (Transmission Control Protocol)**: orientato alla connessione (*connection-oriented*), affidabile, con mantenimento dello stato della connessione, dotato di controllo di flusso e controllo della congestione.

**Caratteristiche comuni a UDP e TCP:**

- **Multiplazione e demultiplazione** dei flussi applicativi mediante le **porte**;
- **Rilevazione (non correzione)** degli errori su intestazione e dati mediante checksum;
- **Scarto delle PDU malformate** o con errori nell'header.

**Funzionalità esclusive di TCP:**

- Instaurazione, gestione e rilascio formale delle connessioni (scambio di segnalazione);
- Recupero degli errori e delle perdite di pacchetti (ritrasmissioni);
- Consegna rigorosamente ordinata dei segmenti;
- Controllo di flusso (azione preventiva per non saturare il buffer del ricevitore);
- Controllo della congestione (reazione dinamica alla saturazione dei buffer interni della rete).

**Altri Protocolli di Trasporto Standardizzati:** Oltre a UDP e TCP, sono stati standardizzati e risultano diffusi:

- **SCTP (Stream Control Transmission Protocol)**: protocollo orientato ai messaggi ma affidabile, con supporto multi-homing e multi-streaming;
- **DCCP (Datagram Congestion Control Protocol)**: per traffico inaffidabile che necessita di controllo di congestione nativo (applicazioni multimediali);
- **Multipath TCP (MPTCP)**: consente l'impiego simultaneo di percorsi multipli per la stessa connessione;
- **QUIC** (RFC 9000): protocollo moderno basato su UDP che unisce affidabilità, crittografia integrata e assenza di blocco di testa tra flussi concorrenti (alla base di HTTP/3);
- Versioni sicure con sotto-livello di protezione dati e crittografia: **TLS** (sopra TCP) e **DTLS** (sopra UDP).

| Caratteristica Funzionale | UDP (RFC 768) | TCP (RFC 793) | QUIC (RFC 9000) |
|---|---|---|---|
| **Modello di Servizio** | Datagramma non connesso | Flusso continuo di byte | Flussi multipli su datagrammi |
| **Garanzia di Consegna** | Nessuna (*best-effort*) | Affidabile al 100% (ARQ) | Affidabile al 100% (ARQ indipendente) |
| **Ordinamento dei Dati** | Non garantito | Ordinamento sequenziale rigido | Ordinato per singolo stream |
| **Handshake Iniziale** | Nessuno (0 RTT) | 3-way handshake (1 RTT) | Connessione + TLS 1.3 (0∇·1 RTT) |
| **Controllo di Flusso** | Assente | Finestra scorrevole (rwnd) | Finestra per stream e per connessione |
| **Controllo Congestione** | Assente | Dinamico (AIMD, Reno, CUBIC) | Dinamico (pluggabile, es. BBR, CUBIC) |
| **Overhead di Intestazione** | 8 byte (fisso) | 20∇ · 60 byte (opzioni) | Variabile (compresso e cifrato) |

Tabella 4.1: Quadro sinottico comparativo tra i modelli di servizio dei protocolli di trasporto in Internet.

## 4.2 Indirizzamento a Livello Trasporto: Multiplazione e Demultiplazione

### 4.2.1 Il Ruolo Fondamentale delle Porte

Gli indirizzi di livello IP vengono usati per l'instradamento dei pacchetti all'interno della rete e identificano univocamente la scheda di rete (*NIC*) dell'host sorgente e destinazione. Quando il pacchetto giunge a destinazione e viene passato al livello di trasporto, emerge il problema:

> *Poiché sul livello di trasporto si appoggiano contemporaneamente più applicazioni, come è possibile distinguere a quale processo consegnare i dati?*

Il destinatario finale non è l'host o la scheda di rete, ma un **processo in esecuzione sull'host**. L'interfaccia tra processi applicativi e strato di trasporto è rappresentata dalla **Porta** (o *Porto*):

**Definizione: Definizione Formale**

Porta di Trasporto Un numero intero su 16 bit (intervallo $[0, 65535]$) presente nell'intestazione di trasporto che identifica univocamente l'applicazione o il processo cui consegnare l'informazione.

**Classificazione delle Porte:**

- **Porte Statiche o Ben Note (*Well-Known Ports*)**: valori inferiori a 1024 (es. porta 80 per WWW/HTTP, 443 per Web cifrato/HTTPS, 25 per posta/SMTP, 53 per DNS). Sono associate ad applicazioni e server standardizzati e pubblici.
- **Porte Dinamiche o Effimere (*Ephemeral Ports*)**: valori superiori a 1024. Sono assegnate dinamicamente dal sistema operativo al momento dell'apertura della comunicazione (utilizzate dai processi client o da servizi non standard).
- La porta sorgente e la porta destinazione sono generalmente distinte.

### 4.2.2 Identificazione di un Flusso: La 5-upla

L'utilizzo delle porte, combinato con gli indirizzi IP sorgente e destinazione, consente il multiplexing e il demultiplexing dei dati. Un flusso o connessione logica tra due applicazioni è identificato in modo assoluto e univoco in qualsiasi punto della rete da una **5-upla**:

$$\mathcal{F} = \left\{ \text{IP}_{\text{Sorgente}},\ \text{IP}_{\text{Destinazione}},\ \text{Protocollo},\ \text{Porta}_{\text{Sorgente}},\ \text{Porta}_{\text{Destinazione}} \right\} \tag{4.1}$$

La 5-upla $\{\text{S-IP}, \text{D-IP}, \text{PROTOCOL}, \text{S-Port}, \text{D-Port}\}$ identifica in modo univoco un flusso in qualsiasi punto della rete, e quindi anche all'interfaccia tra protocollo applicativo e protocollo di trasporto in un host.

La semantica del demultiplexing varia a seconda del protocollo di trasporto:

- **Demultiplazione Connectionless (UDP)**: una socket UDP è legata unicamente alla coppia locale $\{\text{IP}_{\text{Locale}}, \text{Porta}_{\text{Locale}}\}$. Tutti i segmenti UDP in arrivo con quella porta di destinazione vengono recapitati alla stessa socket, indipendentemente dalla sorgente.

- **Demultiplazione Connection-Oriented (TCP)**: una socket TCP stabilita è vincolata all'intera 4-upla $\{\text{IP}_{\text{Sorgente}}, \text{Porta}_{\text{Sorgente}}, \text{IP}_{\text{Destinazione}}, \text{Porta}_{\text{Destinazione}}\}$. Il server in ascolto su una porta ben nota (es. 80) genera una nuova socket dedicata per ciascuna connessione client accolta.

## 4.3 Il Protocollo UDP (User Datagram Protocol — RFC 768)

### 4.3.1 Caratteristiche Fondamentali e Motivazioni d'Uso

Lo **User Datagram Protocol (UDP)**, standardizzato da David P. Reed nella storica `RFC 768` (1980), rappresenta l'incarnazione minimale di un protocollo di livello trasporto. Esso non aggiunge praticamente nulla al servizio offerto dal protocollo IP sottostante, se non la funzione imprescindibile di indirizzamento a livello di processo (multiplexing/demultiplexing mediante porte) e un meccanismo leggero di verifica dell'integrità dei dati (checksum).

Le proprietà cardine di UDP sono:

- **Connectionless (Non connesso)**: non prevede alcuna fase preventiva di segnalazione o handshake per instaurare, mantenere o abbattere la sessione. Il mittente trasmette immediatamente il segmento nel momento in cui l'applicazione genera il messaggio.
- **Non affidabile**: non vi è alcuna garanzia che il datagramma raggiunga la destinazione. I pacchetti possono essere smarriti per congestione, duplicati o recapitati in disordine senza che UDP se ne avveda o intraprenda azioni correttive.
- **Assenza di stato (*Stateless*)**: i terminali non mantengono in memoria alcun parametro relativo alla connessione (nessun buffer di trasmissione o ricezione dinamico, nessun contatore di sequenza, nessun orologio di ritrasmissione).
- **Preservazione dei confini del messaggio (*Message-oriented*)**: ogni invocazione di scrittura applicativa corrisponde esattamente all'invio di un datagramma distinto; il ricevitore estrae l'esatto blocco informativo inviato, senza la frammentazione o fusione tipica del modello a flusso di byte (*stream-oriented*) di TCP.

**Principio / Fondamento: Perché utilizzare UDP se non fornisce affidabilità?**

Spesso ci si interroga sul motivo per cui un'applicazione debba scegliere UDP anziché il potente e sicuro TCP. Le motivazioni tecniche ed economiche sono molteplici:

1. **Nessun ritardo di connessione**: per applicazioni transazionali rapide (come una query di risoluzione dei nomi DNS), eseguire un 3-way handshake prima di scambiare dati raddoppierebbe inutilmente il tempo di risposta totale. Con UDP, la richiesta parte nel primo pacchetto e la risposta ritorna nel secondo (1 solo RTT complessivo).
2. **Semplicità e leggerezza computazionale**: l'assenza di stato rende il server estremamente leggero e scalabile. Un server DNS o DHCP basato su UDP può servire centinaia di migliaia di client contemporanei senza rischiare la saturazione di memoria dovuta a blocchi di controllo connessione (TCB) e risulta intrinsecamente immune ad attacchi di tipo *SYN Flood* mirati all'esaurimento della memoria di stato.
3. **Assenza di blocco da ritrasmissione**: nei flussi interattivi real-time (telefonia VoIP, videoconferenze, videogiochi online, live streaming), la tempestività dell'informazione prevale categoricamente sull'integrità assoluta. Se si perdono 10 ms di campione audio vocale, l'orecchio umano o un algoritmo di mascheramento (*Packet Loss Concealment*) non percepiscono alcuna discontinuità; al contrario, se il protocollo bloccasse il flusso per 100 ms per negoziare la ritrasmissione del pacchetto obsoleto (come farebbe TCP), la comunicazione diverrebbe inaccettabilmente scattosa e inutilizzabile.
4. **Nessun controllo di congestione imposto**: il trasmettitore inietta i dati nel canale esattamente al ritmo desiderato dall'applicazione, senza subire il dimezzamento forzato della finestra di trasmissione tipico delle logiche restrittive di TCP.

### 4.3.2 Perché UDP è relativamente poco diffuso rispetto a TCP?

Nonostante i pregi menzionati, nella maggior parte del traffico Internet odierno domina TCP. I motivi principali sono:

- **Complessità delegata all'applicazione**: UDP scarica l'intera responsabilità dell'affidabilità, della segmentazione dei messaggi lunghi, dell'ordinamento e del controllo di flusso sul programmatore dell'applicazione, rendendo lo sviluppo enormemente più oneroso rispetto a TCP (che offre un'astrazione intuitiva simile alla lettura/scrittura su un file locale su disco).
- **Blocco operato da Firewall e apparati NAT**: in molte reti aziendali o infrastrutture controllate, gli amministratori di rete filtrano e bloccano sistematicamente il traffico UDP in ingresso e uscita (consentendo unicamente le query DNS sulla porta 53), poiché i canali UDP senza stato vengono considerati vettori primari per attacchi DDoS a riflessione/amplificazione o tentativi di scansione furtiva.

### 4.3.3 Formato del Segmento UDP

L'intestazione di un segmento UDP presenta una struttura straordinariamente compatta ed essenziale: ha una dimensione fissa di soli **8 byte** (64 bit), articolata in quattro campi da 16 bit ciascuno.

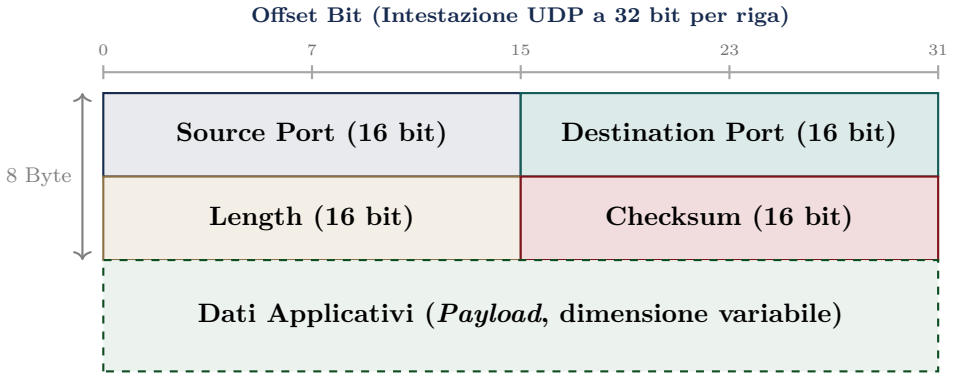



Figura 4.2: Formato dell'intestazione del segmento UDP (RFC 768).

L'analisi dei singoli campi dell'intestazione evidenzia:

1. **Source Port (Porta Sorgente, 16 bit)**: identifica il processo mittente sul client o sul server. È opzionale nel caso di comunicazioni puramente unidirezionali dove non è attesa risposta (in tal caso viene impostata al valore numerico zero, 0); altrimenti contiene la porta effimera o ben nota a cui indirizzare l'eventuale datagramma di ritorno.
2. **Destination Port (Porta Destinazione, 16 bit)**: parametro obbligatorio che identifica il processo applicativo di destinazione sull'host ricevente, consentendo il demultiplexing.
3. **Length (Lunghezza Totale, 16 bit)**: specifica la dimensione complessiva del segmento UDP, espressa in byte, includendo **sia gli 8 byte di intestazione sia l'intero carico utile dei dati**. Poiché il valore minimo è rappresentato dal solo header privo di dati, deve valere tassativamente Length $\geq 8$. Essendo il campo codificato su 16 bit, la dimensione massima teorica di un datagramma UDP è $2^{16} - 1 = 65535$ byte (anche se nella pratica operativa è vincolata dalla MTU del livello di rete per prevenire la frammentazione IP).
4. **Checksum (Somma di Controllo, 16 bit)**: campo preposto alla rilevazione di eventuali alterazioni ed errori occorsi ai bit durante il transito nel canale. In IPv4 il suo impiego è opzionale (se disabilitato viene riempito interamente con zeri); in IPv6 è stato reso **tassativamente obbligatorio**, poiché l'intestazione base di IPv6 ha eliminato il checksum di livello 3 per massimizzare la velocità di commutazione dei router.

### 4.3.4 L'Algoritmo dell'Internet Checksum e la Violazione dello Pseudo-Header

Il calcolo del checksum di UDP segue l'algoritmo standard dell'**Internet Checksum** formalizzato in RFC 1071:

**Definizione: Definizione Formale**

Algoritmo dell'Internet Checksum Il checksum a 16 bit è definito come il **complemento a 1 della somma in complemento a 1 di tutte le parole a 16 bit** componenti il pacchetto:

1. I dati da proteggere vengono trattati come una sequenza ordinata di interi senza segno a 16 bit. Se la lunghezza totale è dispari, viene aggiunto in coda un byte fittizio di zeri (*padding*) ai soli fini del calcolo.
2. Tutte le parole a 16 bit vengono sommate algebricamente.
3. Eventuali riporti eccedenti il 16-esimo bit (*carry-out*) vengono retrocessi e sommati alla cifra meno

significativa del risultato parziale (*end-around carry*).
4. Al termine della somma, si calcola il complemento a uno (inversione bit a bit: gli 0 diventano 1 e i − diventano 0).
5. All'atto della ricezione, l'host calcola la somma in complemento a uno di tutte le parole, includendo il checksum stesso: se il canale è esente da errori, la somma deve risultare identicamente pari a `0xFFFF` (ossia un campo di soli bit 1).

**Principio / Fondamento: Violazione Intenzionale della Stratificazione: Lo Pseudo-Header IP**

Il checksum di UDP non protegge unicamente l'intestazione UDP e il payload applicativo, ma estende la sua copertura a un insieme di campi estratti dal livello di rete sottostante, denominato **Pseudo-Header IP**:

| **Indirizzo IP Sorgente (32 bit)** | |
|---|---|
| **Indirizzo IP Destinazione (32 bit)** | |
| **Tutti Zeri (8 bit)** | **Protocollo (8 bit, valore = 17 per UDP)** |
| **Lunghezza UDP (16 bit, ripetizione del campo Length)** | |

L'inclusione di questi campi costituisce una **formale violazione del principio architetturale di stratificazione protocollare** (*cross-layering*): il livello 4 va a ispezionare direttamente campi che appartengono all'intestazione di livello 3.
Tale violazione è stata introdotta deliberatamente dai progettisti per una ragione difensiva vitale: proteggere l'applicazione contro gli **errori di dis-instradamento**. Se un router intermedio commette un errore di memoria corrompendo l'indirizzo IP di destinazione, il pacchetto verrebbe recapitato a un host errato; se tale host possedesse casualmente una porta aperta identica, il datagramma verrebbe consegnato a un'applicazione ignara. Grazie allo pseudo-header, il checksum calcolato non corrisponderà, e il datagramma errato verrà tempestivamente scartato.

## 4.4 Principi del Trasferimento Dati Affidabile (RDT)

### 4.4.1 La Natura del Problema del Trasporto Affidabile

Il problema del trasporto affidabile consiste nel garantire che l'informazione trasmessa sia corretta al ricevitore **al di là di ogni ragionevole dubbio**. In questi contesti la sicurezza matematica assoluta non esiste: occorre ricorrere a nozioni probabilistiche.

Oltre all'integrità del dato (*correttezza*), il servizio di trasporto affidabile impone vincoli strutturali aggiuntivi:

1. **Consegna in Ordine**: le unità dati devono essere consegnate al processo ricevente nello stesso identico ordine progressivo con cui sono state generate dall'applicazione mittente.
2. **Controllo di Flusso**: la trasmissione deve essere regolata alla velocità che il processo applicativo di ricezione è concretamente in grado di smaltire, evitando di inondare i buffer locali del ricevitore.
3. **Riconoscimento Automatico**: le decisioni di recupero e ritrasmissione devono essere governate in maniera completamente trasparente e autonoma dal protocollo di livello 4, senza richiedere l'intervento dell'applicazione.

La tecnica universale alla base del trasferimento dati affidabile su reti a commutazione di pacchetto è la famiglia di algoritmi **ARQ** (*Automatic Repeat reQuest*).

### 4.4.2 Tassonomia delle Imperfezioni del Canale Fisico

Il canale logico di livello 3 sottostante (IP) è intrinsecamente inaffidabile. Le anomalie che il livello di trasporto deve intercettare e neutralizzare sono:

- **Corruzione di Bit**: rumore termico, interferenze elettromagnetiche o attenuazione del segnale fisico che mutano uno o più bit all'interno del pacchetto.
- **Perdita di Pacchetti (Packet Loss)**: pacchetti distrutti o scartati nel canale. Nei collegamenti punto-punto cablati la perdita da rumore fisico ha una probabilità trascurabile ($P_e \approx 10^{-5} \nabla \cdot 10^{-12}$), mentre **in Internet oltre il 99.9% delle perdite è causato dalla congestione di rete**. Quando si raggiunge il limite della capacità trasmissiva di un link o di un router, l'informazione si accumula nei buffer; quando la memoria si satura (*buffer overflow*), i pacchetti successivi vengono scartati.

- **Duplicazione dei Pacchetti**: pacchetti duplicati da ritrasmissioni indebite o percorsi multipli.
- **Inversione dell'Ordine (Reordering)**: a causa dell'instradamento dinamico per datagrammi, pacchetti successivi possono seguire cammini geografici differenti e arrivare al destinatario disordinati.

## 4.5 Il Meccanismo Base Stop-and-Wait e i Suoi Limiti

### 4.5.1 Funzionamento del Protocollo Stop-and-Wait

Il protocollo più immediato per realizzare un circuito ARQ è il meccanismo **Stop-and-Wait** (analogo al meccanismo di riscontro positivo ACK implementato nello standard wireless IEEE 802.11 Wi-Fi a livello 2). L'algoritmo si struttura secondo regole comportamentali ben precise:

- **Mittente**:
  1. Riceve i dati dall'applicazione, li incapsula in una PDU corredata da un numero di sequenza e ne **crea una copia in memoria locale** (il salvataggio è indispensabile: se la PDU va persa, il livello di trasporto non può richiederla nuovamente all'applicazione).
  2. Invia la singola PDU sul canale e attiva un orologio di temporizzazione (*Timeout*).
  3. Si ferma (*Stop*) e attende l'arrivo del riscontro (*Wait*).
  4. Se l'ACK corrispondente alla PDU giunge prima della scadenza del timeout, il mittente distrugge la copia locale, arresta il timer e si predispone all'invio del pacchetto successivo.
  5. Se il timeout scade senza che sia pervenuta la conferma, il mittente ripete la trasmissione della PDU salvata e re-inizializza il timer.
- **Ricevitore**:
  1. All'arrivo della PDU, ne verifica l'integrità mediante il codice di ridondanza (CRC/Checksum) e ne controlla il numero di sequenza.
  2. Se la PDU è corretta e corrisponde a quella attesa, invia un messaggio di riscontro positivo (**ACK**), consegna il carico informativo al livello applicativo superiore e avanza lo stato di ricezione.
  3. Se la PDU è corrotta, il ricevitore la distrugge silenziosamente (*drop*). L'invio di un riscontro negativo (**NAK**) non è una soluzione robusta, poiché anch'esso potrebbe andare smarrito nel canale, ricadendo inevitabilmente sulla scadenza del timeout al mittente.
  4. Se la PDU è corretta ma duplicata (a causa della perdita del precedente ACK lungo la via del ritorno), il ricevitore rinvia immediatamente l'ACK relativo per sbloccare il trasmettitore, ma scarta il payload dati per non inquinare l'applicazione.

### 4.5.2 Analisi Analitica dell'Efficienza dello Stop-and-Wait

Sebbene lo Stop-and-Wait sia concettualmente impeccabile, a livello di trasporto esso risulta **catastroficamente inefficiente**. Valutiamo le prestazioni mediante i parametri formali della rete:

- $L$: lunghezza della PDU dati [bit];
- $R$: velocità di trasmissione (capacità) del collegamento [bit/s];
- RTT: Round-Trip Time [s], tempo complessivo di andata del pacchetto e ritorno dell'ACK;
- $T_{\text{trans}} = \frac{L}{R}$: tempo necessario per iniettare l'intera PDU sul mezzo fisico [s].

Supponendo trascurabile la dimensione dell'ACK rispetto al pacchetto dati, il ciclo completo di trasmissione richiede un tempo totale pari a:

$$T_{\text{ciclo}} = T_{\text{trans}} + \text{RTT} = \frac{L}{R} + \text{RTT} \tag{4.2}$$

L'**Utilizzazione del canale** ($U$), definita come la frazione di tempo in cui il trasmettitore è effettivamente occupato a immettere bit utili in rete rispetto al tempo totale di attesa, è formulata da:

$$U_{\text{SW}} = \frac{T_{\text{trans}}}{T_{\text{ciclo}}} = \frac{\dfrac{L}{R}}{\dfrac{L}{R} + \text{RTT}} \tag{4.3}$$

Il throughput effettivo sperimentato dall'applicazione risulta di conseguenza:

$$R_{\text{effettivo}} = U_{\text{SW}} \cdot R = \frac{L}{\dfrac{L}{R} + \text{RTT}} \tag{4.4}$$

**Il Paradosso della Rete Gigabit**

Consideriamo il seguente esempio numerico:

- Collegamento in fibra ottica moderno: capacità $R = 1\,\text{Gbit/s} = 10^9\,\text{bit/s}$;
- Ritardo di propagazione geografico (ambito nazionale/europeo, es. Genova–Bologna): RTT $= 30\,\text{ms} = 30 \cdot 10^{-3}\,\text{s}$;
- Pacchetto IP standard: $L = 1\,\text{kbyte} = 8\,\text{kbit} = 8000\,\text{bit}$ (approssimando con $1\,\text{k} = 1000$ per rapidità di calcolo analitico).

Calcoliamo il tempo di trasmissione puro del pacchetto:

$$T_{\text{trans}} = \frac{L}{R} = \frac{8000\,\text{bit}}{10^9\,\text{bit/s}} = 8 \cdot 10^{-6}\,\text{s} = 8\,\mu\text{s} = 0{,}008\,\text{ms} \tag{4.5}$$

Applicando la formula di efficienza (4.3):

$$U_{\text{SW}} = \frac{0{,}008\,\text{ms}}{0{,}008\,\text{ms} + 30\,\text{ms}} = \frac{0{,}008}{30{,}008} \approx 0{,}0002666 \approx 0{,}027\% \tag{4.6}$$

Il throughput effettivo ricavabile dall'utente è:

$$R_{\text{effettivo}} = 0{,}0002666 \cdot 10^9\,\text{bit/s} \approx 266{,}6\,\text{kbit/s} \approx 33\,\text{kbyte/s!} \tag{4.7}$$

Il risultato mostra che la rete fisica non è necessariamente lenta: è il protocollo Stop-and-Wait a limitare drammaticamente l'utilizzo delle risorse fisiche.

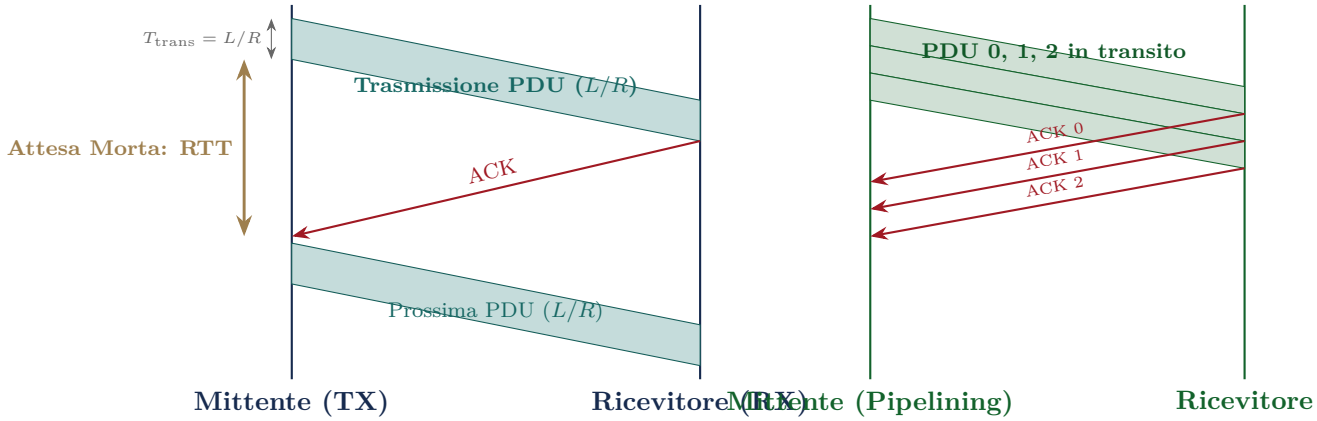



Figura 4.3: Cronogramma comparativo: inefficienza dello Stop-and-Wait con canale inattivo per un intero RTT (a sinistra) rispetto alla saturazione continua tramite tecnica Pipelining (a destra).

## 4.6 Protocolli a Finestra Scorrevole (*Sliding Window*) e Pipelining

### 4.6.1 Il Concetto di Pipelining

L'unica strategia per affrancarsi dalla paralisi dello Stop-and-Wait consiste nell'adottare la tecnica del **Pipelining**: consentire al mittente di trasmettere molteplici pacchetti consecutivi sul canale **prima ancora di ricevere il riscontro relativo al primo di essi**. In questo modo, mentre i riscontri dei primi pacchetti viaggiano lungo la via del ritorno, il trasmettitore continua a occupare il canale con nuovi segmenti.

Se il trasmettitore immette $W_T$ pacchetti prima di bloccarsi in attesa di riscontri, l'utilizzazione del collegamento scala proporzionalmente al fattore $W_T$:

$$U_{\text{pipelined}} = \min\left(1,\ W_T \cdot U_{\text{SW}}\right) = \min\left(1,\ \frac{W_T \cdot \dfrac{L}{R}}{\dfrac{L}{R} + \text{RTT}}\right) \tag{4.8}$$

Per sfruttare interamente la capacità del canale ($U = 100\% = 1$), è sufficiente scegliere una finestra tale che:

$$W_T \geq 1 + \frac{\text{RTT}}{L/R} = \frac{\text{RTT} \cdot R}{L} + 1 \approx \frac{\text{BDP}}{L} \tag{4.9}$$

dove BDP $= R \cdot \text{RTT}$ è il celebre **Bandwidth-Delay Product** (prodotto banda-ritardo), che quantifica il volume massimo di bit che possono essere fisicamente "in volo" all'interno della conduttura di rete in un dato istante.

### 4.6.2 Definizione e Dinamica delle Finestre: $W_T$ e $W_R$

I protocolli di trasporto a pipelining governano la trasmissione mediante due strutture concettuali a finestra scorrevole:

**Definizione: Definizione Formale**

Finestra di Trasmissione e Finestra di Ricezione

- **Finestra di Trasmissione ($W_T$)**: rappresenta la massima quantità di unità dati (PDU o byte) che il trasmettitore ha facoltà di inviare in sequenza senza aver ancora ricevuto alcun riscontro (ACK) da parte del ricevitore.
- **Finestra di Ricezione ($W_R$)**: rappresenta la quantità massima di PDU che il ricevitore è predisposto ad accettare e memorizzare nei propri buffer locali senza doverle immediatamente passare ai livelli applicativi superiori.




Figura 4.4: Struttura della finestra scorrevole (*Sliding Window*) del trasmettitore sul flusso logico delle PDU.

### 4.6.3 Relazione Critica tra le Finestre: Prevenzione delle Perdite Sistematiche

I vincoli di proporzionamento tra le capacità di memoria dei terminali sono essenziali:

**Principio / Fondamento: Condizione di Compatibilità delle Finestre**

Affinché il protocollo operi senza generare perdite di dati artificiose e sistematiche dovute alla sola saturazione della memoria del ricevitore, deve sussistere la condizione:

$$W_R \geq W_T \tag{4.10}$$

**Cosa accade se $W_T > W_R$?** Se la finestra di trasmissione $W_T$ è superiore alla capacità di accumulo del ricevitore $W_R$, il trasmettitore ha facoltà di inviare una raffica continua di $W_T$ pacchetti. Poiché il ricevitore ha spazio per soli $W_R$ elementi, i restanti $W_T - W_R$ pacchetti arrivano a destinazione ma **vengono inesorabilmente distrutti per mancanza di memoria tampone**. Questo causa un disastro prestazionale: il trasmettitore crederà che vi sia congestione nella rete, farà scattare i timeout e ritrasmetterà, finendo per saturare il canale con perdite indotte non dalla rete, bensì dalla lentezza o dall'insufficienza di memoria dell'applicazione destinataria.

### 4.6.4 Il Protocollo Go-Back-N ($W_R = 1$)

Il caso limite minimale dei protocolli con finestra di trasmissione ampia ($W_T > 1$) accoppiata a una finestra di ricezione unitaria ($W_R = 1$) è denominato **Go-Back-N (GBN)**:

- **Ricevitore Elementare**: il ricevitore possiede memoria per una sola PDU. Esso è posizionato rigidamente sul prossimo pacchetto in sequenza (il numero $k$).
- **Scarto del Fuori Sequenza**: se il pacchetto $k$ va smarrito nel canale ma i successivi pacchetti $k+1, k+2, \ldots, k+N-1$ giungono integri al ricevitore, quest'ultimo **li distrugge sistematicamente tutti**, poiché non possiede memoria per bufferizzarli.
- **Ritrasmissione Totale**: il trasmettitore, alla scadenza del timeout relativo al pacchetto $k$, deve "tornare indietro di $N$ posizioni" (*Go Back N*) e **ritrasmettere integralmente l'intero pacchetto $k$ e tutti i successivi $N-1$ pacchetti**, anche se questi ultimi erano già arrivati integri al ricevitore!

**Inefficienza di Go-Back-N e Nicchie d'Uso Industriali**

Go-Back-N manifesta un'efficienza disastrosa non appena la rete presenta una minima probabilità di scarto:

> *«Se la probabilità media di perdita di pacchetto supera $1/N$, il protocollo Go-Back-N non trasferisce più nulla! Prima ancora di aver completato l'invio della finestra di N pacchetti se ne perde uno, il trasmettitore torna indietro e si ritrova a ciclare all'infinito senza fare alcun progresso utile.»*

Perché allora studiare o realizzare Go-Back-N? La logica di mercato ed economica è la seguente:

> *«Go-Back-N si impiega unicamente laddove il dispositivo ricevitore deve essere estremamente economico (target di mercato a bassissimo costo, sensori IoT elementari, periferiche embedded a microcontrollore privo di RAM). Se per implementare una finestra di ricezione maggiore di 1 dovessi spendere 50 centesimi in più di memoria su un dispositivo che ne costa 1, andrei fuori mercato. Allora accetto le scarse prestazioni di Go-Back-N purché la rete sottostante sia molto affidabile (es. $P_{loss} \leq 10^{-5}$).»*

### 4.6.5 Il Protocollo Selective Repeat ($W_R > 1$)

Per conseguire un rendimento ottimale su reti geografiche e canali con perdite non trascurabili, si adotta il protocollo **Selective Repeat (SR)**:

- La finestra di ricezione è estesa: $W_R > 1$.
- Il ricevitore **accetta e memorizza nei buffer interni i pacchetti corretti anche se giungono fuori sequenza**, purché il loro numero di sequenza rientri nei margini della finestra corrente $[R_{\text{base}}, R_{\text{base}} + W_R - 1]$.
- I dati fuori sequenza vengono trattenuti fino a quando il pacchetto mancante non viene recuperato; a quel punto, l'intero blocco contiguo viene consegnato al livello applicativo e la finestra di ricezione scorre in avanti.
- Il trasmettitore **ritrasmette selettivamente solo ed esclusivamente i singoli pacchetti effettivamente smarriti**, preservando tutte le altre PDU già confermate.

### 4.6.6 Semantica degli Acknowledgement: Individuali vs Cumulativi

L'architettura del protocollo Selective Repeat dipende intimamente dalla semantica adottata per i messaggi di riscontro:

1. **ACK Individuali (Selettivi)**: ogni messaggio di riscontro reca un identificativo specifico e conferma l'avvenuta ricezione della sola e singola PDU corrispondente. Il mittente gestisce un timer indipendente per ciascuna PDU.
2. **ACK Cumulativi**: il messaggio reca un numero di sequenza $n$ che attesta formalmente: *«Tutti i dati precedenti al numero n sono stati ricevuti correttamente in sequenza»*. Questa è la soluzione storicamente consacrata in Internet e adottata nel protocollo TCP. Il pregio straordinario dell'ACK cumulativo risiede nella sua **resilienza intrinseca alla perdita**: se l'ACK relativo al pacchetto 1 va perso nel canale ma l'ACK relativo al pacchetto 2 giunge a destinazione, il mittente deduce implicitamente che anche il pacchetto 1 è giunto felicemente a destinazione, evitando ritrasmissioni inutili.
3. **Piggybacking**: nelle comunicazioni bidirezionali, l'informazione di ACK non viene inviata in una PDU separata, ma viene incorporata (*a cavalcioni*, piggybacked) all'interno dell'intestazione del pacchetto di dati applicativi che viaggia nella direzione opposta, dimezzando l'overhead di rete.

### 4.6.7 Politiche di Ritrasmissione al Timeout: Conservativa vs Aggressiva

All'atto della scadenza del timeout di ritrasmissione per una PDU smarrita (la numero $N$), il trasmettitore può adottare due condotte operative distinte:

- **Condotta Conservativa**: il trasmettitore ritrasmette la PDU $N$ e **si arresta completamente**, congelando l'emissione di ulteriori dati fino a quando non ha ottenuto la conferma sicura dell'avvenuta ricezione di $N$.
- **Condotta Aggressiva / Ottimista**: il trasmettitore ritrasmette la PDU $N$ ma, se i margini della finestra di trasmissione $W_T$ lo consentono, **continua a immettere nuove PDU sul canale** senza attendere l'ACK di riparazione.

La ratio ingegneristica alla base di tale scelta è la seguente:

*«La condotta conservativa è la più saggia in Internet: poiché oltre il 99.9% delle perdite è causato da congestione dei buffer nei router, inviare pacchetti aggiuntivi mentre un pacchetto è appena andato perso significa aggravare l'ingorgo di traffico e innescare il collasso per congestione della rete. La condotta aggressiva è sostenibile solo se si ha la certezza che la perdita sia dovuta a un transitorio microscopico o a un errore isolato su linea fisica.»*

## 4.7 Il Teorema dello Spazio di Numerazione Modulare

Nelle implementazioni concrete dei calcolatori, i numeri di sequenza inseriti nelle intestazioni dei pacchetti non possono crescere all'infinito: essi devono essere codificati all'interno di un campo a lunghezza fissa di $k$ bit, definendo uno **spazio di numerazione finito e ciclico** appartenente all'anello modulare:

$$\mathbb{Z}_{2^k} = \Big\{0, 1, 2, \ldots, 2^k - 1\Big\} \tag{4.11}$$

Raggiunto il valore massimo $2^k - 1$, il contatore azzera e ricomincia da 0.

Questa ciclicità introduce un pericolo formale devastante: se le dimensioni delle finestre non sono rigidamente vincolate rispetto allo spazio dei numeri di sequenza disponibili, il ricevitore può cadere in **ambiguità insanabili**, scambiando una PDU duplicata e obsoleta (ritrasmessa a seguito di timeout) per una PDU nuova di zecca appartenente al ciclo successivo!

**Principio / Fondamento: Teorema Fondamentale dello Spazio di Numerazione ARQ**

In un protocollo di trasporto affidabile a finestra scorrevole con contatore di sequenza su $k$ bit (spazio di numerazione $S = 2^k$), la condizione necessaria e sufficiente affinché il protocollo non manifesti mai ambiguità tra nuovi pacchetti e duplicati, in qualsiasi condizione di perdita di dati o di ACK, è:

$$\mathbf{W_T + W_R \leq 2^k} \tag{4.12}$$

**Dimostrazione e Analisi del Disallineamento Estremo.** Analizziamo gli scenari limite di disallineamento temporale tra la finestra del trasmettitore e quella del ricevitore. Sia la finestra del ricevitore inizialmente attestata sull'intervallo di sequenza:

$$\mathcal{W}_R = [r,\, r + W_R - 1] \pmod{2^k} \tag{4.13}$$

Il ricevitore è predisposto ad accogliere unicamente le PDU i cui numeri appartengano a tale intervallo.

Consideriamo la posizione più arretrata possibile per la finestra del trasmettitore $\mathcal{W}_T$:

- Il trasmettitore trasmette l'intera raffica di $W_T$ pacchetti a partire da un numero base $t$: $[t, t + W_T - 1]$.
- Tutti i pacchetti giungono integri al ricevitore, il quale genera i rispettivi ACK e avanza la propria finestra di esattamente $W_T$ posizioni, posizionandosi sul nuovo indice base $r_{\text{nuovo}} = t + W_T$. La nuova finestra di ricezione sarà:

$$\mathcal{W}_R^{\text{nuovo}} = [t + W_T,\; t + W_T + W_R - 1] \pmod{2^k} \tag{4.14}$$

- Ipotizziamo l'evento catastrofico del canale: **tutti i messaggi di ACK inviati dal ricevitore vanno integralmente perduti**.
- Al trasmettitore non perviene alcun riscontro: la sua finestra $\mathcal{W}_T$ rimane inchiodata alla posizione di partenza $[t, t + W_T - 1]$.
- Allo scadere del timeout, il trasmettitore ritrasmette la vecchia PDU numero $t$.

Affinché il ricevitore non commetta un errore fatale accettando la vecchia PDU ritrasmessa $t$ come se fosse una PDU appartenente al nuovo ciclo di numerazione, è inderogabile che il valore $t$ **risulti rigorosamente al di fuori della finestra di ricezione corrente** $\mathcal{W}_R^{\text{nuovo}}$. Dovendo coprire $W_T$ posizioni arretrate del trasmettitore e $W_R$ posizioni avanzate del ricevitore senza che gli estremi si sovrappongano ciclicamente nel modulo $2^k$, l'unione dei due intervalli di incertezza deve risultare strettamente contenuta nell'ampiezza dell'anello:

$$W_T + W_R \leq 2^k \tag{4.15}$$

Se $W_T + W_R > 2^k$, l'intervallo della finestra di ricezione avanza fino a coprire nuovamente per avvolgimento ciclico l'indice $t$, provocando l'accoglimento del duplicato come dato inedito.

**Dimensionamento Ottimale per Go-Back-N e Selective Repeat**

Dal teorema discendono due regole architetturali capitali:

1. **Go-Back-N** ($W_R = 1$):
$$W_T + 1 \leq 2^k \implies W_T \leq 2^k - 1 \tag{4.16}$$
In Go-Back-N, con un contatore a $k$ bit, la finestra di trasmissione massima consentita è $2^k - 1$.
2. **Selective Repeat Simmetrico** ($W_T = W_R$):
$$2W_T \leq 2^k \implies W_T \leq 2^{k-1} = \frac{2^k}{2} \tag{4.17}$$
In Selective Repeat con finestre paritetiche, la dimensione della finestra non può mai superare la metà esatta dello spazio di numerazione disponibile.

### 4.7.1 Analisi Dettagliata del Controesempio Critico: $W_T = 3$, $W_R = 2$, $k = 2$

Si analizzi il caso in cui la condizione (4.12) viene violata. Impostiamo i parametri dell'esperimento:

- Contatore su $k = 2$ bit $\implies$ spazio di numerazione $2^2 = 4$ simboli: $\{0, 1, 2, 3\}$;
- Finestra di trasmissione: $W_T = 3$;
- Finestra di ricezione: $W_R = 2$;
- Controllo di compatibilità: $W_T + W_R = 3 + 2 = 5 > 4 = 2^k$ (**VIOLAZIONE DEL TEOREMA**).

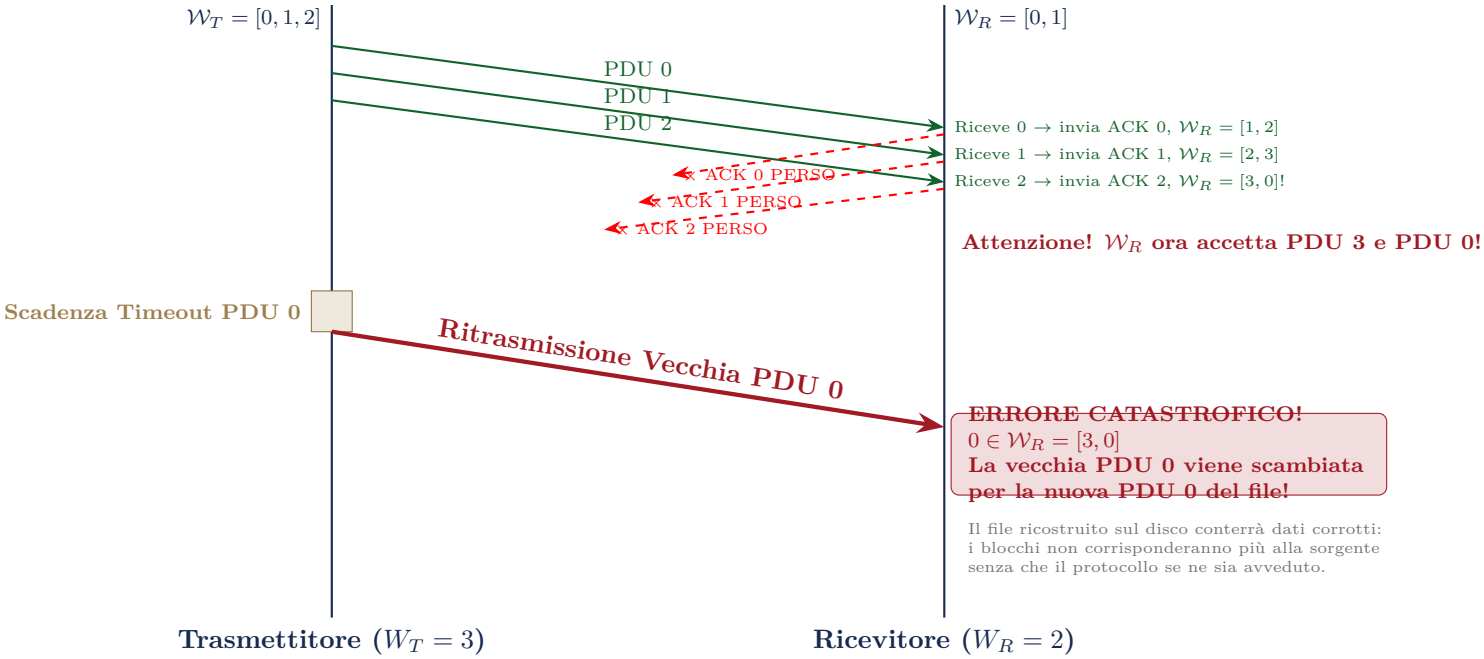



Figura 4.5: Cronologia del fallimento architetturale per $W_T = 3, W_R = 2, k = 2$ ($W_T + W_R = 5 > 4$): la perdita di ACK induce il ricevitore ad accettare un duplicato obsoleto come blocco inedito.

Seguiamo passo dopo passo l'evoluzione dinamica:

1. Il trasmettitore invia le tre PDU autorizzate dalla propria finestra: $0, 1, 2$.
2. Il ricevitore riceve la PDU 0: è attesa, la accetta, invia ACK(0) e scorre in avanti la propria finestra: $\mathcal{W}_R = [1, 2]$.
3. Il ricevitore riceve la PDU 1: è attesa, la accetta, invia ACK(1) e scorre la finestra: $\mathcal{W}_R = [2, 3]$.
4. Il ricevitore riceve la PDU 2: è attesa, la accetta, invia ACK(2) e scorre la finestra sul ciclo successivo:
$$\mathcal{W}_R = [3, 0] \pmod 4$$
5. Supponiamo che tutti e tre gli ACK vadano smarriti lungo il tragitto di ritorno verso il mittente.

6. Al trasmettitore scade l'orologio di timeout relativo alla PDU 0. Il mittente, non avendo ricevuto alcun riscontro, presume che la PDU 0 non sia mai giunta e la **ritrasmette**.
7. Quando la vecchia PDU 0 ritrasmessa giunge al ricevitore, quest'ultimo ispeziona la propria finestra di ricezione: $\mathcal{W}_R = [3, 0]$. Poiché il simbolo 0 fa parte della finestra corrente, il ricevitore compie un'azione rovinosa: **ritiene che quella PDU sia il quinto blocco del file** (il blocco numero 4, che per modularità ciclica prende il nome di 0), collocandolo nella porzione avanzata del file su disco!

Ne consegue che:

> *«Se sto trasferendo un file di grandi dimensioni (es. un'immagine ISO o un database da 10 Gigabyte), un errore di questo tipo posiziona dati vecchi al posto di dati nuovi. Il protocollo ha fallito: l'applicazione se ne accorgerà solamente alla fine controllando l'hash complessivo del file, costringendo l'utente a riscaricare l'intero file da capo. La condizione $W_T + W_R \leq 2^k$ è una barriera logica invalicabile.»*

## 4.8 Approfondimento Operativo: Switch Cut-Through nei Data Center

Un'osservazione applicativa di rilievo collega i concetti di ritardo di propagazione e memorizzazione delle PDU con le moderne architetture hardware dei **Data Center**.

### 4.8.1 Commutazione Store-and-Forward vs Cut-Through

Nel trattare i dispositivi di commutazione locali (switch Ethernet di Livello 2), il paradigma convenzionale adottato nella didattica è il modello **Store-and-Forward**:

- Lo switch attende di ricevere l'intera trama fisica dal cavo d'ingresso;
- Memorizza la trama integralmente nel proprio buffer interno;
- Calcola e verifica il codice di ridondanza ciclica (CRC) a 32 bit posto nella coda della trama;
- Se la trama è integra, esegue il lookup nella tabella MAC (*Forwarding Table*) e la trasmette sul link di uscita; se è corrotta, la distrugge all'istante.

Questo approccio garantisce che le trame alterate non propaghino nella rete, ma introduce un **ritardo di serializzazione addizionale in ogni singolo nodo** pari a $L/R$.

Nelle architetture di calcolo parallelo e nei Data Center iperscalari (es. Google, Microsoft Azure, AWS), le macchine server sono dislocate a pochi metri di distanza le une dalle altre, collegate da interconnessioni in fibra ottica ad altissima velocità ($40\,\text{Gbit/s}, 100\,\text{Gbit/s}, 400\,\text{Gbit/s}$). In questo contesto, accumulare anche solo pochi microsecondi di ritardo di memorizzazione per ogni switch attraversato degrada vistosamente le prestazioni delle comunicazioni tra cluster.

Per abbattere radicalmente tale latenza, si adottano apparati basati sulla tecnologia **Cut-Through**:

**Definizione: Definizione Formale**

Switching Cut-Through Uno switch con commutazione **Cut-Through** non attende la ricezione dell'intera trama:

1. Appena riceve i primi byte dell'intestazione (i 6 byte dell'indirizzo MAC di destinazione, collocati all'inizio del frame Ethernet);
2. Consulta la tabella di inoltro hardware (ASIC/CAM);
3. Apre un canale logico diretto all'interno della matrice di commutazione (*switching fabric*);
4. Inizia immediatamente a inoltrare i bit sulla porta di uscita, prima ancora che i byte successivi della trama siano stati ricevuti dalla porta d'ingresso!

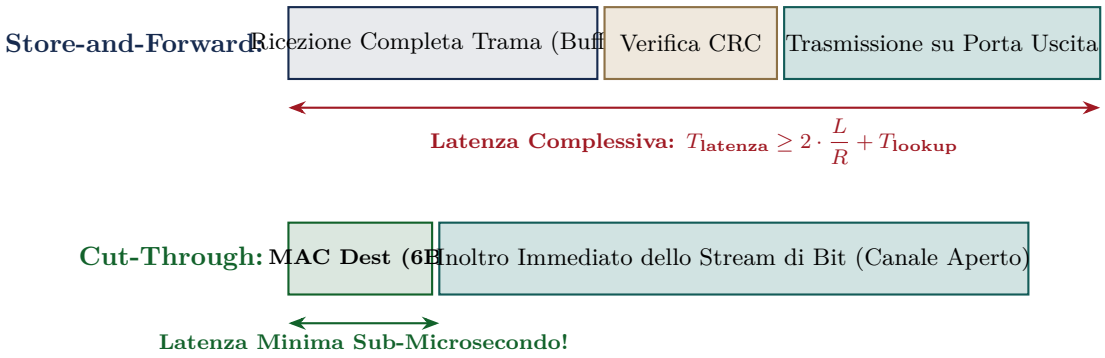



Figura 4.6: Confronto architetturale tra commutazione Store-and-Forward convenzionale e tecnologia Cut-Through a bassissima latenza.

### 4.8.2 Il Compromesso di Affidabilità e l'Ordine di Grandezza Fisico

Qual è il potenziale difetto strutturale di uno switch Cut-Through? Si osserva che:

> *«Il difetto risiede nel fatto che lo switch Cut-Through* ***non può controllare la correttezza della trama prima di inoltrarla****: esso inizia a trasmettere i dati prima ancora che il campo di controllo CRC (posto in coda) sia giunto all'ingresso. Se la trama era corrotta, lo switch la propagherà comunque in rete, sprecando banda.»*

Tuttavia, l'ingegneria dei sistemi di telecomunicazione si fonda su un'analisi probabilistica rigorosa:

- Su 3 metri di fibra ottica monomodale o multimodale all'interno di un armadio rack di un data center, la probabilità di errore sui bit da cause fisiche non è zero (la probabilità zero non esiste nella fisica reale), ma assume ordini di grandezza compresi tra:

$$P_e \approx 10^{-12} \nabla \cdot 10^{-18} \tag{4.18}$$

- Con una probabilità di guasto così infinitesimalmente remota, sacrificare le prestazioni di milioni di elaborazioni al secondo per verificare preventivamente ogni singolo frame è irrazionale. La velocità quasi istantanea (latenza sub-microsecondo) compensa ampiamente il rischio trascurabile di propagare un errore isolato, il quale verrà comunque intercettato e sanato al termine della catena dal checksum di livello 4 dell'host destinatario!

**PARTE SECONDA: IL PROTOCOLLO TCP E IL CONTROLLO DI CONGESTIONE**

Dopo aver compreso i limiti strutturali del protocollo senza connessione `UDP` e i principi teorici del trasferimento affidabile (*Stop-and-Wait*, *Go-Back-N*, *Selective Repeat*), affrontiamo ora la disamina integrale del protocollo principe della suite Internet: il **TCP** (`Transmission Control Protocol`, standardizzato originariamente nella storica `RFC 793`).
La presente trattazione espone l'architettura end-to-end, il formato dell'header a 32 bit, la gestione del ciclo di vita della connessione (*Three-Way Handshake* e chiusura *gentle*), gli algoritmi di stima del ritardo di andata e ritorno (*RTT*), il controllo di flusso e l'intero impianto matematico e operativo del controllo di congestione (*Slow Start*, *Congestion Avoidance*, *Fast Retransmit*, *Fast Recovery* ed equità di rete), seguendo fedelmente il percorso didattico del Prof. Renato Lo Cigno.

## 4.9 Il Protocollo TCP: Architettura, Servizi e Ruoli End-to-End

### 4.9.1 Caratteristiche Fondamentali del Canale TCP

Il protocollo TCP trasforma il canale sottostante offerto da IP — per sua natura inaffidabile, privo di connessione e soggetto a perdite, duplicazioni e disordini di pacchetto — in un'astrazione di comunicazione perfetta. I postulati architetturali del servizio TCP sono definiti come segue:

**Definizione: Proprietà Fondamentali della Connessione TCP**

Una connessione TCP è un servizio di trasporto logico che garantisce simultaneamente:

1. **Punto-a-Punto (*Point-to-Point*)**: la comunicazione avviene sempre e solo tra due entità terminali ben definite (un singolo trasmettitore e un singolo ricevitore). TCP non supporta intrinsecamente la trasmissione multicast o broadcast (che è invece appannaggio di UDP su IP).
2. **Orientato alla Connessione (*Connection-Oriented*)**: prima che i processi applicativi possano scambiarsi anche un solo byte di carico utile (*payload*), i due host remoti devono instaurare una sessione logica preliminare (**Three-Way Handshake**) volta a negoziare i parametri operativi iniziali e sincronizzare i reciproci numeri di sequenza.
3. **Flusso di Byte Continuo (*Byte-Stream*)**: per TCP non esistono i confini di messaggio applicativo. Il protocollo vede i dati come un flusso continuo e amorfo di singoli ottetti (byte). Spetta al livello di applicazione definire e interpretare la delimitazione interna dei propri messaggi.
4. **Affidabilità Assoluta e Conservazione dell'Ordine**: tutti i byte immessi dal processo mittente vengono consegnati al processo destinatario privi di corruzioni, senza duplicati e nell'esatto ordine sequenziale con cui sono stati generati.
5. **Trasmissione Bidirezionale Simultanea (*Full-Duplex*)**: all'interno della stessa connessione, i dati applicativi possono fluire in entrambe le direzioni nello stesso istante temporale. Ne consegue che gli annunci di riscontro (*ACK*) e le informazioni di controllo possono viaggiare incapsulati direttamente all'interno dei segmenti di dati diretti in senso opposto (tecnica del **piggybacking**).

**Principio / Fondamento: Identificazione Univoca a 4-upla**

Ogni connessione TCP attiva all'interno del sistema operativo è identificata univocamente nel tempo da una coppia di **Socket** (*Socket Pair*), ovvero dalla 4-upla:

$$\langle \text{IP Sorgente, Porta Sorgente, IP Destinazione, Porta Destinazione} \rangle \tag{4.19}$$

Due connessioni che condividono la medesima porta di destinazione su un server (ad esempio la porta 80 o 443 di un server Web) rimangono rigorosamente isolate e instradate a contesti di memoria distinti grazie alla differenza nell'indirizzo IP o nella porta effimera del client mittente.

### 4.9.2 Architettura Funzionale del Trasmettitore e del Ricevitore

L'implementazione concreta di TCP all'interno del kernel del sistema operativo si articola nella cooperazione tra due macchine a stati finite asimmetriche ma coordinate:

- **Il Trasmettitore (*Sender*)**:
  - Riceve i dati dal processo applicativo sovrastante e li immagazzina nel proprio **Send Buffer** (buffer di trasmissione);
  - Segmenta il flusso continuo di byte in blocchi discreti denominati **Segmenti TCP**, la cui dimensione massima non può eccedere il parametro **MSS** (*Maximum Segment Size*);
  - Alloca e gestisce le finestre di trasmissione (*Send Window*), che vincolano la quantità massima di byte che possono essere immessi in rete senza aver ancora ricevuto il relativo riscontro (*bytes in flight*);
  - Mantiene attivo un **singolo timer logico di ritrasmissione** (**RTO**, *Retransmission Timeout*), associato formalmente al segmento più vecchio che si trova in volo ma non è ancora stato riscontrato (*Head-of-Line segment*);
  - Raccoglie i campioni temporali di ritardo per aggiornare costantemente la stima del tempo di andata e ritorno ($RTT$).
- **Il Ricevitore (*Receiver*)**:
  - Intercetta i segmenti IP in arrivo e ne verifica l'integrità tramite il campo *Checksum*; se il checksum fallisce, il segmento viene scartato silenziosamente;
  - Se il segmento è integro, verifica che non sia un duplicato di dati già acquisiti;
  - Gestisce l'eventuale arrivo di segmenti **fuori ordine**: anziché scartarli rozzamente (come farebbe un puro schema *Go-Back-N*), TCP mantiene un **Receive Buffer** in cui i segmenti fuori ordine

vengono memorizzati temporaneamente in attesa che i byte mancanti (il "gap") vengano ritrasmessi dal mittente;
- Notifica al mittente lo stato della ricezione per mezzo di **ACK Cumulativi**, specificando non l'ultimo byte ricevuto, bensì il **numero del prossimo byte che si attende di ricevere**;
- Monitora lo spazio libero residuo nel proprio buffer di ricezione e lo comunica al mittente attraverso il campo *Receive Window* (*rwnd*), realizzando così il controllo di flusso.

### 4.9.3 Regole Formali per la Generazione degli ACK

Per evitare di intasare la rete con un traffico parassita costituito da soli pacchetti di controllo privi di dati, l'IETF (`RFC 1122` e `RFC 2581`) ha definito quattro regole rigorose che vincolano il ricevitore TCP nella generazione dei riscontri:

| Stato dell'Evento al Ricevitore | Azione Prescritta dal Protocollo TCP |
|---|---|
| Arrivo di un segmento **in ordine**, con numero di sequenza atteso. Tutti i dati precedenti sono già stati regolarmente riscontrati. | **ACK Ritardato (*Delayed ACK*)**: il ricevitore non invia immediatamente l'ACK, ma avvia un timer (fino a un massimo di 500 ms, tipicamente 200 ms). Se entro tale finestra giunge un secondo segmento in ordine o se l'applicazione locale genera dati da inviare in senso opposto, l'ACK viene generato (cumulativo o in piggybacking). Se il timer scade, l'ACK viene inviato da solo. |
| Arrivo di un segmento **in ordine**, ma c'è già un precedente segmento in ordine in attesa della scadenza del timer di ACK ritardato. | **Invio Immediato di ACK Cumulativo**: per evitare che il timer del mittente scada inutilmente, si genera seduta stante un singolo ACK che riscontra cumulativamente entrambi i segmenti in ordine. |
| Arrivo di un segmento **fuori ordine**, con numero di sequenza superiore a quello atteso (rilevazione di un *gap* o buco). | **Invio Immediato di un ACK Duplicato (*Duplicate ACK*)**: il segmento fuori ordine viene salvato nel buffer e il ricevitore genera all'istante un ACK che reitera esattamente il numero di sequenza del byte mancante atteso. |
| Arrivo di un segmento che **colma parzialmente o totalmente il gap** dei dati mancanti. | **Invio Immediato di ACK Aggiornato**: se il segmento colma il vuoto, il ricevitore emette immediatamente un ACK cumulativo che notifica il nuovo limite superiore dei dati ordinati contigui memorizzati nel buffer. |

Tabella 4.2: Politica standard di generazione degli Acknowledgement secondo RFC 1122 e RFC 2581.

## 4.10 La PDU di TCP: Struttura Dettagliata dell'Header a 32 bit

### 4.10.1 Rappresentazione Architetturale dell'Header

La Protocol Data Unit (PDU) di livello 4 generata da TCP prende il nome formale di **Segmento TCP**. Un segmento è composto da un'intestazione (*Header*) a lunghezza variabile (da un minimo di 20 byte a un massimo di 60 byte) seguita dall'eventuale campo dati applicativo.

La visualizzazione standard canonica di un header TCP è strutturata su righe contigue da 32 bit (4 byte):

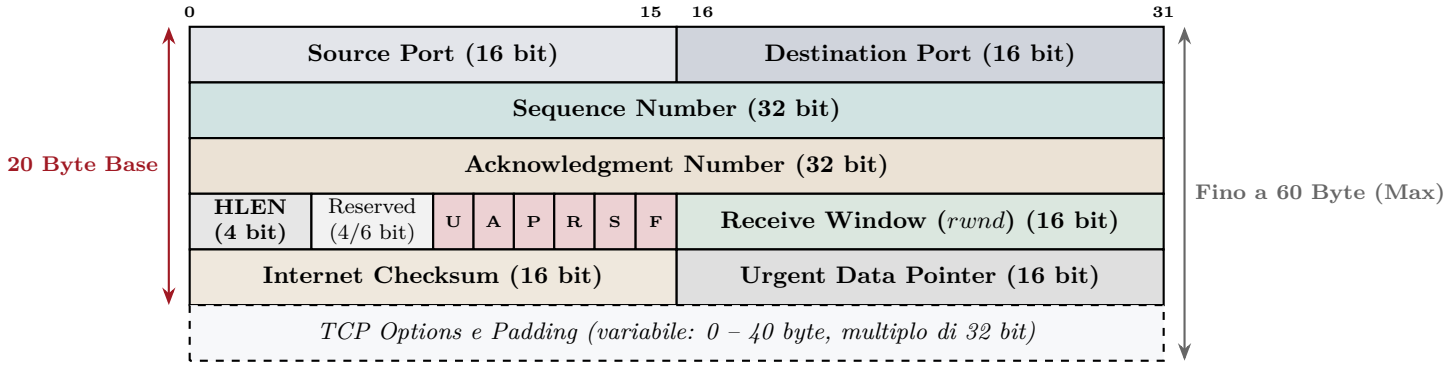



Figura 4.7: Formato analitico dell'Header TCP (RFC 793 e successive estensioni IETF).

### 4.10.2 Disamina Analitica dei Singoli Campi

Analizziamo nel dettaglio la semantica di ciascun elemento dell'header:

1. **Source Port e Destination Port (16 bit ciascuna)**: identificano i rispettivi punti di accesso ai servizi (**Socket**) sui due host terminali. Determinano la demultiplazione del segmento verso il processo di competenza.

2. **Sequence Number (32 bit)**: rappresenta il numero d'ordine del **primo byte di dati** contenuto in questo segmento rispetto all'inizio del flusso.

   **La Natura del Sequence Number in TCP**

   Mentre nei protocolli didattici visti in precedenza (*Go-Back-N*, *Selective Repeat*) il numero di sequenza contava i *pacchetti*, in TCP il Sequence Number conta rigorosamente i **byte**! Se un segmento trasporta 1000 byte di dati e il suo Sequence Number vale 5000, i byte contenuti sono quelli indicizzati da 5000 a 5999. Il segmento successivo avrà Sequence Number pari a 6000.

3. **Initial Sequence Number (ISN)**: all'apertura di una connessione, il Sequence Number iniziale non parte quasi mai da zero! Viene selezionato in modo pseudo-casuale o agganciato a un contatore temporale di clock hardware. Questa scelta architetturale risponde a due imperativi:
   - *Prevenzione dei conflitti con segmenti fantasma*: evita che frammenti ritardati di una connessione precedente (chiusa di recente ma insistente sulla medesima 4-upla) vengano scambiati per dati validi della nuova connessione;
   - *Sicurezza contro il Session Hijacking*: impedisce a un attaccante esterno che non si trovi sul percorso di intercettazione di indovinare il Sequence Number corrente per iniettare segmenti malevoli forgiati (*TCP sequence prediction attack*).
4. **Acknowledgment Number (32 bit)**: ha valore valido solo se il flag `ACK` è impostato ad 1 (ossia in tutti i pacchetti tranne il primo SYN di apertura). Esprime una semantica rigorosamente **cumulativa**: indica il numero di sequenza del **prossimo byte che il mittente dell'ACK si aspetta di ricevere**. Se un host invia un pacchetto con Ack = 1461, sta formalmente certificando di aver già ricevuto e archiviato correttamente tutti i byte fino al 1460 compreso.

   **Esempio Operativo d'Aula: Sessione Telnet/SSH e Piggybacking**

   Per cogliere l'interazione microscopica tra *Sequence Number*, *Acknowledgment Number* e dati applicativi, consideriamo lo scenario didattico classico presentato dal docente relativo a una sessione interattiva di terminale remoto (`Telnet` o `SSH`):
   - Un utente su Host A digita sulla propria tastiera il singolo carattere `'C'`.
   - **Messaggio 1 (Host A → Host B)**: il client incapsula il carattere e invia il segmento:

     $$Seq = 42, \quad Ack = 79, \quad \text{dati} = \text{'C'} \quad (1\,\text{byte})$$

     $Seq = 42$ è l'indice del byte ''C'' nello stream A → B; $Ack = 79$ certifica che A ha già ricevuto tutti i byte da B fino al 78 compreso.
   - **Messaggio 2 (Host B → Host A) — Il Meccanismo del Piggybacking**: il server riceve il carattere, lo passa alla shell remota che genera l'eco a video del carattere stesso. Il server non spreca banda inviando due pacchetti separati (un ACK e poi i dati), ma li fonde in un unico segmento:

     $$Seq = 79, \quad Ack = 43, \quad \text{dati} = \text{'C'} \quad (1\,\text{byte})$$

     $Ack = 43$ riscontra il byte 42 ricevuto ($42 + 1 = 43$); $Seq = 79$ trasmette il byte di eco dello stream B → A.
   - **Messaggio 3 (Host A → Host B)**: il client riceve l'eco a video e conferma la ricezione:

     $$Seq = 43, \quad Ack = 80, \quad \text{dati} = \emptyset \quad (0\,\text{byte})$$

     $Ack = 80$ riscontra il byte 79 ricevuto ($79 + 1 = 80$); non essendovi nuovi dati da inviare, $Seq$ resta pari a 43.
5. **Header Length / Data Offset (4 bit)**: specifica la lunghezza complessiva dell'header TCP espressa in **parole da 32 bit** (4 byte). Poiché il campo è su 4 bit, il valore massimo rappresentabile è 15. Ne consegue che:
   - Il valore minimo consentito è 5 (header di base privo di opzioni: $5 \times 4 = 20$ byte);
   - Il valore massimo è 15 ($15 \times 4 = 60$ byte), lasciando un margine fino a 40 byte per i campi di opzione aggiuntivi.

6. **Flag di Controllo (6 bit)**: ciascun bit pilota una specifica modalità operativa:
   - `URG` (*Urgent*): se asserito ad 1, specifica che il segmento contiene dati con priorità assoluta il cui offset è indicato nel campo *Urgent Pointer* (meccanismo poco diffuso nei moderni stack);
   - `ACK` (*Acknowledgment*): indica che il campo *Acknowledgment Number* contiene un valore valido;
   - `PSH` (*Push*): ordina al ricevitore di consegnare immediatamente i dati presenti nel buffer all'applicazione locale senza attendere di riempire un intero blocco di memoria;
   - `RST` (*Reset*): provoca l'interruzione brutale e immediata della connessione senza alcuna negoziazione (utilizzato in caso di errore irrecuperabile, connessione a una porta chiusa o ripristino di risorse);
   - `SYN` (*Synchronize*): utilizzato tassativamente durante la fase di *Three-Way Handshake* per sincronizzare i numeri di sequenza iniziali tra le parti;
   - `FIN` (*Finish*): notifica che il trasmettitore ha esaurito i dati da inviare e richiede una chiusura ordinata (*gentle close*) della propria direzione di trasmissione.
7. **Receive Window (*rwnd*, 16 bit)**: comunica quanti byte il ricevitore è attualmente disposto ad accettare nel proprio buffer di ricezione. Costituisce il perno del meccanismo di **Controllo di Flusso**. Poiché il campo è codificato su 16 bit, la finestra massima annunciabile nativamente è limitata a:

$$rwnd_{\max} = 2^{16} - 1 = 65535\,\text{byte} \approx 64\,\text{KB} \tag{4.20}$$

   Nelle moderne reti ad altissima velocità e lungo raggio (grandi prodotti Banda-Ritardo, $B \cdot D$), 64 KB sono insufficienti a saturare il canale fisico. Per superare questa barriera, la `RFC 1323` ha introdotto l'opzione **Window Scale**, che consente al ricevitore di definire un fattore di scala moltiplicativo $2^S$ (con $S \leq 14$), elevando la finestra massima virtuale fino a quasi 1 GB.
8. **Internet Checksum (16 bit)**: calcola il complemento a uno della somma a 16 bit di tutti i campi dell'header TCP e del payload. Include nel computo anche un'intestazione fittizia denominata **Pseudo-Header IP** (costituita da IP sorgente, IP destinazione, protocollo fisso 6 e lunghezza totale del segmento). Questa deliberata violazione dell'isolamento dei livelli ISO/OSI garantisce che un segmento non venga consegnato all'host sbagliato in caso di corruzione silente del livello di rete.
9. **Urgent Pointer (16 bit)**: se il flag `URG` è attivo, indica l'offset di byte a partire dal Sequence Number oltre il quale i dati urgenti cessano.
10. **Options (Lunghezza Variabile, fino a 40 byte)**: possono includere:
    - **MSS (*Maximum Segment Size*)**: negoziato obbligatoriamente durante il SYN iniziale;
    - **Window Scale**: moltiplicatore di scala per la finestra *rwnd*;
    - **SACK Permitted e Blocchi SACK (*Selective Acknowledgment*)**: consentono al ricevitore di notificare blocchi di byte ricevuti correttamente a valle di un buco (RFC 2018);
    - **TCP Timestamps**: inclusione del tempo di trasmissione per la misura continua e inequivocabile dell'RTT (RFC 1323).

## 4.11 Gestione del Ciclo di Vita della Connessione

### 4.11.1 Apertura della Connessione: Il Three-Way Handshake (3WHS)

Prima di iniziare il trasferimento dati, client e server devono concordare i parametri di stato. La procedura di apertura prende il nome di **Three-Way Handshake** (stretta di mano a tre vie) ed è strutturata in tre passi sequenziali rigorosi:

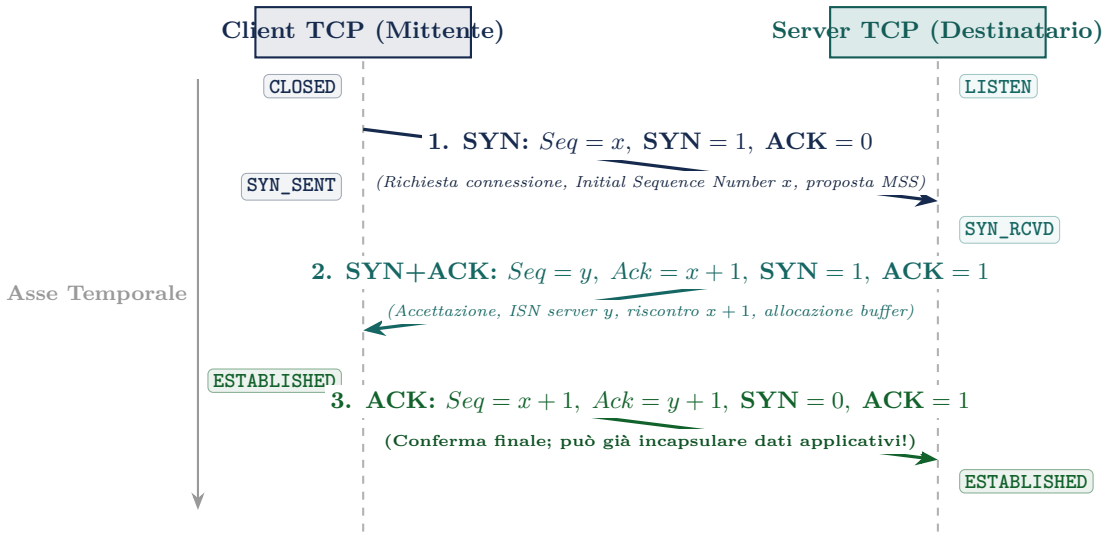



Figura 4.8: Diagramma temporale dello scambio a tre vie (*Three-Way Handshake*) per l'apertura della sessione.

1. **Passo 1 (Client → Server, segmento SYN)**: Il client seleziona un proprio Initial Sequence Number $x$ e invia un segmento speciale con il flag `SYN` asserito (SYN $= 1$, ACK $= 0$). Questo pacchetto non contiene dati applicativi, ma consuma formalmente un'unità di sequenza. All'interno delle opzioni, il client propone il proprio parametro MSS.
2. **Passo 2 (Server → Client, segmento SYN+ACK)**: Il server, trovandosi in stato di ascolto passivo (`LISTEN`), riceve il SYN e alloca nella memoria del kernel i buffer e le strutture di stato (*Transmission Control Block*, TCB). Risponde inviando un segmento congiunto di riscontro e sincronizzazione: sceglie il proprio ISN indipendente $y$, imposta sia il flag `SYN` sia il flag `ACK` (SYN $= 1$, ACK $= 1$), e imposta il riscontro a:
$$Ack = x + 1 \tag{4.21}$$
3. **Passo 3 (Client → Server, segmento ACK)**: Il client riceve la risposta, passa allo stato operativo `ESTABLISHED` e completa la stretta di mano inviando un ACK con $Seq = x + 1$ e $Ack = y + 1$ (SYN $= 0$, ACK $= 1$).

> **Principio / Fondamento: Trasporto Dati nel Terzo Pacchetto dell'Handshake**
>
> È fondamentale notare che, poiché il client ha già ricevuto la conferma del server, **il terzo segmento dell'handshake può già incorporare dati applicativi utili** (ad esempio la prima richiesta `GET` di un client HTTP).

### 4.11.2 Dimensionamento dell'MSS e Path MTU Discovery

Il parametro **MSS** (*Maximum Segment Size*) rappresenta la quantità massima di **soli dati applicativi** (escludendo gli header TCP e IP) che un segmento può trasportare.

**Definizione: Relazione tra MTU ed MSS**

Dato il vincolo imposto dalla **MTU** (*Maximum Transmission Unit*) del livello di collegamento sottostante, l'MSS massimo ammissibile è pari a:

$$MSS = MTU - \text{Lunghezza Header IP} - \text{Lunghezza Header TCP} \tag{4.22}$$

Poiché sulla tecnologia Ethernet lo standard universale fissa $MTU = 1500$ byte, considerando gli header base di 20 byte per IPv4 e 20 byte per TCP (totale 40 byte di overhead), si ricava il celebre valore:

$$MSS_{\text{Ethernet}} = 1500 - 20 - 20 = 1460 \text{ byte} \tag{4.23}$$

Per evitare frammentazioni intermedie a livello IP lungo la rotta (che degraderebbero pesantemente le prestazioni), i nodi moderni eseguono la procedura di **PMTUD** (*Path MTU Discovery*): trasmettono pacchetti con il bit IP `DF` (*Don't Fragment*) attivo; se un router intermedio incontra un link con MTU inferiore, scarta il pacchetto e restituisce una notifica ICMP *Fragmentation Needed*, permettendo al mittente di ricalibrare l'MSS alla strettoia fisica del percorso.

### 4.11.3 Chiusura della Connessione: Il Teardown a Quattro Vie (Half-Close)

Poiché una connessione TCP è full-duplex, la chiusura dei due canali di trasmissione è logicamente indipendente. Quando un processo ha terminato la spedizione dei dati, invia un segmento speciale con il flag `FIN` asserito:

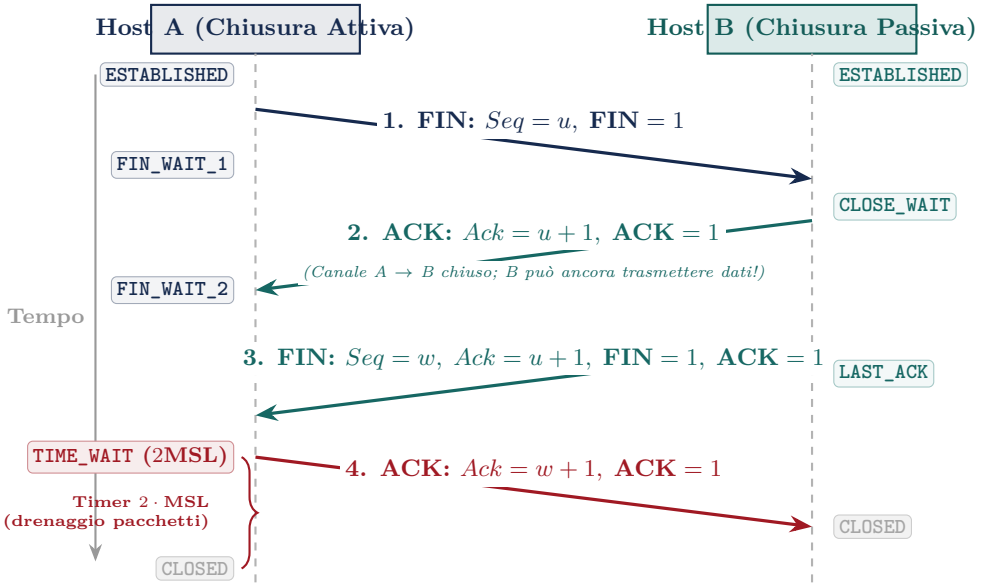



Figura 4.9: Diagramma temporale della chiusura simmetrica a quattro vie (*Gentle Teardown*) e stato TIME_WAIT.

**La Necessità Ingegneristica dello Stato TIME_WAIT (2MSL)**

Perché l'host che intraprende la chiusura attiva non passa immediatamente allo stato `CLOSED`, bensì attende una durata pari al doppio del **MSL** (*Maximum Segment Lifetime*, tipicamente stimato tra 30 secondi e 2 minuti)? Le motivazioni tecniche sono due:

1. **Garantire l'affidabilità della consegna dell'ultimo ACK**: se il quarto messaggio (l'ACK inviato da A) viene smarrito dalla rete, Host B ritrasmetterà il proprio segmento FIN dopo la scadenza del proprio timer. Se Host A fosse già passato a `CLOSED`, intercetterebbe il FIN ritrasmesso considerandolo inatteso e risponderebbe con un pacchetto di errore `RST`, segnalando un'interruzione anomala all'applicazione su B. Rimanendo in `TIME_WAIT`, Host A può invece riconoscere il FIN ritrasmesso e reinviare pacificamente l'ACK finale.
2. **Drenaggio dei pacchetti orfani (Quiet Time)**: assicura che tutti i segmenti e i duplicati appartenenti alla vecchia connessione ancora vaganti nei buffer dei router abbiano il tempo fisico di esaurire il proprio *Time-To-Live* (TTL) ed essere scartati. In questo modo si impedisce che un segmento ritardato possa riapparire a sorpresa e corrompere i dati di una successiva sessione aperta sulla medesima 4-upla!

### 4.11.4 Chiusura Forzata tramite Flag RST (Reset)

A differenza della chiusura aggraziata a quattro vie, il protocollo TCP prevede un meccanismo di aborto immediato tramite il flag `RST` (*Reset*):

- **Destinazione non raggiungibile o porta inattiva**: se un host riceve un pacchetto TCP indirizzato a una porta logica su cui nessun processo è in ascolto (*closed port*), lo stack del kernel risponde immediatamente con un pacchetto `RST` per notificare all'istante l'impossibilità di stabilire il dialogo (equivalente al codice ICMP *Port Unreachable* in UDP).
- **Ottimizzazione delle risorse sui server Web**: nei data center e sui server ad altissimo carico (milioni di connessioni brevi al secondo), mantenere aperte decine di migliaia di socket nello stato `TIME_WAIT` per due minuti comporterebbe l'esaurimento della tabella delle porte effimere e della memoria kernel. In tali scenari controllati, il server forza frequentemente la chiusura tramite un pacchetto `RST`, riciclando la socket istantaneamente.

## 4.12 Stima del Round-Trip Time e Calcolo Adattativo del Timeout (RTO)

### 4.12.1 La Natura Stocastica dell'RTT nelle Reti a Pacchetto

In una rete a commutazione di pacchetto, il tempo impiegato da un segmento per raggiungere il destinatario e far pervenire il relativo riscontro (**Round-Trip Time**, *RTT*) non è un valore deterministico o costante: esso oscilla continuamente a causa dell'instabilità dei tempi di accodamento nei buffer dei router intermedi.

Impostare il timeout di ritrasmissione (**RTO**, *Retransmission Timeout*) a un valore statico è disastroso:

- Se *RTO* è **troppo breve**: un minimo picco transitorio nel ritardo provoca lo scatto prematuro del timer, causando ritrasmissioni inutili di pacchetti che erano semplicemente ancora in viaggio;
- Se *RTO* è **troppo lungo**: a fronte di un'effettiva perdita fisica, il mittente rimane bloccato per un tempo eccessivo prima di ritrasmettere, abbattendo drasticamente il throughput.

Ne consegue che l'RTO deve essere **adattivo**, calcolato dinamicamente a runtime in base alle misure campionate sul canale.

### 4.12.2 Il Dilemma di Karn-Partridge e i Timestamp TCP

La misura base del tempo di andata e ritorno è il **SampleRTT**, definito come l'intervallo temporale che intercorre tra l'emissione di un segmento e la ricezione del corrispondente ACK.

**Principio / Fondamento: L'Ambiguità di Karn-Partridge**

Cosa accade se un segmento subisce un ritardo tale da far scadere l'RTO, il mittente lo ritrasmette, e poco dopo giunge un ACK?

A quale emissione appartiene l'ACK? Alla prima (con RTT molto lungo) o alla seconda (con RTT brevissimo)? (4.24)

Poiché l'header TCP convenzionale riscontra il numero di byte ma non reca traccia del numero di ritrasmissione, tale ambiguità è irrisolvibile. L'**Algoritmo di Karn-Partridge** impone una regola perentoria:

> *«È vietato calcolare campioni di SampleRTT su segmenti che sono stati ritrasmessi! Inoltre, in caso di timeout, non si applica la formula standard, ma si raddoppia ciecamente il valore dell'RTO corrente (**Exponential Backoff**).»*

La soluzione ingegneristica moderna per superare definitivamente questo vincolo risiede nell'opzione **TCP Timestamps** (`RFC 1323`): il trasmettitore inserisce un marcatore temporale hardware a 32 bit (*Timestamp Value*, `TSval`) all'interno delle opzioni del segmento; il ricevitore si limita a copiare fedelmente tale valore nel campo *Timestamp Echo Reply* (`TSecr`) dell'ACK. All'arrivo dell'ACK, il mittente calcola l'RTT esatto sottraendo `TSecr` dal clock corrente: l'ambiguità decade e ogni singolo pacchetto può essere campionato.

### 4.12.3 L'Algoritmo di Jacobson per la Stima di RTO (RFC 2988)

Per filtrare le oscillazioni casuali ad alta frequenza senza perdere la capacità di tracciare le derive a lungo termine del canale, TCP adotta una media mobile esponenziale pesata (**EWMA**, *Exponentially Weighted Moving Average*).

**Definizione: Formule di Jacobson / RFC 2988 per la Stima dell'RTO**

L'algoritmo opera in tre fasi matematiche distinte:

1. **Stima del Ritardo Medio Smussato ($SRTT$, *Smoothed RTT*)**:

$$SRTT \leftarrow (1-\alpha) \cdot SRTT + \alpha \cdot SampleRTT \tag{4.25}$$

   Il peso consigliato dallo standard è $\alpha = \frac{1}{8} = 0.125$, conferendo l'87.5% dell'importanza alla storia passata e il 12.5% all'ultimo campione.

2. **Stima della Dispersione e Variabilità ($RTTVAR$, *RTT Variation*)**:

$$RTTVAR \leftarrow (1-\beta) \cdot RTTVAR + \beta \cdot |SampleRTT - SRTT| \tag{4.26}$$

   Il coefficiente standard è $\beta = \frac{1}{4} = 0.25$. La deviazione media assoluta cattura la varianza e l'ampiezza delle code nella rete.

3. **Computo Finale dell'Intervallo di Timeout ($RTO$)**:

$$RTO = SRTT + \max(G,\ 4 \cdot RTTVAR) \tag{4.27}$$

dove $G$ rappresenta la granularità temporale dell'infrastruttura di clock del sistema operativo (nelle dispense d'aula del Prof. Lo Cigno, $G$ corrisponde convenzionalmente a 3 volte il quanto di temporizzazione con cui il kernel schedula il processo TCP, con valori tipici compresi tra 150 ms e 1 s).

**Inizializzazione e Regole Operative del Timer TCP**

L'implementazione concreta del timer nel kernel segue regole tassative dettate dalle specifiche didattiche ed RFC:

1. **Valori Iniziali alla Connessione**: all'apertura del canale non si calcola $SRTT$ durante il *Three-Way Handshake*. Si imposta convenzionalmente:
   - $RTO = 1$ s se l'handshake si è concluso al primo tentativo senza perdite;
   - $RTO = 3$ s qualora l'handshake abbia richiesto ritrasmissioni;
   - Al primo campione valido misurato $RTT(1)$: si pone $SRTT(1) = RTT(1)$ e $RTTVAR(1) = \frac{RTT(1)}{2}$.
2. **Principio del Timer Singolo (Head-of-Line)**: per minimizzare il sovraccarico di sistema della CPU, TCP mantiene **un solo timer attivo per connessione**. L'istante di decorrenza dell'RTO è calcolato in base all'invio del segmento non riscontrato più vecchio attualmente in volo.
3. **Backoff Esponenziale su Perdite Consecutive**: alla $i$-esima ritrasmissione consecutiva del medesimo segmento, l'intervallo viene moltiplicato:

$$RTO^{(i)} = 2^{i-1} \cdot RTO \tag{4.28}$$

4. **Ripristino del Valore Nominale**: l'RTO viene riportato al suo valore calcolato ordinario (senza fattore moltiplicativo di backoff) **solo dopo la trasmissione andata a buon fine di un segmento mai trasmesso in precedenza**, aggiornando prima $SRTT$ e $RTTVAR$ con il nuovo campione di ritardo non ambiguo.
5. **Politica di Rinuncia**: TCP ripete la ritrasmissione raddoppiando l'RTO per 10 volte; dopodiché effettua ulteriori 6 tentativi mantenendo fisso l'RTO massimo raggiunto. Se dopo 16 tentativi complessivi non si riceve alcun riscontro, la socket si arrende e la connessione viene terminata per timeout irreversibile.

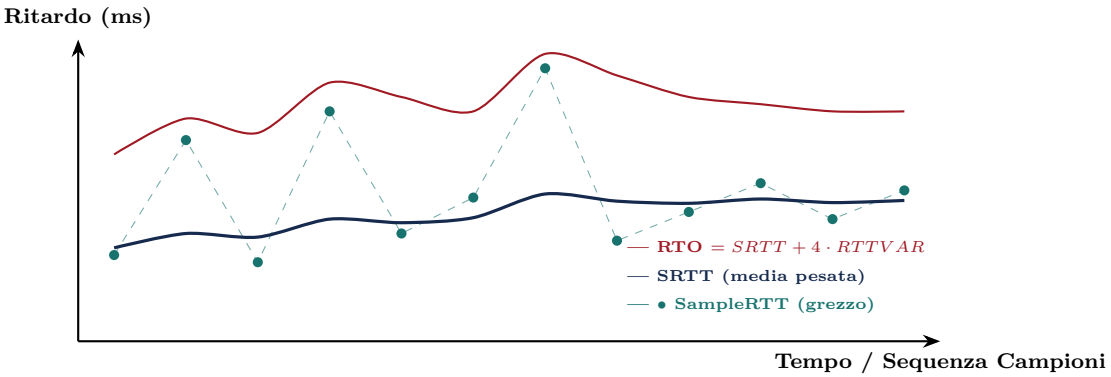



Figura 4.10: Dinamica adattativa di Jacobson: inseguimento della media smussata e calcolo del margine di sicurezza per l'RTO.

Il termine $+4 \cdot RTTVAR$ ha un fondamento probabilistico ineccepibile: applicando la disuguaglianza di Chebychev, posizionare il margine a 4 volte la deviazione garantisce che la probabilità che un ritardo casuale superi la soglia provocando un falso allarme sia inferiore al 6%, azzerando le ritrasmissioni premature.

## 4.13 Il Controllo di Flusso End-to-End

### 4.13.1 Protezione del Buffer del Ricevitore

È vitale non confondere il **Controllo di Flusso** con il **Controllo di Congestione**:

- Il **Controllo di Congestione** protegge la *rete interna* (router intermedi, code dei commutatori);
- Il **Controllo di Flusso** protegge esclusivamente il *ricevitore terminale*, impedendo che un trasmettitore troppo veloce saturi il **Receive Buffer** dell'host destinatario mandandolo inesorabilmente in overflow.

L'applicazione in esecuzione sul destinatario preleva i byte dal buffer tramite chiamate di sistema (es. `read()` o `recv()`) con una velocità che dipende dal carico computazionale della CPU locale, del tutto disaccoppiata dalla velocità con cui i pacchetti giungono dalla scheda di rete:




Figura 4.11: Dinamica interna del Receive Buffer e calcolo della finestra libera annunciata (*rwnd*).

**Definizione: Formulazione del Controllo di Flusso**

Definiti i puntatori espressi in numeri di byte:

- *LastByteRead*: indice dell'ultimo byte prelevato dal processo applicativo;
- *LastByteRcvd*: indice dell'ultimo byte ricevuto dalla rete e memorizzato nel buffer;

il volume di dati attualmente stazionante nel buffer è dato dalla differenza ($LastByteRcvd - LastByteRead$). Affinché non si verifichi alcuno scarto per overflow, deve valere l'invariante:

$$LastByteRcvd - LastByteRead \leq RcvBuffer \tag{4.29}$$

La **Receive Window** (*rwnd*), comunicata nell'header TCP ad ogni riscontro, quantifica esattamente lo spazio residuo libero:

$$rwnd = RcvBuffer - (LastByteRcvd - LastByteRead) \tag{4.30}$$

Il trasmettitore vincola la propria trasmissione imponendo che la quantità di byte inviati ma non ancora riscontrati (*bytes in flight*) non ecceda mai la finestra annunciata dal destinatario:

$$LastByteSent - LastByteAcked \leq rwnd \tag{4.31}$$

### 4.13.2 Il Problema dello Stallo con Finestra a Zero e i Pacchetti Sonda (Probing)

Cosa accade se il processo ricevente smette temporaneamente di leggere i dati e il buffer si riempie del tutto? Il ricevitore invia un ACK con $rwnd = 0$. Il mittente riceve la notifica e, rispettando il vincolo, blocca immediatamente ogni ulteriore trasmissione.

Successivamente, l'applicazione ricevente si risveglia, esegue una `read()` e libera 10 KB di memoria nel buffer. Il kernel del ricevitore genera allora un nuovo segmento di riscontro per annunciare l'avvenuta riapertura della finestra ($rwnd = 10240$ B, noto come *Window Update ACK*).

**Principio / Fondamento: Il Deadlock da Smarrimento del Window Update**

Un pacchetto di solo riscontro non contiene dati applicativi e non è a sua volta riscontrato. Se questo *Window Update ACK* viene perso nella rete:

- Il ricevitore rimane in attesa di dati, convinto di aver aperto la finestra;
- Il trasmettitore rimane bloccato all'infinito, convinto che la finestra sia ancora zero!

Il sistema entrerebbe in un mortale blocco reciproco (**Deadlock**).

Per scongiurare questo collasso, TCP prescrive il meccanismo di **Zero-Window Probing**:

- Non appena il mittente riceve $rwnd = 0$, attiva un timer speciale denominato **Persist Timer**;

- Alla scadenza del timer, il trasmettitore invia forzatamente un pacchetto di sonda (*Probe Packet*) contenente **un singolo byte di dati**;
- Il ricevitore non può scartare la sonda senza rispondere: è formalmente obbligato a emettere un ACK, all'interno del quale è costretto a reiterare il valore aggiornato di *rwnd*;
- Se *rwnd* è ancora zero, il persist timer raddoppia con backoff esponenziale e la procedura si ripete ciclicamente fino alla riapertura definitiva del canale.

## 4.14 I Fondamenti del Controllo di Congestione: Cause, Costi ed Approccio End-to-End

### 4.14.1 Che cos'è la Congestione di Rete?

Mentre il controllo di flusso opera bilateralmente tra due host, la **congestione** è una patologia sistemica della rete: essa si verifica quando un insieme concorrente di sorgenti tenta di trasmettere dati a una velocità aggregata superiore a quella che i canali fisici o i buffer dei router intermedi possono sostenere:

$$\sum \lambda_{\text{sorgenti}} > C_{\text{collo di bottiglia}} \tag{4.32}$$

Le conseguenze dirette sono il riempimento delle code nei router, l'esplosione dei ritardi di accodamento e, infine, la perdita fisica di pacchetti per esaurimento dei buffer (*buffer drop*).

### 4.14.2 Analisi dei Costi della Congestione: I Tre Scenari Canonici

Per comprendere la gravità del fenomeno, analizziamo i tre scenari teorici progressivi presentati dal docente:

1. **Scenario 1: Due trasmettitori, un router con buffer infiniti**: Ipotizziamo una rete ideale in cui i router dispongano di una quantità infinita di memoria. In tale contesto non vi è alcuna perdita di pacchetti ($P_{\text{loss}} = 0$). Tuttavia, all'avvicinarsi della frequenza di arrivo aggregata alla capacità fisica del canale ($\lambda_{in} \to C$), la teoria delle code (modello $M/M/1$) dimostra che il tempo medio di accodamento diverge asintoticamente:
$$T_{\text{coda}} = \frac{1}{\mu - \lambda} \xrightarrow{\lambda \to \mu} \infty \tag{4.33}$$
**Primo costo sistemico**: ritardi di recapito asintoticamente infiniti all'approssimarsi della piena saturazione.
2. **Scenario 2: Un router con buffer finiti e ritrasmissioni**: Poiché nella realtà i buffer sono finiti, non appena la coda si satura i pacchetti in arrivo vengono distrutti. I mittenti, non ricevendo gli ACK entro l'RTO, sono costretti a ritrasmettere. Ne consegue che il carico complessivo immesso nella rete ($\lambda'_{in}$) non è più formato solo dai dati originali ($\lambda_{in}$), ma include una quota pesante di dati ritrasmessi:
$$\lambda'_{in} = \lambda_{in} + \lambda_{\text{ritrasmissioni}} \tag{4.34}$$
Se i timer RTO scattano prematuramente, la rete viene invasa da duplicati di pacchetti che erano ancora in coda, sprecando banda per recapitare informazioni ridondanti. **Secondo costo sistemico**: crollo del *goodput* (la frazione di bit utili consegnati con successo all'applicazione) rispetto alla capacità del link.
3. **Scenario 3: Nodi multipli con percorsi incrociati e Collasso da Congestione**: Consideriamo una topologia distribuita con più router in cascata attraversati da flussi di traffico ortogonali. Immaginiamo che un pacchetto attraversi con successo i primi tre router consumando capacità di trasmissione sui rispettivi canali, e giunto al quarto router venga scartato per buffer pieno.

> **Principio / Fondamento: Lo Spreco a Monte e il Congestive Collapse**
>
> Quando un pacchetto viene scartato al salto $k$-esimo, **tutto il lavoro di trasmissione, la banda e le risorse di accodamento consumate nei salti da 1 a $k-1$ sono stati completamente gettati al vento**! Se tutti i mittenti continuano ad aumentare ciecamente il carico, la capacità della rete viene interamente divorata per trasportare pacchetti destinati a essere distrutti poche tratte più avanti: il throughput utile globale della rete crolla verticalmente a zero! Questo fenomeno distruttivo prende il nome storico di **Collasso da Congestione** (*Congestive Collapse*).

### 4.14.3 La Filosofia End-to-End di TCP

Come può Internet prevenire il collasso da congestione? In linea di principio esisterebbero due approcci:

- *Controllo assistito dalla rete*: i router interni monitorano le proprie code e inviano pacchetti espliciti di strozzamento alle sorgenti (*Choke Packets*) o marcano bit nell'header (tecnologia **ECN**, *Explicit Congestion Notification*);
- **Controllo End-to-End (scelta fondante di TCP)**: coerentemente con il principio architetturale della rete IP stupida e terminali intelligenti, i router intermedi non collaborano attivamente al controllo. Sono i **sistemi terminali** a inferire la congestione della rete analizzando gli effetti collaterali osservabili:
  1. La comparsa di **perdite di pacchetti** (scadenza del timer RTO o arrivo di ACK duplicati);
  2. L'aumento progressivo dell'RTT misurato.

## 4.15 Finestra di Congestione e Algoritmi di Crescita: Slow Start e Congestion Avoidance

### 4.15.1 La Variabile Dinamica CWND e il Principio del Self-Clocking

Per implementare il controllo di congestione, ogni istanza TCP operante come trasmettitore definisce una variabile di stato fondamentale: la **Finestra di Congestione** (**CWND**, *Congestion Window*), espressa in byte (ma tipicamente concettualizzata come multiplo di MSS).

**Definizione: Finestra Trasmissiva Effettiva**

La quantità massima di dati che il mittente è autorizzato a mantenere in volo nella rete (*bytes in flight*) è vincolata contemporaneamente dal ricevitore (flusso) e dalla rete (congestione):

$$W = \min(CWND,\ rwnd) \tag{4.35}$$

Nelle sezioni seguenti, ipotizzeremo che il ricevitore disponga di un buffer sufficientemente capiente da rendere $rwnd \gg CWND$, cosicché il collo di bottiglia operativo sia dettato unicamente da $CWND$.

TCP è un protocollo basato sul **Self-Clocking**: la velocità di emissione dei segmenti non è scandita da un timer hardware periodico, ma è sincronizzata all'arrivo degli ACK di ritorno. La ricezione di un ACK attesta che un segmento è uscito dalla rete ed è stato consegnato, autorizzando il trasmettitore a immettere nuovi dati.

L'algoritmo standard TCP (**RFC 2581** e successive) articola la gestione di $CWND$ in due regimi operativi distinti governati da una soglia critica denominata **ssthresh** (*Slow Start Threshold*):

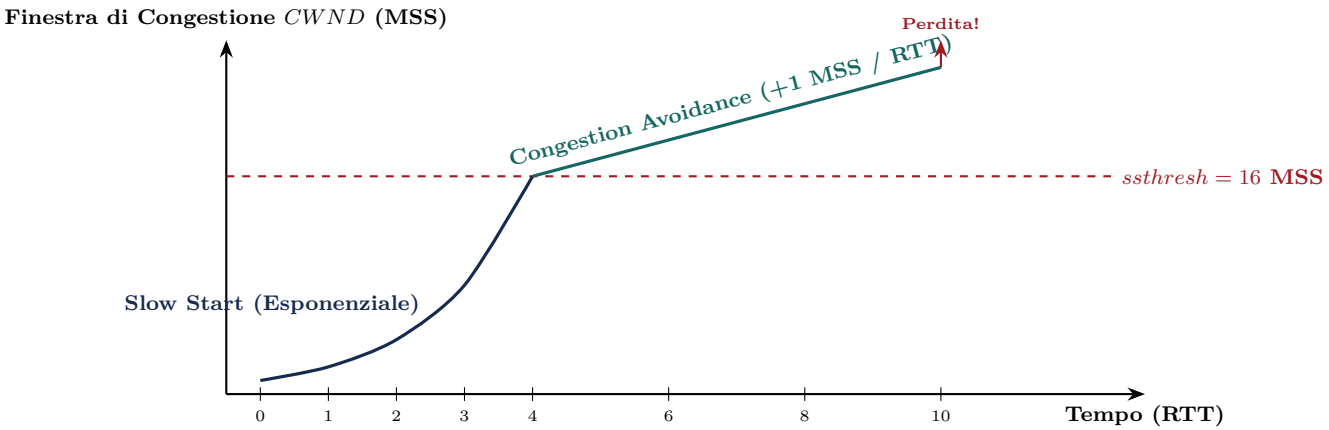



Figura 4.12: Transizione fisiologica da Slow Start (crescita esponenziale) a Congestion Avoidance (crescita lineare additiva al superamento di *ssthresh*).

### 4.15.2 L'Algoritmo di Slow Start (Partenza Lenta)

All'avvio della connessione, il trasmettitore ignora quale sia la reale capacità del percorso. Se iniettasse istantaneamente una grande quantità di segmenti, potrebbe mandare in saturazione immediata il canale.

La fase di **Slow Start** ha lo scopo di sondare rapidamente la capacità della rete:

1. **Inizializzazione**: si imposta la finestra iniziale a un valore conservativo:

$$CWND = 1 \text{ MSS} \quad \text{(nelle RFC moderne tipicamente da 2 a 10 MSS)} \tag{4.36}$$

2. **Regola di Incremento per Singolo ACK**: per **ogni riscontro non duplicato** ricevuto, la finestra viene incrementata di un intero MSS:

$$CWND \leftarrow CWND + 1 \text{ MSS} \tag{4.37}$$

3. **Dinamica Esponenziale su Scala RTT**: Supponiamo che $CWND = 1$ MSS. Il mittente trasmette 1 segmento. Dopo un RTT giunge l'ACK $\implies$ $CWND = 2$ MSS. Il mittente immette ora 2 segmenti. Dopo un RTT giungono 2 ACK. Ciascun ACK incrementa $CWND$ di 1 MSS $\implies$ $CWND = 2+1+1 =$ 4 MSS. Al ciclo successivo giungono 4 ACK $\implies$ $CWND = 8$ MSS.

**Il Paradosso del Nome "Slow Start"**

Sebbene sia chiamato "partenza lenta" (perché inizia cautelativamente da un solo segmento), il regime di incremento è in realtà **esponenziale**: ad ogni tempo di andata e ritorno ($RTT$), la finestra di congestione raddoppia:

$$CWND(t + RTT) = 2 \cdot CWND(t) \tag{4.38}$$

La fase di Slow Start prosegue imperterrita fino a quando non si verifica una delle seguenti due condizioni:

- $CWND \geq ssthresh$: la finestra ha raggiunto la soglia di sicurezza, il protocollo passa in **Congestion Avoidance**;
- Viene rilevata una perdita di pacchetti (timeout RTO o 3 ACK duplicati).

### 4.15.3 L'Algoritmo di Congestion Avoidance (Prevenzione della Congestione)

Non appena $CWND$ raggiunge o supera la soglia $ssthresh$, continuare a raddoppiare esponenzialmente la finestra significherebbe causare una collisione massiva nei buffer dei router.

Il protocollo commuta nella fase di **Congestion Avoidance**: la crescita diventa **lineare e conservativa**. L'obiettivo progettuale è incrementare la finestra di **un solo MSS per ogni RTT**:

$$CWND(t + RTT) \approx CWND(t) + 1 \text{ MSS} \tag{4.39}$$

Poiché lo stack TCP del kernel elabora gli eventi all'arrivo dei singoli ACK e non a blocchi di RTT, come si implementa matematicamente questo incremento unitario distribuito su tutti gli ACK della finestra?

**Definizione: Aggiornamento di Congestion Avoidance per Singolo ACK**

All'arrivo di ogni ACK valido non duplicato, $CWND$ viene aggiornata secondo la formula frazionaria:

$$CWND \leftarrow CWND + \text{MSS} \cdot \frac{\text{MSS}}{CWND} \tag{4.40}$$

Poiché all'interno di un RTT il mittente riceve un numero di ACK pari a $\frac{CWND}{\text{MSS}}$, l'incremento cumulato al termine dell'RTT risulta esattamente:

$$\Delta CWND = \sum_{i=1}^{\frac{CWND}{\text{MSS}}} \left( \text{MSS} \cdot \frac{\text{MSS}}{CWND} \right) = \left( \frac{CWND}{\text{MSS}} \right) \cdot \left( \text{MSS} \cdot \frac{\text{MSS}}{CWND} \right) = 1 \text{ MSS} \tag{4.41}$$

### 4.15.4 Tracciamento Passo-Passo dei Puntatori (LP e UP) ed Esempio d'Aula

Nelle dispense analitiche del Prof. Lo Cigno, il funzionamento interno del trasmettitore viene formalizzato attraverso il movimento di due puntatori di confine nel buffer:

- **Lower Pointer (LP)**: punta al numero di sequenza del primo byte non ancora riscontrato dall'ACK più recente ricevuto;
- **Upper Pointer (UP)**: punta al numero di sequenza dell'ultimo byte fisicamente immesso sul canale.

La differenza ($UP - LP$) quantifica rigorosamente il volume istantaneo di dati in volo (*bytes in flight*).

**Definizione: Algoritmo di Aggiornamento di Finestra e Puntatori**

Ad ogni ricezione di un riscontro valido ACK($n$), lo stack del trasmettitore aggiorna le variabili di stato e invia nuovi dati secondo il seguente algoritmo formale:

```
if (valid ACK(n)) {
if (CWND < SSTHRESH) {
CWND = CWND + MSS;   // Fase di Slow Start (esponenziale)
} else {
CWND = CWND + MSS * (MSS / CWND);   // Congestion Avoidance (lineare)
}
TXWND = min(CWND, RCWND);
LP = n;
UP = n + TXWND - 1;
if (LP <= UnsentData <= UP) {
TransmitData();   // Invia segmenti non ancora trasmessi ammessi dalla finestra
}
}
```

**Evoluzione Dettagliata della Finestra e dei Puntatori**

Analizziamo la dinamica temporale dell'algoritmo ipotizzando: dati misurati in unità di segmenti, tempo di trasmissione trascurabile rispetto all'RTT ($RTT \gg T_{\text{tx}}$), finestra di ricezione fissa $RCWND = 12$ segmenti, e soglia iniziale $ssthresh = RCWND/2 = 6$ segmenti.

1. **Inizio ($t = 0$, RTT 0)**: $TXWND = 1$, puntatori $LP = 1$, $UP = 1$. Il mittente trasmette il segmento 1.
2. **RTT 1**: giunge Ack(2). Poiché $CWND = 1 < ssthresh = 6$, siamo in *Slow Start*:

$$CWND \leftarrow 1 + 1 = 2, \quad TXWND = 2, \quad LP = 2, \quad UP = 2 + 2 - 1 = 3$$

   Vengono immessi in rete i segmenti 2 e 3.
3. **RTT 2**:
   - Giunge Ack(3) $\implies$ $CWND \leftarrow 2 + 1 = 3$, $TXWND = 3$, $LP = 3$, $UP = 5$. Vengono trasmessi i segmenti 4 e 5.
   - Giunge Ack(4) $\implies$ $CWND \leftarrow 3 + 1 = 4$, $TXWND = 4$, $LP = 4$, $UP = 7$. Vengono trasmessi i segmenti 6 e 7.
4. **RTT 3**:
   - Giunge Ack(5) $\implies$ $CWND \leftarrow 5$, $LP = 5$, $UP = 9$. Vengono trasmessi i segmenti 8 e 9.
   - Giunge Ack(6) $\implies$ $CWND \leftarrow 6 = ssthresh$, $LP = 6$, $UP = 11$. Vengono trasmessi i segmenti 10 e 11.
   - Giunge Ack(7): la soglia è raggiunta ($CWND \geq ssthresh$), transizione in **Congestion Avoidance**:

$$CWND \leftarrow 6 + \frac{1}{6} \approx 6.17, \quad TXWND = \lfloor 6.17 \rfloor = 6, \quad LP = 7, \quad UP = 12$$

     Viene trasmesso il solo segmento 12.
   - Giunge Ack(8) $\implies$ $CWND \leftarrow 6.17 + \frac{1}{6} \approx 6.33$, $TXWND = 6$, $LP = 8$, $UP = 13$. Viene trasmesso il segmento 13.
5. **RTT 4**:
   - Giungono gli Ack$(9, 10, 11)$ $\implies$ $CWND$ cresce a $6.50, 6.67, 6.83$. La parte intera della finestra non cambia ($TXWND = 6$), vengono trasmessi i segmenti 14, 15, 16.
   - Giunge Ack(12) $\implies$ $CWND \leftarrow 7.00$, la finestra di trasmissione sale a $TXWND = 7$, $LP = 12$, $UP = 18$. Vengono trasmessi i segmenti 17 e 18.
6. **RTT 5–9**: la finestra continua la crescita lineare additiva (+1 segmento ogni intero ciclo RTT) passando per $CWND = 8, 9, 10, 11, 12$.
7. **A Regime ($RTT \geq 9$)**: la finestra raggiunge il limite imposto dal ricevitore ($CWND = RCWND = 12$). Da questo istante in poi $TXWND = \min(CWND, RCWND) = 12$: il trasmettitore cessa di espandere la finestra e immette esattamente 12 segmenti per ogni RTT in condizioni stazionarie!

## 4.16 Meccanismi di Recupero delle Perdite: RTO, Fast Retransmit e Fast Recovery

### 4.16.1 Tassonomia degli Eventi di Perdita: Timeout vs Duplicate ACK

Nel corso del trasferimento dati, la congestione si manifesta inevitabilmente con la perdita di uno o più segmenti. Nel modello TCP, una perdita viene notificata al mittente attraverso due manifestazioni fenomenologiche profondamente diverse:

1. **Scadenza del Timer di Ritrasmissione (RTO)**: Il timer scade senza che sia giunto alcun ACK per il segmento in testa alla coda. Questo evento diagnostica un'anomalia severa: la rete è entrata in **congestione massiva** oppure si è verificata un'interruzione fisica del link. Non vi è alcun pacchetto che riesca ad attraversare la strettoia.
2. **Ricezione di 3 ACK Duplicati (Triple Duplicate ACK)**: Il trasmettitore riceve un primo ACK seguito da ulteriori 3 ACK identici (per un totale di 4 ACK identici consecutivi).

> **Principio / Fondamento: La Firma Telemetrica dei 3 ACK Duplicati**
>
> Se il ricevitore ha emesso 3 ACK duplicati reiterando il medesimo numero di sequenza, significa che **almeno altri tre segmenti successivi a quello mancante sono transitati con successo attraverso la rete e sono giunti a destinazione** (innescando le regole di generazione ACK per pacchetti fuori ordine viste nella Tabella 4.2)! La rete, pertanto, **non è bloccata**: ha perso un singolo pacchetto isolato, ma continua a drenare traffico.

### 4.16.2 La Reazione Drastica al Timeout (RTO)

Quando scade il timer RTO, la reazione di TCP è categorica e conservativa:

1. La soglia di Slow Start viene dimezzata rispetto alla finestra corrente:

$$ssthresh \leftarrow \max\left(\frac{CWND}{2},\ 2\ \text{MSS}\right) \tag{4.42}$$

2. La finestra di congestione viene azzerata al valore iniziale di partenza:

$$CWND \leftarrow 1\ \text{MSS} \tag{4.43}$$

3. Si raddoppia l'intervallo di timeout con legge esponenziale (*Exponential Backoff*):

$$RTO \leftarrow 2 \cdot RTO \tag{4.44}$$

4. Il trasmettitore ritrasmette il segmento perso e riparte da zero in regime di **Slow Start** esponenziale fino al raggiungimento della nuova soglia *ssthresh*.

### 4.16.3 L'Algoritmo di Fast Retransmit (Ritrasmissione Rapida)

Attendere la scadenza del timer RTO per recuperare una perdita isolata comporta un'intollerabile perdita di tempo (spesso dell'ordine di centinaia di millisecondi).

L'algoritmo di **Fast Retransmit** (`RFC 2581`) intercetta la ricezione del **terzo ACK duplicato** come trigger immediato: non appena il contatore dei duplicati raggiunge quota 3, il mittente deduce senza ombra di dubbio che il pacchetto corrispondente all'ACK è andato smarrito ed **esegue immediatamente la ritrasmissione del segmento mancante, prima ancora che il timer RTO abbia il tempo di scadere**!

### 4.16.4 Evoluzione Storica: Il Confronto tra TCP Tahoe e TCP Reno

La gestione successiva al Fast Retransmit ha segnato l'evoluzione storica dell'architettura Internet, incarnata nelle due storiche versioni del protocollo:

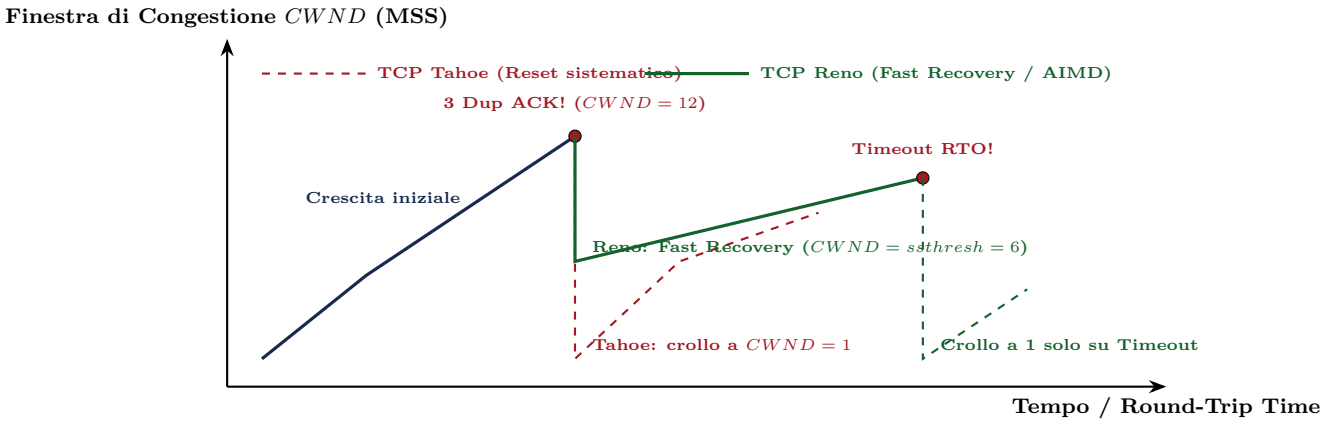



Figura 4.13: Dinamica comparativa tra TCP Tahoe (azzeramento sia su timeout che su 3 Dup ACK) e TCP Reno (Fast Recovery su 3 Dup ACK con conservazione dello stato).

1. **TCP Tahoe (1988, Van Jacobson) – Recupero con Fast Retransmit**: Tahoe non faceva distinzione di principio tra timeout ed eventi di perdita multipli o singoli. Alla ricezione del 3° ACK duplicato (4 ACK identici), Tahoe esegue la seguente sequenza algoritmica formale ():
   a. Registra lo stato al momento della congestione e memorizza l'estremo superiore dei segmenti già trasmessi nella variabile ausiliaria `UPLoss`:
   $$ssthresh \leftarrow \frac{CWND}{2}, \qquad UPLoss \leftarrow UP \tag{4.45}$$
   b. Crolla la finestra di congestione a un solo segmento e azzera lo stato:
   $$CWND \leftarrow 1 \text{ MSS} \tag{4.46}$$
   c. Ritrasmette immediatamente il primo segmento della finestra (il pacchetto mancante indicato dall'ACK cumulativo);
   d. Ignora e scarta sistematicamente tutti gli ulteriori ACK duplicati che continuano a giungere;
   e. Alla ricezione di un nuovo ACK non duplicato, aggiorna il puntatore di base:
   $$LP \leftarrow \text{ACK_Number}, \qquad UP \leftarrow LP + CWND \tag{4.47}$$
   f. **Controllo di Uscita**:
      - Se ACK_Number $> UPLoss$: l'intero buco di trasmissione è stato colmato. Il protocollo esce dalla procedura di recupero e riprende il normale funzionamento in **Slow Start**;
      - Se ACK_Number $\leq UPLoss$: si è verificata una perdita multipla all'interno della medesima finestra. Tahoe ritrasmette il segmento successivo mancante ricominciando dal passo (c);
   g. Se durante l'attesa scade il timer RTO, la procedura viene abortita e si ricade nella drastica reazione a timeout.

   Sebbene Tahoe evitasse l'attesa del timeout per il primo pacchetto perso, la distruzione della finestra ($CWND \leftarrow 1$) causava un funzionamento discontinuo "a singhiozzo", azzerando il flusso nei buffer e sprecando banda.
2. **TCP Reno (1990) – L'Algoritmo di Fast Recovery**: Reno introduce la geniale intuizione della **Fast Recovery** (): ogni ACK duplicato successivo al terzo è la prova inconfutabile che un segmento successivo ha attraversato la rete ed è uscito dal collo di bottiglia, liberando spazio nei buffer. Anziché fermare la trasmissione e azzerare la finestra, Reno "gonfia" la finestra e mantiene la connessione attiva:
   a. Alla ricezione del 3° ACK duplicato, dimezza la soglia e salva l'estremo superiore:
   $$ssthresh \leftarrow \frac{CWND}{2}, \qquad UPLoss \leftarrow UP \tag{4.48}$$
   b. Ritrasmette immediatamente il primo segmento mancante della finestra;
   c. "Gonfia" artificialmente la finestra di congestione per contabilizzare i 3 segmenti che hanno generato

i 3 ACK duplicati lasciando la rete:

$$CWND \leftarrow ssthresh + 3 \text{ MSS} \tag{4.49}$$

Se la nuova finestra lo consente ($UP - LP < CWND$), trasmette nuovi segmenti;

d. Per ogni ulteriore ACK duplicato ricevuto mentre la ritrasmissione è in volo:

$$CWND \leftarrow CWND + 1 \text{ MSS} \tag{4.50}$$

e, se consentito dal margine di finestra, **immette un nuovo segmento nella rete**. Questo garantisce il mantenimento del "clock dei pacchetti" (*Packet Clocking*), evitando che la tubatura (*pipe*) di trasmissione si svuoti!

e. Quando finalmente giunge un ACK non duplicato, aggiorna $LP \leftarrow$ ACK_Number;

f. **Controllo di Uscita e Sgonfiamento (*Deflating ACK*)**:

- Se ACK_Number $> UPLoss$: tutti i segmenti inviati prima della perdita sono stati riscontrati con successo. Reno "sgonfia" all'istante la finestra di congestione riportandola al valore di regime:
$$CWND \leftarrow ssthresh, \qquad UP \leftarrow LP + CWND \tag{4.51}$$
ed esce dalla Fast Recovery entrando direttamente in **Congestion Avoidance** (crescita lineare additiva), **senza transitare per la Slow Start**!
- Se ACK_Number $\leq UPLoss$ (cosiddetto *Partial ACK*): vi sono ulteriori pacchetti persi nel medesimo volo di dati. Reno ritrasmette il segmento mancante e rimane in Fast Recovery continuando a elaborare gli ACK duplicati.

g. Se scade il timer RTO (ad esempio per perdita del segmento ritrasmesso), Reno abbandona la Fast Recovery e ricade nel recupero standard con azzeramento $CWND \leftarrow 1$ MSS.

**Principio / Fondamento: La Regola Aurea AIMD (Additive Increase Multiplicative Decrease)**

L'alternanza operativa di TCP Reno tra incremento lineare e dimezzamento su perdita isolata realizza la legge di controllo universale **AIMD**:

- **Additive Increase (+1 MSS / RTT)**: incrementa la finestra di un'unità costante per ciclo RTT per esplorare cautamente la banda residua disponibile;
- **Multiplicative Decrease ($-50\%$)**: dimezza moltiplicativamente la finestra non appena rileva una perdita isolata, liberando all'istante il 50% della capacità per far decomprimere i buffer dei router.

Il grafico di $CWND(t)$ assume così il celeberrimo andamento periodico a **dente di sega** (*sawtooth pattern*).

### 4.16.5 Cenni alle Evoluzioni Moderne: TCP CUBIC e TCP Compound

Sebbene Reno rappresenti il riferimento didattico cardine, nelle reti moderne ad altissima capacità e lungo raggio (grandi prodotti $B \cdot D$, ad esempio link a 10 Gbps transoceanici) un incremento di un solo MSS per RTT richiederebbe decine di minuti per risaturare il canale dopo un dimezzamento. Per questo motivo:

- **TCP NewReno (`RFC 3782`)**: perfeziona la Fast Recovery gestendo perdite multiple all'interno della stessa finestra senza ricadere in timeout;
- **TCP SACK (`RFC 2018/2883`)**: notifica blocchi discreti ricevuti permettendo di ritrasmettere solo i buchi effettivi;
- **TCP CUBIC**: algoritmo predefinito del kernel Linux (a partire dalla versione 2.6.19). Sostituisce la crescita lineare con una funzione cubica del tempo trascorso dall'ultima congestione, consentendo una crescita ultra-rapida lontano dal punto critico e un appiattimento in prossimità della saturazione;
- **TCP Compound**: algoritmo integrato nei sistemi operativi Windows, che combina una finestra basata sulle perdite (loss-based) con una seconda finestra basata sulle variazioni stimate del ritardo di accodamento (delay-based).

## 4.17 Modellazione Analitica del Throughput ed Equità (*Fairness*) di Rete

### 4.17.1 Dalla Finestra Fissa alla Finestra Variabile: L'Integrale del Throughput

Nel caso elementare di un protocollo a finestra statica (come un circuito Stop-and-Wait o Go-Back-N a dimensione fissa $D_F$ byte), il throughput effettivo scambiato lungo una connessione di durata totale $T$ secondi è dato banalmente da ():

$$\text{Thr} = \frac{D_F \cdot 8}{T} \qquad [\,\text{bit/s}] \tag{4.52}$$

In TCP, tuttavia, la dimensione della finestra di trasmissione varia continuamente nel tempo in risposta alla telemetria degli ACK e degli eventi di congestione: $D_F = D_F(t) = CWND(t) \cdot \text{MSS}$. Il throughput istantaneo varia istante per istante; su un orizzonte temporale macroscopico $[t_0, t_1]$ di durata $T = t_1 - t_0$, il **throughput medio continuo** corrisponde formalmente all'**integrale della finestra nel tempo** ():

$$\text{Thr} = \frac{1}{T}\int_{t_0}^{t_1} D_F(t)\,dt = \frac{A_{DF}}{t_1 - t_0} \qquad [\,\text{bit/s}] \tag{4.53}$$

dove $A_{DF}$ rappresenta l'area geometrica complessiva sottesa dalla curva temporale della finestra di trasmissione (espressa in bit o in byte per secondo).

**Il Ruolo Sistemico della Finestra come Freno di Congestione ()**

Perché è imperativo per TCP limitare matematicamente la dimensione della finestra di trasmissione anziché iniettare i segmenti alla massima velocità dell'interfaccia locale? Se il trasmettitore impostasse una finestra eccessivamente sovradimensionata rispetto alla capacità del canale ($CWND \gg B \cdot D$), i pacchetti in eccesso non raggiungerebbero prima la destinazione, ma andrebbero ad accumularsi nei buffer di accodamento dei router intermedi lungo il percorso. Questo accumulo fa lievitare drasticamente il tempo di andata e ritorno ($RTT = RTT_{\text{prop}} + RTT_{\text{queue}}$): l'aumento dell'RTT rallenta la frequenza di arrivo degli ACK, prolungando paradossalmente il tempo totale necessario a completare il trasferimento! Mantenere la finestra tarata sulla banda disponibile permette ai pacchetti eccedenti di sostare comodamente nei buffer del sistema operativo dell'host sorgente (che dispone di gigabyte di RAM), proteggendo la memoria limitata dei commutatori di rete.

### 4.17.2 Stima Macroscopica del Throughput Medio a Regime AIMD

Consideriamo una sessione TCP Reno operante a regime stazionario, in assenza di timeout distruttivi e con perdite isolate recuperate con successo mediante Fast Retransmit / Fast Recovery.

La finestra di congestione oscilla periodicamente secondo il tipico andamento a dente di sega compreso tra un minimo pari a $\frac{W_{\max}}{2}$ (subito dopo il dimezzamento moltiplicativo) e il picco di saturazione $W_{\max}$ (raggiunto al termine della salita lineare in Congestion Avoidance). Il valore medio temporale della finestra coincide con il valor medio aritmetico degli estremi:

$$\bar{W} = \frac{\frac{W_{\max}}{2} + W_{\max}}{2} = \frac{3}{4}W_{\max} = 0.75 \cdot W_{\max} \tag{4.54}$$

Ne consegue che il throughput medio continuo scambiato dalla connessione è espresso da:

$$\bar{R}_{\text{TCP}} \approx \frac{0.75 \cdot W_{\max} \cdot \text{MSS}}{RTT} \tag{4.55}$$

### 4.17.3 La Relazione con la Probabilità di Perdita: La Formula di Mathis

Esiste una correlazione fisica fondamentale tra la probabilità di perdita di un pacchetto nella rete ($p$) e il throughput sostenibile da una connessione TCP.

In ciascun ciclo completo a dente di sega, la finestra sale linearmente da $W_{\max}/2$ a $W_{\max}$ compiendo un passo da 1 MSS per ogni $RTT$. Il numero di RTT necessari a percorrere il ciclo è:

$$N_{\text{RTT}} = W_{\max} - \frac{W_{\max}}{2} = \frac{W_{\max}}{2} \tag{4.56}$$

Il volume complessivo di pacchetti iniettati nella rete durante il ciclo corrisponde all'area del trapezio sotteso:

$$N_{\text{pacchetti}} = \sum_{k=0}^{W_{\max}/2} \left( \frac{W_{\max}}{2} + k \right) \approx \frac{\frac{W_{\max}}{2} + W_{\max}}{2} \cdot \frac{W_{\max}}{2} = \frac{3}{8} W_{\max}^2 \tag{4.57}$$

Poiché ciascun ciclo a dente di sega si chiude con l'occorrenza di esattamente una perdita isolata che innesca il dimezzamento, la frequenza statistica di perdita $p$ coincide con l'inverso del numero totale di pacchetti trasmessi nel ciclo:

$$p = \frac{1}{N_{\text{pacchetti}}} = \frac{8}{3W_{\max}^2} \implies W_{\max} = \sqrt{\frac{8}{3p}} \tag{4.58}$$

Sostituendo l'espressione di $W_{\max}$ nella formula del throughput medio, si ottiene la celebre **Formula Macroscopica di Mathis** (1997):

$$\text{Throughput} \approx \frac{\text{MSS}}{RTT} \cdot \frac{\sqrt{3/8} \cdot 1.5}{\sqrt{p}} \approx \frac{1.22 \cdot \text{MSS}}{RTT \cdot \sqrt{p}} \qquad [\text{Byte/s}] \tag{4.59}$$

**Principio / Fondamento: Implicazioni Fisiche della Formula di Mathis**

La formula di Mathis stabilisce due pilastri architetturali del trasporto su Internet:

1. Il throughput è **inversamente proporzionale all'RTT**: flussi con percorsi lunghi o ad alta latenza (es. tratte satellitari geostazionarie con $RTT \approx 500\,\text{ms}$) ottengono un throughput drasticamente inferiore rispetto a flussi locali a parità di banda del collegamento;
2. Il throughput è **inversamente proporzionale alla radice quadrata delle perdite** ($\sqrt{p}$): per raddoppiare il throughput su una tratta a latenza fissa, non basta dimezzare il tasso di perdita, ma occorre abbatterlo di ben un **fattore 4**!

### 4.17.4 L'Equità di Rete: La Dimostrazione Geometrica di Chiu-Jain

Un protocollo di trasporto si definisce **Equo** (*Fair*) se, quando $K$ connessioni indipendenti condividono un medesimo collegamento di strozzatura avente capacità $C$, ciascuna connessione converge asintoticamente a ricevere una quota paritaria di banda ():

$$R_i \approx \frac{C}{K} \tag{4.60}$$

Perché il controllo di congestione basato sulla legge **AIMD** garantisce la convergenza stabile all'equità, mentre alternative apparentemente ragionevoli (come l'incremento moltiplicativo o il decremento additivo) falliscono? La risposta formale venne dimostrata nel 1989 da Chiu e Jain mediante l'analisi del **Piano delle Finestre** (): Si considerino due flussi TCP concorrenti ($Flusso_1$ e $Flusso_2$) che condividono una linea a capacità $C$ e che possiedano il medesimo tempo di andata e ritorno ($RTT_1 = RTT_2 = RTT$). Lo stato del sistema in ogni istante è individuato dal punto cartesiano $(W_1, W_2)$:

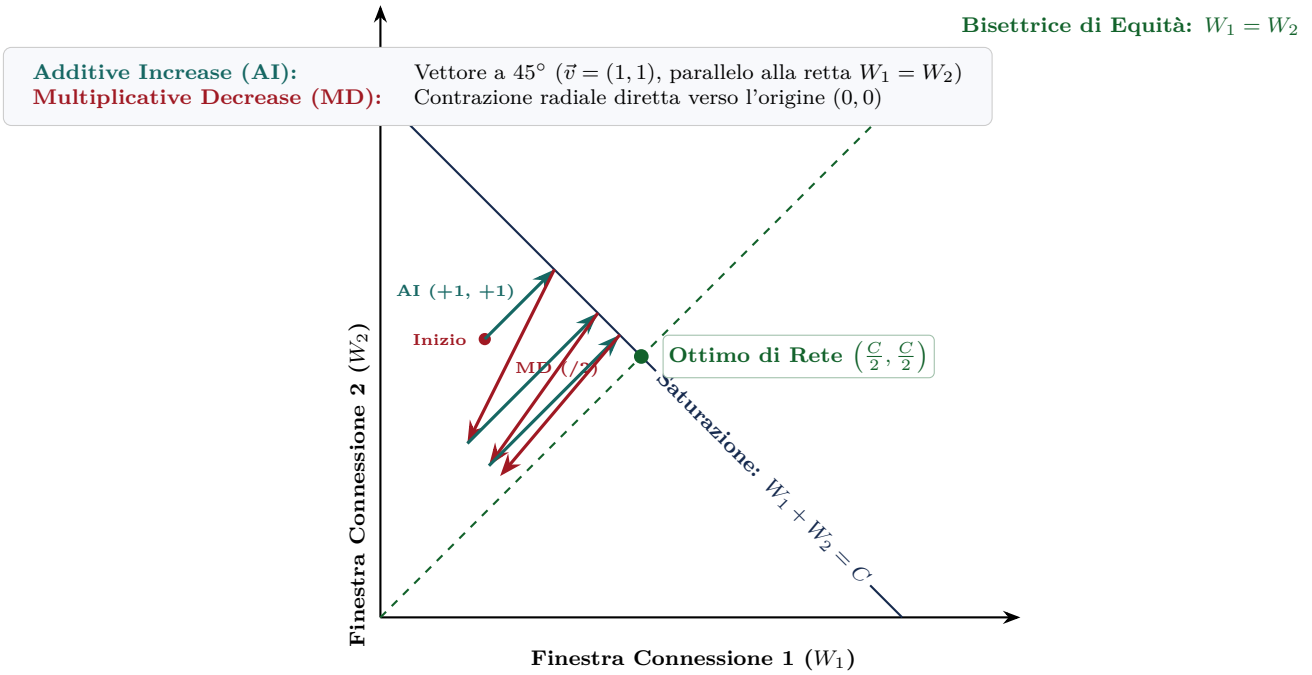



Figura 4.14: Dimostrazione geometrica di Chiu-Jain nel caso di RTT omogenei ($RTT_1 = RTT_2$): la combinazione di traslazione a 45° e contrazione radiale forza la convergenza stabile verso il punto di ottimo $\left(\frac{C}{2}, \frac{C}{2}\right)$.

**Principio / Fondamento: Meccanica Geometrica della Convergenza all'Equità**

L'analisi vettoriale nel piano delle finestre spiega la convergenza asintotica:

1. **Additive Increase (AI)**: entrambi i flussi aumentano di 1 MSS per RTT. Il vettore di incremento è $\vec{v}_{\text{AI}} = (1,1)$, con pendenza pari a 1 (angolo di 45°). Esso è **perfettamente parallelo alla bisettrice di equità** ($W_1 = W_2$). L'AI mantiene costante la differenza assoluta ($W_2 - W_1$), ma riduce progressivamente il rapporto relativo $\frac{W_2}{W_1}$;
2. **Multiplicative Decrease (MD)**: quando la somma supera la capacità ($W_1 + W_2 > C$), si verifica la perdita. Entrambi i flussi dimezzano la propria finestra. Il vettore di contrazione punta **esattamente verso l'origine** $(0,0)$. La contrazione preserva il rapporto $\frac{W_2}{W_1}$, ma dimezza la distanza euclidea dalla bisettrice!
3. **Convergenza a Spirale**: l'interazione tra la salita a 45° e il puntamento all'origine costringe il sistema in un'orbita che si stringe geometricamente attorno alla retta $W_1 = W_2$, stabilizzandosi nell'intorno dell'ottimo $\left(\frac{C}{2}, \frac{C}{2}\right)$.

### 4.17.5 L'Iniquità Strutturale di TCP con RTT Eterogenei ()

La dimostrazione classica di Chiu-Jain presuppone che entrambi i flussi abbiano il medesimo RTT. Cosa accade nella realtà quando due connessioni concorrenti presentano tempi di propagazione differenti?

Si consideri il caso formale discusso a lezione () in cui due connessioni condividono la medesima strozzatura ma con:

$$RTT_2 = 2 \cdot RTT_1 \tag{4.61}$$

Poiché la regola di Congestion Avoidance incrementa la finestra di +1 MSS ad ogni RTT del rispettivo flusso, la velocità temporale di incremento della finestra risulta:

$$\frac{dW_i}{dt} \approx \frac{\text{MSS}}{RTT_i} \tag{4.62}$$

In un intervallo di tempo pari a $\Delta t = RTT_2$, la connessione 1 compie **due passi completi di incremento additivo** ($\Delta W_1 = +2$ MSS) mentre la connessione 2 ne compie **uno solo** ($\Delta W_2 = +1$ MSS)!

Il vettore di incremento temporale nel piano delle finestre non è più parallelo alla bisettrice di equità a 45°, bensì ha pendenza:

$$\vec{v}_{\text{AI}} = (2,1) \implies \text{pendenza} = \frac{\Delta W_2}{\Delta W_1} = \frac{1}{2} \quad (\theta \approx 26.5°) \tag{4.63}$$

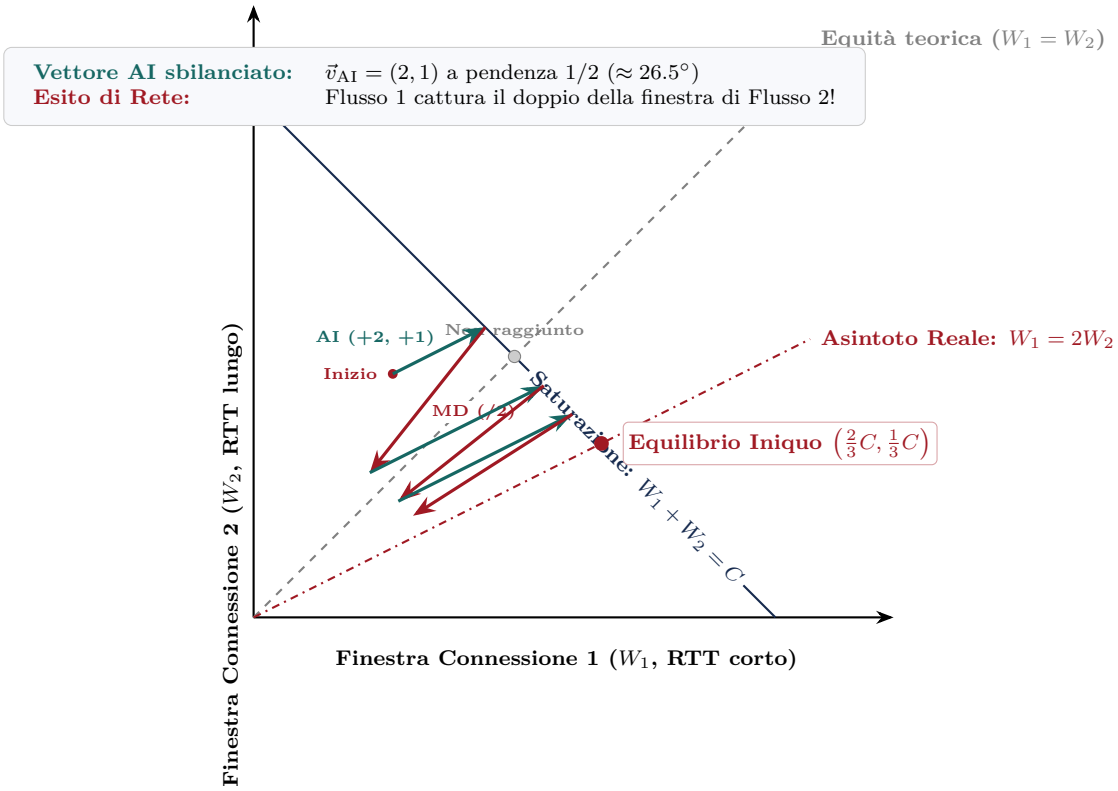



Figura 4.15: Dimostrazione geometrica di Chiu-Jain nel caso di RTT eterogenei ($RTT_2 = 2 \cdot RTT_1$, ): la pendenza dell'Additive Increase si appiattisce a 1/2, deviando stabilmente la convergenza verso un equilibrio iniquo a favore della connessione con latenza inferiore.

### L'Iniquità Quadratica nel Throughput

Come illustrato nella Figura 4.15, all'equilibrio le finestre di saturazione rispettano:

$$\frac{W_1}{W_2} \approx \frac{RTT_2}{RTT_1} = 2 \implies W_1 = \frac{2}{3}C, \quad W_2 = \frac{1}{3}C \tag{4.64}$$

Se ora calcoliamo il throughput effettivo conseguito dai due flussi, l'iniquità non è lineare, ma si amplifica con **legge quadratica**:

$$\frac{R_1}{R_2} = \frac{\frac{W_1 \cdot \text{MSS}}{RTT_1}}{\frac{W_2 \cdot \text{MSS}}{RTT_2}} = \left(\frac{W_1}{W_2}\right) \cdot \left(\frac{RTT_2}{RTT_1}\right) = 2 \cdot 2 = 4 \tag{4.65}$$

La connessione locale (con $RTT$ dimezzato) ottiene un throughput **quattro volte superiore** rispetto alla connessione remota! Questo evidenzia il più grande limite architetturale di TCP Reno nelle reti geografiche estese: la contesa della banda non riflette il fabbisogno applicativo, ma punisce sistematicamente gli host fisicamente più lontani.

# Capitolo 5

# Livello di Applicazione (*Application Layer*)

Il **livello di applicazione** (*Application Layer*, Livello 5 nell'architettura TCP/IP e vertice del modello di riferimento ISO/OSI) rappresenta il punto di incontro immediato tra la rete di telecomunicazioni e l'utente finale. Mentre i livelli inferiori (fisico, collegamento, rete e trasporto) operano congiuntamente per predisporre un canale di comunicazione logico, trasparente e privo di errori tra entità remote, il livello applicativo contiene tutti i protocolli e i programmi che realizzano i servizi concreti richiesti dagli utenti: navigazione Web, risoluzione dei nomi di dominio, recapito della posta elettronica, trasferimento file e flussi multimediali in tempo reale.

Nello stack TCP/IP, il livello applicativo ingloba e fonde anche i compiti che il modello formale ISO/OSI attribuiva separatamente ai livelli di *Presentazione* (codifica e formattazione dei dati) e di *Sessione* (sincronizzazione e delimitazione del dialogo), lasciando agli sviluppatori piena libertà progettuale e standardizzando l'accesso alla rete per mezzo delle **Socket API**.

## 5.1 Principi delle Applicazioni di Rete e Paradigmi Architetturali

### 5.1.1 Ruolo e Funzione dei Protocolli Applicativi

È imperativo operare una distinzione concettuale netta tra un'**applicazione** (o servizio) e il **protocollo a livello applicativo**:

- L'**applicazione di rete** è il software complessivo che gira sui sistemi terminali (*end systems*) e che l'utente percepisce e interagisce direttamente (ad esempio un browser Web come Google Chrome, un server Web come Apache o Nginx, un client di posta come Mozilla Thunderbird).
- Il **protocollo a livello applicativo** è la specifica formale e standardizzata (definita tipicamente tramite RFC da IETF) che governa lo scambio dei messaggi tra i processi applicativi: ne prescrive la sintassi dei campi, la semantica dei comandi, le regole di temporizzazione e la codifica dei dati. Per esempio, `HTTP` è il protocollo, non il browser; `SMTP` è il protocollo, non la casella di posta.

Un principio cardine dell'architettura Internet risiede nel fatto che **il software applicativo risiede e gira esclusivamente nei sistemi terminali (*hosts*)**:

> **Principio / Fondamento: Principio End-to-End nello Sviluppo Applicativo**
>
> I router e gli switch intermedi della rete a commutazione di pacchetto non eseguono codice di livello applicativo e non hanno visibilità del contenuto dei messaggi scambiati. Tale disaccoppiamento architetturale garantisce un'elevata velocità di innovazione: chiunque può concepire, sviluppare e distribuire una nuova applicazione di rete globale senza dover richiedere modifiche o autorizzazioni agli operatori dell'infrastruttura di telecomunicazioni.

### 5.1.2 Evoluzione del Traffico e Dinamiche di Rete

La natura del traffico circolante su Internet ha subito profonde mutazioni storiche:

- Nelle prime decadi della rete, il traffico era dominato da applicazioni testuali a basso bit-rate quali `Telnet` (emulazione terminale remoto) e `FTP` (trasferimento file).
- A partire dalla metà degli anni '90, l'avvento del World Wide Web (`HTTP`) ha polarizzato il traffico, portando a una crescita esplosiva.
- Nei decenni successivi si sono succedute ondate guidate dal traffico Peer-to-Peer (`BitTorrent`), dalle piattaforme social e, in misura preponderante, dallo streaming video continuo ad alta definizione (YouTube, Netflix) e dalle comunicazioni interattive in tempo reale.

Il volume complessivo del traffico di rete è cresciuto storicamente raddoppiando a intervalli di circa $9\nabla\cdot11$ mesi (un ritmo persino superiore alla celebre legge di Moore per i semiconduttori). Se la capacità trasmissiva delle

fibre ottiche è riuscita a reggere tale espansione grazie all'innovazione optoelettronica e al silicio, i nodi di commutazione interni affrontano sfide costanti di scalabilità.

### 5.1.3 Tassonomia delle Architetture Applicative

Le applicazioni di rete si strutturano secondo quattro paradigmi architetturali fondamentali:

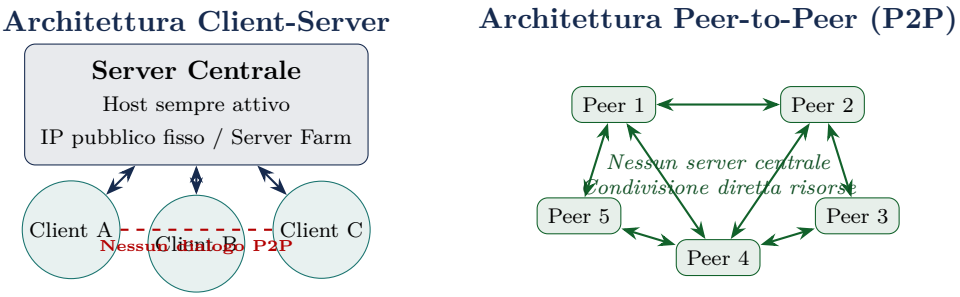



Figura 5.1: Confronto topologico tra il modello Client-Server centralizzato e il modello Peer-to-Peer distribuito.

#### 1. Architettura Client-Server

Costituisce il modello dominante sul Web e nella maggioranza dei servizi tradizionali:

- **Server**: è un host perennemente attivo ed esposto sulla rete, dotato di un indirizzo IP fisso, pubblico e noto a priori. Per sopportare carichi elevatissimi e prevenire colli di bottiglia, i server non sono singole macchine fisiche ma *Server Farm* o cluster virtualizzati a cui corrisponde un singolo nome logico.
- **Client**: sistemi terminali che avviano l'interazione inoltrando richieste al server. Possono collegarsi a intermittenza, disporre di indirizzi IP dinamici e mutevoli nel tempo assegnati da DHCP, e **non comunicano mai direttamente tra di loro**.
- **Limiti**: il server costituisce un potenziale punto unico di congestione o di guasto (*single point of failure*). La capacità di servizio è intrinsecamente asimmetrica: al crescere degli utenti, il carico sul server aumenta linearmente, richiedendo ingenti investimenti infrastrutturali.

#### 2. Architettura Peer-to-Peer (P2P) Pura

Nel modello P2P puro (esempi storici: `Gnutella`, `BitTorrent`):

- Non esiste un server centrale sempre attivo né un'infrastruttura fissa dedicata.
- Coppie arbitrarie di host connessi alla periferia della rete (**peer**) comunicano direttamente tra loro per scambiarsi dati e porzioni di file.
- I nodi sono intermittenti (*churn*): entrano ed escono dalla rete liberamente cambiando indirizzo IP.
- **Autoscalabilità**: ogni nuovo nodo che si collega non introduce solo domanda di traffico (download), ma apporta contemporaneamente capacità di servizio e banda trasmissiva a beneficio degli altri partecipanti (upload).
- **Criticità**: se da un lato offre elevata resilienza e tutela della riservatezza contro censure centralizzate, dall'altro risulta estremamente complessa da gestire (problema della ricerca distribuita delle risorse o *lookup*) e ha sollevato storicamente gravi problematiche legali legate alla violazione del diritto d'autore (*copyright*).

#### 3. Architetture Ibride (Mix Client-Server e P2P)

Molti servizi di successo combinano la robustezza del client-server con l'efficienza del P2P:

- Una componente centrale (erogata tramite web service / server centralizzati) cura la localizzazione degli utenti, la registrazione dello stato, l'autenticazione e la scoperta dei servizi (**Service Discovery**).
- La componente ad alto volume di traffico (ad esempio i flussi audio/video o il trasferimento di flussi multimediali tra interlocutori) avviene direttamente **peer-to-peer** tra i terminali, una volta messi in contatto dal server centrale.
- È il modello tipico delle moderne piattaforme di videoconferenza (Google Meet, Microsoft Teams, Zoom, Jitsi) e delle soluzioni basate su `WebRTC`. Questo approccio risolve elegantemente la scoperta del servizio, ma consente al fornitore centrale di raccogliere metadati sulle comunicazioni.

#### 4. Modello Cloud, Fog ed Edge Computing

Nelle applicazioni contemporanee, il paradigma client-server si è trasformato radicalmente con il modello **Cloud**:

- Il server non è più identificabile con una macchina fisica locale, bensì con un'infrastruttura astratta di calcolo, archiviazione e virtualizzazione distribuita su scala planetaria.
- L'utente fruisce del servizio interamente online, con backup e sincronizzazione trasparente.

**Cloud, Piattaforme e Normativa GDPR**

Un profilo di rilievo riguarda le implicazioni giuridiche e di sovranità del modello a piattaforme Cloud:

> *«Nei modelli dominanti attuali, l'utente ha barattato la propria sovranità con la gratuità del servizio. La posta e i documenti risiedono interamente su server farm estere di colossi multinazionali. A differenza della posta tradizionale o dei sistemi gestiti in locale, dove il messaggio recapitato diventa proprietà esclusiva del destinatario e la violazione della corrispondenza costituisce reato penale, nel cloud i dati restano a totale appannaggio della piattaforma. Tale centralizzazione estera si pone sovente in netto e insanabile contrasto con la normativa europea di protezione dei dati personali (**GDPR**), la quale qualifica la riservatezza e la titolarità del dato come diritti individuali inalienabili.»*

### 5.1.4 Processi Comunicanti e Socket

Un programma in esecuzione su un sistema terminale costituisce un **processo**. All'interno del medesimo sistema operativo, due processi comunicano tramite primitive di comunicazione interprocesso (IPC, memorie condivise, pipe). Quando i processi risiedono su host differenti e interconnessi dalla rete, essi comunicano scambiandosi messaggi attraverso la rete.

In ogni interazione applicativa distinguiamo:

- **Processo Client**: l'istanza che avvia l'interazione inviando la prima richiesta.
- **Processo Server**: l'istanza passiva che si pone in ascolto su una porta nota, in attesa di essere contattata.

Nelle architetture P2P, un nodo agisce contemporaneamente da client (quando richiede blocchi ad altri peer) e da server (quando invia i blocchi memorizzati a chi li richiede).

**Definizione: Socket**

La **Socket** è l'interfaccia programmabile (API) del sistema operativo che funge da punto di accesso al servizio di trasporto (*Service Access Point*, SAP). Essa rappresenta la metaforica "porta" attraverso cui il processo applicativo immette i messaggi destinati alla rete e da cui preleva i messaggi in ingresso inoltrati dallo stack protocollare sottostante.

Per recapitare un messaggio a un processo specifico, l'infrastruttura di rete necessita di due coordinate identificative:

1. L'**Indirizzo IP** dell'host destinatario (operante a Livello 3 per individuare l'interfaccia di rete);
2. Il **Numero di Porta** (operante a Livello 4 per individuare il socket/processo di destinazione all'interno dell'host). Ad esempio, la porta 80 identifica convenzionalmente il server Web HTTP, la porta 25 il server SMTP, la porta 53 il server DNS.

### 5.1.5 Requisiti di Servizio Richiesti al Livello di Trasporto

Applicazioni differenti presentano requisiti eterogenei nei confronti della rete:

- **Tolleranza alla perdita di dati**: applicazioni come il trasferimento di file (`FTP`), la posta (`SMTP`) o il web (`HTTP`) non tollerano alcuna perdita di byte (richiedono affidabilità al 100%, delegata a `TCP`); applicazioni multimediali di streaming o conversazione real-time possono tollerare modeste frazioni di pacchetti persi senza pregiudicare la fruizione.
- **Sensibilità al tempo (Latenza e Jitter)**: la telefonia via Internet (VoIP) e i videogiochi interattivi esigono ritardi inferiori a poche centinaia di millisecondi ($< 150$–$200$ ms) per risultare efficaci e realistici, mentre il download di pagine o file è insensibile al ritardo puntuale (applicazioni elastiche).
- **Throughput**: lo streaming video necessita di una banda garantita minima (da centinaia di kbps a svariati Mbps per video 4K), mentre le applicazioni elastiche saturano dinamicamente la capacità disponibile offerta dalla rete.
- **Sicurezza**: cifratura a tutela della riservatezza, integrità crittografica e autenticazione mutua (realizzati tipicamente tramite TLS sopra TCP).

| Applicazione | Tolleranza Perdite | Sensibilità ai Ritardi | Throughput Richiesto |
|---|---|---|---|
| Trasferimento File | No (affidabilità 100%) | No | Elastico / Variabile |
| Posta Elettronica | No (affidabilità 100%) | No | Elastico / Variabile |
| Navigazione Web | No (affidabilità 100%) | Modesta (pochi secondi) | Elastico / Variabile |
| Audio/Video Real-Time | Sì (tolleranza perdite) | Sì (stringente, $\leq 150\,\text{ms}$) | Costante / Minimo garantito |
| Audio/Video a Richiesta | Modesta tolleranza | Moderata (pochi secondi) | Sostenuto ($2\nabla \cdot 25\,\text{Mbps}$) |
| Giochi Interattivi | Minima | Molto alta ($< 100\,\text{ms}$) | Ridotto (pochi kbps) |
| Messaggistica | No | Bassa / Moderata | Molto ridotto |

Tabella 5.1: Requisiti di trasporto delle principali classi di applicazioni di rete.

## 5.2 Il Sistema dei Nomi di Dominio (DNS – Domain Name System)

### 5.2.1 Motivazioni e Definizione del Servizio

Gli apparati di rete a livello 3 identificano le interfacce per mezzo di indirizzi numerici a lunghezza fissa (gli indirizzi `IPv4` a 32 bit e `IPv6` a 128 bit), strutture ideali per l'elaborazione hardware dei router nelle tabelle di routing. Gli esseri umani, tuttavia, memorizzano con difficoltà sequenze numeriche binarie o decimali puntate e prediligono stringhe alfanumeriche mnemoniche dotate di significato gerarchico (es. `www.unibs.it`).

**Definizione: DNS – Domain Name System**

Il **DNS** è un servizio fondamentale dell'architettura Internet preposto alla mappatura bidirezionale tra identificatori mnemonici (**nomi a dominio** o *hostname*) e indirizzi binari (**indirizzi IP**). Architetturalmente si compone di:

1. Un **database distribuito e gerarchico** implementato su una molteplicità di server DNS dislocati a livello planetario;
2. Un **protocollo a livello applicativo** (operante di norma su `UDP` alla porta destinazione 53) che consente agli host e ai server di interrogarsi reciprocamente per risolvere i nomi.

Oltre alla risoluzione canonica, il DNS eroga funzioni ausiliarie indispensabili:

- **Host Aliasing**: associazione di uno o più nomi alias a un nome canonico (ufficiale). Ad esempio, l'host con nome canonico complesso `servereast.backup2.ibm.com` può essere reso accessibile all'utenza tramite il semplice alias `www.ibm.com`.
- **Mail Server Aliasing**: identificazione del server di posta elettronica incaricato di ricevere messaggi per uno specifico dominio (es. recapitare la posta per l'indirizzo `utente@unibs.it` all'effettivo server `mail.unibs.it`).
- **Bilanciamento del Carico (*Load Distribution*)**: per i grandi siti web replicati su scala globale, a un unico nome a dominio corrisponde un insieme di molteplici indirizzi IP appartenenti a server differenti. Il server DNS ruota l'ordine degli indirizzi forniti nelle risposte (*DNS round-robin*), distribuendo le connessioni dei client tra le varie macchine della server farm.

### 5.2.2 Perché il DNS non è Centralizzato?

La centralizzazione del database DNS in un singolo supercomputer mondiale è stata scartata a priori dai progettisti della rete per le seguenti ragioni critiche:

- **Punto Unico di Guasto (*Single Point of Failure*)**: il blocco del server centrale paralizzerebbe l'intera Internet globale;
- **Volume di Traffico Incontrollabile**: miliardi di query al secondo convergerebbero su un unico apparato o fornitore;
- **Distanza Geografica e Latenza**: client situati agli antipodi del pianeta subirebbero ritardi di propagazione inaccettabili per ogni singola transazione web;
- **Impossibilità di Scalabilità e Gestione**: nessun singolo database centralizzato può gestire aggiornamenti continui e concorrenti da ogni registrar mondiale.

### 5.2.3 Nomi di Dominio: FQDN e PQDN

I nomi a dominio presentano una struttura gerarchica ad albero delimitata da punti (`.`):

- **FQDN (*Fully Qualified Domain Name*)**: nome completo che individua in modo univoco e non ambiguo uno specifico host o server farm nella gerarchia globale, ad esempio:

  `web.unibs.it`

  Come rimarcato nelle lezioni, **l'FQDN si legge da sinistra verso destra, ma la gerarchia logica si sviluppa strettamente da destra verso sinistra**:
  - `.it` rappresenta il *Top-Level Domain* (TLD nazionale);
  - `unibs.it` rappresenta il dominio locale accreditato;
  - `web.unibs.it` rappresenta l'host specifico (la server farm interna).
- **PQDN (*Partially Qualified Domain Name*)**: una porzione parziale di un nome di dominio, oppure un nome alias generico che non corrisponde direttamente all'host di destinazione finale. Ad esempio, il docente ha spiegato che `unibs.it` identifica un dominio ma non ha un singolo indirizzo IP associato; il portale accademico `www.unibs.it` è in realtà un alias (PQDN) che i server DNS ridirezionano all'FQDN operativo `portalidrupal-prod-1.cineca.it` attestato sui sistemi del consorzio CINECA.

### 5.2.4 Gerarchia dei Server DNS

La base dati globale è distribuita su quattro categorie strutturali di server:

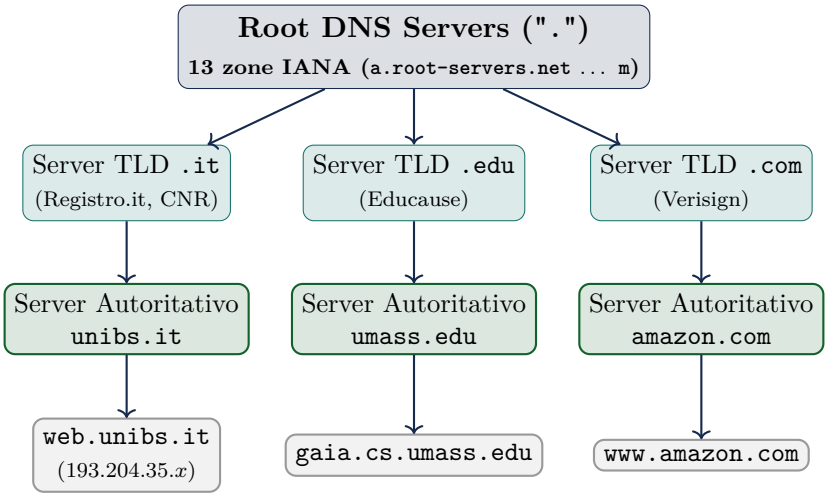



Figura 5.2: Struttura ad albero gerarchica del database globale del Domain Name System.

1. **Root DNS Servers (Server Radice)**: costituiscono il vertice dell'albero. Esistono storicamente **13 indirizzi logici** radice designati dalle lettere da `a` a `m` (es. `a.root-servers.net`), coordinati dall'ente IANA/ICANN e gestiti da 12 operatori indipendenti (tra cui Verisign, NASA, US DoD, RIPE NCC, WIDE). Ciascun indirizzo logico è in realtà replicato in centinaia di server fisici tramite routing Anycast. I root server forniscono i puntatori ai server competenti per i TLD.
2. **TLD Servers (Top-Level Domain)**: gestiscono i domini generici di primo livello (`.com`, `.org`, `.net`, `.edu`) e i domini geografici nazionali (*ccTLD*: `.it`, `.fr`, `.uk`, `.jp`). Per esempio, la società Verisign gestisce i server per i domini `.com`, mentre il consorzio Educause gestisce il TLD `.edu`.
3. **Authoritative DNS Servers (Server di Competenza o Autoritativi)**: ogni organizzazione o istituzione che mantiene host accessibili pubblicamente su Internet (es. `unibs.it`) è tenuta a fornire server autoritativi che contengono le mappature ufficiali e definitive tra i propri nomi host e gli indirizzi IP assegnati.
4. **Local DNS Server (Server DNS Locale o Default Nameserver)**: non appartiene formalmente all'albero di deleghe della gerarchia. È il server assegnato all'host richiedente (tramite DHCP o configurazione manuale) dal proprio ISP o amministratore di rete aziendale/universitario. Esso agisce da intermediario (**proxy resolver**): intercetta le query DNS generate dai client locali e si incarica di navigare la gerarchia globale per trovare la risposta.

### 5.2.5 Modalità di Risoluzione: Query Iterative vs. Ricorsive

La traduzione di un nome host in indirizzo IP può svolgersi secondo due modelli algoritmici:

- **Query Ricorsiva**: il client delega totalmente l'onere della ricerca al server DNS contattato. Tale server, se non dispone della risposta, si impegna a contattare personalmente i livelli successivi della gerarchia, attendere i risultati e restituire all'host originario la risposta finale consolidata (logica del: *«Io non conosco questo nome, ma mi incarico di procurartelo»*).
- **Query Iterativa**: il server contattato risponde indicando al richiedente l'indirizzo del server successivo da interrogare nella catena gerarchica (logica del: *«Io non conosco questo nome, ma puoi chiederlo a questo altro server che ti indico»*). L'onere di inviare le chiamate successive resta in capo al nodo richiedente.

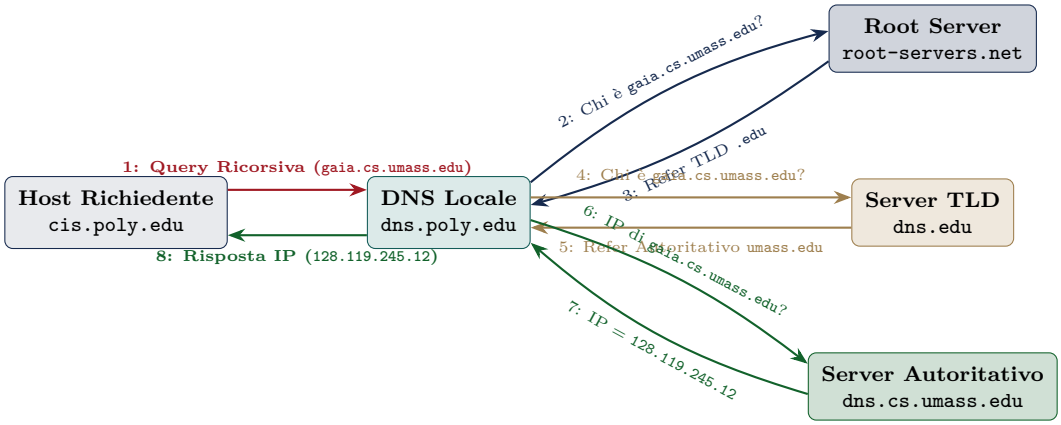



Figura 5.3: Risoluzione standard DNS: query ricorsiva tra host e server locale, e query iterative tra server locale e gerarchia mondiale.

**Principio / Fondamento: Prassi Operativa della Risoluzione DNS**

Nella pratica ingegneristica di Internet:

1. La richiesta del sistema terminale verso il proprio **Server DNS Locale è tipicamente ricorsiva**: l'applicazione affida interamente la risoluzione al resolver di sistema.
2. Le interrogazioni inoltrate dal Server DNS Locale verso i Root server e i TLD server **sono rigorosamente iterative**. I server radice e di TLD rifiutano tassativamente di gestire sessioni ricorsive, poiché mantenere lo stato e attendere le risposte di terzi comporterebbe un overhead di memoria e computazione capace di collassare le dorsali mondiali.

### 5.2.6 Caching e Aggiornamento dei Record

Per ridurre drasticamente il traffico verso la gerarchia e tagliare i tempi di risposta, il DNS si fonda capillarmente sul meccanismo di **Caching**:

- Non appena un Server DNS Locale apprende l'associazione tra un nome a dominio e il rispettivo indirizzo IP (o apprende l'indirizzo IP dei server TLD di competenza), la memorizza all'interno della propria memoria cache locale.
- Le query successive generate da qualsiasi host attestato sulla medesima sottorete ottengono risposta immediata dal server locale, senza compiere alcuna navigazione della gerarchia esterna.
- Poiché i server locali tengono in cache gli indirizzi IP dei server TLD, i **Root DNS Server vengono consultati solo in una frazione infinitesima delle richieste globali**.
- Ogni record inserito in cache è vincolato a un parametro temporale denominato **TTL** (*Time-To-Live*): scaduto il TTL, la voce viene invalidata e scartata, garantendo che eventuali cambi di indirizzo decisi dai server autoritativi vengano recepiti dalla rete.

### 5.2.7 Record di Risorsa DNS (Resource Record – RR)

Il database DNS memorizza le informazioni sotto forma di tuple strutturate a 4 campi denominate **Resource Record** (RR):

$$\text{RR} = (\text{Name}, \text{Value}, \text{Type}, \text{TTL}) \tag{5.1}$$

Il significato dei campi `Name` e `Value` dipende in modo determinante dal tipo (`Type`):

| Type | Significato Name | Significato Value | Descrizione e Casi d'Uso |
|---|---|---|---|
| `A` | Nome dell'host (*Hostname*) | Indirizzo IPv4 (32 bit) | Risoluzione base IPv4 standard (es. `web.unibs.it` → `193.204.35.80`). |
| `AAAA` | Nome dell'host (*Hostname*) | Indirizzo IPv6 (128 bit) | Risoluzione standard per il protocollo IPv6. |
| `NS` | Nome del dominio | Hostname del server autoritativo | Indica quale server è di competenza per il dominio specificato (es. `unibs.it` → `dns.unibs.it`). |
| `CNAME` | Nome alias | Nome canonico (vero hostname) | Consente l'aliasing di un host (es. `www.ibm.com` → `servereast.backup2.ibm.com`). |
| `MX` | Nome del dominio | Nome del Mail Server | Specifica l'hostname del server preposto alla posta del dominio (es. `unibs.it` → `mail.unibs.it`). |

Tabella 5.2: Principali tipologie di Record di Risorsa (RR) nel database DNS.

### 5.2.8 Formato dei Messaggi DNS

Le query e i messaggi di risposta DNS condividono la medesima struttura di pacchetto a 12 byte di intestazione fissa seguita da 4 sezioni a lunghezza variabile:

- **Identificazione (16 bit)**: numero progressivo generato dal client all'atto della domanda. La risposta proveniente dal server riporterà il medesimo identificativo a 16 bit, consentendo al client di abbinare univocamente le risposte alle rispettive interrogazioni asincrone trasmesse su UDP.
- **Flag di Controllo (16 bit)**:
  - Bit *Domanda/Risposta* (0 = query, 1 = reply);
  - Bit *Risposta di Competenza (Authoritative)*: asserito a 1 se il server che risponde è autoritativo per il nome;
  - Bit *Richiesta di Ricorsione*: impostato dal client per richiedere che il server operi in modalità ricorsiva;
  - Bit *Ricorsione Disponibile*: asserito dal server per notificare che supporta la modalità ricorsiva.
- **Sezioni Dati**: contengono il numero e il corpo delle Domande (*Questions*), Risposte (*Answers*), Record di Autorità (*Authority*) e Record Aggiuntivi (*Additional Information*).

## 5.3 Il Web e il Protocollo HTTP

### 5.3.1 Distinzione Fondamentale tra HTML e HTTP

Come sottolineato con vigore dal prof. Lo Cigno nelle lezioni dedicate (2 Dicembre), la comprensione del Web esige l'assoluta non-confusione tra i due pilastri fondanti:

- **HTML (*HyperText Markup Language*)**: è un linguaggio dichiarativo di formattazione e marcatura ipertestuale che descrive la struttura, l'aspetto visivo e i collegamenti di un documento web.
- **HTTP (*HyperText Transfer Protocol*)**: è il protocollo di livello applicativo che definisce come tali oggetti vengono trasportati, negoziati e scambiati attraverso la rete tra processi client e server.

Una **pagina web** si compone di molteplici **oggetti**. Di base, vi è un file HTML iniziale (denominato convenzionalmente `index.html` o file base) che contiene il testo e i riferimenti ipertestuali a diversi altri oggetti ausiliari (immagini JPEG, fogli di stile CSS, file JavaScript, applet). Ciascun oggetto è referenziato e localizzato in rete tramite un identificatore universale:

**Definizione: URL e URI**

L'**URL** (*Uniform Resource Locator*) identifica la collocazione fisica e la modalità di accesso a un oggetto presente su un server web:

$$\underbrace{\texttt{http://}}_{\text{Protocollo}}\ \underbrace{\texttt{www.someschool.edu}}_{\text{Nome Host (risolto dal DNS)}}\ \underbrace{\texttt{/someDept/pic.gif}}_{\text{Percorso file sul server}}$$

L'**URI** (*Uniform Resource Identifier*) è una nozione più ampia che identifica in modo univoco una risorsa logica (ad esempio un indirizzo SIP: `sip:locigno@unibs.it`).

### 5.3.2 Modello di Servizio e Assenza di Stato (*Statelessness*)

HTTP opera secondo il paradigma **client-server**:

- Il **Client** (il browser web, es. Chrome, Firefox, Safari) avvia la connessione, inoltra richieste e visualizza gli oggetti all'utente.
- Il **Server** (es. Apache, Nginx, Tomcat) resta in ascolto passivo sulla porta convenzionale **TCP 80** (o porta 443 per `HTTPS`), accetta connessioni ed eroga gli oggetti richiesti.

> **Principio / Fondamento: Natura Priva di Stato (*Stateless*) di HTTP**
>
> HTTP è un protocollo rigorosamente **senza stato** (*stateless*): il server web non memorizza alcuna informazione storica in merito alle richieste soddisfatte in precedenza verso lo stesso client. Se un medesimo host richiede il medesimo file venti volte in un minuto, il server tratta ogni invocazione come un evento atomico del tutto inedito, reinviando l'oggetto senza tenere traccia della sessione pregressa.

I protocolli provvisti di stato (*stateful*) presentano una complessità esponenzialmente maggiore: devono allocare registri storici, monitorare le disconnessioni e riconciliare lo stato in caso di crash improvviso del server o del client. L'assenza di stato ha consentito a HTTP di scalare a livello planetario garantendo semplicità e robustezza.

### 5.3.3 Connessioni Non Persistenti vs. Connessioni Persistenti

Uno dei profili più rilevanti per le prestazioni e la latenza del Web riguarda la gestione delle connessioni `TCP` sottostanti.

#### 1. Connessioni Non Persistenti (HTTP/1.0)

In `HTTP/1.0`, **ciascuna connessione TCP serve a trasferire un unico oggetto web**, dopodiché la connessione viene chiusa dal server:

1. Il client avvia l'handshake a 3 vie TCP con il server sulla porta 80.
2. Il client invia il messaggio di richiesta HTTP per il file base HTML.
3. Il server estrae l'oggetto, compone la risposta, la trasmette al client e chiude la connessione.
4. Il client analizza il file HTML e individua, ad esempio, 10 immagini JPEG referenziate.
5. I passi da 1 a 4 devono essere **ripetuti integralmente per ciascuno dei 10 oggetti**!

**Calcolo del Tempo di Risposta per le Connessioni Non Persistenti:** Per trasferire un singolo oggetto, il tempo complessivo di andata e ritorno è dominato dall'**RTT** (*Round-Trip Time*):

- 1 RTT per completare l'handshake iniziale di TCP (invio SYN e ricezione SYN-ACK);
- 1 RTT per inoltrare la richiesta HTTP e ricevere i primi byte della risposta;
- $T_{\text{tx}}$, tempo materiale di trasmissione dell'oggetto sul canale.

Il ritardo per singolo oggetto vale dunque:

$$T_{\text{oggetto}} = 2 \cdot \text{RTT} + T_{\text{tx}} \tag{5.2}$$

Se la pagina web è formata da un file HTML e da $N$ oggetti referenziati, con connessioni sequenziali non persistenti il tempo complessivo risulta:

$$T_{\text{totale, non persistente}} = (N + 1) \cdot (2 \cdot \text{RTT} + T_{\text{tx}}) \tag{5.3}$$

Tale approccio comporta un overhead gravoso per il sistema operativo (creazione e distruzione continua di socket) e penalizza gravemente le prestazioni a causa della fase di *Slow-Start* di TCP, che riparte da una finestra di congestione minima ad ogni singola immagine.

#### 2. Connessioni Persistenti (HTTP/1.1)

Per ovviare all'inefficienza delle connessioni non persistenti, `HTTP/1.1` introduce le **connessioni persistenti**:

- Il server **mantiene aperta la connessione TCP** dopo aver erogato la risposta al file base.
- I successivi oggetti referenziati appartenenti allo stesso host possono essere richiesti e trasferiti lungo il medesimo canale già aperto e stabilizzato.

- **Senza Pipelining**: il client richiede un nuovo oggetto solo dopo aver ricevuto la risposta dell'oggetto precedente ($1\,\mathrm{RTT} + T_{\mathrm{tx}}$ per ciascun oggetto aggiuntivo).
- **Con Pipelining**: non appena il client analizza il file HTML, invia **tutte le richieste degli oggetti referenziati in sequenza continua**, senza attendere le risposte intermedie. Il server accoda le richieste e risponde trasmettendo tutti gli oggetti in successione.

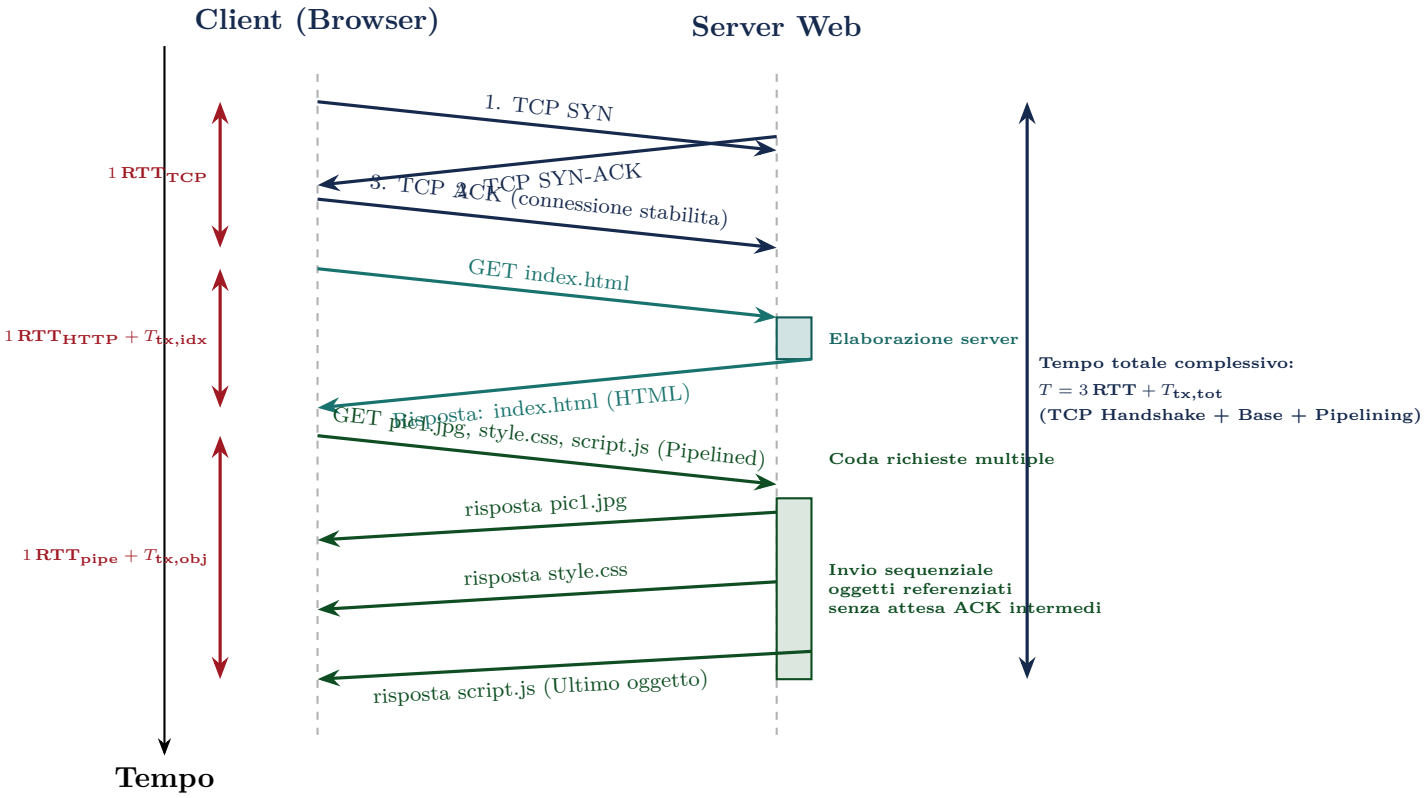



Figura 5.4: Diagramma temporale di HTTP/1.1 con connessione persistente, handshake TCP completo e pipelining delle richieste per gli oggetti referenziati.

**Calcolo del Tempo di Risposta in Pipelining:** Il tempo di completamento per l'intera pagina web con pipelining richiede unicamente:

$$T_{\text{totale, pipelining}} = 3 \cdot \mathrm{RTT} + T_{\mathrm{tx}}(\text{index}) + \sum_{i=1}^{N} T_{\mathrm{tx}}(\text{oggetto}_i) \tag{5.4}$$

ossia: 1 RTT per la connessione TCP, 1 RTT per richiedere e ricevere la pagina base, e solo 1 RTT cumulativo per inviare tutte le altre richieste in cascata e ricevere i primi byte del primo oggetto, seguito dal puro tempo di trasmissione.

### 5.3.4 Formato dei Messaggi HTTP

I messaggi HTTP sono interamente composti da testo in codifica **ASCII**, suddivisi in righe terminate dalla sequenza `CRLF` (*Carriage Return* `\r`, *Line Feed* `\n`).

#### 1. Messaggio di Richiesta

Si articola in tre componenti:

1. **Riga di Richiesta**: contiene il **Metodo**, l'**URL della risorsa** e la **Versione del protocollo**.
2. **Righe di Intestazione (*Header Lines*)**: coppie `Nome-Campo: Valore`.
3. **Riga Vuota (CRLF)**: sentinella tassativa che segnala al parser la terminazione delle intestazioni.
4. **Corpo del Messaggio (*Entity Body / Payload*)**: opzionale, impiegato per trasferire dati inseriti dall'utente (es. form HTML).

**Esempio Formale di Richiesta HTTP**

```
GET /somedir/page.html HTTP/1.1\r\n
Host: www.someschool.edu\r\n
User-agent: Mozilla/4.0\r\n
Connection: close\r\n
Accept-language: fr\r\n
\r\n % Riga vuota di delimitazione
```

**Metodi HTTP e Gestione dell'Input Utente (GET vs. POST):**

- **Metodo GET**: impiegato per richiedere risorse. Se la richiesta include parametri di ricerca digitati in un form web, tali dati vengono concatenati direttamente all'**interno dell'URL** dopo il punto interrogativo (es. `www.somesite.com/search?a=2&b=5`).
- **Metodo POST**: i parametri inseriti dall'utente nel form non compaiono nell'URL, ma vengono incapsulati all'interno del **payload** (corpo) del messaggio HTTP.
- **Metodo HEAD**: analogo a GET, ma ordina al server di omettere l'oggetto richiesto nella risposta, restituendo unicamente le righe di intestazione (utilizzato per diagnostica e test di validità cache).
- **Metodi HTTP/1.1 aggiuntivi**: `PUT` (invia un file per essere memorizzato nel percorso specificato dall'URL) e `DELETE` (rimuove il file specificato nell'URL).

### 2. Messaggio di Risposta e Codici di Stato

La risposta del server inizia con la **Riga di Stato**:

Protocollo Codice di Stato Espressione di Stato

**Esempio Formale di Risposta HTTP**

```
HTTP/1.1 200 OK\r\n
Connection: close\r\n
Date: Thu, 06 Aug 1998 12:00:15 GMT\r\n
Server: Apache/1.3.0 (Unix)\r\n
Last-Modified: Mon, 22 Jun 1998 09:23:24 GMT\r\n
Content-Length: 6821\r\n
Content-Type: text/html\r\n
\r\n
<!DOCTYPE html><html>... [dati HTML trasferiti] ...</html>
```

| Codice | Espressione | Descrizione Tecnica e Comportamento |
|---|---|---|
| 200 | OK | Richiesta soddisfatta con successo. L'oggetto richiesto è contenuto nel corpo. |
| 301 | Moved Permanently | Risorsa trasferita stabilmente a un nuovo URL, indicato nell'header `Location:`. |
| 304 | Not Modified | L'oggetto in cache è ancora valido; il payload non viene reinviato (risparmio banda). |
| 400 | Bad Request | La sintassi della richiesta non è stata compresa dal server. |
| 404 | Not Found | La risorsa richiesta non è presente sul server. |
| 505 | Version Not Supported | La versione di protocollo HTTP dichiarata dal client non è supportata dal server. |

Tabella 5.3: Principali codici di stato della risposta HTTP.

## 5.3.5 Gestione dello Stato: i Cookie

Poiché HTTP è intrinsecamente stateless, le applicazioni commerciali (autenticazione, carrelli per acquisti elettronici, profilazione e suggerimenti personalizzati) necessitano di preservare la memoria dello stato utente tra transazioni distinte. Questo è ottenuto per mezzo dei **Cookie**.

### I Quattro Componenti Fondamentali del Meccanismo dei Cookie

1. Un'**intestazione nella risposta HTTP**: `Set-Cookie: <valore_id>`;
2. Un'**intestazione nella richiesta HTTP**: `Cookie: <valore_id>`;

3. Un **file gestito localmente dal browser** dell'utente che memorizza le coppie (Dominio, ID);
4. Un **database sul server** che associa all'ID le informazioni storiche, l'anagrafica, il carrello o le preferenze dell'utente.




Figura 5.5: Ciclo di vita e interazione client-server mediante cookie HTTP.

**Cookie e Tracciamento della Privacy**

I cookie costituiscono una vera e propria introduzione di un livello di sessione surrettizio al di sopra di un protocollo stateless. Essi consentono ai siti web di profilare in modo pervasivo le preferenze di navigazione dell'utente, determinando rischi rilevanti di tracciamento commerciale e manipolazione comportamentale.

### 5.3.6 Web Caching, Proxy Server e Richieste Condizionali

#### Funzionamento del Web Cache (Proxy)

Un **Web Cache** (o **Server Proxy**) è un'entità di rete che si interpone tra il browser dell'utente e il server web d'origine:

- Il browser viene configurato per inoltrare tutte le richieste HTTP al server proxy.
- Se l'oggetto richiesto risiede già nella memoria locale del proxy, quest'ultimo lo eroga immediatamente al client (**risparmio di tempo e di banda sul link di accesso dell'ISP**).
- Se l'oggetto è assente, il proxy interroga il server d'origine, ne riceve la copia, la memorizza nella propria cache e la restituisce all'utente.

#### Il Meccanismo della Richiesta Condizionale (Conditional GET)

Una copia conservata in cache rischia di diventare obsoleta se la risorsa originale sul server subisce modifiche. Per conciliare l'aggiornamento con l'efficienza, HTTP introduce la **GET Condizionale**:

1. Quando il browser o il proxy effettua la richiesta, include l'header:

   ```
   If-Modified-Since: <data_copia_in_cache>
   ```

2. Il server d'origine verifica la data di modifica della risorsa:
   - Se l'oggetto **non è mutato**, il server restituisce una risposta brevissima priva di payload:

     ```
     HTTP/1.1 304 Not Modified
     ```

     L'entità richiedente riutilizza la copia locale in cache, azzerando il consumo di banda di trasmissione!
   - Se l'oggetto **è stato modificato**, il server risponde con:

     ```
     HTTP/1.1 200 OK
     ```

     e include i nuovi byte aggiornati nel corpo della risposta.

**Declino Storico dei Proxy e Diffusione di HTTPS**

Come annotato dal prof. Lo Cigno, l'impiego dei server proxy per compiti di interconnessione e caching trasparente è andato incontro a un progressivo **declino per obsolescenza**:

- L'interconnessione tra indirizzi IP privati e pubblici è stata assorbita e standardizzata a livello di rete dagli apparati `NAT` (*Network Address Translation*).

- La transizione capillare dell'intero Web al protocollo crittografato **HTTPS** (*HTTP over TLS*) cifra end-to-end sia gli URL sia i contenuti tra browser e server finale. I proxy intermedi non sono più in grado di decifrare o ispezionare il payload per fare caching o filtraggio, costringendo i client a prelevare l'informazione crittografata direttamente dalla sorgente.

### 5.3.7 Evoluzione ad Alte Prestazioni: HTTP/2 (RFC 7540)

Nel maggio 2015, la IETF ha approvato la specifica formale di **HTTP/2** (**RFC 7540**), originata dai progetti di sperimentazione industriale condotti da Google con il protocollo **SPDY**.

HTTP/2 introduce una radicale rivoluzione architetturale:

- **Codifica Binaria e Compressione HPACK**: abbandono del formato testuale ASCII a favore di un framing binario rigidamente tipizzato, con compressione formale delle righe di intestazione (*HPACK*) per ridurre l'overhead dei byte trasmessi.
- **Singola Connessione TCP Persistente con Multiplexing**: l'intero dialogo tra un client e un dato server viene incapsulato all'interno di una **sola connessione TCP persistente**. All'interno di essa vengono aperti molteplici **stream** logici bidirezionali indipendenti, suddivisi in piccoli **frame** binari interlacciati tra loro.
- **Mitigazione dell'Head-of-the-Line (HoL) Blocking Applicativo**: in HTTP/1.1 con pipelining, se il primo oggetto in coda richiede un tempo di elaborazione lunghissimo sul server, tutti gli oggetti successivi rimangono bloccati in attesa. In HTTP/2, i frame dei vari stream viaggiano interlacciati in tempo reale: la lentezza di un singolo elemento non paralizza il rendering degli altri.
- **Server Push**: il server web, all'atto dell'invio del file `index.html`, può decidere di inviare spontaneamente in anticipo al client anche i file CSS e JS indispensabili, anticipando la successiva esplicita richiesta del browser.

**Criticità di HTTP/2 e Server Push**

Nelle lezioni frontali, il docente ha evidenziato le ombre applicative di HTTP/2: il *Server Push* riduce il controllo dell'utente, consentendo ai server di dare priorità arbitraria ai propri contenuti (inclusa la pubblicità o script di profilazione) a discapito della volontà dell'agente locale. Inoltre, poiché HTTP/2 poggia ancora su una singola connessione TCP, in presenza di un canale rumoroso la perdita di un singolo segmento fisico a livello 4 genera un **blocco HoL a livello di trasporto**, paralizzando temporaneamente tutti gli stream multiplexati (problema che la recente evoluzione di **HTTP/3** su **QUIC** risolve sostituendo TCP con flussi UDP nativi).

## 5.4 Altri Protocolli di Rete: Posta Elettronica, FTP e Flussi Multimediali

### 5.4.1 L'Architettura della Posta Elettronica

La posta elettronica rappresenta uno dei servizi applicativi storici di Internet. Contrariamente a quanto percepito nell'uso comune, il sistema si fonda su un'architettura cooperativa distribuita a tre componenti essenziali:




Figura 5.6: Architettura e protocolli del servizio di Posta Elettronica: interazione Push (SMTP) per l'invio e l'inoltro tra server, e Pull (POP3/IMAP o HTTP) per la consultazione da parte del destinatario.

1. **User Agent (Agente Utente)**: l'applicazione locale utilizzata dall'utente per la composizione, la lettura, l'editing e la cifratura dei messaggi (es. Thunderbird, Outlook, Apple Mail). Il messaggio viene redatto in locale e inviato al proprio server di riferimento solo alla pressione del comando di invio.
2. **Mail Server (Server di Posta)**: apparati sempre attivi che costituiscono l'infrastruttura dorsale del servizio. Ogni server ospita:
   - La **Casella di Posta (*Mailbox*)**: memoria su disco in cui vengono depositati i messaggi pervenuti per ciascun utente locale accreditato;

- La **Coda dei Messaggi in Uscita (*Message Queue*)**: struttura bufferizzata in cui sostano i messaggi da trasmettere verso i mail server di altre organizzazioni.

3. **Protocolli Applicativi**:
   - `SMTP`: impiegato per l'**invio** (da client a server) e per l'**instradamento cooperativo tra server**;
   - `POP3` / `IMAP` / `HTTP`: impiegati per l'**accesso e prelievo** dei messaggi depositati nella mailbox remota.

### 5.4.2 Il Protocollo SMTP (RFC 2821)

Il protocollo **SMTP** (*Simple Mail Transfer Protocol*, standardizzato originariamente nell'RFC 821 e aggiornato nell'RFC 2821) governa il trasferimento dei messaggi su connessioni `TCP` alla porta destinazione convenzionale **25**:

- **Natura Rigorosamente Push**: SMTP è un protocollo **push**. Il processo mittente (client SMTP) apre la connessione verso il processo ricevente (server SMTP) ed «immette» forzatamente i dati verso la destinazione.
- **Trasferimento Diretto tra Server**: di norma, il server del mittente interroga il DNS (record `MX`) per individuare il mail server della destinazione e vi instaura una connessione TCP diretta, senza stadi intermedi (salvo la presenza di *mail relay* aziendali o gateway di sicurezza).
- **Connessioni Persistenti**: se un mail server detiene in coda più messaggi destinati allo stesso dominio remoto, apre una sola sessione TCP e vi trasferisce tutti i messaggi in sequenza.
- **Le Tre Fasi del Trasferimento**:
  1. **Handshake Applicativo**: apertura TCP, emissione del banner `220` da parte del server ricevente e saluto `HELO` (o `EHLO`) da parte del mittente con validazione `250`;
  2. **Trasferimento Messaggi**: dichiarazione del mittente (`MAIL FROM`), del destinatario (`RCPT TO`), comando di corpo dati (`DATA`) e trasmissione riga per riga;
  3. **Chiusura**: comando `QUIT` e conferma di distruzione canale `221`.

**Trascrizione Formale di una Sessione SMTP**

```
% Il server di hamburger.edu risponde dopo l'apertura TCP:
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr...  Sender ok
C: RCPT TO: <rob@hamburger.edu>
S: 250 rob@hamburger.edu...  Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```

**Peculiarità Storiche di SMTP e Sentinella di Fine Messaggio**

Nelle registrazioni d'aula, il docente ha illustrato con ironia i tratti arcaici di SMTP:

- Il comando di handshake è deliberatamente digitato `HELO` anziché *Hello*, per renderlo una parola chiave sintattica univoca e non un termine lessicale comune.
- La terminazione del corpo dati avviene tramite la sequenza `CRLF.CRLF` (un punto isolato su una riga vuota). Si tratta di un'eredità storica derivata dal protocollo telegrafico e dai telegrammi d'epoca (*«STOP»*). Questa convenzione introduce una fragilità strutturale: se un utente inserisce un punto isolato nel testo, rischia di troncare anticipatamente il messaggio, costringendo i client a un meccanismo di escape (*dot-stuffing*).
- **Assenza Storica di Autenticazione tra Server**: originariamente i server non richiedevano

alcuna convalida crittografica per accettare comunicazioni alla porta 25 (permettendo a chiunque di inviare messaggi spacciandosi per terzi, es. Napoleone Bonaparte). Tale omissione progettuale dell'IETF ha generato la piaga del **Phishing** e dello **Spam**, che a tutt'oggi costituisce una quota rilevantissima del traffico mondiale di posta, contrastata unicamente da filtri euristici antispam, record DNS dedicati (SPF, DKIM, DMARC) e chiusura delle porte 25 aperte al pubblico.

### 5.4.3 Confronto Architetturale: HTTP vs. SMTP

Entrambi i protocolli operano al livello applicativo, impiegano codici di stato numerici a tre cifre e si fondano su dialoghi ASCII su connessioni TCP persistenti. Tuttavia, essi divergono sotto tre aspetti sistemici:

| Caratteristica | HTTP | SMTP |
|---|---|---|
| **Direzione del Flusso** | Protocollo rigorosamente **PULL**: il client richiede l'oggetto e lo riceve. | Protocollo rigorosamente **PUSH**: il client invia il dato al server. |
| **Incapsulamento Oggetti** | Ciascun oggetto web viaggia nel proprio messaggio di risposta autonomo. | Tutti gli oggetti e gli allegati vengono fusi e accorpati in un unico messaggio complessivo. |
| **Set di Caratteri** | Nessuna restrizione sui dati (binario nativo o ASCII a 8 bit). | Esclusivamente testo **ASCII a 7 bit**; dati binari richiedono codifica esterna. |

Tabella 5.4: Confronto sistematico tra i protocolli applicativi HTTP e SMTP.

### 5.4.4 Formato del Messaggio di Posta ed Estensioni MIME

I comandi SMTP (`MAIL FROM`, `RCPT TO`) non devono essere confusi con le **righe di intestazione** del messaggio vero e proprio. Un messaggio di posta si compone di:

1. **Righe di Intestazione dell'Applicazione**: `From:`, `To:`, `Subject:`, `Date:`;
2. **Riga Vuota**;
3. **Corpo del Messaggio**: stringhe di testo.

#### Estensioni Multimediali MIME (RFC 2045, 2056)

Poiché SMTP ammette esclusivamente caratteri ASCII standard a 7 bit (incapace persino di codificare vocali accentate italiane o caratteri cirillici ed orientali), per allegare immagini, file binari o documenti complessi è stato introdotto lo standard **MIME** (*Multipurpose Internet Mail Extensions*):

- Vengono aggiunte intestazioni formali che dichiarano il tipo di contenuto e la codifica:

  `MIME-Version: 1.0` e `Content-Type: image/jpeg`

- **Codifica Base64**: specifica che mappa blocchi di byte binari a 8 bit in un alfabeto ristretto di 64 caratteri ASCII stampabili sicuri (lettere maiuscole, minuscole, cifre numeriche, `+` e `/`).
- Tale codifica produce un'**espansione del volume dati**: 3 byte binari (24 bit) vengono mappati in 4 caratteri ASCII (32 bit), determinando un overhead trasmissivo del +33%, prezzo obbligato per attraversare server SMTP arcaici.

### 5.4.5 Protocolli di Accesso alla Posta: POP3, IMAP e Webmail

Poiché un client utente non è un host sempre attivo dotato di IP fisso, esso **non può ricevere posta mediante SMTP** (che è un protocollo push). Per consultare la mailbox remota, l'agente locale necessita di un protocollo **pull**:

#### 1. POP3 (Post Office Protocol – Versione 3, RFC 1939)

Modello basilare ed essenziale, analogo al ritiro fisico di corrispondenza dalla buca delle lettere:

- Si articola in tre fasi: Autorizzazione (scambio credenziali utente e password), Transazione (scaricamento dei messaggi) e Aggiornamento (chiusura della sessione).
- **Modalità Scarica e Cancella**: il client preleva i messaggi e ordina al server di eliminarli dalla mailbox. Se l'utente tenta di accedere da un secondo terminale (es. smartphone), la posta non risulta più reperibile.
- **Modalità Scarica e Mantieni**: i messaggi rimangono sul server, ma il protocollo non prevede alcun allineamento o sincronizzazione dello stato (letto/non letto) tra dispositivi eterogenei.

**2. IMAP (Internet Mail Access Protocol, RFC 1730)**

Protocollo moderno e strutturato, progettato specificamente per l'accesso multipiattaforma:

- Mantiene tutti i messaggi permanentemente archiviati sul server all'interno di una gerarchia di cartelle personalizzabili dall'utente.
- Consente a molteplici client concorrenti (computer, smartphone, tablet) di connettersi simultaneamente alla medesima casella mantenendo lo stato perfettamente **sincronizzato in tempo reale** (contrassegno dei messaggi letti, cancellati o spostati).
- Permette il download parziale o differito dei soli header o delle sole parti testuali di un messaggio, posticipando lo scaricamento di allegati pesanti.
- Risulta indispensabile nei contesti lavorativi condivisi (es. caselle di help-desk o presidi aziendali accessibili da turni di operatori differenti).

**3. Webmail (Accesso Basato su HTTP)**

In questo modello ampiamente diffuso (es. Google Gmail, Outlook Online), il browser dell'utente agisce da terminale di visualizzazione comunicando via `HTTP`/`HTTPS` con il web server del fornitore. La posta rimane confinata e gestita all'interno dell'infrastruttura Cloud del provider.

### 5.4.6 Il Protocollo FTP (File Transfer Protocol, RFC 959)

Il protocollo **FTP** costituisce una pietra miliare nello sviluppo delle reti di calcolatori:

**Definizione: FTP – Architettura Out-of-Band e Connessioni Multiple**

FTP è il protocollo concepito per trasferire file tra sistemi con architettura client-server. La sua caratteristica strutturale risiede nell'impiego simultaneo di **due connessioni TCP distinte**:

1. **Connessione di Controllo (Porta TCP 21)**: connessione **fuori banda** (*out-of-band*) e permanente per l'intera durata della sessione. Trasporta i comandi dell'utente (autenticazione, navigazione directory con `CWD`, comandi `LIST`, `RETR`, `STOR`) e i relativi codici di stato di risposta del server.
2. **Connessione Dati (Porta TCP 20)**: connessione aperta dinamicamente per il solo tempo necessario a trasferire il contenuto di un singolo file o l'elenco dei file di una cartella, e chiusa immediatamente al termine di ciascun trasferimento.

Inoltre, il server FTP è uno dei rari protocolli applicativi tradizionali che **mantiene lo stato della sessione**: esso traccia costantemente quale utente sia autenticato e quale sia la directory di lavoro corrente all'interno dell'albero del file system remoto. Sebbene oggi sia stato in gran parte soppiantato dall'interfaccia Web HTTP e da canali protetti basati su SSH (`SFTP`), FTP conserva un eccezionale valore didattico per l'astrazione di controllo fuori banda.

### 5.4.7 Cenni a Protocolli Applicativi Moderni: Messaggistica e Streaming

In chiusura del corso, il docente ha fornito una panoramica sui protocolli che governano le applicazioni emergenti contemporanee:

- **Messaggistica Istantanea**:
  - `XMPP (Extensible Messaging and Presence Protocol)`: protocollo aperto basato sulla serializzazione di stream XML, originariamente impiegato dai servizi Jabber e alla base delle prime versioni di WhatsApp.
  - `MTProto`: protocollo crittografico proprietario a bassissimo overhead incapsulato su HTTPS/TCP (o UDP) impiegato dall'infrastruttura di Telegram.
- **Comunicazioni Multimediali e Telefonia su Internet (VoIP)**:
  - `SIP (Session Initiation Protocol)`: protocollo di segnalazione testuale (ispirato a HTTP e SMTP) standardizzato da IETF per instaurare, modificare e abbattere sessioni di comunicazione multimediale bidirezionale.
  - `RTP (Real-time Transport Protocol)` e `RTCP`: protocolli operanti su UDP dedicati rispettivamente al trasporto dei pacchetti audio/video con numerazione progressiva e timestamp, e al monitoraggio diagnostico della qualità del flusso (misura del jitter e della perdita di pacchetti).
  - `RTSP (Real-Time Streaming Protocol)`: funge da telecomando di rete per flussi video a richiesta (comandi `PLAY`, `PAUSE`, `RECORD`).

- **WebRTC (*Web Real-Time Communication*)**: Architettura promossa a standard congiunto W3C/IETF che consente ai browser web di instaurare comunicazioni voce, video e dati peer-to-peer ad altissime prestazioni senza necessità di installare plugin o software esterni, avvalendosi di un server centrale unicamente per la negoziazione iniziale dei parametri e integrandosi perfettamente nell'ecosistema Web moderno.